

Claude Code 2026 입문 가이드

에이전틱 코딩 도구를 입문부터 실무까지 (Windows 11 기준)

Claude Code 2026

목차

1장. Claude Code란 무엇인가: 에이전틱 코딩의 시작 [입문]

- 에이전틱 코딩(agentic coding)이란 - 자동완성을 넘어서
- Claude Code가 동작하는 표면(CLI·IDE·데스크톱·웹)
- 무엇을 할 수 있나 - 읽기·편집·실행·검색 역량 개관
- 에이전트 루프(agent loop)와 사람의 역할
- 이 책의 학습 지도와 안전 실습 원칙(Windows 11 기준)

2장. 설치·인증·첫 세션 (Windows 11) [입문]

- 설치 준비 - Windows 11 환경 점검
- 설치하기 - npm과 네이티브 설치 관리자
- 로그인과 인증(authentication) - 계정·구독·API 키
- 첫 세션 시작·종료·재개 (C:\w 프로젝트 폴더에서)
- 화면 구성·상태 줄(status line)·기본 단축키

3장. 기본 흐름: 대화·파일 편집·터미널·권한 첫걸음 [입문]

- 대화로 일 시키기(prompting) - 명확한 요청 만들기
- 파일 읽기·편집과 변경 미리보기(diff)
- 터미널 명령 실행과 결과 해석
- 권한 요청과 승인의 첫 만남
- 컨텍스트 관리 기초 - 압축(compaction)과 세션 위생

4장. 권한·승인 모드 심화와 안전 [기초]

- 권한 모델(permission model)의 원리

- 승인 모드 비교 - 매번 묻기·편집 자동수락·계획 모드·우회
- 허용·거부·질문 규칙(allow/deny/ask) 작성
- 위험 작업 가드 - 민감 경로·파괴적 명령 차단
- Windows 11에서 안전 실습 환경 구성

5장. CLAUDE.md: 프로젝트·개인·계층 메모리 [기초]

- 메모리(memory)란 - 반복 설명을 없애는 지속 컨텍스트
- 메모리 계층과 우선순위(전역·프로젝트·하위 폴더)
- 잘 쓰는 CLAUDE.md - 구조·규칙·금지·임포트
- 개인 메모리와 팀 공유 메모리 분리
- 메모리 디버깅 - 무엇이 로드됐는지 확인

6장. 슬래시 커맨드: 내장과 커스텀 [기초]

- 슬래시 커맨드(slash command)와 내장 명령 둘러보기
- 커스텀 명령 만들기(.claude/commands)
- 인자·파일참조·bash 실행을 명령에 넣기
- 명령에서 도구 권한·모델 지정(frontmatter)
- 팀과 명령 공유·플러그인(plugin)으로 배포

7장. 서브에이전트 설계와 위임 (.claude/agents) [중급]

- 서브에이전트(subagent)와 독립 컨텍스트의 원리
- 에이전트 정의하기 - frontmatter 구조
- 자동 위임 vs 명시 호출 - description 설계
- 도구 권한 최소화과 역할 프롬프트 설계
- 멀티 에이전트 협업 패턴(에이전트 팀)과 한계

8장. Hooks: 이벤트 자동화와 가드레일 [중급]

- 훅(hook)이란 - 확정적 가드레일의 원리
- 훅 이벤트 전체 지도(PreToolUse·PostToolUse·UserPromptSubmit 외)
- 훅 입출력 계약 - JSON·exit code·decision
- 실전 훅 - 민감파일 차단·저장 시 포매팅·로깅
- Windows에서 훅 작성 시 주의(PowerShell·경로·인코딩)

9장. MCP 서버 연동과 직접 작성 [중급]

- MCP(Model Context Protocol)의 구조와 의미
- MCP 서버 추가와 설정 범위(stdio·SSE·HTTP)

- MCP 도구·리소스·프롬프트 사용하기
- 직접 만드는 MCP 서버 - 최소 구현
- MCP 보안 - 신뢰·권한·비밀값

10장. settings.json 거버넌스·환경변수·팀 공유 정책 [실무]

- settings.json 전체 구조와 병합 우선순위
- 환경변수와 비밀값 관리
- 관리형 정책으로 조직 거버넌스 강제
- 모델·상태 줄·출력 스타일·도구 권한 다듬기
- 설정을 저장소에 담기 - 공유 vs 로컬 분리

11장. 여러 인터페이스와 팀 협업·CI 자동화 [실무]

- 인터페이스별 강점과 연속성(CLI·IDE·데스크톱·웹)
- 비대화형 실행(headless)과 CI 파이프라인 통합
- GitHub Actions·자동 코드 리뷰 워크플로
- 팀 표준화 - 명령·에이전트·훅·플러그인 배포
- 비용·관측·감사(audit)와 사용량 거버넌스

12장. Agent SDK 실전·고급 워크플로·트러블슈팅 [실무]

- 고급 워크플로 - 계획 모드·백그라운드·멀티에이전트
- Agent SDK 개요(Python·TypeScript)와 query 루프
- SDK로 커스텀 에이전트 만들기 - 도구·권한·서브에이전트
- 트러블슈팅 - 인증·권한·MCP·컨텍스트 진단
- 다음 단계 - 심화 자료·커뮤니티·계속 학습

Claude Code란 무엇인가: 에이전틱 코딩의 시작

학습 목표 - 에이전틱 코딩(agentic coding)이 기존 자동완성과 무엇이 다른지 한 문장으로 설명할 수 있습니다. - Claude Code가 제공되는 여러 표면(CLI·IDE·데스크톱·웹)을 나열하고 공통 엔진 개념을 이해합니다. - Claude Code가 잘하는 일과 사람이 챙겨야 하는 일을 구분할 수 있습니다. - 에이전트 루프(agent loop)의 4단계와 사람이 개입하는 지점을 설명할 수 있습니다. - 이 책의 12장 학습 지도와 Windows 11 안전 실습 원칙을 이해합니다.

이 장은 도구를 설치하기 전에 "**Claude Code가 도대체 무엇이고, 왜 지금까지 쓰던 도구와 다른가**"를 먼저 그림으로 잡는 장입니다. 용어를 처음 듣는 분도 따라올 수 있도록 일상 비유를 충분히 들겠습니다. 설치와 첫 실행은 다음 장에서 다루므로, 이 장에서는 손으로 따라 하기보다 **개념을 머릿속에 정확히 심는 것이 목표**입니다.

먼저 이 책에서 자주 나오는 두 낱말을 짧게 풀어 두겠습니다. **터미널(terminal)**은 그림 버튼 대신 글자(명령어)를 입력해 컴퓨터에 일을 시키는 검은 화면입니다. Windows 11에서는 보통 **PowerShell**이라는 터미널을 씁니다. **CLI(Command Line Interface, 명령줄 인터페이스)**는 바로 그렇게 "글자로 명령을 주고받는 방식"을 가리키는 말입니다. 이 책의 명령 화면에 자주 보이는 `PS C:\Users\사용자\my-project>`는 "지금 PowerShell에서 `my-project` 폴더에 들어와 명령을 기다리는 중"이라는 표시입니다.

에이전틱 코딩(agentic coding)이란 - 자동완성을 넘어서

도입

워드에서 "안녕하" 까지 치면 "안녕하세요"를 회색 글씨로 제안해 주는 기능을 본 적이 있을 것입니다. 코드 편집기의 **자동완성(autocomplete)**도 그와 같습니다. 사람이 한 줄씩 입력하면 그 다음에 올 법한 글자를 제안할 뿐, 일을 대신 끝내 주지는 않습니다. 에이전틱 코딩은 여기서 한 걸음 더 나아가 **목표를 통째로 맡기는** 방식입니다.

핵심 개념 설명

에이전틱 코딩 (agentic coding)

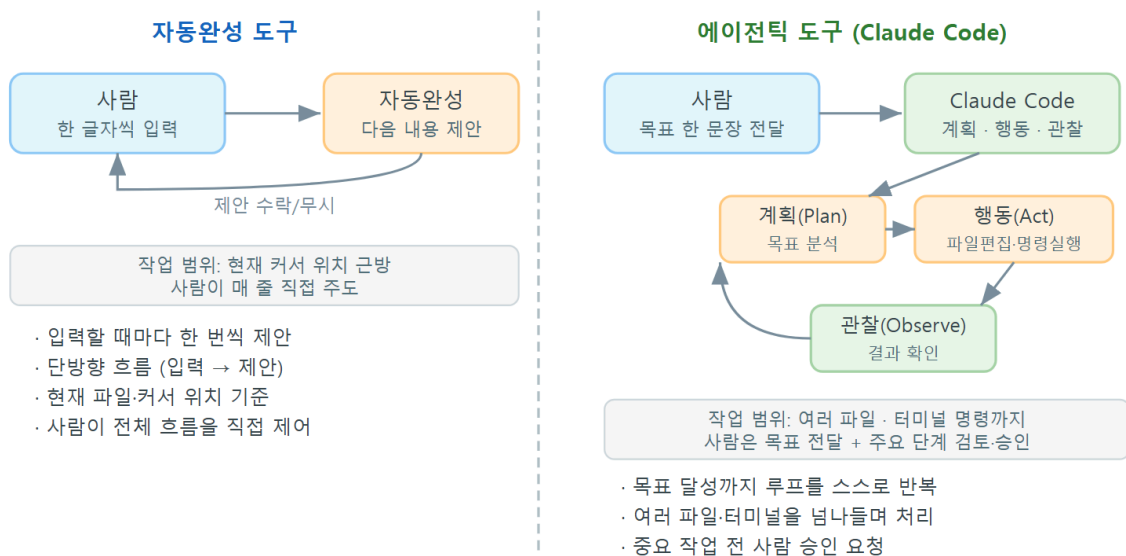
에이전틱 코딩(agentic coding)이란, 도구가 사람에게서 **목표**를 받아 **스스로 파일을 읽고, 고치고, 명령을 실행하고, 그 결과를 보고 다음에 무엇을 할지 정하는** 방식의 코딩을 말합니다. 여기서 '에이전트(agent)'는 '대리인', 즉 나를 대신해 여러 단계를 알아서 처리해 주는 일꾼이라는 뜻입니다.

비유로 풀면 이렇습니다. 자동완성은 **한 줄씩 받아쓰게 하는 비서**입니다. 내가 부르는 만큼만 받아 적고, 다음 문장은 내가 또 불러 줘야 합니다. 반면 에이전틱 도구는 **"이번 분기 보고서 정리해 줘"**라고 말기면 **자료를 찾아 표를 만들고 초안까지 가져오는 비서**입니다. 내가 매 글자를 부르지 않아도, 목표만 주면 중간 단계를 스스로 밟습니다.

조금 더 들어가 보겠습니다. 자동완성과 에이전틱 코딩의 진짜 차이는 **"누가 다음 행동을 정하는가"**에 있습니다.

- **왜(why) 필요한가:** 실제 일은 한 줄로 끝나지 않습니다. "파일을 찾고 → 고치고 → 실행해 보고 → 틀리면 다시 고치는" 여러 단계가 이어집니다. 그 반복을 사람이 매번 손으로 하면 느리고 지칩니다. 에이전틱 코딩은 이 반복을 도구에 맡깁니다.
- **언제(when) 빛나는가:** 단계가 여러 개로 이어지는 일, 예를 들어 "여러 파일에 흩어진 같은 오타를 모두 고치기", "코드를 고친 뒤 테스트까지 돌려 보기" 처럼 *과정*이 있는 작업에서 강합니다.
- **어떻게(how) 다른가:** 자동완성은 사람이 행동하고 도구는 제안만 합니다. 에이전틱 코딩은 사람이 목표를 말하고, 도구가 행동하며, 사람은 그 행동을 확인·승인합니다.

자동완성 도구 vs 에이전틱 도구 비교



ch_01_s01 · 에이전틱 코딩(agentic coding)이란

이렇게 동작합니다

다음은 자동완성과 에이전틱 코딩에서 "사람이 하는 말"이 어떻게 다른지를 보여 주는 예입니다. 왼쪽처럼 글자를 이어 주는 것이 아니라, 오른쪽처럼 **목표 한 문장**을 건넵니다. 아래는 실제 입력이 아니라 차이를 보여 주기 위한 비교용 예시입니다.

[자동완성에 기대하는 것]

사람: def add(a, b):
 return a + <- 여기서 "b" 를 회색으로 제안

[에이전틱 코딩에 말하는 것]

사람: 장바구니 합계가 음수로 나오는 버그를 찾아서 고치고, 테스트도 돌려 줘

Claude Code에게 목표를 건네면, 실제 화면에서는 다음과 비슷하게 도구가 스스로 단계를 밟아 나갑니다. 대괄호로 표시된 [Read], [Edit] 같은 표기는 Claude Code가 **지금 어떤 작업(읽기·편집 등)을 하는 중인지**를 알려 주는 안내입니다.

PS C:\Users\사용자\my-project> claude

> 장바구니 합계가 음수로 나오는 버그를 찾아서 고치고, 테스트도 돌려 줘

[Read] cart.py 를 읽었습니다

[Search] "total" 이 쓰인 곳 3군데를 찾았습니다

[Edit] cart.py 의 빼기(-)를 더하기(+)로 고치겠습니다 (승인하시겠어요?)

[Bash] pytest 실행 ... 통과 5건

완료했습니다. 합계 계산의 부호 오류를 고치고 테스트를 통과했습니다.

화면 읽는 법

표시	의미
> ...	사람이 Claude Code에게 건넨 목표(지시) 입니다
[Read]	파일을 읽는 중이라는 안내입니다
[Search]	코드베이스에서 단어·패턴을 검색 하는 중입니다
[Edit]	파일을 고치기 직전 사람에게 확인을 구하는 단계입니다
[Bash]	테스트 같은 명령을 실행 하는 중입니다

팁: 좋은 목표 문장에는 "무엇을(버그 수정)"과 "어떻게 확인할지(테스트 통과)"가 함께 들어갑니다. 끝 상태를 함께 알려 주면 도구가 언제 멈춰야 할지 스스로 판단하기 쉽습니다.

흔한 오해 두 가지 - "그냥 자동완성이 똑똑해진 것 아닌가요?" → 아닙니다. 자동완성은 *제안*만 하고, 에이전틱 코딩은 *행동*까지 합니다. 핵심 차이는 '여러 단계를 스스로 반복하느냐'입니다. - "그럼 사람은 손을 떼도 되나요?" → 아닙니다. 파일을 고치거나 명령을 실행하기 전에 사람이 확인·승인하는 지점이 있습니다(자세한 내용은 뒤의 에이전트 루프 절에서 다룹니다).

절 요약

- 자동완성은 한 줄씩 *제안*하고, 에이전틱 코딩은 목표를 받아 여러 단계를 스스로 *행동*합니다.
- 사람의 역할은 "글자 입력"에서 "목표 제시와 확인·승인"으로 바뀝니다.
- 단계가 이어지는 반복 작업에서 가장 큰 효과를 냅니다.

Claude Code가 동작하는 표면(CLI·IDE·데스크톱·웹)

도입

같은 은행 계좌라도 ATM에서, 휴대폰 앱에서, 창구에서 모두 꺼내 쓸 수 있습니다. 창구가 여러 개라고 통장이 여러 개인 것은 아닙니다. Claude Code도 마찬가지로, **하나의 엔진을 여러 창구에서 쓰는 구조**입니다. 이 절에서는 그 창구들의 종류와, 이 책이 어느 창구를 중심으로 가르치는지를 정리합니다.

핵심 개념 설명

인터페이스 표면 (interface surface)

인터페이스 표면(interface surface)이란, 같은 Claude Code 엔진을 사용하는 **서로 다른 사용 창구**를 가리킵니다. 여기서 '표면'은 사람이 도구와 맞는 바깥쪽 화면이라는 뜻입니다. 대표적인 표면은 다음과 같습니다.

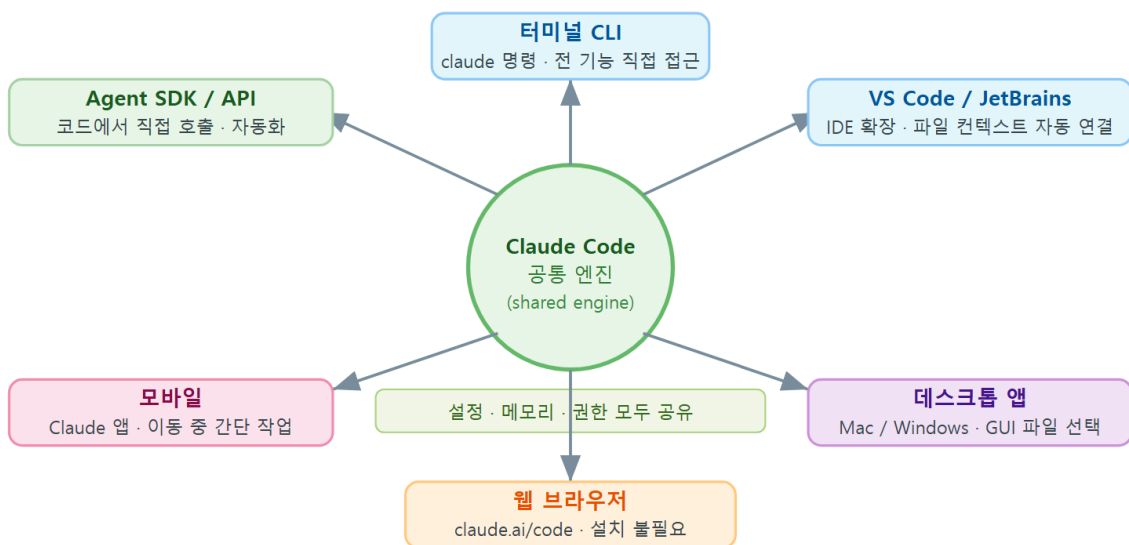
- **CLI(명령줄)**: 터미널에서 `claude` 라고 입력해 글자로 대화하는 방식입니다. 가장 가볍고, 모든 기능이 가장 빨리 도착하는 기본 창구입니다.
- **IDE 확장**: VS Code, JetBrains 같은 **코드 편집기 안에서** Claude Code를 부르는 방식입니다. IDE(Integrated Development Environment, 통합 개발 환경)는 코드를 쓰고 실행하는 통합 작업 공간을 뜻합니다.
- **데스크톱 앱**: 별도 창으로 띄워 쓰는 프로그램 형태의 창구입니다.
- **웹**: 브라우저에서 접속해 쓰는 창구입니다.

공통 엔진 (shared engine)

네 창구가 따로 노는 것이 아니라, 그 아래에는 **공통 엔진(shared engine)** 하나가 있습니다. 그래서 어느 창구에서 설정하든 **같은 설정·메모리·권한**을 공유합니다. 예를 들어 프로젝트 규칙을 적어 두는 `CLAUDE.md` 파일(뒤에서 자세히 배웁니다)을 한 번 만들어 두면, CLI에서 쓰든 IDE에서 쓰든 똑같이 적용됩니다.

- **왜(why) 이렇게 나눠 두었나:** 사람마다 일하는 자리가 다릅니다. 터미널이 편한 사람, 편집기를 떠나기 싫은 사람, 가볍게 브라우저로만 보고 싶은 사람이 모두 같은 도구를 자기 방식대로 쓰게 하려는 것입니다.
- **언제(when) 어떤 표면을 쓰나:** 빠르게 한 가지 일을 시킬 때는 CLI, 코드를 보면서 같이 고칠 때는 IDE 확장, 자리에 앉아 길게 작업할 때는 데스크톱, 잠깐 확인만 할 때는 웹이 편합니다.
- **어떻게(how) 고르나:** 처음 배우는 단계에서는 **하나만 깊게** 익히는 편이 좋습니다. 창구마다 화면은 달라도 동작 원리는 같기 때문에, 한 표면을 제대로 익히면 나머지는 금방 옮겨탈 수 있습니다.

Claude Code 인터페이스 표면 — 허브-스포크 구조



ch_01_s02 · 인터페이스 표면(interface surface) · 공통 엔진(shared engine)

표면별 한눈에 보기

다음 표는 네 표면을 언제 쓰면 좋은지 정리한 것입니다.

표면	사용하는 곳	어울리는 상황	입문자 추천도
CLI(명령줄)	PowerShell 등 터미널	빠른 한 가지 작업, 자동화, 학습	가장 먼저
IDE 확장	VS Code·JetBrains	코드를 보며 함께 편집	익숙해진 뒤
데스크톱 앱	별도 프로그램 창	자리에 앉아 길게 작업	선택
웹	브라우저	가벼운 확인·짧은 작업	선택

이 책은 **CLI(명령줄)**를 중심으로 가르칩니다. 이유는 두 가지입니다. 첫째, CLI는 가장 가볍고 어디서나 똑같이 동작하므로 개념을 또렷하게 익히기 좋습니다. 둘째, 새 기능이 가장 먼저 도착하는 창구가 CLI입니다. 여기서 익힌 개념은 IDE·데스크톱·웹에서도 그대로 통합니다.

흔한 오해: "표면마다 완전히 다른 프로그램이겠지" 라고 생각하기 쉽습니다. 그러나 표면은 겉 모습만 다를 뿐, 그 아래 엔진과 설정은 하나입니다. 그래서 한 곳에서 만든 규칙·권한이 다른 곳에도 그대로 적용됩니다.

절 요약

- 표면(CLI·IDE·데스크톱·웹)은 같은 엔진을 쓰는 서로 다른 창구입니다.
- 설정·메모리·권한은 표면을 넘나들며 공유됩니다.
- 이 책은 CLI를 중심으로 배우며, 여기서 익힌 개념은 다른 표면에도 통합니다.

무엇을 할 수 있나 - 읽기·편집·실행·검색 역량 개관

도입

새 직원이 들어오면 "이 친구는 무엇을 맡길 수 있고, 무엇이 아직 손대면 안 되는가"를 먼저 파악합니다. Claude Code도 똑같습니다. 이 절에서는 Claude Code가 가진 **네 가지 기본 역량**과, 잘하는 일·사람이 챙겨야 할 일을 나눕니다.

핵심 개념 설명

에이전트 도구 (agent tools)

Claude Code가 목표를 처리하려고 사용하는 능력을 **도구(agent tools)** 라고 부릅니다. 사람이 손과 눈으로 일하듯, Claude Code는 다음 네 가지 도구로 일합니다.

- **읽기(Read)**: 파일 내용을 읽어 상황을 파악합니다. 사람으로 치면 "자료를 먼저 훑어보는" 단계입니다.
- **편집(Edit)**: 파일을 고치거나 새로 만듭니다. 실제로 변화를 만드는 단계입니다.
- **실행(Run/Bash)**: 테스트나 명령을 실행해 결과를 직접 확인합니다.
- **검색(Search)**: 코드·문서 더미에서 필요한 단어나 패턴이 어디 있는지 찾습니다.

이 네 가지가 합쳐지면 "찾아서(검색) → 읽고(읽기) → 고치고(편집) → 돌려 보는(실행)" 한 흐름이 됩니다. 바로 이 흐름이 다음 절에서 배울 **에이전트 루프**의 재료입니다.

이렇게 동작합니다

다음은 "오래된 함수 이름을 새 이름으로 모두 바꿔 달라"는 목표를 줬을 때, 네 역량이 차례로 쓰이는 예상 화면입니다.

```
PS C:\Users\사용자\my-project> claude
> getUser 라는 함수 이름을 fetchUser 로 프로젝트 전체에서 바꿔 줘

[Search] "getUser" 가 쓰인 곳 4개 파일, 9군데를 찾았습니다
[Read]   해당 파일들을 읽어 사용 맥락을 확인했습니다
[Edit]   9군데를 fetchUser 로 바꾸겠습니다 (변경 내용을 보여드립니다)
[Bash]   npm test 실행 ... 통과
완료했습니다. 9군데를 모두 바꾸고 테스트로 깨진 곳이 없는지 확인했습니다.
```

네 역량이 하는 일

역량	화면 표시	사람으로 치면
검색	[Search]	흩어진 자료에서 필요한 곳 찾기
읽기	[Read]	자료를 훑어 맥락 파악하기
편집	[Edit]	실제로 고쳐 쓰기
실행	[Bash]	고친 결과를 직접 돌려 확인하기

잘하는 일과 사람이 챙겨야 하는 일

역량과 한계 (capabilities and limits)

도구를 잘 쓰려면 **잘하는 일**과 **사람이 챙겨야 하는 일**을 구분해야 합니다. 이를 **역량과 한계 (capabilities and limits)** 라고 합니다.

구분	내용
잘하는 일	여러 파일에 걸친 반복 수정, 코드베이스 탐색, 정해진 절차의 테스트 실행, 초안 작성
사람이 챙길 일	"무엇이 옳은 결과인가" 판단, 보안·민감 정보 결정, 사업적 우선순위, 최종 승인

- **왜(why) 한계를 알아야 하나:** 도구는 "어떻게(how)"를 빠르게 처리하지만, "무엇이 맞는가(what is right)"는 사람의 몫입니다. 한계를 모르면 결과를 그대로 믿어 버리는 실수를 합니다.
- **언제(when) 사람이 꼭 봐야 하나:** 파일을 지우거나, 외부에 영향을 주는 명령을 실행하거나, 민감한 데이터를 다룰 때는 반드시 사람이 확인합니다.
- **어떻게(how) 챙기나:** 변경 내용을 미리 보여 달라고 하고, 테스트로 검증하고, 중요한 변경은 사람이 직접 읽고 승인하는 습관을 들입니다.

베스트 프랙티스: Claude Code에게 일을 맡길 때는 "고친 다음 테스트로 확인까지 해 줘" 처럼 **검증 방법**을 함께 요청합니다. 도구가 스스로 자기 결과를 점검하게 만드는 것이 결과 품질을 높이는 가장 쉬운 방법입니다.

절 요약

- Claude Code는 읽기·편집·실행·검색 네 역량으로 일합니다.
- 네 역량이 합쳐져 "찾고-읽고-고치고-돌려 보는" 흐름이 됩니다.
- "어떻게"는 도구가, "무엇이 옳은가"는 사람이 책임집니다.

에이전트 루프(agent loop)와 사람의 역할

도입

국을 끓일 때 우리는 간을 보고, 부족하면 소금을 더 넣고, 다시 맛보기를 완성될 때까지 반복합니다. "맛보기 → 조절 → 다시 맛보기"의 이 반복이 바로 Claude Code가 일하는 방식, **에이전트 루프(agent loop)** 와 똑 닮았습니다. 이 절은 이 책에서 **가장 중요한 개념**을 다룹니다.

핵심 개념 설명

에이전트 루프 (agent loop)

에이전트 루프(agent loop)란, **계획(plan) → 행동(act) → 관찰(observe) → 다시 계획**으로 도는 반복 과정이며, **목표가 달성되면 멈춥니다**. 여기서 '루프(loop)'는 '되풀이되는 고리'라는 뜻입니다. 네 단계를 풀면 다음과 같습니다.

- **계획(plan)**: 목표를 보고 "무엇부터 할지" 순서를 세웁니다.
- **행동(act)**: 파일을 편집하거나 명령을 실행합니다. 실제로 무언가를 바꾸는 단계입니다.
- **관찰(observe)**: 행동의 결과를 확인합니다. 테스트가 통과했는지, 오류가 났는지 봅니다.
- **다시 계획**: 결과를 보고 "끝났으면 멈추고, 아니면 다음 행동"을 정합니다.

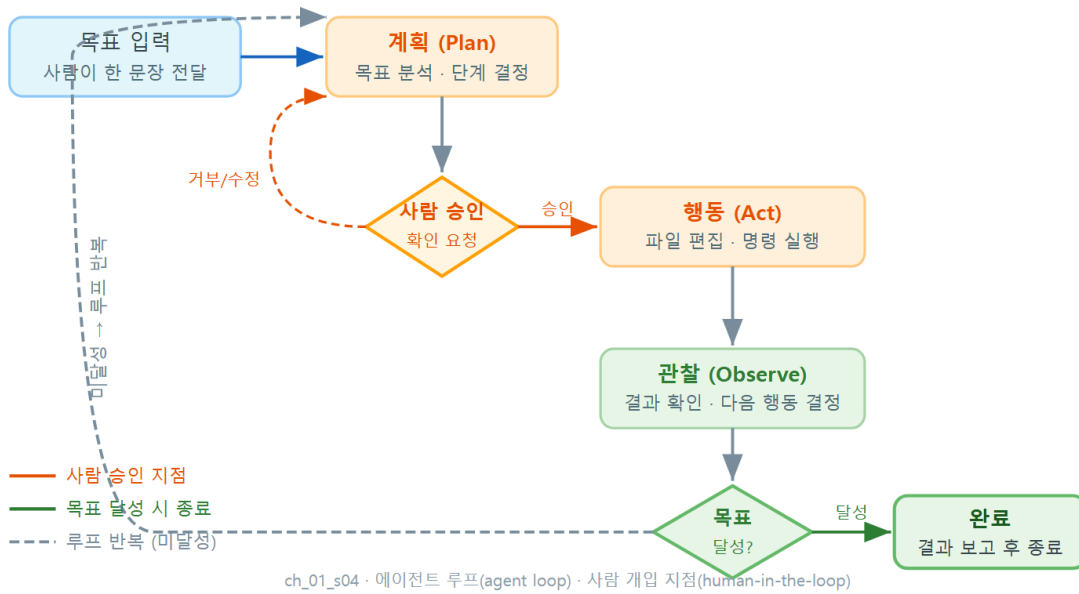
요리 비유로 다시 보면, 계획은 "간을 맞춰야겠다", 행동은 "소금을 넣는다", 관찰은 "맛을 본다", 다시 계획은 "충분하면 끝, 모자라면 한 번 더"입니다. 한 번에 끝나는 것이 아니라 **목표에 닿을 때까지 고리를 돈다**는 점이 핵심입니다.

사람 개입 지점 (human-in-the-loop)

루프가 사람 없이 무한정 도는 것은 아닙니다. 중요한 **행동(act) 직전**에 사람이 확인·승인하는 지점이 있습니다. 이를 **사람 개입 지점(human-in-the-loop)** 이라고 합니다. 'human-in-the-loop'는 "고리 안에 사람이 들어가 있다", 즉 자동 흐름 중간에 사람의 판단이 끼어든다는 뜻입니다.

- **왜(why) 필요한가**: 파일 삭제나 외부에 영향을 주는 명령처럼 되돌리기 어려운 행동은, 실행 전에 사람이 한 번 막아 줄 수 있어야 안전합니다.
- **언제(when) 끼어드나**: 보통 파일을 고치거나([Edit]), 명령을 실행하기([Bash]) 직전에 승인을 묻습니다. 단순히 읽기만 할 때는 묻지 않는 경우가 많습니다.
- **어떻게(how) 조절하나**: 매번 묻게 할 수도, 안전한 작업은 자동 허용하게 할 수도 있습니다. 이 승인 방식(권한 설정)은 뒤 장에서 자세히 배웁니다.

에이전트 루프 (Agent Loop)



이렇게 동작합니다

다음은 "README 파일에 설치 방법 항목을 추가해 줘"라는 목표가 루프를 따라 처리되는 예상 화면입니다. 각 줄 옆에 어떤 단계인지 표시해 두었습니다.

```

PS C:\Users\사용자\my-project> claude
> README 에 '설치 방법' 항목을 추가하고, 형식이 깨지지 않았는지 확인해 줘

[계획]  README 를 읽고 추가할 위치를 정하겠습니다
[Read]  README.md 를 읽었습니다                <- 관찰(현재 상태 파악)
[Edit]  '설치 방법' 항목을 추가하겠습니다 (승인하시겠어요?) <- 행동 + 사람 개입 지점
사람:    네, 진행해 주세요
[Bash]  markdown 형식 검사 실행 ... 이상 없음   <- 관찰(결과 확인)
[계획]  목표를 달성했으므로 종료합니다          <- 다시 계획 -> 종료 분기
완료했습니다.
    
```

루프 단계와 화면 대응

화면	루프 단계	설명
[계획]	계획(plan)	다음에 할 일의 순서를 세웁니다
[Read] · [Bash] 결과	관찰(observe)	현재 상태·실행 결과를 확인합니다
[Edit] · [Bash]	행동(act)	실제로 파일을 고치거나 명령을 실행합니다
승인 요청 표시	사람 개입 지점	행동 직전 사람이 확인·승인합니다
종료합니다	종료 분기	목표가 달성되면 루프를 빠져나옵니다

흔한 오해 두 가지 - "한 번 지시하면 한 번만 동작하겠지" → 아닙니다. 목표에 닿을 때까지 계획-행동-관찰을 여러 번 반복합니다. - "사람 확인 없이 무한히 돌까 봐 무섭다" → 아닙니다. 중요한 행동 전에는 승인을 묻고, 목표를 달성하면 스스로 멈춥니다.

절 요약

- 에이전트 루프는 계획→행동→관찰→다시 계획으로 돌며, 목표를 이루면 멈춥니다.
- 행동 직전에 사람이 확인하는 '사람 개입 지점'이 있어 안전합니다.
- 이 루프를 이해하면 Claude Code의 모든 동작이 한 그림으로 보입니다.

이 책의 학습 지도와 안전 실습 원칙(Windows 11 기준)

도입

운전을 처음 배울 때는 복잡한 도로가 아니라 한적한 연습장에서 시작합니다. 도구를 배울 때도 똑같이, **안전한 연습 환경**부터 마련해야 합니다. 이 절에서는 이 책 전체의 학습 지도와, Windows 11에서 안전하게 실습하기 위한 원칙을 정리합니다.

핵심 개념 설명

학습 로드맵 (learning roadmap)

이 책은 12개 장을 **입문 → 기초 → 중급 → 실무** 네 단계로 나눠 차근차근 올라가도록 짰습니다. 이 단계 구성을 **학습 로드맵(learning roadmap)** 이라고 합니다. '로드맵'은 "어디서 출발해 어디로 가는지 보여 주는 길 안내도"라는 뜻입니다.

단계	장	배우는 내용
입문	1~3장	개념 이해, 설치·인증·첫 세션, 기본 흐름(대화·편집·권한)
기초	4~6장	개념을 연결하며 왜 그렇게 쓰는지 이해
중급	7~9장	설정의 깊이와 내부 동작 원리
실무	10~12장	실제 업무 사례·팀 적용·선택의 트레이드오프

지금 읽고 있는 1장은 **입문 단계의 첫 장**으로, 개념의 큰 그림을 잡는 데 집중합니다. 손으로 직접 해 보는 실습은 도구를 설치하는 2장부터 본격적으로 시작됩니다.

안전 실습 (safe practice)

안전 실습(safe practice)이란, 도구를 안전하게 익히기 위한 기본 원칙입니다. 핵심은 세 가지입니다.

- **실습 전용 폴더 분리**: 중요한 실제 프로젝트가 아니라, 망가져도 괜찮은 연습용 폴더에서 시작합니다.
- **변경 전 백업·버전 관리**: 고치기 전 상태를 저장해 두면 언제든지 되돌릴 수 있습니다.
- **권한 확인 습관**: 도구가 무엇을 하려는지 확인하고 승인하는 습관을 들입니다.

이렇게 준비합니다

다음은 Windows 11의 PowerShell에서 **실습 전용 폴더를 따로 만드는** 예시입니다. 사용자가 직접 입력하는 명령이므로 PowerShell 형식으로 보여 드립니다. (# 으로 시작하는 줄은 설명용 주석이며, 실행되지 않습니다.)

```
# 홈 폴더로 이동합니다 ($env:USERPROFILE 는 C:\Users\사용자 를 가리킵니다)
cd $env:USERPROFILE

# 'claude-practice' 라는 실습 전용 폴더를 만듭니다
mkdir claude-practice

# 만든 폴더 안으로 들어갑니다
cd claude-practice
```

위 명령을 실행하면 프롬프트가 다음처럼 바뀌어, 지금 실습 폴더 안에 있음을 보여 줍니다.

```
PS C:\Users\사용자\claude-practice>
```


명령 설명

명령	의미
<code>cd \$env:USERPROFILE</code>	내 홈 폴더(C:\Users\사용자)로 이동합니다
<code>mkdir claude-practice</code>	claude-practice 라는 새 폴더를 만듭니다
<code>cd claude-practice</code>	방금 만든 폴더 안으로 들어갑니다

주의: 회사의 실제 운영 코드나 중요한 문서가 들어 있는 폴더에서 처음부터 연습하지 마십시오. 실습 단계에서는 반드시 위처럼 **연습 전용 폴더**를 따로 만들어 그 안에서만 시도합니다. 익숙해진 뒤 실제 프로젝트로 옮겨도 늦지 않습니다.

팁: 가능하면 실습 폴더를 Git 같은 버전 관리로 묶어 두면, 도구가 고친 내용을 한눈에 비교하고 마음에 들지 않으면 되돌릴 수 있습니다. Git이 무엇인지는 아직 몰라도 됩니다. 뒤 장에서 다룹니다.

절 요약

- 이 책은 입문→기초→중급→실무 12장으로 단계적으로 올라갑니다.
- 안전 실습의 핵심은 전용 폴더 분리, 백업·버전 관리, 권한 확인 습관입니다.
- Windows 11에서는 PowerShell과 `C:\Users\사용자` 경로를 기준으로 실습합니다.

실습 과제

해보기: 내 업무에 맞는 활용 시나리오 지도 그리기

아직 도구를 설치하지 않았어도 할 수 있는, 머리로 정리하는 과제입니다. 이 장에서 배운 *에이전트 루프*와 *사람 개입 지점*을 내 일에 대입해 봅니다.

- **단계 1:** 자신의 실제 업무(코드·문서·데이터 정리 등)에서 Claude Code에게 맡기고 싶은 작업 **5가지**를 적습니다. 예: "회의록 초안 정리", "여러 파일에서 같은 오타 찾아 고치기".
- **단계 2:** 각 작업이 에이전트 루프의 **어느 단계에서 사람 확인이 필요한지** 아래 표 형식으로 정리합니다.
- **점검:** 5가지 중 "사람 확인 없이 도구에 통째로 맡겨도 안전한 것"과 "반드시 사람이 승인해야 하는 것"을 구분해 표시할 수 있으면 성공입니다.

맡기고 싶은 작업	기대하는 행동(편집·실행·검색)	사람 확인이 필요한 지점
(예) 오래된 표 형식 통일	여러 문서 편집	최종 저장 직전 한 번
...	...	...

FAQ

Q1. 코딩을 전혀 모르는데 Claude Code를 쓸 수 있나요? → A1. 네. Claude Code는 코드뿐 아니라 문서·데이터 정리 같은 일도 다룰 수 있고, 무엇보다 **사람의 말(목표)** 로 지시합니다. 다만 결과가 옳은지 판단하는 눈은 사람이 길러야 하므로, 이 책의 안전 실습 원칙을 따르며 차근차근 익히는 것을 권합니다.

Q2. 자동완성과 에이전틱 코딩, 둘 중 하나만 써야 하나요? → A2. 아닙니다. 한 줄을 빠르게 채우는 데는 자동완성이, "찾아서 고치고 확인까지"의 여러 단계 작업에는 에이전틱 코딩이 어울립니다. 둘은 경쟁이 아니라 상황에 따라 나눠 쓰는 도구입니다.

Q3. Claude Code가 제 파일을 마음대로 지우거나 망가뜨리지는 않나요? → A3. 중요한 행동(파일 편집·명령 실행) 직전에는 사람에게 확인을 구하는 '사람 개입 지점'이 있습니다. 또한 실습 전용 폴더를 분리하고 버전 관리를 함께 쓰면 위험을 크게 줄일 수 있습니다. 권한 설정은 뒤 장에서 자세히 배웁니다.

장 요약

이번 장에서는 Claude Code를 손대기 전에 큰 그림을 잡았습니다. 에이전틱 코딩은 한 줄씩 제안하는 자동완성과 달리, **목표를 받아 읽기·편집·실행·검색을 스스로 반복**하는 방식입니다. 이 모든 표면(CLI·IDE·데스크톱·웹)은 하나의 공통 엔진을 공유하며, 이 책은 그중 CLI를 중심으로 가르칩니다. Claude Code의 동작은 **계획→행동→관찰→다시 계획**으로 도는 에이전트 루프 한 그림으로 이해할 수 있고, 중요한 행동 전에는 사람이 확인하는 개입 지점이 있어 안전합니다. 마지막으로, 실습은 언제나 전용 폴더 분리·백업·권한 확인이라는 안전 원칙 위에서 시작합니다.

핵심 키워드

에이전틱 코딩(agentic coding), 인터페이스 표면(interface surface), 공통 엔진(shared engine), 에이전트 루프(agent loop), 사람 개입 지점(human-in-the-loop), 안전 실습(safe practice)

다음 장 미리보기

다음 장에서는 드디어 Claude Code를 **Windows 11에 직접 설치하고 인증한 뒤, 첫 세션을 실행**해 봅니다. 이 장에서 머리로 그린 에이전트 루프를 실제 화면에서 처음 마주하게 됩니다.

설치·인증·첫 세션 (Windows 11)

학습 목표 - Windows 11에서 Claude Code 설치 전에 필요한 환경(Node·터미널·Git)을 스스로 점검할 수 있습니다. - Claude Code를 설치하고 버전을 확인하며 업데이트 방식을 이해합니다. - 구독 로그인과 API 키 인증의 차이를 설명하고 첫 세션을 시작·종료·재개할 수 있습니다. - 상태 줄(status line)의 정보를 해석하고 기본 단축키를 사용할 수 있습니다.

이 장은 1장에서 머릿속에 그린 개념을 **실제 내 PC에서 동작하게 만드는** 장입니다. 설치 준비부터 인증, 첫 세션 열기, 화면 읽는 법까지 차례대로 따라가면 끝에는 본인 컴퓨터에서 `claude` 한 줄로 대화를 시작할 수 있게 됩니다. 명령은 모두 Windows 11의 기본 터미널인 **PowerShell**(파워셸) 기준으로 적었습니다.

이 책의 명령 화면에 자주 보이는 `PS C:\Users\사용자>` 는 "지금 PowerShell이 사용자 계정 폴더에서 명령을 기다리는 중"이라는 표시입니다. 여기서 사용자 자리에는 본인 Windows 계정 이름이 들어갑니다. 직접 입력하는 부분은 이 프롬프트 표시 **뒤에 오는 글자**뿐이며, 프롬프트 자체는 따라 칠 필요가 없습니다.

설치 준비 - Windows 11 환경 점검

도입

새 전자레인지로 사 왔다고 바로 쓸 수 있는 것은 아닙니다. 콘센트가 있는지, 전압이 맞는지부터 확인합니다. Claude Code도 똑같습니다. 설치 버튼을 누르기 전에 **이 도구가 기대는 바탕 프로그램**이 내 컴퓨터에 갖춰져 있는지 먼저 살펴야 합니다. 이 절에서는 그 점검 목록을 하나씩 확인합니다.

핵심 개념 설명

사전 요구사항 (prerequisites)

사전 요구사항(prerequisites) 이란, Claude Code를 설치하고 실행하기 위해 **미리 갖춰져 있어야 하는 환경**을 말합니다. 새 가전을 쓰기 전 콘센트와 전압을 확인하는 것과 같습니다.

Windows 11에서 점검할 항목은 크게 네 가지입니다.

- **터미널(terminal):** 글자로 명령을 입력하는 검은 화면입니다. Windows 11에는 **Windows Terminal**(윈도우 터미널)이 기본 설치되어 있고, 그 안에서 **PowerShell**(파워셸)이 열립니다. Claude Code는 이 터미널 안에서 동작합니다.
- **Node.js(노드):** 자바스크립트로 만든 프로그램을 실행해 주는 바탕 프로그램입니다. 뒤에서 설명할 두 가지 설치 방법 중 **npm 방식**을 쓸 때만 필요합니다. npm(엔피엠, Node Package Manager)은 Node와 함께 깔리는 "프로그램 설치 도우미"입니다.
- **Git(깃):** 파일 변경 이력을 기록·되돌리는 버전 관리 도구입니다. 필수는 아니지만, Claude Code가 변경 사항을 추적하고 안전하게 되돌리는 데 큰 도움이 되므로 권장합니다.
- **PowerShell 실행 정책(execution policy):** PowerShell이 외부 스크립트를 실행해도 되는지 정하는 보안 설정입니다. 네이티브 설치 스크립트를 쓰려면 이 설정이 막혀 있지 않아야 합니다.

조금 더 들어가 보겠습니다. 왜 이 점검을 먼저 하는지(why), 언제 무엇이 필요한지(when), 어떻게 확인하는지(how)를 나눠 보겠습니다.

- **왜(why) 점검하나:** 빠진 항목이 있으면 설치 도중 "명령을 찾을 수 없습니다" 같은 오류가 납니다. 미리 확인하면 그 좌절을 피할 수 있습니다.
- **언제(when) 무엇이 필요한가:** 다음 절에서 고를 설치 방법에 따라 다릅니다. **네이티브 설치 관리자**를 쓰면 Node 없이도 됩니다. **npm 방식**을 쓰면 Node가 반드시 있어야 합니다.
- **어떻게(how) 확인하나:** 아래 명령으로 각 프로그램의 버전을 물어봅니다. 버전 번호가 나오면 설치된 것이고, 오류가 나면 없는 것입니다.

이렇게 점검합니다

PowerShell을 열고 다음 명령을 한 줄씩 입력합니다. (Windows Terminal은 시작 메뉴에서 "터미널" 또는 "PowerShell"을 검색해 열 수 있습니다.)

```
# 현재 PowerShell 버전을 확인합니다
$PSVersionTable.PSVersion

# Node.js 가 설치돼 있는지 확인합니다 (npm 방식을 쓸 때 필요)
node --version

# Git 이 설치돼 있는지 확인합니다 (권장)
git --version
```

예상 화면(설치돼 있는 경우):

```
PS C:\Users\사용자> node --version
v20.11.1
PS C:\Users\사용자> git --version
git version 2.43.0.windows.1
```

만약 `node` 가 없다면 다음과 비슷한 메시지가 나옵니다. 이때는 npm 방식을 쓸 수 없으니, 다음 절에서 **네이티브 설치 관리자**를 고르거나 Node를 먼저 설치하면 됩니다.

```
PS C:\Users\사용자> node --version
node : 'node' 용어가 cmdlet, 함수, 스크립트 파일 또는 실행 가능한 프로그램 이름으로 인식되지 않음
```

점검 명령 읽는 법

명령	확인하는 것	정상 신호
<code>\$PSVersionTable.PSVersion</code>	PowerShell 버전	5.1 이상 또는 7.x 숫자가 보임
<code>node --version</code>	Node.js 설치 여부	v18 이상 버전 번호가 보임
<code>git --version</code>	Git 설치 여부	<code>git version ...</code> 이 보임

이어서 PowerShell **실행 정책**을 확인합니다. 네이티브 설치 스크립트를 안전하게 내려받아 실행하려면 이 값이 너무 엄격하지 않아야 합니다.

```
# 현재 실행 정책을 확인합니다
Get-ExecutionPolicy

# 막혀 있다면(Restricted), 현재 사용자에게 한해 외부 스크립트 실행을 허용합니다
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

주의: 실행 정책은 보안 장치입니다. `RemoteSigned` 는 "내가 직접 만든 스크립트와, 신뢰된 발행자가 서명한 외부 스크립트만 허용"한다는 뜻으로, 무분별한 `Unrestricted` 보다 안전한 선택입니다. 회사 PC라면 관리 정책상 변경이 막혀 있을 수 있으니, 그때는 IT 담당자에게 문의합니다.

마지막으로 **작업 폴더**를 정합니다. Claude Code는 "지금 어느 폴더에 있는지"를 기준으로 그 폴더의 파일을 읽고 다룹니다. 그래서 연습용으로는 시스템 폴더가 아닌 **새 실습 폴더**를 따로 만들어 두는 편이 안전합니다.

```
# 홈 폴더 아래에 실습 전용 폴더를 만듭니다
mkdir C:\claude-lab
```

팁: 실습은 `C:\Windows` 나 `C:\Program Files` 같은 시스템 폴더가 아니라, `C:\claude-lab` 처럼 비어 있는 전용 폴더에서 하는 것이 안전합니다. 처음에는 도구가 무엇을 바꾸는지 익히는 단계라, 중요한 파일과 떨어진 곳에서 연습하는 것이 좋습니다.

절 요약

- 설치 전 **터미널·Node·Git·실행 정책** 네 가지를 점검합니다.
- 네이티브 설치 방식은 Node가 없어도 되고, npm 방식은 Node가 반드시 필요합니다.
- 연습은 시스템 폴더가 아닌 `C:\claude-lab` 같은 **전용 폴더**에서 합니다.

설치하기 - npm과 네이티브 설치 관리자

도입

준비가 끝났으니 이제 실제로 설치합니다. 같은 앱이라도 공식 스토어에서 받을 수도, 설치 파일을 직접 받을 수도 있는 것처럼, Claude Code도 설치하는 길이 두 가지입니다. 어느 길을 택하든 결과는 같은 `claude` 명령입니다. 이 절에서는 두 방법을 비교하고, 설치가 잘 됐는지 확인하는 법까지 봅니다.

핵심 개념 설명

설치 (installation)

설치(installation) 란, Claude Code 프로그램을 내 컴퓨터에 내려받아 어디서든 `claude` 라고 부르면 실행되도록 등록하는 과정입니다. Windows 11에서 쓸 수 있는 방법은 두 가지입니다.

- **네이티브 설치 관리자(native installer):** Anthropic이 제공하는 설치 스크립트를 PowerShell에서 한 줄로 실행하는 방식입니다. Node.js가 없어도 되고, 2026년 현재 **권장되는 방법**입니다.
- **npm 방식:** Node와 함께 깔린 npm으로 패키지를 전역 설치하는 방식입니다. 예전부터 쓰이던 방법이며 여전히 동작하지만, 2026년 기준으로는 공식적으로 **권장에서 물러난(deprecated)** 상태입니다.

업데이트 관리 (update management)

업데이트 관리(update management)란, 설치한 Claude Code를 최신 버전으로 유지하는 방법을 말합니다. Claude Code는 새 기능과 보안 수정이 자주 나오므로, 어떤 방식으로 최신 상태를 유지하는지 알아 두는 것이 좋습니다. 네이티브 설치는 보통 **자동 업데이트**를 지원하고, npm 방식은 **수동 업데이트**가 기본입니다.

이렇게 설치합니다

방법 1 — 네이티브 설치 관리자(권장). PowerShell을 열고 다음 한 줄을 실행합니다. `irm` 은 인터넷에서 스크립트를 받아오는 명령(Invoke-RestMethod)이고, `iex` 는 받아온 스크립트를 실행하는 명령(Invoke-Expression)입니다.

```
# Anthropic 공식 설치 스크립트를 받아 실행합니다 (네이티브 방식)
irm https://claude.ai/install.ps1 | iex
```

```
Downloading Claude Code...
Installing to C:\Users\사용자\.local\bin ...
Updating PATH...
Claude Code installed successfully. Restart your terminal to continue.
```

방법 2 — npm 방식. Node가 설치돼 있다면 다음 명령으로도 설치할 수 있습니다.

```
# npm 으로 Claude Code 를 전역(-g) 설치합니다 (구식, 동작은 함)
npm install -g @anthropic-ai/claude-code
```

설치가 끝나면 **반드시 현재 터미널을 닫고 새 PowerShell 창을 엽니다.** 설치 과정에서 바뀐 PATH(프로그램을 찾는 경로 목록)는 새로 연 창부터 적용되기 때문입니다. 그다음 버전을 확인합니다.

```
# 설치된 버전을 확인합니다
claude --version

# 설치 환경에 문제가 없는지 자가 진단합니다
claude doctor
```

```
PS C:\Users\사용자> claude --version
2.0.14 (Claude Code)
```

확인 명령 정리

명령	하는 일
<code>claude --version</code>	설치된 Claude Code의 버전 번호를 출력합니다
<code>claude doctor</code>	설치·PATH·인증 상태에 문제가 없는지 점검합니다
<code>claude update</code>	최신 버전이 있으면 내려받아 갱신합니다

업데이트는 설치 방식에 따라 다릅니다. 네이티브 설치는 보통 자동으로 최신을 받아 두며, 필요하면 `claude update` 로 직접 갱신할 수 있습니다. npm 방식은 `npm update -g @anthropic-ai/claude-code` 로 수동 갱신합니다.

항목	네이티브 설치 관리자	npm 방식
Node.js 필요	필요 없음	반드시 필요
권장 여부 (2026)	권장	권장에서 물러남
설치 명령	<code>irm https://claude.ai/install.ps1 \& iex</code>	<code>npm install -g @anthropic-ai/claude-code</code>
업데이트	자동(또는 <code>claude update</code>)	수동(<code>npm update -g ...</code>)
적합한 사람	대부분의 입문자	이미 Node 환경에 익숙한 사용자

흔한 실수 두 가지 - "설치했는데 `claude` 가 인식되지 않습니다" → 집중팔구 **새 터미널을 열지 않아서**입니다. 현재 창을 닫고 새 PowerShell 창에서 다시 시도합니다. - 네이티브와 npm 두 방식을 **둘 다 설치**하면 어느 쪽이 실행되는지 헷갈릴 수 있습니다. 한 가지 방식만 쓰는 것을 권합니다. 이미 둘 다 깔았다면 `claude doctor` 로 어느 경로가 쓰이는지 확인합니다.

절 요약

- 설치하는 **네이티브 설치 관리자(권장)** 와 **npm 방식** 두 갈래이며, 결과는 같은 `claude` 명령입니다.
- 설치 후에는 **새 터미널을 열고** `claude --version` 으로 확인합니다.
- 업데이트는 네이티브가 자동, npm이 수동이며 `claude doctor` 로 상태를 점검할 수 있습니다.

로그인과 인증(authentication) - 계정·구독·API 키

도입

설치가 끝나도 도구가 곧장 일하지는 않습니다. "당신이 누구인지, 사용할 권한이 있는지"를 먼저 확인합니다. 큰 건물에 처음 들어갈 때 출입증을 발급받아 두면 다음부터는 태그만 찍고 들어가는 것과 같습니다. 이 절에서 다루는 **인증(authentication)** 이 바로 그 출입증을 받는 과정입니다.

핵심 개념 설명

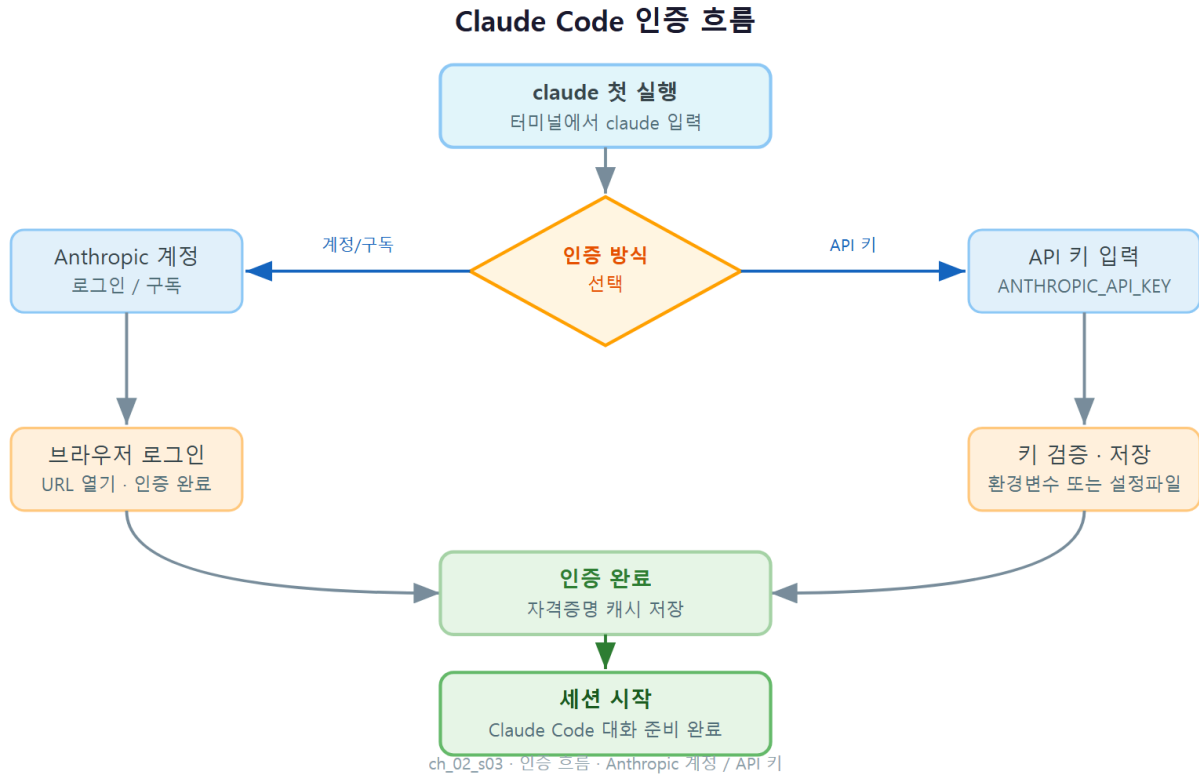
계정 인증 (account authentication)

계정 인증(authentication) 이란, Claude Code가 내 계정과 사용 권한을 확인하는 **로그인 절차**입니다. 한 번 성공하면 **토큰(token)** 이 내 컴퓨터에 저장됩니다. 토큰은 "이 사람은 이미 확인된 사용자"임을 증명하는 디지털 출입증으로, 이후 세션에서는 이 토큰을 재사용하므로 매번 다시 로그인하지 않아도 됩니다.

인증 방식 (구독 vs API 키)

인증하는 길은 두 가지입니다. 이 둘은 **요금이 매겨지는 방식**이 다릅니다.

- **구독(subscription) 로그인**: Claude Pro·Max 같은 **월 정액 구독** 계정으로 로그인하는 방식입니다. 정해진 한도 안에서 추가 과금 없이 사용합니다. 개인 입문자에게 가장 단순한 선택입니다.
- **API 키(API key) 방식**: **API 키**라는 긴 비밀 문자열을 등록해 쓰는 방식입니다. API(Application Programming Interface)는 프로그램끼리 주고받는 약속된 창구를 뜻하고, API 키는 그 창구를 여는 열쇠입니다. 사용한 만큼 요금이 매겨지는 **종량제**이며, 회사·팀에서 사용량을 관리할 때 주로 씁니다.



이렇게 인증합니다

설치 후 처음 `claude` 를 실행하면 자동으로 로그인 안내가 나타납니다. 화면 안내를 따라 인증 방식을 고르면 됩니다.

```

PS C:\Users\사용자> claude

Welcome to Claude Code

How would you like to log in?
> 1. Claude account with subscription (Pro/Max)  <- 구독 로그인
   2. Anthropic Console account (API usage billing) <- API 키/종량제

(브라우저가 열립니다. 로그인 후 '권한 허용'을 누르면 됩니다.)
Login successful. Credentials saved.
    
```

구독 로그인을 고르면 브라우저 창이 열리고, 평소 쓰던 Claude 계정으로 로그인한 뒤 권한을 허용하면 끝납니다. 로그인 상태는 세션 안에서 슬래시 명령으로도 관리할 수 있습니다.

`/login` — 다시 로그인하거나 다른 계정으로 전환합니다 `/logout` — 현재 계정에서 로그아웃하고 저장된 토큰을 지웁니다 `/status` — 현재 로그인된 계정과 인증 방식을 보여 줍니다

API 키 방식을 쓴다면, 먼저 Anthropic Console(console.anthropic.com) 화면에서 키를 발급받습니다. 그 키를 환경 변수로 등록해 두면 Claude Code가 자동으로 인식합니다.

```
# 현재 사용자 환경 변수로 API 키를 등록합니다 (값은 본인 키로 교체)
setx ANTHROPIC_API_KEY "sk-ant-..."
```

인증 관련 항목 정리

항목	의미
토큰(token)	로그인 성공 후 저장되는 디지털 출입증. 재로그인을 줄여 줍니다
ANTHROPIC_API_KEY	API 키 방식에서 키를 담는 환경 변수 이름입니다
setx	환경 변수를 영구적으로 등록하는 Windows 명령입니다
저장 위치	자격 증명은 사용자 홈의 C:\Users\사용자\.claude 영역 등에 안전하게 보관됩니다

비교 항목	구독 로그인	API 키 방식
과금 방식	월 정액(한도 내 추가 과금 없음)	종량제(쓴 만큼 청구)
준비물	Claude Pro·Max 계정	Console에서 발급한 API 키
로그인 절차	브라우저로 계정 로그인	API 키를 환경 변수로 등록
적합한 대상	개인 입문자	팀·조직, 사용량 관리가 필요한 경우

흔한 오해 두 가지 - "쓸 때마다 매번 다시 로그인해야 하나요?" → 아닙니다. 한 번 인증하면 토큰이 저장되어 이후 세션에서 재사용됩니다. 다시 묻는다면 토큰이 만료됐거나 지워진 경우입니다. - "구독과 API 키가 같은 건가요?" → 다릅니다. 구독은 월 정액, API 키는 종량제입니다. 한쪽으로 로그인했다면 그 방식의 요금 체계가 적용됩니다.

위험: API 키는 비밀번호와 같습니다. 채팅·코드·공개 저장소에 **절대 붙여 넣지 마세요**. 키가 노출되면 누군가 내 계정으로 종량제 요금을 발생시킬 수 있습니다. 키는 환경 변수로만 다루고, 노출됐다면 즉시 Console에서 폐기·재발급합니다.

절 요약

- 처음 claude 실행 시 **구독 로그인** 또는 **API 키** 중에서 인증 방식을 고릅니다.
- 한 번 인증하면 **토큰**이 저장되어 매번 다시 로그인하지 않아도 됩니다.
- API 키는 비밀번호처럼 다루며, 환경 변수로만 등록하고 외부에 노출하지 않습니다.

첫 세션 시작·종료·재개 (C:\w 프로젝트 폴더에서)

도입

이제 실제로 대화를 시작합니다. Claude Code와의 대화 한 묶음을 **세션(session)** 이라고 부릅니다. 통화 한 통과 비슷합니다. 통화를 끊어도 통화 기록을 보고 다시 이어 걸 수 있듯, 세션도 종료한 뒤 다시 이어서 재개할 수 있습니다. 이 절에서는 세션을 열고, 닫고, 다시 이어 가는 법을 익힙니다.

핵심 개념 설명

세션 (session)

세션(session) 이란, Claude Code를 한 번 실행해 작업을 주고받는 **대화 단위**입니다. `claude` 를 입력해 시작하고, 그 안에서 여러 차례 지시·응답을 주고받다가, 끝내면 닫힙니다. 중요한 점은 **세션을 닫아도 대화 기록이 사라지지 않는다**는 것입니다. 종료 후에도 `--continue` 나 `--resume` 으로 그대로 이어 갈 수 있습니다.

작업 디렉터리 (working directory)

작업 디렉터리(working directory) 란, 세션을 시작한 "**지금 그 폴더**" 를 말합니다. Claude Code는 이 폴더를 기준으로 파일을 읽고 다룹니다. 그래서 어떤 프로젝트를 다루고 싶다면, 먼저 그 프로젝트 폴더로 이동한 뒤에 `claude` 를 실행해야 합니다. 엉뚱한 폴더에서 열면 도구가 엉뚱한 파일들을 보게 됩니다.

이렇게 시작합니다

앞서 만든 실습 폴더로 이동한 뒤 `claude` 를 실행합니다. `cd` 는 폴더를 이동하는 명령(change directory)입니다.

```
# 실습 폴더로 이동합니다
cd C:\claude-lab

# 이 폴더를 작업 디렉터리로 삼아 세션을 시작합니다
claude
```

세션이 열리면 입력창에 평소 말하듯 지시를 적습니다. 예를 들어 폴더에 어떤 파일이 있는지 물어볼 수 있습니다.

```
PS C:\claude-lab> claude
> 이 폴더에 어떤 파일들이 있는지 목록으로 보여 줘

[Bash] 현재 폴더의 파일을 확인하겠습니다
이 폴더에는 다음 파일이 있습니다:
- notes.txt
- todo.md
무엇을 도와드릴까요?
```

세션을 끝낼 때는 입력창에 `/exit` 를 입력하거나, 키보드에서 `Ctrl + C` 를 두 번 누릅니다. 종료해도 대화는 저장됩니다. 다음에 같은 폴더에서 이어 가려면 `--continue` 를, 여러 과거 세션 중에서 고르려면 `--resume` 을 씁니다.

`claude` — 현재 폴더에서 **새 세션**을 시작합니다 `claude --continue` — 이 폴더의 **가장 최근 세션**을 그대로 이어서 엽니다 `claude --resume` — 과거 세션 **목록을 보여 주고** 골라서 재개합니다

```
PS C:\claude-lab> claude --continue
(가장 최근 대화를 불러옵니다)
> 아까 보던 todo.md 에서 끝낸 항목을 지워 줘
```

세션 명령·종료 방법 정리

입력	하는 일
<code>cd C:\claude-lab</code>	작업할 폴더로 이동합니다
<code>claude</code>	그 폴더에서 새 세션을 시작합니다
<code>/exit</code>	현재 세션을 종료합니다(기록은 저장됨)
<code>Ctrl + C</code> (두 번)	진행 중 작업을 멈추고 세션을 빠져나옵니다
<code>claude --continue</code>	가장 최근 세션을 이어서 엽니다
<code>claude --resume</code>	과거 세션을 골라 재개합니다

팁: `--continue` 는 "바로 직전 대화 이어 가기", `--resume` 은 "여러 대화 중에서 골라 이어 가기"로 기억하면 쉽습니다. 어제 하던 작업을 오늘 이어 가고 싶다면 `--continue` 가 가장 빠릅니다.

흔한 실수 두 가지 - "창을 닫았더니 대화가 다 날아갔어요" → 대부분 사라지지 않았습니다. 같은 폴더에서 `claude --resume` 으로 과거 세션을 찾을 수 있습니다. - "분명 파일이 있는데 못 본다고 해요" → **작업 디렉터리가 다른** 경우가 많습니다. 세션을 시작하기 전에 `cd` 로 올바른 폴더에 들어왔는지 확인합니다.

절 요약

- 세션은 **프로젝트 폴더로 이동한 뒤** `claude` 로 시작합니다. 작업 디렉터리가 곧 도구가 보는 범위입니다.
- 종료는 `/exit` 또는 `Ctrl + C` 두 번이며, 대화 기록은 사라지지 않습니다.
- 이어 가기는 최근 세션이면 `--continue`, 고를 때는 `--resume` 을 씁니다.

화면 구성·상태 줄(status line)·기본 단축키

도입

자동차 계기판은 속도·연료·온도를 한눈에 보여 줍니다. 그 표시를 읽을 줄 알면 운전이 한결 편해집니다. Claude Code 화면 아래쪽에도 비슷한 계기판이 있습니다. 바로 **상태 줄(status line)** 입니다. 이 절에서는 화면이 어떻게 구성되는지, 상태 줄의 숫자가 무엇을 뜻하는지, 그리고 자주 쓰는 단축키를 정리합니다.

핵심 개념 설명

상태 줄 (status line)

상태 줄(status line) 이란, CLI 화면 **하단**에서 지금 세션의 상태를 한눈에 보여 주는 표시줄입니다. 자동차 계기판처럼, 지금 어떤 모델을 쓰는지·어느 폴더에서 일하는지·얼마나 많은 대화가 쌓였는지를 알려 줍니다. 단순 장식이 아니라, 도구의 상태를 읽고 다음 행동을 정하는 **계기판**입니다.

컨텍스트 사용량 (context usage)

컨텍스트(context) 란 Claude Code가 지금 대화에서 "기억하고 있는 내용 전체"를 말합니다. 이 기억 공간에는 한계가 있는데, 그 한계를 **컨텍스트 윈도우(context window)** 라고 합니다. **컨텍스트 사용량(context usage)** 은 그 공간을 지금 얼마나 채웠는지를 가리킵니다. 컵에 물이 차오르

는 것과 같아서, 가득 차면 오래된 내용부터 자리를 비워야 합니다(이 정리 과정은 3장에서 자세히 다룹니다).

Claude Code 화면 구성



화면 이렇게 읽습니다

세션 화면은 크게 세 부분으로 나뉩니다. 위쪽 **대화 영역**(주고받은 내용), 아래쪽 **입력창**(내가 지시를 적는 곳), 그리고 그 사이의 **상태 줄**입니다. 작업 중 권한이 필요한 순간에는 **권한 프롬프트**가 따로 떼서 승인을 묻습니다.

> 이 폴더의 파일을 정리해 줘

[Read] notes.txt 를 읽었습니다
...(대화 영역)...

claude-sonnet-4.5 | C:\claude-lab | context 32%

> _

(입력창)

상태 줄 항목 읽는 법

표시	의미
claude-sonnet-4.5	지금 사용 중인 모델 이름입니다
C:\claude-lab	현재 작업 디렉터리 입니다(도구가 보는 폴더)
context 32%	컨텍스트 사용량 입니다. 100%에 가까울수록 정리가 필요합니다

자주 쓰는 단축키와 슬래시 명령은 다음과 같습니다. 슬래시 명령은 입력창에 `/` 로 시작해 입력합니다.

단축키·명령	하는 일
<code>/help</code>	사용할 수 있는 명령과 도움말을 보여 줍니다
<code>/status</code>	모델·계정·작업 폴더 등 현재 상태를 자세히 보여 줍니다
<code>/model</code>	사용할 모델을 확인하거나 바꿉니다
<code>/clear</code>	대화를 비워 컨텍스트를 새로 시작합니다
Esc	진행 중인 작업을 중단 합니다
Shift + Tab	승인 모드를 전환합니다(자세한 내용은 4장)
↑ (위 화살표)	이전에 입력한 지시를 다시 불러옵니다

팁: 무엇을 할 수 있는지 막힐 때는 일단 `/help` 를 입력해 보세요. 사용 가능한 모든 명령이 한 눈에 보입니다. 처음에는 `/help`, `/status`, `/clear` 세 가지만 기억해도 충분합니다.

흔한 실수: 상태 줄의 `context` 수치를 무시한 채 한 세션에서 계속 일하면, 컨텍스트가 가득 차 도구가 앞부분 내용을 잊어버릴 수 있습니다. 작업이 한 단락 끝났다면 `/clear` 로 깨끗이 비우고 새 주제를 시작하는 습관이 좋습니다.

절 요약

- 화면은 **대화 영역·상태 줄·입력창**으로 구성되며, 권한이 필요하면 별도 프롬프트가 뜹니다.
- 상태 줄은 **모델·작업 디렉터리·컨텍스트 사용량**을 보여 주는 계기판입니다.
- 막힐 때는 `/help`, 상태 확인은 `/status`, 정리는 `/clear` 를 기억합니다.

실습 과제

해보기: 내 PC에 설치하고 첫 세션 점검표 완성하기

이번 장에서 배운 흐름을 본인 컴퓨터에서 한 바퀴 돌려 봅니다.

- 단계 1 — 점검: PowerShell을 열고 `node --version`, `git --version`, `Get-ExecutionPolicy`를 실행해 결과를 적어 둡니다.
- 단계 2 — 설치: 네이티브 설치 관리자(`irm https://claude.ai/install.ps1 | iex`)로 설치한 뒤, 새 터미널을 열어 `claude --version`으로 버전을 확인합니다.
- 단계 3 — 인증: `claude`를 처음 실행해 구독 로그인 또는 API 키로 인증을 마칩니다.
- 단계 4 — 첫 세션: `mkdir C:\claude-lab`로 폴더를 만들고, `cd C:\claude-lab`후 `claude`를 실행해 "이 폴더에 어떤 파일이 있는지 알려 줘"라고 물어봅니다.
- 단계 5 — 점검표: 아래 네 칸을 직접 채워 봅니다.

점검 항목	내 값
설치된 버전(<code>claude --version</code>)	
인증 방식(구독/API 키)	
작업 디렉터리(상태 줄)	
컨텍스트 사용량(상태 줄)	

- 점검: `/exit`로 세션을 닫은 뒤 `claude --continue`로 다시 열어 대화가 이어지는지 확인합니다.

FAQ

Q1. 네이티브 설치 관리자와 npm, 둘 중 무엇으로 설치해야 하나요? → **A1.** 입문자라면 **네이티브 설치 관리자**를 권합니다. Node.js가 없어도 되고 자동 업데이트가 편하기 때문입니다. 이미 Node 환경에 익숙하고 npm 관리가 편하다면 npm 방식도 동작하지만, 2026년 기준 공식 권장에서는 물려난 방법입니다. 두 방식을 동시에 설치하는 것은 혼란을 줄 수 있어 피하는 편이 좋습니다.

Q2. 설치를 마쳤는데 `claude` 가 "인식되지 않습니다"라고 나옵니다. → A2. 거의 대부분 **새 터미널을 열지 않아서** 생기는 문제입니다. 설치 중 바뀐 PATH(프로그램 경로 목록)는 새로 연 창부터 적용됩니다. 현재 PowerShell 창을 닫고 새 창을 연 뒤 다시 시도하세요. 그래도 안 되면 `claude doctor` 로 설치 상태를 점검합니다.

Q3. 구독이 없어도 API 키만으로 쓸 수 있나요? 그리고 키는 어디에 두나요? → A3. 네, **API 키 방식**으로 종량제(쓴 만큼 과금)로 사용할 수 있습니다. 키는 Anthropic Console에서 발급받아 `setx ANTHROPIC_API_KEY "..."` 처럼 **환경 변수**로만 등록합니다. 키는 비밀번호와 같으니 코드·채팅·공개 저장소에 절대 붙여 넣지 마세요. 노출됐다면 즉시 Console에서 폐기·재발급합니다.

Q4. 어제 하던 대화를 오늘 이어서 할 수 있나요? → A4. 네. 같은 폴더에서 `claude --continue` 를 실행하면 가장 최근 세션을 그대로 이어 갑니다. 여러 과거 세션 중에서 고르고 싶다면 `claude --resume` 으로 목록을 보고 선택합니다. 세션을 종료해도 대화 기록은 사라지지 않습니다.

장 요약

이번 장에서는 Windows 11에서 Claude Code를 실제로 쓸 수 있게 만드는 과정을 처음부터 끝까지 따라갔습니다. 먼저 터미널·Node·Git·실행 정책을 점검하고, 네이티브 설치 관리자(권장)나 npm으로 설치한 뒤 `claude --version` 으로 확인했습니다. 이어 구독 로그인 또는 API 키로 인증해 출입증(토큰)을 받아 두었고, 프로젝트 폴더로 이동해 첫 세션을 열고 종료·재개하는 법을 익혔습니다. 마지막으로 상태 줄로 모델·작업 폴더·컨텍스트 사용량을 읽고 기본 단축키를 다루는 법까지 살펴보았습니다. 이제 도구를 켜고 대화를 시작할 준비가 끝났습니다.

핵심 키워드

사전 요구사항(prerequisites), 설치(installation), 계정 인증(authentication), 토큰(token), 세션(session), 작업 디렉터리(working directory), 상태 줄(status line), 컨텍스트 사용량(context usage)

다음 장 미리보기

다음 장에서는 열어 둔 세션 안에서 **실제로 일을 시키는 기본 흐름**(대화로 요청하기·파일 읽기와 편집·터미널 명령 실행·권한 승인의 첫 만남)을 다룹니다. 이번 장이 도구를 켜는 법이었다면, 다음 장은 도구로 일하는 법입니다.

기본 흐름: 대화·파일 편집·터미널·권한 첫걸음

학습 목표 - 원하는 작업을 맥락이 포함된 명확한 프롬프트(prompt)로 표현할 수 있습니다. - 파일 편집(file edit)이 변경 미리보기(diff)와 승인을 거치는 과정을 설명할 수 있습니다. - 명령 실행 결과와 오류 출력을 읽고 다음 행동을 정할 수 있습니다. - 권한 요청(permission prompt)이 언제 뜨는지, 이번만·항상 허용·거부를 어떻게 고르는지 압니다. - 컨텍스트가 길어질 때 압축(compaction)과 `/clear` 로 세션을 관리할 수 있습니다.

앞 장에서 Claude Code를 설치하고 첫 세션을 열어 보았습니다. 이 장은 그 위에서 **실제로 일을 시키는 한 바퀴**를 처음부터 끝까지 따라가는 장입니다. 일의 흐름은 늘 같은 모양을 띕니다. **말로 요청하고(대화) → Claude Code가 파일을 읽고 고칠 안을 보여 주면 확인하고(편집·미리보기) → 필요하면 명령을 실행해 결과를 보고(터미널) → 위험한 행동 전에는 허락을 묻는(권한)** 순서입니다.

이 장은 입문 단계인 만큼 손으로 따라 하기보다 **각 단계에서 화면에 무엇이 보이고, 내가 무엇을 눌러야 하는지**를 뚜렷이 익히는 데 집중합니다. 이 책은 Windows 11의 **PowerShell(파워셸)**을 기준으로 합니다. PowerShell은 글자로 명령을 입력해 컴퓨터에 일을 시키는 검은 화면(터미널)이며, 화면 맨 앞의 `PS C:\Users\사용자\claude-practice>` 표시는 "지금 `claude-practice` 폴더에서 명령을 기다리는 중"이라는 안내입니다.

대화로 일 시키기(prompting) - 명확한 요청 만들기

도입

식당에서 "맛있는 거 주세요"라고 하면 무엇이 나올지 알 수 없습니다. 반면 "덜 맵게, 곱빼기로, 김치는 따로"라고 하면 원하는 음식이 정확히 나옵니다. Claude Code에게 일을 시키는 것도 똑 같습니다. 이 절에서는 도구가 헛다리를 짚지 않도록 **요청을 뚜렷하게 만드는 법**을 배웁니다.

핵심 개념 설명

프롬프트 (prompt)

프롬프트(prompt)란, Claude Code에게 *무엇을 어떻게 해 달라고* 전하는 요청 한 덩어리를 말합니다. 우리말로 풀면 "지시문" 또는 "주문서"에 가깝습니다. 동료에게 일을 부탁할 때 배경과 기대 결과를 함께 알려 주면 더 잘 처리되듯, 프롬프트도 **맥락·예시·제약을 함께 줄수록** 결과가 정확해집니다.

맥락 제공 (providing context)

맥락 제공(providing context)이란, 요청에 "어떤 파일을, 어떤 상황에서, 어떤 조건으로" 처리할지 배경 정보를 함께 붙여 주는 것을 말합니다. Claude Code는 내 머릿속 사정을 모르므로, 내가 당연하게 여기는 전제를 글로 풀어 줘야 합니다.

여기서 입문자가 가장 자주 하는 오해 하나를 짚겠습니다. **"짧게 한 단어만 던져도 알아서 채워 주겠지"**라는 기대입니다. 도구는 똑똑하지만 독심술사는 아닙니다. 정보가 모자라면 도구는 추측으로 빈칸을 메우고, 그 추측이 내 의도와 어긋나면 결과를 다시 고쳐야 합니다.

- **왜(why) 명확해야 하나:** 모호한 요청은 도구가 추측하게 만듭니다. 추측이 빗나가면 결국 다시 설명하고 다시 시키는 왕복이 늘어, 오히려 더 느려집니다.
- **언제(when) 특히 중요한가:** 고칠 파일이 여러 개거나, 결과 형식이 정해져 있거나(예: 표로), 건드리면 안 되는 부분이 있을 때입니다.
- **어떻게(how) 만드나:** "무엇을 + 어떤 파일에 + 어떤 조건으로 + 어떻게 확인할지"를 한 문장 또는 짧은 목록으로 적습니다.

이렇게 요청합니다

다음은 같은 일을 **모호하게** 시킨 경우와 **명확하게** 시킨 경우를 나란히 둔 예시입니다. 아래 화면은 실제 입력이 아니라 차이를 보여 주기 위한 비교용입니다. > 로 시작하는 줄이 내가 Claude Code에게 건넨 프롬프트입니다.

[모호한 요청]
> 코드 좀 고쳐 줘

[명확한 요청]
> notes.txt 파일에서 날짜 형식을 2026-06-16 처럼 'YYYY-MM-DD'로 통일해 줘.
본문 내용은 바꾸지 말고 날짜 부분만 고쳐 줘. 다 고치면 몇 군데 바꿨는지 알려 줘.

명확한 요청을 받으면 Windows 11의 PowerShell에서 다음과 비슷하게 진행됩니다. 대괄호로 표시된 [Read], [Edit] 는 Claude Code가 지금 무슨 작업을 하는 중인지 알려 주는 안내입니다.

```
PS C:\Users\사용자\claude-practice> claude
> notes.txt 의 날짜를 'YYYY-MM-DD' 형식으로 통일해 줘. 본문은 그대로 두고,
   다 고치면 몇 군데 바꿨는지 알려 줘.

[Read]  notes.txt 를 읽었습니다
[Edit]  날짜 표기 3군데를 'YYYY-MM-DD' 형식으로 바꾸겠습니다 (변경 내용을 보여드립니다)
완료했습니다. 본문은 건드리지 않고 날짜 3군데만 통일했습니다.
```

명확한 요청에 들어간 네 가지 재료

재료	예시에서 해당 부분	하는 일
무엇을	"날짜 형식을 통일"	할 일을 한 가지로 못박습니다
어떤 파일에	"notes.txt 에서"	대상 범위를 좁혀 줍니다
어떤 제약으로	"본문은 바꾸지 말고"	건드리면 안 되는 곳을 알려 줍니다
어떻게 확인할지	"몇 군데 바꿨는지 알려 줘"	끝났는지 사람이 점검할 기준을 줍니다

팁: 좋은 프롬프트에는 보통 **대상(어떤 파일)** 과 **제약(무엇은 건드리지 말 것)** 이 함께 들어갑니다. 특히 "이건 바꾸지 마"라는 한 줄이 의도와 다른 수정 사고를 크게 줄여 줍니다.

흔한 실수 두 가지 - 한 번에 너무 많은 일을 욕여넣기 → "고치고, 정리하고, 이름도 바꾸고, 배포까지" 처럼 묶으면 어디서 어긋났는지 알기 어렵습니다. 일은 작게 쪼개 시킵니다. - 파일 이름을 안 알려 주기 → "그 파일"이라고만 하면 도구가 엉뚱한 파일을 고를 수 있습니다. 대상 파일 이름을 분명히 적습니다.

절 요약

- 프롬프트는 Claude Code에게 건네는 주문서이며, 맥락·예시·제약을 함께 줄수록 정확해집니다.
- 명확한 요청에는 "무엇을·어떤 파일에·어떤 제약으로·어떻게 확인할지"가 들어갑니다.
- 일은 한 번에 욕여넣지 말고 작게 쪼개 시키는 편이 안전합니다.

파일 읽기·편집과 변경 미리보기(diff)

도입

중요한 문서를 동료에게 맡길 때, 우리는 보통 "고친 부분을 표시해서 보여 줘. 확인하고 반영할 게"라고 합니다. 곧바로 덮어쓰지 않고 **무엇이 달라지는지 먼저 검토**하는 것입니다. Claude Code의 파일 편집도 정확히 이 방식입니다. 이 절에서는 도구가 파일을 어떻게 읽고, 어떻게 변경안을 보여 주며, 내가 승인해야 비로소 적용되는 과정을 봅니다.

핵심 개념 설명

파일 편집 (file edit)

파일 편집(file edit)이란, Claude Code가 파일의 내용을 고치거나 새로 만드는 작업입니다. 다만 도구는 곧바로 파일을 덮어쓰지 않습니다. 먼저 **어떻게 바꿀지 안을 만들어 사람에게 보여 주고**, 사람이 승인한 뒤에야 실제 파일에 반영합니다.

변경 미리보기 (diff)

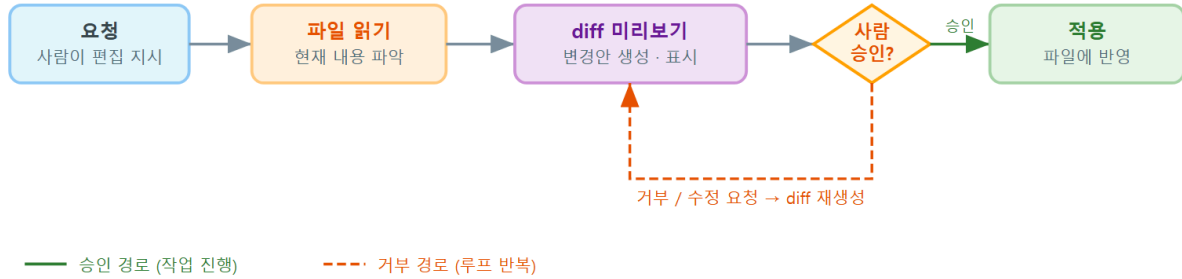
변경 미리보기(diff)란, 파일을 실제로 바꾸기 전에 *어떤 줄이 어떻게 달라지는지*를 보여 주는 화면입니다. 'diff'는 영어 'difference(차이)'를 줄인 말로, "이전과 이후의 차이를 나란히 보여 주는 것"을 뜻합니다. 위드의 "변경 내용 표시"로 무엇이 달라졌는지 먼저 검토하는 것과 똑같습니다.

여기서 가장 흔한 오해는 **"제안하면 곧바로 파일에 반영된다"**는 생각입니다. 그렇지 않습니다. 미리보기는 말 그대로 *미리 보기*일 뿐, 내가 승인하기 전까지 원본 파일은 그대로 있습니다.

- **왜(why) 미리보기를 거치나**: 파일을 한 번 덮어쓰면 되돌리기 번거롭습니다. 적용 전에 차이를 눈으로 확인하면, 의도와 다른 변경을 미리 막을 수 있습니다.
- **언제(when) 보게 되나**: Claude Code가 파일을 고치려는 모든 순간입니다. 읽기만 할 때는 미리보기가 뜨지 않고, 고치려 할 때만 차이를 보여 줍니다.
- **어떻게(how) 읽나**: 보통 없어지는 줄 앞에 빼기 기호(-), 새로 들어가는 줄 앞에 더하기 기호(+)가 붙습니다. 빼기 줄이 없어지고 더하기 줄로 바뀐다고 읽으면 됩니다.

파일 편집 파이프라인

요청 → 파일 읽기 → diff 미리보기 → 사람 승인 → 적용



ch_03_s02 · 파일 편집 파이프라인 · diff 미리보기 · 사람 승인

이렇게 보입니다

먼저 파일을 읽고 **설명하게** 하는 경우입니다. 읽기는 파일을 바꾸지 않으므로 미리보기도, 승인 창도 뜨지 않습니다.

```

PS C:\Users\사용자\claude-practice> claude
> config.json 파일이 무슨 역할을 하는지 한 문단으로 설명해 줘

[Read] config.json 을 읽었습니다
이 파일은 앱의 기본 설정을 담고 있습니다. 화면 테마(theme)와 표시 언어(language),
자동 저장 간격(autosave) 세 가지 값을 정의합니다.
  
```

이번에는 **고치게** 하는 경우입니다. Claude Code는 곧바로 바꾸지 않고 변경 미리보기(diff)를 보여 준 뒤 승인을 묻습니다.

```

> config.json 의 theme 값을 "light"에서 "dark"로 바꿔 줘

[Edit] config.json 변경 미리보기:

{
  "theme": "light",      <- 이 줄을 없애고
+  "theme": "dark",      <- 이 줄로 바꿉니다
  "language": "ko",
  "autosave": 60
}

이 변경을 적용할까요? [ 1. 이번만 허용  2. 항상 허용  3. 거부 ]
  
```

빼기/더하기 표시를 더 또렷한 diff 형태로 보면 다음과 같습니다.


```
- "theme": "light",
+ "theme": "dark",
```

미리보기 화면 읽는 법

표시	의미
[Read]	파일을 읽기만 하는 중입니다. 파일은 바뀌지 않습니다
[Edit] + 미리보기	고칠 안을 보여 주는 중입니다. 아직 파일은 그대로입니다
앞에 - 가 붙은 줄	없어지는(이전) 줄입니다
앞에 + 가 붙은 줄	새로 들어가는(이후) 줄입니다
[1. 이번만 허용 ...]	사람이 승인·거부를 고르는 지점입니다

베스트 프랙티스: 미리보기가 떴을 때 + 와 - 줄을 한 줄씩 눈으로 짚어 본 뒤 승인합니다. 특히 내가 "건드리지 말라"고 한 부분이 그대로 남아 있는지 확인하는 습관이 사고를 막습니다.

흔한 실수: 미리보기를 읽지 않고 습관적으로 승인하기. 미리보기는 사람이 마지막으로 점검하는 안전장치입니다. 변경 폭이 예상보다 크면(생각보다 많은 줄이 바뀌면) 일단 거부하고, 요청을 더 좁혀 다시 시키는 편이 안전합니다.

절 요약

- 파일 편집은 곧바로 덮어쓰지 않고, 변경 미리보기(diff)와 승인을 거칩니다.
- diff에서 - 는 없어지는 줄, + 는 새로 들어가는 줄을 뜻합니다.
- 미리보기는 사람이 마지막으로 점검하는 안전장치이므로, 읽고 나서 승인합니다.

터미널 명령 실행과 결과 해석

도입

요리를 마친 뒤에는 한 입 맛을 봐서 간이 맞는지 확인합니다. 코드나 문서를 고친 뒤에도 마찬가지로 "제대로 됐는지" 확인하는 단계가 필요합니다. Claude Code는 이 확인을 위해 **터미널 명**

령을 직접 실행하고 그 결과를 읽을 수 있습니다. 이 절에서는 명령 실행이 무엇이고, 특히 오류가 났을 때 출력을 어떻게 읽어 다음 행동을 정하는지를 봅니다.

핵심 개념 설명

명령 실행 (command execution)

명령 실행(command execution)이란, Claude Code가 테스트·빌드 같은 명령을 터미널에서 직접 돌려 결과를 확인하는 작업입니다. 사람이 "고쳤으니 한번 돌려 볼게"라고 직접 실행하던 일을, 도구가 대신 해 주는 것입니다. 여기서 '빌드(build)'는 코드를 실제로 실행 가능한 형태로 조립하는 과정, '테스트(test)'는 고친 코드가 규칙대로 동작하는지 자동으로 점검하는 과정을 뜻합니다.

결과 해석 (output reading)

결과 해석(output reading)이란, 명령을 실행한 뒤 화면에 찍힌 출력(글자 결과)을 읽고 "성공인지, 실패인지, 다음에 무엇을 해야 하는지"를 판단하는 일입니다. Claude Code는 이 출력을 스스로 읽고, 실패하면 원인을 찾아 다시 고치는 다음 행동으로 이어 갑니다.

- **왜(why) 실행해 보나:** 고친 것이 정말 맞는지는 돌려 봐야 합니다. 눈으로만 본 변경은 빠뜨린 곳이 있을 수 있습니다.
- **언제(when) 실행하나:** 보통 파일을 고친 뒤, "테스트로 확인까지 해 줘"라고 함께 시켰을 때입니다. 실행은 컴퓨터에 영향을 주는 행동이므로 권한 요청이 함께 뜨는 경우가 많습니다(다음 절).
- **어떻게(how) 결과를 읽나:** 출력에는 보통 "통과/실패", 실패한 위치, 오류 메시지가 들어 있습니다. 사람도 이 세 가지를 같이 읽으면 다음 행동을 정할 수 있습니다.

이렇게 실행하고 읽습니다

먼저 명령이 **성공**한 경우입니다. 마지막 줄에 통과 건수가 보입니다.

```
> 방금 고친 코드가 잘 동작하는지 테스트를 돌려서 확인해 줘

[Bash]  pytest 실행 중 ...
[Bash]  결과:
        ===== 5 passed in 0.42s =====
완료했습니다. 테스트 5건이 모두 통과했습니다.
```

다음은 **실패**한 경우입니다. 여기서 결과 해석이 빛을 발합니다. Claude Code는 오류 출력을 읽고, 무엇이 문제인지 짚은 뒤 다음 행동을 제안합니다.

```
[Bash] pytest 실행 중 ...
[Bash] 결과:
FAILED test_cart.py::test_total - AssertionError: 합계가 -10 입니다
===== 1 failed, 4 passed =====
```

테스트 1건이 실패했습니다. 합계가 음수(-10)로 나오는 것을 보니
cart.py 의 더하기 계산이 빠기로 잘못 들어간 것 같습니다.
cart.py 의 해당 부분을 고칠까요?

오류 출력에서 읽어야 할 세 가지

출력 조각	무슨 정보인가	다음 행동 힌트
FAILED test_cart.py::test_total	실패한 위치(어느 테스트)	어디를 봐야 할지 알려 줍니다
AssertionError: 합계가 -10 입니다	실패한 이유(메시지)	무엇이 잘못됐는지 알려 줍니다
1 failed, 4 passed	전체 요약(몇 건 실패)	문제의 크기를 알려 줍니다

팁: 오류 메시지가 길어도 겁먹을 필요 없습니다. 대개 **맨 아래 몇 줄**에 핵심 원인과 위치가 적혀 있습니다. Claude Code에게 "이 오류가 무슨 뜻인지 쉽게 풀어 줘"라고 물으면, 출력을 대신 읽어 설명해 줍니다.

흔한 실수: 실패한 출력을 보고 곧바로 "그냥 통과되게 해 줘"라고 시키기. 이렇게 하면 원인은 그대로 둔 채 점검만 무력화될 수 있습니다. 실패했을 때는 **왜 실패했는지부터** 묻고, 원인을 고치는 방향으로 시키는 것이 바른 순서입니다.

절 요약

- 명령 실행은 고친 결과가 맞는지 도구가 직접 돌려 확인하는 단계입니다.
- 오류 출력에서는 실패 위치·실패 이유·전체 요약 세 가지를 읽습니다.
- 실패했을 때는 점검을 무력화하지 말고 원인부터 고치도록 시킵니다.

권한 요청과 승인의 첫 만남

도입

택배 기사는 문을 열고 들어오기 전에 초인종을 눌러 확인을 받습니다. 집 안에 영향을 주는 행동이기 때문입니다. Claude Code도 파일을 고치거나 명령을 실행하는 것처럼 **컴퓨터에 영향을 주는 행동 직전**에는 사람에게 허락을 묻습니다. 이 절에서는 그 확인 창이 언제 뜨고, 어떤 선택지가 있으며, 무엇을 골라야 하는지를 봅니다.

핵심 개념 설명

권한 요청 (permission prompt)

권한 요청(permission prompt) 이란, Claude Code가 영향이 있는 작업을 하기 전에 사용자에게 "이거 해도 될까요?"라고 묻는 확인 창입니다. 'prompt(프롬프트)'는 여기서 "사용자에게 답을 요구하는 화면"이라는 뜻으로, 앞서 1절의 '주문서로서의 프롬프트'와는 다른 쓰임입니다. 읽기처럼 안전한 작업에는 보통 뜨지 않고, 파일 편집이나 명령 실행처럼 **되돌리기 어렵거나 외부에 영향을 주는 작업**에서 뜹니다.

이번만 허용·항상 허용 (allow once / allow always)

권한 요청 창에는 보통 세 가지 선택지가 있습니다. **이번만 허용(allow once)** 은 지금 이 한 번만 허락하는 것이고, **항상 허용(allow always)** 은 같은 종류의 작업을 앞으로 묻지 않고 자동으로 허락하는 것이며, **거부(deny)** 는 하지 못하게 막는 것입니다.

여기서 입문자가 흔히 하는 오해가 있습니다. "**한 번 허용하면 모든 작업이 영구 허용된다**" 는 생각입니다. 그렇지 않습니다. '이번만 허용'은 딱 그 작업 한 번에만 적용되고, 다음에 비슷한 작업을 또 하려 하면 다시 묻습니다. '항상 허용'을 골랐을 때만 그 종류의 작업이 자동으로 허락됩니다.

- **왜(why) 묻나**: 파일 삭제나 외부에 영향을 주는 명령은 되돌리기 어렵습니다. 실행 전에 사람이 한 번 막아 줄 수 있어야 안전합니다.
- **언제(when) 뜨나**: 주로 파일 편집([Edit])과 명령 실행([Bash]) 직전입니다. 단순 읽기([Read])에서는 대개 뜨지 않습니다.
- **어떻게(how) 고르나**: 처음에는 **이번만 허용**을 기본으로 삼아 매번 내용을 확인합니다. 안전이 확실하고 자주 반복되는 작업만 신중하게 항상 허용으로 둡니다.

이렇게 보입니다

다음은 파일을 고치려 할 때 뜨는 권한 요청 창의 예상 화면입니다.

[Edit] notes.txt 를 수정하려고 합니다.

변경 미리보기를 확인했습니다. 이 편집을 적용할까요?

1. 이번만 허용 (allow once) - 지금 이 한 번만 적용합니다
2. 항상 허용 (allow always) - 앞으로 파일 편집을 묻지 않습니다
3. 거부 (deny) - 적용하지 않습니다

선택 >

명령을 실행하려 할 때도 비슷한 창이 뜹니다. 어떤 명령을 돌리려는지 함께 보여 줍니다.

[Bash] 다음 명령을 실행하려고 합니다:

pytest

실행할까요?

1. 이번만 허용
2. 항상 허용
3. 거부

선택 >

세 가지 선택지 비교

선택지	적용 범위	언제 고르나	주의점
이번만 허용	지금 이 작업 한 번	내용을 매번 확인하고 싶을 때(기본 권장)	비슷한 작업마다 다시 묻습니다
항상 허용	같은 종류의 작업 전체	안전이 확실하고 자주 반복될 때	이후 묻지 않으므로 신중히
거부	적용 안 함	의도와 다르거나 위험할 때	도구는 다른 방법을 다시 제안합니다

주의: 파일을 지우거나(Remove-Item), 외부로 무언가를 보내거나, 시스템 설정을 바꾸는 명령에 대해서는 **항상 허용을 함부로 고르지 마십시오**. 한 번 자동 허용으로 열어 두면 이후 같은 종류의 위험한 작업도 묻지 않고 진행될 수 있습니다. 위험한 작업은 매번 내용을 보고 '이번만 허용'으로 다루는 편이 안전합니다.

팁: 무엇을 골라야 할지 망설여지면 일단 **거부**해도 됩니다. 거부한다고 작업이 망가지지 않습니다. Claude Code는 거부를 받으면 다른 방법을 제안하거나, 더 자세히 설명해 달라고 합니다. 승인 방식을 미리 정해 두는 자세한 설정은 다음 4장에서 다룹니다.

절 요약

- 권한 요청은 파일 편집 명령 실행처럼 영향이 있는 작업 직전에 뜹니다.
- '이번만 허용'은 한 번만, '항상 허용'은 같은 종류를 계속, '거부'는 막는 것입니다.
- 처음에는 이번만 허용을 기본으로, 위험한 작업은 항상 허용을 피합니다.

컨텍스트 관리 기초 - 압축(compaction)과 세션 위생

도입

책상 위에 서류가 끝없이 쌓이면 어느 순간 정작 필요한 종이를 찾기 어려워집니다. 그럴 때 우리는 핵심만 요약 메모로 남기고 나머지는 치웁니다. Claude Code와의 대화도 길어지면 비슷한 일이 벌어집니다. 이 절에서는 대화가 길어질 때 무슨 일이 생기는지, 그리고 `/compact` 와 `/clear` 로 어떻게 정리하는지를 배웁니다.

핵심 개념 설명

컨텍스트 윈도우 (context window)

컨텍스트 윈도우(context window)란, Claude Code가 한 번에 기억하고 참고할 수 있는 **대화·파일 내용의 총량**을 담는 그릇입니다. '컨텍스트(context)'는 "지금까지 나눈 맥락", '윈도(window)'는 "그 맥락을 담아 두는 한정된 공간"을 뜻합니다. 이 그릇에는 정해진 크기(한도)가 있어서, 대화와 읽은 파일이 쌓일수록 점점 차오릅니다.

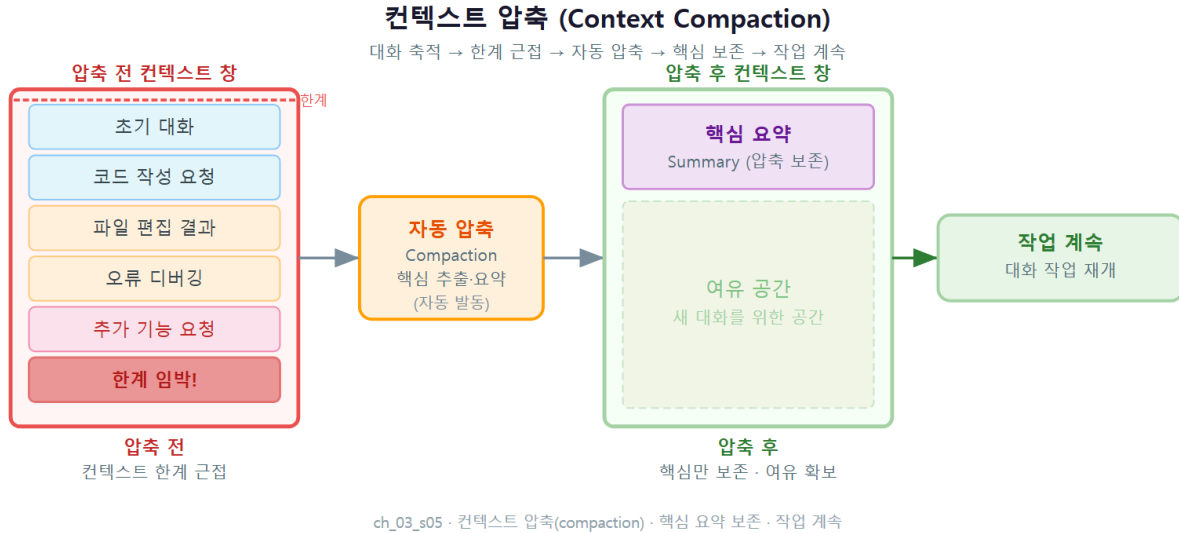
여기서 흔한 오해가 둘 있습니다. 하나는 **"대화는 무한히 기억된다"** 는 생각입니다. 그릇에는 한도가 있어, 너무 차면 오래된 내용부터 밀려납니다. 다른 하나는 뒤에 나올 압축과 관련해 **"압축 되면 모든 내용이 사라진다"** 는 오해인데, 이것도 사실이 아닙니다.

컨텍스트 압축 (compaction)

컨텍스트 압축(compaction)이란, 그릇이 차오를 때 **과거 대화를 핵심만 요약해 공간을 회복**하는 기능입니다. 책상이 가득 차면 긴 서류를 한 장 요약 메모로 바꿔 자리를 비우는 것과 같습니다. 중요한 점은, 압축은 **지우는 것이 아니라 줄이는 것**이라는 데 있습니다. 핵심 맥락은 요약으로 남고, 세세한 군더더기만 정리됩니다. 압축은 한도에 가까워지면 **자동으로** 일어나기도 하고, 내가 `/compact` 명령으로 직접 실행할 수도 있습니다.

세션 위생 (session hygiene)

세션 위생(session hygiene)이란, 작업 주제가 바뀌거나 대화가 너무 길어졌을 때 불필요한 맥락을 정리해 깨끗한 상태로 일하는 습관을 말합니다. '위생'이라는 말처럼, 책상을 주기적으로 치워 두면 다음 일을 더 정확하게 할 수 있습니다.



- **왜(why) 관리하나:** 그릇이 가득 차면 오래된 맥락이 밀려나 도구가 앞 내용을 놓칠 수 있고, 서로 관계없는 옛 대화가 남아 있으면 엉뚱한 참고를 할 수 있습니다.
- **언제(when) 정리하나:** 한 작업을 끝내고 **전혀 다른 작업**으로 넘어갈 때는 `/clear`, 같은 작업을 계속하지만 대화가 너무 길어졌을 때는 `/compact` 가 어울립니다.
- **어떻게(how) 정리하나:** 아래 두 명령을 상황에 맞게 씁니다.

이렇게 정리합니다

다음은 대화 도중 사용하는 두 명령의 예상 화면입니다. `/` 로 시작하는 입력은 Claude Code 세션 안에서 쓰는 **슬래시 명령**입니다.

```
> /compact
```

대화가 길어져 컨텍스트를 정리합니다 ...
지금까지의 작업(notes.txt 날짜 통일, config.json 테마 변경)을 요약으로 남기고
세부 대화는 정리했습니다. 핵심 맥락은 유지됩니다. 계속 이어서 작업할 수 있습니다.

```
> /clear
```

대화 내용을 모두 비웠습니다. 새 작업을 시작합니다.
(이전 대화 맥락은 더 이상 참고하지 않습니다.)

두 명령의 차이

명령	하는 일	이전 맥락	언제 쓰나
<code>/compact</code>	과거 대화를 요약해 공간 회복	핵심은 요약으로 유지	같은 작업을 길게 이어 갈 때
<code>/clear</code>	대화를 통째로 비우고 새 시작	더 이상 참고 안 함	완전히 다른 작업으로 넘어 갈 때

팁: 새 주제로 넘어가기 전에 `/clear` 로 책상을 비우면, 앞 작업의 맥락이 새 작업에 섞여 드는 혼선을 막을 수 있습니다. 반대로 같은 작업을 계속할 때 도구가 앞 내용을 흐릿하게 기억하는 느낌이면 `/compact` 가 도움이 됩니다.

흔한 실수: 아직 진행 중인 작업의 중요한 맥락이 대화에 남아 있는데 `/clear` 로 비워 버리기. `/clear` 는 이전 대화를 더 이상 참고하지 않으므로, 작업이 끝났는지 먼저 확인하고 사용합니다. 헛갈리면 비우기 전에 "지금까지 한 일을 요약해 줘"라고 한 번 정리해 두는 것도 방법입니다.

절 요약

- 컨텍스트 윈도우는 한 번에 기억하는 대화·파일의 총량을 담는 한정된 그릇입니다.
- 압축(compaction)은 과거 대화를 요약해 공간을 회복하며, 지우는 것이 아니라 줄이는 것입니다.
- 같은 작업이 길어지면 `/compact` , 다른 작업으로 넘어가면 `/clear` 로 세션을 정리합니다.

실습 과제

해보기: 샘플 폴더에서 읽기-수정-실행 한 바퀴 돌리기

앞 장에서 만든 실습 폴더(`claude-practice`)에 작은 파일 하나를 두고, 이 장에서 배운 흐름을 처음부터 끝까지 한 바퀴 돌려 봅니다.

- **단계 1 (준비):** 실습 폴더에 줄마다 날짜가 적힌 간단한 텍스트 파일 `notes.txt` 를 하나 만듭니다(예: "회의 6/16 ...", "마감 2026.6.20 ..."). 형식이 제각각이어도 괜찮습니다.
- **단계 2 (대화):** Claude Code에게 **명확한 프롬프트**로 요청합니다. 예: "notes.txt 의 날짜를 'YYYY-MM-DD' 형식으로 통일해 줘. 본문은 그대로 두고, 다 고치면 몇 군데 바꿨는지 알려 줘."
- **단계 3 (미리보기·승인):** 변경 미리보기(diff)가 뜨면 `+` 와 `-` 줄을 한 줄씩 확인하고, 의도와 맞으면 **이번만 허용**을 고릅니다.

- **단계 4 (관찰):** 적용 후 파일을 다시 읽어 달라고 해서 결과를 확인합니다.
- **점검:** 아래 표를 채워 보고, 각 단계에서 권한 요청이 떴는지 안 떴는지 표시할 수 있으면 성공입니다.

단계	내가 입력한 프롬프트	본 화면(미리보기/권한/결과)
읽기	(예) notes.txt 설명해 줘	권한 요청 없음, 설명만 출력
수정	...	diff 미리보기 + 권한 요청
확인	...	...

FAQ

Q1. 프롬프트를 한국어로 줘도 잘 알아듣나요? → A1. 네. Claude Code는 한국어 요청을 잘 이해합니다. 중요한 것은 언어가 아니라 **명확함**입니다. 어떤 파일을, 어떤 조건으로, 무엇을 건드리지 말지 한국어로 또렷이 적으면 충분합니다.

Q2. 변경 미리보기를 봤는데 마음에 안 들면 어떻게 하나요? → A2. 권한 요청에서 **거부**를 고르면 됩니다. 파일은 바뀌지 않습니다. 그다음 "이번에는 이 부분만 고쳐 줘"처럼 요청을 더 좁혀 다시 시키면, 도구가 새 미리보기를 보여 줍니다.

Q3. 실수로 `/clear` 를 눌렀습니다. 파일도 사라지나요? → A3. 아닙니다. `/clear` 는 **대화 맥락만** 비울 뿐, 디스크에 저장된 파일은 그대로 있습니다. 다만 이전 대화 내용은 더 이상 참고하지 않으므로, 이어서 작업하려면 필요한 맥락을 다시 알려 줘야 합니다.

장 요약

이번 장에서는 Claude Code로 일을 시키는 기본 한 바퀴를 익혔습니다. 시작은 늘 **명확한 프롬프트**입니다. 무엇을.어떤 파일에.어떤 제약으로.어떻게 확인할지를 담으면 도구가 헛다리를 짚지 않습니다. 파일을 고칠 때는 곧바로 덮어쓰지 않고 **변경 미리보기(diff)** 를 보여 주며, `-` (없어질 줄)와 `+` (들어갈 줄)를 확인하고 승인해야 적용됩니다. 고친 뒤에는 **터미널 명령을 실행**해 결과를 읽는데, 실패 출력에서는 실패 위치·이유·요약 세 가지를 읽어 다음 행동을 정합니다. 영향이 있는 작업 직전에는 **권한 요청**이 떴서 이번만·항상 허용·거부를 고를 수 있고, 입문 단계에서는

이번만 허용을 기본으로 삼습니다. 마지막으로 대화가 길어지면 **컨텍스트 윈도우**가 차오르므로, `/compact` 로 요약하거나 `/clear` 로 비우는 **세션 위생**으로 깨끗한 상태를 유지합니다.

핵심 키워드

프롬프트(prompt), 맥락 제공(providing context), 변경 미리보기(diff), 명령 실행(command execution), 권한 요청(permission prompt), 컨텍스트 윈도우(context window), 압축(compaction), 세션 위생(session hygiene)

다음 장 미리보기

다음 장에서는 이번 장에서 처음 만난 **권한**을 본격적으로 다룹니다. 승인을 매번 묻게 할지, 어떤 작업을 자동 허용할지 미리 정해 두는 권한 모드와 설정을 배워, 안전과 편함 사이의 균형을 스스로 맞추게 됩니다.

권한·승인 모드 심화와 안전

학습 목표 - Claude Code의 권한 모델(permission model)이 도구별로 통제되는 원리를 설명할 수 있습니다. - 네 가지 승인 모드(approval mode)의 차이를 비교하고 상황에 맞게 고를 수 있습니다. - 허용·거부·질문(allow/deny/ask) 규칙을 명령·경로 패턴으로 작성할 수 있습니다. - 민감 경로와 파괴적 명령을 차단하고, Windows 11에서 안전한 실습 환경을 갖출 수 있습니다.

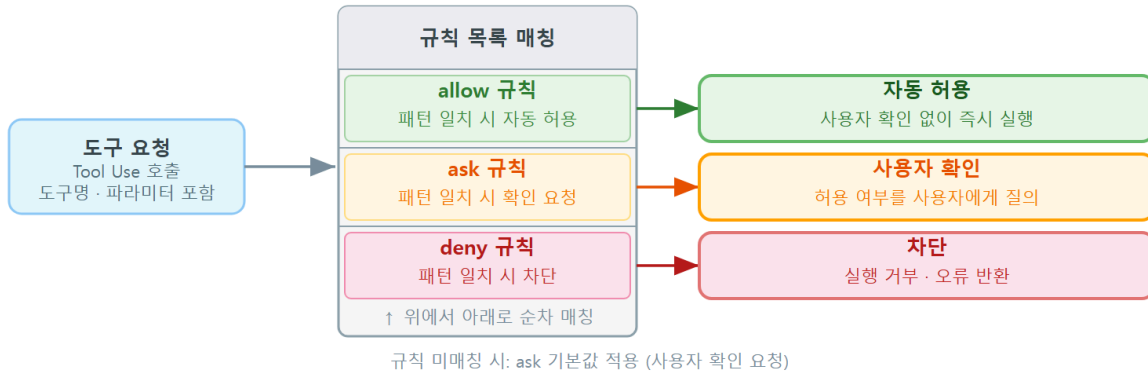
앞 장에서 권한 요청(permission prompt)이 *언제* 뜨고 이번만·항상 허용·거부를 어떻게 고르는지 처음 만나 보았습니다. 그때는 화면에 뜬 질문에 그때그때 답하는 수준이었습니다. 이 장은 한 걸음 더 들어갑니다. **"왜 이 작업에서는 묻고, 저 작업에서는 안 묻는가"** 라는 규칙의 속을 들여다보고, 그 규칙을 내가 직접 정하는 법을 배웁니다.

핵심 질문은 하나입니다. **편함과 안전을 어떻게 동시에 가져갈 것인가.** 매번 모든 것을 물으면 안전하지만 번거롭고, 아무것도 안 물으면 편하지만 위험합니다. Claude Code는 이 둘 사이를 **도구별 권한 규칙**과 **승인 모드**라는 두 손잡이로 조절합니다. 이 장을 마치면 그 두 손잡이를 내 상황에 맞게 돌릴 수 있게 됩니다.

이 책은 Windows 11의 **PowerShell(파워셸)** 을 기준으로 합니다. 화면 맨 앞의 `PS C:\Users\사용자>` 표시는 "지금 명령을 기다리는 중"이라는 안내입니다. 설정 파일 경로도 Windows 기준으로 `C:\Users\사용자\.claude\settings.json` 처럼 적습니다.

권한 모델(permission model)의 원리

권한 모델 구조 — 도구 요청 처리 흐름



ch_04_s01 · 권한 모델(permission model) · allow / ask / deny 규칙

도입

회사 출입증을 떠올려 보겠습니다. 같은 출입증이라도 로비는 그냥 통과, 사무실은 태그 확인, 서버실은 아예 출입 금지처럼 **구역마다 권한이 다릅니다**. Claude Code의 권한도 똑같습니다. 파일을 읽는 일, 고치는 일, 명령을 실행하는 일에 **서로 다른 통제 수준**을 둡니다. 이 절에서는 그 통제가 어떤 원리로 작동하는지 살펴봅니다.

핵심 개념 설명

권한 모델 (permission model)

권한 모델(permission model)이란, 읽기·쓰기·명령 실행 같은 **도구별로** 허용·질문·거부를 다르게 통제하는 체계를 말합니다. 여기서 "도구"는 Claude Code가 일을 할 때 쓰는 손과 같습니다. 파일을 읽는 손(Read), 파일을 고치는 손(Edit), 터미널 명령을 실행하는 손(Bash)이 각각 따로 있고, 손마다 통제 수준을 다르게 줄 수 있습니다.

입문자가 가장 자주 하는 오해는 **"모든 작업이 하나의 켜고 끄는 스위치로 통제된다"**는 생각입니다. 실제로는 그렇지 않습니다. 읽기는 자유롭게 풀어 두면서 명령 실행만 매번 묻게 하는 식으로, 작업마다 다르게 정할 수 있습니다. 이 **"도구별 통제(per-tool control)"**가 권한 모델의 핵심입니다.

왜 이렇게 나눠 통제할까요? **위험의 크기가 작업마다 다르기 때문입니다**. 파일을 읽기만 하는 일은 무언가를 망가뜨리지 않습니다. 반면 파일을 고치거나 명령을 실행하는 일은 되돌리기 어

려운 결과를 낼 수 있습니다. 위험이 큰 손일수록 더 자주 물어보게 하는 것이 합리적입니다.

권한 판정의 순서 (어떻게 결정되나)

Claude Code가 어떤 도구를 쓰려 할 때마다, 그 요청은 **정해진 순서를 따라 판정**됩니다. 이 순서를 알면 "왜 이 작업이 막혔지?" 또는 "왜 안 묻고 그냥 실행됐지?"를 스스로 설명할 수 있습니다.

1. **거부(deny) 규칙**을 먼저 확인합니다. 걸리면 그 자리에서 **차단**합니다. 거부는 가장 강력해서 다른 무엇보다 우선합니다.
2. 다음으로 **허용(allow) 규칙**을 확인합니다. 걸리면 묻지 않고 **자동 실행**합니다.
3. 다음으로 **질문(ask) 규칙**을 확인합니다. 걸리면 사용자에게 **물어봅니다**.
4. 어느 규칙에도 걸리지 않으면, **현재 승인 모드**의 기본 동작을 따릅니다(다음 절에서 다룹니다).

여기서 꼭 기억할 점은 **거부가 항상 이긴다**는 사실입니다. 허용 규칙과 거부 규칙이 같은 작업에 동시에 걸리면, 결과는 언제나 차단입니다. "거부 규칙보다 허용 규칙이 우선한다"는 흔한 오해와 정반대이니, 이 순서를 거꾸로 외워 두면 안전한 쪽으로 실수하게 됩니다.

이렇게 동작합니다

다음은 같은 세션 안에서 세 가지 작업이 **서로 다르게 처리되는** 모습입니다. 아래 화면은 실제 입력이 아니라 권한 모델의 동작을 보여 주기 위한 예시입니다. 대괄호로 표시된 **[Read]**, **[Edit]**, **[Bash]** 는 Claude Code가 지금 어떤 손을 쓰려는지 알려 주는 안내입니다.

```
PS C:\Users\사용자\claude-practice> claude
> README.md 를 읽고, 오타를 고친 뒤, 테스트를 실행해 줘

[Read] README.md
(자동 실행 — 읽기는 허용되어 있어 묻지 않음)

[Edit] README.md 3줄 변경
이 편집을 적용할까요? (1) 이번만 허용 (2) 항상 허용 (3) 거부

[Bash] npm test
이 명령을 실행할까요? (1) 이번만 허용 (2) 항상 허용 (3) 거부
```

읽기는 묻지 않고 지나갔지만, 편집과 명령 실행에서는 멈춰 물어봅니다. 같은 한 번의 요청 안에서도 **손마다 통제 수준이 다르다**는 것을 화면으로 확인할 수 있습니다.

화면에서 읽어야 할 것

표시	의미
[Read] 뒤에 곧바로 결과	읽기 작업이 허용되어 질문 없이 실행됨
[Edit] ... 3줄 변경	파일을 고치기 직전, 무엇이 바뀌는지 보여 주고 멈춤
(1) 이번만 / (2) 항상 / (3) 거부	이번 한 번만, 앞으로 계속, 또는 막기 중 선택
[Bash] npm test	터미널 명령을 실행하기 직전의 확인

팁: "항상 허용"을 고르면 그 작업에 대한 **허용(allow) 규칙이 자동으로 추가**됩니다. 다음부터는 같은 작업에서 묻지 않습니다. 편하지만, 한 번 고르면 계속 적용되므로 명령 실행 같은 위험한 손에는 신중하게 고릅니다.

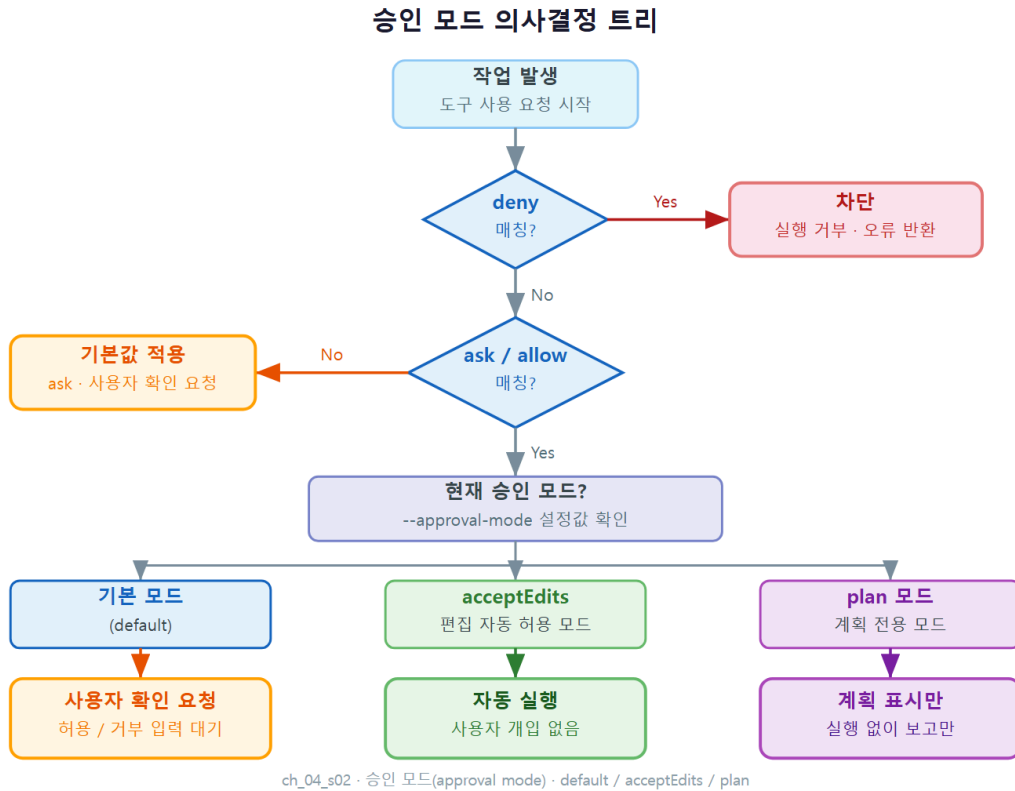
신뢰와 통제의 균형

권한 모델을 한마디로 요약하면 **"신뢰하는 만큼 풀어 주고, 불안한 만큼 붙잡아 둔다"** 입니다. 읽기처럼 안전한 일은 풀어 주어 흐름을 끊지 않고, 명령 실행처럼 위험한 일은 붙잡아 두어 사람이 한 번 더 확인합니다. 이 균형을 내 손으로 조절하는 구체적인 방법이 바로 다음에 배울 **승인 모드와 권한 규칙**입니다.

절 요약

- 권한은 하나의 스위치가 아니라 **읽기·편집·명령 실행 등 도구별로** 따로 통제됩니다.
- 판정 순서는 **거부 → 허용 → 질문 → 승인 모드** 기본이며, 거부가 항상 이깁니다.
- 위험이 큰 작업일수록 더 자주 묻게 하는 것이 권한 모델의 기본 사고방식입니다.

승인 모드 비교 - 매번 묻기·편집 자동수락·계획 모드·우회



도입

자동차에 비유하면 권한 규칙이 "누가 어디까지 운전할 수 있는가"라는 면허라면, **승인 모드 (approval mode)**는 "지금 이 순간 얼마나 자동으로 달릴 것인가"라는 주행 모드입니다. 수동으로 한 단계씩 확인할 수도 있고, 일부는 자동에 맡길 수도 있습니다. 이 절에서는 네 가지 주행 모드를 비교하고, 상황별로 안전하게 고르는 법을 배웁니다.

핵심 개념 설명

승인 모드 (approval mode)

승인 모드(approval mode)란, 도구 사용 요청을 전체적으로 어떻게 처리할지 정하는 모드입니다. 앞 절의 권한 규칙이 "이 작업은 이렇게"라는 개별 규칙이라면, 승인 모드는 규칙에 걸리지 않은 나머지 작업을 **무엇이 기본인지** 정하는 큰 틀입니다. 네 가지가 있습니다.

- **기본(default) - 매번 묻기:** 편집과 명령 실행처럼 영향이 있는 작업마다 멈춰 물어봅니다. 가장 안전하며, 입문 단계의 기본값입니다.
- **편집 자동수락(acceptEdits):** 파일 읽기와 편집은 묻지 않고 자동 적용하되, 터미널 명령 실행은 여전히 물어봅니다. 문서 정리처럼 편집이 많고 명령이 적은 일에 잘 맞습니다.
- **계획 모드(plan):** 무엇을 할지 계획만 세워 보여 주고, 파일을 고치거나 명령을 실행하지는 않습니다. 큰 작업을 시작하기 전에 "이렇게 진행하겠다"는 안을 먼저 검토할 때 씁니다.

다.

- **전체 우회(bypassPermissions):** 모든 권한 질문과 안전 점검을 끄고 **곧바로 실행**합니다. 가장 빠르지만 가장 위험합니다.

계획 모드 (plan mode)

네 모드 중 **계획 모드(plan mode)** 는 따로 강조할 가치가 있습니다. 이 모드에서 Claude Code 는 파일을 읽어 상황을 파악하고 **"이런 순서로 이런 파일을 이렇게 바꾸겠습니다"** 라는 계획을 글로 제시합니다. 그러나 실제로 손대지는 않습니다. 사용자가 계획을 읽고 동의해야 비로소 실행 단계로 넘어갑니다.

왜 유용할까요? **되돌리기 어려운 큰 작업일수록, 시작하기 전에 방향을 맞추는 비용이 가장 쌉니다.** 도구가 엉뚱한 방향으로 10개 파일을 고친 뒤 되돌리는 것보다, 계획 단계에서 "그 파일은 건드리지 마"라고 한마디 하는 편이 훨씬 빠릅니다.

흔한 오해 하나를 짚겠습니다. **"자동 수락이 항상 편하다"** 는 생각입니다. 편집 자동수락이나 전체 우회는 분명 빠릅니다. 하지만 도구가 잘못 판단했을 때 **멈춰 줄 사람이 없으면** 잘못이 그대로 쌓입니다. 편함의 대가로 안전을 내주는 거래임을 잊지 않아야 합니다.

한눈에 비교하기

다음 표는 네 가지 승인 모드가 읽기·편집·명령 실행을 각각 어떻게 처리하는지 정리한 것입니다.

승인 모드	읽기	편집	명령 실행	잘 맞는 상황
기본(default)	자동	질문	질문	입문·익숙하지 않은 작업, 안전 우선
편집 자동수락(acceptEdits)	자동	자동	질문	편집이 많고 명령이 적은 문서·코드 정리
계획 모드(plan)	자동	안 함	안 함	큰 작업 전 방향 검토, 영향 범위 파악
전체 우회(bypassPermissions)	자동	자동	자동	격리된 환경의 자동화(권장하지 않음)

표에서 위에서 아래로 내려갈수록 **사람의 개입이 줄고 속도가 빨라지지만, 안전망도 같이 사라**진다는 흐름을 읽을 수 있습니다.

이렇게 모드를 바꿉니다

CLI 화면에서는 **Shift+Tab** 키를 누를 때마다 모드가 차례로 바뀌고, 지금 어떤 모드인지는 입력창 근처에 표시됩니다. 아래는 모드 표시가 바뀌는 모습을 보여 주는 예시 화면입니다.

```
PS C:\Users\사용자\claude-practice> claude
> (Shift+Tab 을 눌러 모드를 순환합니다)

-- 기본 (매번 묻기) --
-- accept edits on (편집 자동수락) --
-- plan mode on (계획 모드) --
```

세션을 시작할 때부터 특정 모드로 열고 싶다면, 설정 파일에 시작 모드를 적어 둘 수 있습니다. 다음은 항상 "매번 묻기"로 시작하도록 정해 두는 설정 예시입니다.

```
{
  "permissions": {
    "defaultMode": "default"
  }
}
```

설정 키 설명

키	의미
permissions	권한 관련 설정을 담는 묶음입니다
defaultMode	세션을 시작할 때의 승인 모드입니다. default · acceptEdits · plan · bypassPermissions 중 하나를 적습니다

주의: 익숙해지기 전에는 `defaultMode` 를 `default` (매번 묻기)로 두기를 권합니다. 편집 자동수락은 편집 결과를 검토할 자신이 생긴 뒤에 켜도 늦지 않습니다.

상황별로 안전하게 고르기

두 가지 시나리오로 감을 잡아 보겠습니다.

- **시나리오 A** — 처음 보는 코드를 정리할 때: 무엇이 어떻게 바뀔지 예측이 어렵습니다. **기본(매번 묻기)** 으로 두고 편집 미리보기를 하나씩 확인하는 편이 안전합니다.
- **시나리오 B** — **오타-들여쓰기처럼 단순 편집을 대량으로 할 때**: 매번 묻는 것이 오히려 흐름을 끊습니다. 잠시 **편집 자동수락**으로 바꿔 빠르게 처리하고, 끝나면 다시 기본으로 돌아옵니다. 단, 명령 실행은 자동수락 모드에서도 여전히 물어보므로 한 겹의 안전망은 남습니다.

절 요약

- 승인 모드는 **기본·편집 자동수락·계획 모드·전체 우회** 네 가지이며, 아래로 갈수록 빠르지만 안전망이 줄어듭니다.
- **계획 모드**는 큰 작업 전에 방향을 먼저 맞추는 가장 값싼 안전장치입니다.
- CLI에서는 `Shift+Tab` 으로 모드를 순환하고, `defaultMode` 로 시작 모드를 정할 수 있습니다.

허용·거부·질문 규칙(allow/deny/ask) 작성

도입

앞 절의 승인 모드가 "큰 틀"이라면, 이 절의 **권한 규칙(permission rule)**은 그 틀 안에서 "이 명령은 통과, 저 경로는 차단"처럼 **예외를 콕 집어 정하는** 도구입니다. 건물 입구의 출입 명단에 비유할 수 있습니다. 명단에 따라 누구는 그냥 통과, 누구는 확인, 누구는 차단으로 미리 정해 두는 것과 같습니다. 이 절에서는 그 명단을 직접 작성합니다.

핵심 개념 설명

권한 규칙 (permission rule)

권한 규칙(permission rule)이란, 특정 명령이나 경로 패턴에 대해 허용(allow)·질문(ask)·거부(deny) 중 하나를 지정하는 규칙입니다. 이 규칙들은 설정 파일 `settings.json`의 `permissions` 안에 세 개의 목록으로 적습니다.

- **allow**: 묻지 않고 자동 실행할 작업의 목록입니다.
- **ask**: 실행 전에 항상 물어볼 작업의 목록입니다.
- **deny**: 절대 실행하지 않고 차단할 작업의 목록입니다.

앞서 배운 판정 순서를 떠올리면, **거부(deny)**가 가장 강하고, 그다음 허용, 그다음 질문 순으로 평가됩니다. 따라서 "이건 절대 안 됨"은 `deny` 에, "이건 안심하고 맡김"은 `allow` 에, "헛갈리면 물어봐"는 `ask` 에 적는다고 기억하면 됩니다.

규칙 패턴 (rule pattern)

각 규칙은 **도구 이름과 괄호 안의 패턴**으로 적습니다. 예를 들어 `Bash(git status:*)` 는 "git status 로 시작하는 모든 명령"을 뜻합니다. 여기서 `:*` 는 "그 뒤에 무엇이 와도 상관없음"을 나타내는 **와일드카드(wildcard, 빈칸 채우기 기호)** 입니다.

도구이름(패턴) — 권한 규칙 한 줄의 형태

- `Bash(git status:*)` — `git status` 로 시작하는 명령
- `Read(./src/**)` — `src` 폴더 아래 모든 파일 읽기
- `Edit(./docs/**)` — `docs` 폴더 아래 모든 파일 편집

여기서 `**` 는 "하위 폴더까지 전부"를 뜻하고, `*` 는 "한 단계 안에서 아무 이름"을 뜻합니다. 패턴을 좁게 쓸수록 통제가 정밀해지고, 넓게 쓸수록 한 번에 많이 풀리거나 막힙니다.

이렇게 작성합니다

다음은 실습 프로젝트에서 자주 쓰는 안전한 출발점입니다. 읽기와 안전한 조회 명령은 허용하고, 위험한 명령과 민감한 경로는 거부하며, 나머지 변경 작업은 물어보게 합니다.

```
{
  "permissions": {
    "allow": [
      "Read(./**)",
      "Bash(git status:*)",
      "Bash(git diff:*)",
      "Bash(npm test:*)"
    ],
    "ask": [
      "Edit(./**)",
      "Bash(git commit:*)"
    ],
    "deny": [
      "Read(./env)",
      "Bash(git push:*)"
    ]
  }
}
```

이 설정으로 세션을 열면, 파일을 읽거나 `git status` 를 실행하는 일은 조용히 지나가고, 파일을 고치거나 커밋하는 일은 한 번 물어보며, `.env` 파일을 읽거나 원격에 강제로 올리는 일은 시도조차 막힙니다.

규칙별 의미

규칙	분류	의미
<code>Read(./**)</code>	allow	프로젝트 안 모든 파일 읽기를 자동 허용
<code>Bash(npm test:*)</code>	allow	테스트 실행은 안전하다고 보고 자동 허용
<code>Edit(./**)</code>	ask	모든 파일 편집은 실행 전 한 번 물어봄
<code>Bash(git commit:*)</code>	ask	커밋은 되돌리기 번거로우니 물어봄
<code>Read(./env)</code>	deny	비밀 정보가 담긴 <code>.env</code> 읽기를 차단
<code>Bash(git push:*)</code>	deny	원격 저장소로 올리는 명령을 차단

팁: 설정 파일은 위치에 따라 적용 범위가 다릅니다. `C:\Users\사용자\.claude\settings.json` 은 모든 프로젝트에 적용되는 **내 개인 설정**이고, 프로젝트 폴더 안의 `.claude\settings.json` 은 **그 프로젝트에만** 적용됩니다. 팀과 공유하고 싶은 규칙은 프로젝트 쪽에 둡니다.

주의: `allow` 에 넓은 패턴을 적을수록 편하지만, 그만큼 자동으로 실행되는 범위가 넓어집니다. 특히 `Bash` 규칙은 한 글자 차이로 위험이 크게 달라지므로, 조회성 명령(`git status`, `git diff`)부터 좁게 허용하는 습관을 들입니다.

특정 명령만 허용, 민감 경로는 거부

규칙 작성의 두 가지 흔한 목표를 시나리오로 정리합니다.

- **시나리오 A — 자주 쓰는 안전한 명령만 풀어 주기:** 매번 묻기 귀찮은 조회 명령(`git status`, `git log`, `npm test`)만 `allow` 에 넣습니다. 나머지 명령은 규칙에 없으므로 승인 모드 기본 동작(물어보기)을 따릅니다.
- **시나리오 B — 절대 건드리면 안 되는 곳 잠그기:** `.env`, `secrets` 폴더, 빌드 산출물처럼 손대면 곤란한 경로를 `deny` 에 넣습니다. 거부는 가장 강하므로, 실수로 허용 규칙이 겹쳐도 안전하게 막힙니다.

흔한 실수 하나를 짚겠습니다. `allow` 에 넣었으니 안심이라고 생각하다, 같은 대상이 `deny` 에도 걸려 있어 "왜 안 되지?"라고 당황하는 경우입니다. 이때는 판정 순서를 떠올리면 됩니다. 거부 허용을 이깁니다. 막힌다면 먼저 `deny` 목록부터 확인합니다.

절 요약

- 권한 규칙은 `settings.json` 의 `permissions` 안 `allow · ask · deny` 세 목록으로 작성합니다.
- 규칙은 도구이름(패턴) 형태이며, `*` 와 `**` 로 범위를 넓히거나 좁힙니다.
- 거부가 가장 강하므로, 위험한 것은 `deny` 에 확실히 넣어 두면 안전 쪽으로 실수하게 됩니다.

위험 작업 가드 - 민감 경로·파괴적 명령 차단

도입

부엌에서 칼은 꼭 필요하지만, 아이 손이 닿는 곳에 두지는 않습니다. Claude Code도 마찬가지입니다. 강력한 만큼 되돌리기 어려운 일을 할 수 있으므로, 그런 일을 미리 잠가 두는 안전 가드가 필요합니다. 이 절에서는 파괴적인 명령과 민감한 경로를 어떻게 막는지 배웁니다.

핵심 개념 설명

파괴적 작업 (destructive action)

파괴적 작업(destructive action) 이란, 실행하고 나면 되돌리기 어렵거나 불가능한 작업을 말합니다. 대표적인 예는 다음과 같습니다.

- 파일·폴더를 통째로 지우는 명령(예: PowerShell의 `Remove-Item`, Git Bash의 `rm -rf`)
- 원격 저장소의 기록을 덮어쓰는 강제 푸시(`git push --force`)
- 작업 내용을 한꺼번에 버리는 초기화(`git reset --hard`)

이런 작업의 공통점은 **사고가 나면 복구 비용이 매우 크다**는 것입니다. 그래서 자동 허용 대상에서 빼고, 가능하면 거부 목록에 넣어 둡니다.

보호 경로 (protected path)

보호 경로(protected path) 란, 도구가 함부로 읽거나 고치면 안 되는 폴더·파일을 가리킵니다. 대표적으로 비밀번호·접속 키가 담긴 `.env` 파일, 인증 정보가 모인 `secrets` 폴더가 있습니다.

이런 경로는 *위키조차* 위험할 수 있습니다. 비밀 정보가 대화 맥락에 실려 흘러나갈 수 있기 때문입니다.

왜 따로 잠가야 할까요? 권한 규칙을 폭넓게 풀어 두었더라도, **단 한 곳이라도 절대 건드리면 안 되는 곳**은 따로 못을 박아 두어야 안심할 수 있기 때문입니다. 거부 규칙은 그 못의 역할을 합니다.

이렇게 잠급니다

다음은 파괴적 명령과 보호 경로를 거부 목록으로 잠그는 설정 예시입니다.

```
{
  "permissions": {
    "deny": [
      "Read(./env)",
      "Read(./env.*)",
      "Read(/secrets/**)",
      "Edit(/secrets/**)",
      "Bash(git push --force:*)",
      "Bash(git reset --hard:*)",
      "Bash(rm -rf:*)"
    ]
  }
}
```

가드별 의미

규칙	막는 것
<code>Read(./env)</code> · <code>Read(./env.*)</code>	비밀 정보가 담긴 환경 변수 파일 읽기
<code>Read(/secrets/**)</code> · <code>Edit(/secrets/**)</code>	인증 정보 폴더의 읽기·편집
<code>Bash(git push --force:*)</code>	원격 기록을 덮어쓰는 강제 푸시
<code>Bash(git reset --hard:*)</code>	작업 내용을 통째로 버리는 초기화
<code>Bash(rm -rf:*)</code>	폴더를 강제로 삭제하는 명령

이 설정이 걸려 있으면, 도구가 해당 작업을 시도할 때 실행되지 않고 차단되었다는 안내가 나옵니다. 아래는 차단 화면의 예시입니다.

```
PS C:\Users\사용자\claude-practice> claude
> .env 파일을 읽어서 설정값을 확인해 줘
```

```
[Read] .env
차단됨: 이 경로는 거부(deny) 규칙으로 보호되어 있습니다.
(작업이 실행되지 않았습니다)
```

위험: `--dangerously-skip-permissions` 옵션(전체 우회 모드)은 **모든 권한 질문과 안전 점검을 끕니다**. 이름에 "dangerously(위험하게)"가 들어간 데는 이유가 있습니다. 이 모드에서는 파괴적 명령도 묻지 않고 실행됩니다. 인터넷에 연결된 평소 작업 환경에서는 사용하지 않습니다. 꼭 필요하다면 외부와 격리되고 백업이 보장된 환경에서만 씁니다.

베스트 프랙티스: 거부 규칙은 "혹시 모를 사고"를 막는 **마지막 안전망**입니다. 승인 모드를 잠시 풀어 두더라도 거부 규칙만큼은 그대로 두면, 가장 위험한 작업은 어떤 상황에서도 막힙니다.

흔한 실수와 시나리오

- **시나리오 A — 비밀 파일이 새는 사고 막기:** `.env` 를 읽어 디버깅하려다 그 내용이 대화에 남는 일을 막으려면, 처음부터 `Read(./env)` 를 `deny` 에 넣어 둡니다. 필요한 값은 사람이 직접 확인해 일부만 알려 주는 편이 안전합니다.
- **시나리오 B — 실수 삭제 막기:** "임시 파일 정리해 줘" 같은 요청이 의도보다 넓게 해석되어 중요한 파일까지 지우는 일을 막으려면, 광범위 삭제 명령을 `deny` 에 넣습니다.

흔한 실수는 **거부 규칙을 너무 좁게 적는 것**입니다. 예를 들어 `Read(./env)` 만 막고 `Read(./env.local)` 은 빠뜨리면, 비슷한 이름의 다른 비밀 파일이 그대로 노출됩니다. `Read(./env.*)` 처럼 변형까지 함께 막는 습관이 필요합니다.

절 요약

- 파괴적 작업(삭제·강제 푸시·강제 초기화)과 보호 경로(`.env`·`secrets`)는 `deny` 로 잠급니다.
- `--dangerously-skip-permissions` (전체 우회)는 평소 환경에서 쓰지 않습니다.
- 거부 규칙은 승인 모드와 무관하게 작동하는 **마지막 안전망**입니다.

Windows 11에서 안전 실습 환경 구성

도입

운전을 배울 때 도로가 아니라 한적한 연습장에서 시작하듯, Claude Code도 **연습장**에서 익히는 것이 안전합니다. 이 절에서는 Windows 11에서 실습 전용 폴더를 만들고, 변경을 언제든지 되돌릴 수 있는 안전망을 갖추는 법을 배웁니다.

핵심 개념 설명

샌드박스 폴더 (sandbox folder)

샌드박스 폴더(sandbox folder)란, 무슨 일이 생겨도 다른 곳에 피해를 주지 않는 **실습 전용 격리 폴더**를 말합니다. 모래밭(sandbox)에서는 마음껏 쏟고 부셔도 괜찮은 것과 같습니다. 실제 업무 파일이나 중요한 프로젝트와 **물리적으로 떨어진 별도 폴더**에서 연습하면, 실수가 나도 그 폴더 안에서 끝납니다.

버전 관리 안전망 (git safety net)

버전 관리 안전망(git safety net)이란, 변경을 시작하기 전에 현재 상태를 **Git(깃)**으로 **저장(커밋)해 두어**, 잘못되면 그 시점으로 되돌리는 습관을 말합니다. 여기서 Git은 파일의 변경 이력을 사진처럼 찍어 두는 도구이고, 커밋(commit)은 "지금 이 순간을 한 장 찍어 둬"에 해당합니다. 찍어 둔 시점이 있으면 언제든지 그리로 돌아갈 수 있습니다.

왜 둘을 함께 쓸까요? 샌드박스 폴더가 **피해 범위를 가두는 울타리**라면, 버전 관리는 **시간을 되돌리는 안전망**입니다. 울타리 안에서도 실수는 나지만, 되돌릴 수 있으면 두렵지 않습니다.

이렇게 구성합니다

다음은 Windows 11의 PowerShell에서 실습 전용 폴더를 만들고 Git으로 첫 시점을 찍어 두는 절차입니다. > 가 아니라 `PS C:\...>` 로 시작하는 줄은 내가 직접 PowerShell에 입력하는 명령입니다.

```
# 1) 홈 폴더 아래에 실습 전용 폴더를 만들고 그 안으로 이동합니다
PS C:\Users\사용자> mkdir claud-practice
PS C:\Users\사용자> cd claud-practice

# 2) 이 폴더를 Git 저장소로 초기화합니다
PS C:\Users\사용자\claud-practice> git init

# 3) 비밀 파일이 실수로 올라가지 않도록 무시 목록을 만듭니다
PS C:\Users\사용자\claud-practice> ni .gitignore

# 4) 현재 상태를 첫 시점으로 저장(커밋)합니다
PS C:\Users\사용자\claud-practice> git add .
PS C:\Users\사용자\claud-practice> git commit -m "실습 시작 시점"
```


명령별 의미

명령	의미
<code>mkdir claude-practice</code>	<code>claude-practice</code> 라는 실습 폴더를 새로 만듭니다
<code>cd claude-practice</code>	그 폴더 안으로 이동합니다
<code>git init</code>	이 폴더에서 변경 이력 관리를 시작합니다
<code>ni .gitignore</code>	무시할 파일 목록 파일을 만듭니다(<code>ni</code> 는 <code>New-Item</code>)
<code>git add . · git commit -m "..."</code>	현재 상태를 한 시점으로 찍어 둡니다

`.gitignore` 파일에는 버전 관리에서 빼고 싶은 비밀 파일을 적습니다. 다음은 실습용 최소 예시입니다.

```
# 비밀 정보가 담긴 파일은 버전 관리에서 제외합니다
.env
.env.*
secrets/
```

팁: 작업을 시작하기 전에 `git status` 로 현재 상태를 확인하고, 한 덩어리의 작업이 끝날 때마다 커밋하는 습관을 들이면, 무엇이 언제 바뀌었는지 또렷이 남습니다. 잘못됐을 때 `git restore .` 로 마지막 커밋 시점의 파일로 되돌릴 수 있습니다.

주의: 실습 폴더는 바탕화면이나 문서함처럼 **다른 중요한 파일과 섞이지 않는 위치**에 따로 둡니다. 회사 공용 폴더나 동기화되는 클라우드 폴더 안에서 연습하면, 실수가 다른 사람에게까지 번질 수 있습니다.

변경 전 커밋·복구 습관

두 가지 시나리오로 안전망의 효과를 봅니다.

- **시나리오 A — 큰 변경 전에 시점 찍기:** 여러 파일을 한꺼번에 고치는 작업 전에 커밋해 두면, 결과가 마음에 안 들 때 `git restore .` 한 번으로 통째로 되돌립니다. 도구에게 일을 맡기기 전 "지금 커밋부터 하자"가 몸에 배면 사고가 사고로 끝나지 않습니다.
- **시나리오 B — 권한 설정도 폴더와 함께 관리:** 앞 절들에서 만든 `.claude\settings.json` 권한 규칙을 실습 폴더 안에 두고 함께 커밋하면, 권한 설정의 변경 이력도 남아 "언제 어떤 규칙을 풀었는지" 추적할 수 있습니다.

흔한 실수는 커밋 없이 한참 작업하다 한 번에 망치는 것입니다. 되돌릴 시점이 없으면 안전망도 없습니다. 작게 자주 커밋하는 것이 가장 든든한 습관입니다.

절 요약

- 실습은 다른 파일과 격리된 샌드박스 폴더에서 시작합니다.
- `git init` 과 커밋으로 되돌릴 시점을 만들어 두면 실수가 두렵지 않습니다.
- `.gitignore` 로 비밀 파일을 버전 관리에서 빼고, 권한 설정도 함께 관리합니다.

실습 과제

해보기: 내 프로젝트용 권한 규칙 세트 설계하기

실습 폴더에 맞춰 권한 규칙과 승인 모드 사용 계획을 직접 설계해 봅니다.

- **단계 1:** PowerShell에서 `claude-practice` 샌드박스 폴더를 만들고 `git init` 후 첫 커밋을 남깁니다.
- **단계 2:** 종이나 메모장에 작업을 세 칸으로 나눠 적습니다. (가) 항상 허용해도 안전한 명령, (나) 항상 물어볼 작업, (다) 절대 거부할 경로·명령.
- **단계 3:** 위 분류를 `.claude\settings.json` 의 `allow·ask·deny` 로 옮겨 적습니다. `.env` 와 `secrets/` 읽기, 강제 푸시·강제 삭제는 반드시 `deny` 에 넣습니다.
- **단계 4:** 어떤 상황에서 **계획 모드**를 쓰고 어떤 상황에서 **편집 자동수락**을 쓸지, 각각 한 문장 시나리오로 적습니다.
- **점검:** 같은 작업이 `allow` 와 `deny` 에 동시에 걸리면 어느 쪽이 이기는지 설명할 수 있나요? (정답: 거부가 이깁니다.)

FAQ

Q1. 권한 규칙을 바꿨는데 적용이 안 되는 것 같습니다. 왜 그런가요? → **A1.** 가장 흔한 원인은 **거부(deny) 규칙과 겹치는 경우**입니다. 판정 순서상 거부가 허용을 이기므로, 허용에 넣어도 같은 대상이 거부에 있으면 막힙니다. 먼저 `deny` 목록을 확인하세요. 또한 설정 파일을 저장한 위치(개인 설정인지 프로젝트 설정인지)와 패턴의 오타도 점검합니다.

Q2. 편집 자동수락(acceptEdits) 모드면 명령도 자동으로 실행되나요? → A2. 아닙니다. 편집 자동수락은 **파일 읽기와 편집만** 묻지 않고 진행하며, **터미널 명령 실행은 여전히 물어봅니다**. 그래서 편집이 많은 작업을 빠르게 처리하면서도, 위험이 큰 명령 실행에는 한 겹의 안전망이 남습니다.

Q3. `--dangerously-skip-permissions` 는 언제 써도 되나요? → A3. 인터넷에 연결된 평소 작업 환경에서는 쓰지 않기를 권합니다. 이 옵션은 모든 권한 질문과 안전 점검을 끄므로, 파괴적 명령도 묻지 않고 실행됩니다. 꼭 필요하다면 외부와 격리되고 백업이 보장된 환경에서만 제한적으로 사용합니다.

Q4. 계획 모드(plan)에서 파일이 실제로 바뀌나요? → A4. 바뀌지 않습니다. 계획 모드는 "이렇게 진행하겠다"는 안만 보여 주고, 파일 편집이나 명령 실행은 하지 않습니다. 계획을 검토하고 동의해야 비로소 실제 작업 단계로 넘어갑니다. 큰 작업 전에 방향을 맞추는 용도로 가장 안전합니다.

장 요약

이번 장에서는 안전과 편함의 균형을 내 손으로 맞추는 법을 배웠습니다. 권한은 하나의 스위치가 아니라 **읽기·편집·명령 실행 등 도구별로** 따로 통제되며, 모든 도구 요청은 **거부 → 허용 → 질문 → 승인 모드 기본** 순서로 판정되고 거부가 항상 이깁니다. 큰 틀을 정하는 **승인 모드**는 매번 묻기·편집 자동수락·계획 모드·전체 우회 네 가지이며, 아래로 갈수록 빠르지만 안전망이 줄어듭니다. 예외를 꼭 짚는 **권한 규칙**은 `settings.json` 의 `allow·ask·deny` 목록에 **도구이름(패턴)** 형태로 작성합니다. 되돌리기 어려운 **파괴적 작업**과 비밀이 담긴 **보호 경로**는 `deny` 로 잠가 마지막 안전망을 만들고, 실습은 격리된 **샌드박스 폴더**에서 **Git 커밋**으로 되돌릴 시점을 확보한 채 진행합니다.

핵심 키워드

권한 모델(permission model), 도구별 통제(per-tool control), 승인 모드(approval mode), 계획 모드(plan mode), 권한 규칙(permission rule), 규칙 패턴(rule pattern), 파괴적 작업(destructive action), 보호 경로(protected path), 샌드박스 폴더(sandbox folder), 버전 관리 안전망(git safety net)

다음 장 미리보기

다음 장에서는 Claude Code에게 **프로젝트의 맥락을 미리 알려 주는 법**을 다룹니다. `CLAUDE.md` 같은 메모리 파일에 코딩 규칙과 프로젝트 정보를 적어 두면, 매번 설명하지 않아도 도구가 그 맥락을 바탕으로 더 똑똑하게 일하게 됩니다.

CLAUDE.md: 프로젝트·개인·계층 메모리

학습 목표 - CLAUDE.md가 반복 설명을 줄여 주는 지속 컨텍스트(persistent context)임을 설명할 수 있습니다. - 전역(개인)·프로젝트·하위 폴더 메모리의 계층과 우선순위를 설명할 수 있습니다. - 구조·규칙·금지·임포트(@)를 갖춘 효과적인 CLAUDE.md를 작성할 수 있습니다. - 개인 메모리와 공유 메모리를 분리하고 `/memory` 로 무엇이 로드됐는지 점검할 수 있습니다.

앞 장들에서 우리는 Claude Code에게 명확한 프롬프트(prompt)로 일을 시키고, 권한을 다루는 법을 익혔습니다. 그런데 한 가지 답답한 일이 남습니다. 세션을 새로 열 때마다 "우리 프로젝트는 들여쓰기 네 칸이야", "커밋 메시지는 이런 형식으로 써", "이 폴더는 건드리지 마" 같은 **같은 설명을 매번 되풀이**해야 한다는 점입니다.

이 장은 그 반복을 없애 주는 장치, **CLAUDE.md(클로드 점 엠디)** 를 다룹니다. CLAUDE.md는 프로젝트 폴더에 두는 한 장의 안내문으로, Claude Code가 세션을 시작할 때 **자동으로 읽어 들이는** 파일입니다. 한 번 잘 적어 두면 다음부터는 설명 없이도 우리 규칙을 지킨 채 일합니다. 이 장은 기초 단계인 만큼, 무엇을 적는지보다 **왜 메모리가 필요하고 여러 메모리가 어떻게 합쳐지는지**, 그 원리를 또렷이 잡는 데 집중합니다.

이 책은 Windows 11의 **PowerShell(파워셸)** 을 기준으로 합니다. 화면 맨 앞의 `PS C:\Users\사용자\my-project>` 표시는 "지금 `my-project` 폴더에서 명령을 기다리는 중"이라는 안내입니다. 개인 메모리가 놓이는 `~` (물결표)는 Windows에서 내 사용자 폴더, 즉 `C:\Users\사용자` 를 가리킵니다.

메모리(memory)란 - 반복 설명을 없애는 지속 컨텍스트

도입

새 직원이 올 때마다 "우리 회사는 출근하면 이것부터 합니다"를 말로 매번 설명한다면 무척 번거롭습니다. 그래서 회사는 **업무 매뉴얼**을 한 번 잘 써 두고 신입에게 건넵니다. CLAUDE.md가 바로 그 매뉴얼입니다. 이 절에서는 Claude Code의 '메모리'가 무엇이고, 왜 그것이 대화로 말한 규칙보다 든든한지를 배웁니다.

핵심 개념 설명

프로젝트 메모리 (project memory)

프로젝트 메모리(project memory)란, 프로젝트 폴더에 둔 `CLAUDE.md` 파일에 적어 두면 **세션을 시작할 때마다 자동으로 로드되어** 반복 설명 없이 규칙과 맥락을 적용하게 해 주는 지속 컨텍스트입니다. '메모리'라는 말이 거창하게 들리지만, 실제로는 사람이 읽고 쓸 수 있는 **평범한 텍스트 파일 한 장**입니다.

여기서 입문자가 가장 자주 하는 오해 하나를 짚겠습니다. **"어제 대화로 말해 둔 규칙을 오늘도 기억하겠지"** 라는 기대입니다. 그렇지 않습니다. 앞 장에서 본 것처럼 대화 맥락은 세션이 끝나거나 `/clear` 로 비우면 사라집니다. 다음에 다시 열면 도구는 그 약속을 모릅니다. 반면 `CLAUDE.md`에 적은 내용은 파일로 남아 **매번 다시 읽히므로** 세션이 바뀌어도 유지됩니다. 이 차이가 메모리의 핵심입니다.

지속 컨텍스트 (persistent context)

지속 컨텍스트(persistent context)란, 한 번의 대화가 끝나도 사라지지 않고 **여러 세션에 걸쳐 계속 적용되는 맥락**을 말합니다. '컨텍스트(context)'는 "도구가 참고하는 배경 정보", '지속(persistent)'은 "사라지지 않고 이어진다"는 뜻입니다. 대화 맥락이 '휘발성 메모'라면, `CLAUDE.md`는 '책상 위에 붙여 둔 고정 메모'에 가깝습니다.

`CLAUDE.md`가 자동으로 로드되는 원리는 단순합니다. Claude Code는 세션을 시작할 때 **현재 작업 폴더에서 위로 거슬러 올라가며** `CLAUDE.md` 파일을 찾아, 발견하면 대화 맨 앞에 그 내용을 깔아 둡니다. 그래서 우리가 첫 마디를 꺼내기도 전에 도구는 이미 "이 프로젝트의 규칙은 이런 구나"를 알고 시작합니다.

- **왜(why) 쓰나:** 같은 설명을 매 세션 되풀이하는 낭비를 없애고, 누가 언제 열어도 **일관된 규칙**으로 일하게 하기 위해서입니다. 사람의 기억이나 그날의 기분에 의존하지 않습니다.
- **언제(when) 쓰나:** 프로젝트마다 지켜야 할 코딩 규칙, 자주 쓰는 명령, 건드리면 안 되는 영역, 도메인 용어처럼 **반복적으로 알려 줘야 하는 정보**가 생길 때입니다.
- **어떻게(how) 만드나:** 프로젝트 루트(가장 바깥 폴더)에 `CLAUDE.md` 라는 이름의 텍스트 파일을 만들어 규칙을 적습니다. 세션 안에서 `/init` 명령을 쓰면 Claude Code가 프로젝트를 훑어 초안을 만들어 주기도 합니다.

이렇게 만듭니다

다음은 실습 프로젝트 루트에 두는 가장 단순한 `CLAUDE.md` 의 예시입니다(실제로 파일에 적는 내용입니다). 마크다운(markdown)이라는 가벼운 문서 형식으로 적으며, `#` 한 개로 만든 줄이 가장 큰 제목입니다.

프로젝트 안내 (CLAUDE.md)**## 개요**

이 프로젝트는 사내 일정 관리용 작은 웹 앱입니다.

코딩 규칙

- 들여쓰기는 공백 4칸을 씁니다.
- 함수와 변수 이름은 영어 snake_case로 짓습니다.
- 새 기능에는 반드시 테스트를 함께 추가합니다.

검증 명령

- 테스트: ``npm test``
- 형식 검사: ``npm run lint``

이 파일을 둔 폴더에서 Claude Code를 실행하면, 예상 화면은 다음과 같습니다. 시작과 동시에 메모리를 읽어 들였다는 안내가 보입니다.

```
PS C:\Users\사용자\my-project> claude
```

프로젝트 메모리를 불러왔습니다: CLAUDE.md
무엇을 도와드릴까요?

> 로그인 함수 하나 만들어 줘

(이미 CLAUDE.md를 읽었으므로, 들여쓰기 4칸·snake_case·테스트 동반 규칙을 따로 말하지 않아도 그대로 지켜 코드를 제안합니다.)

이 CLAUDE.md에 담은 것

항목	적은 내용	도구가 얻는 것
개요	프로젝트가 무슨 일을 하는지	작업의 큰 맥락을 잡습니다
코딩 규칙	들여쓰기·이름 규칙·테스트	매번 말하지 않아도 규칙대로 코딩합니다
검증 명령	테스트·형식 검사 명령	일을 끝낸 뒤 스스로 점검할 방법을 압니다

팁: CLAUDE.md는 짧고 또렷할수록 좋습니다. 매 세션 함께 읽히므로 분량이 길면 그만큼 공간(컨텍스트)을 차지합니다. "꼭 매번 알려 줘야 하는 것"만 추리고, 한 번 쓰는 배경 설명은 대화로 그때그때 주는 편이 낫습니다.

흔한 실수: 규칙을 대화창에만 말해 두고 CLAUDE.md에 적지 않기. 그 자리에서는 잘 지켜지지만, 세션을 닫거나 `/clear`로 비우면 도구는 약속을 잊습니다. "다음에도 지켜야 하는 규칙"이라면 말로 끝내지 말고 CLAUDE.md에 옮겨 적습니다.

절 요약

- 프로젝트 메모리(CLAUDE.md)는 세션마다 자동 로드되어 반복 설명을 없애 줍니다.
- 대화 맥락은 휘발성이지만, CLAUDE.md는 파일로 남아 세션이 바뀌어도 유지됩니다.
- 꼭 매번 필요한 규칙·명령·맥락만 짧고 또렷하게 담습니다.

메모리 계층과 우선순위(전역·프로젝트·하위 폴더)

도입

우리가 지키는 규칙은 한 겹이 아닙니다. 나라의 법이 있고, 그 위에 우리 동네의 조례가 있고, 또 내가 다니는 회사의 사규가 있습니다. 이들은 충돌하지 않는 한 **함께** 적용되고, 부딪칠 때는 더 가까운(구체적인) 규칙이 우선합니다. Claude Code의 메모리도 똑같이 여러 겹으로 쌓입니다. 이 절에서는 그 계층 구조와, 규칙이 충돌할 때 무엇이 이기는지를 배웁니다.

핵심 개념 설명

메모리 계층 (memory hierarchy)

메모리 계층(memory hierarchy)이란, 서로 다른 위치에 놓인 여러 CLAUDE.md가 **동시에 로드**되어 하나의 유효 컨텍스트로 **합쳐지는** 구조를 말합니다. Claude Code는 보통 다음 세 곳(필요하면 네 곳)을 함께 읽습니다.

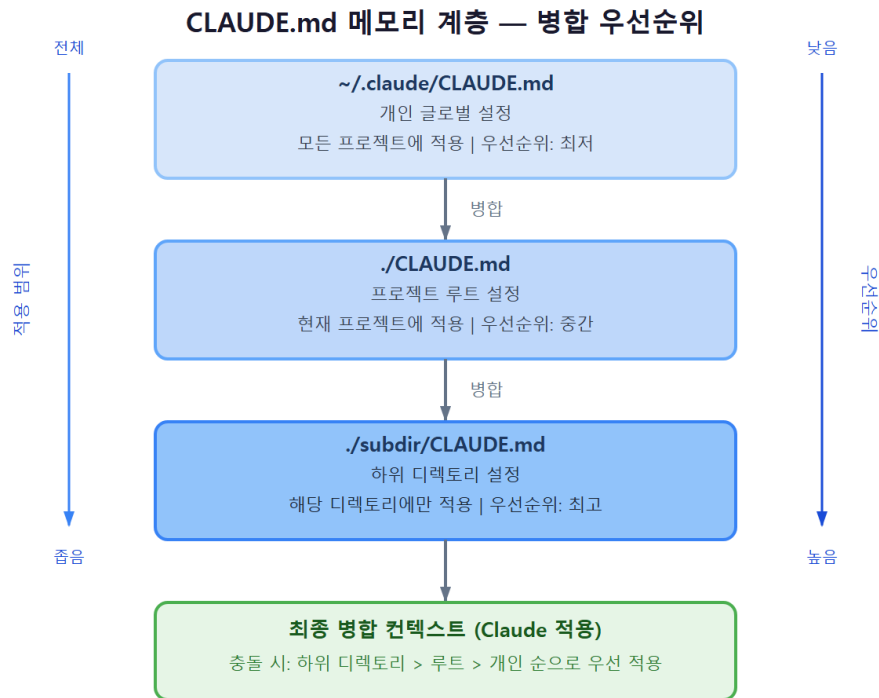
- **개인(전역) 메모리:** `C:\Users\사용자\.claude\CLAUDE.md`. 내가 쓰는 **모든 프로젝트에 공통**으로 적용되는 나만의 메모리입니다. `~/.claude/CLAUDE.md` 라고도 적는데, 여기서 `~`는 Windows의 내 사용자 폴더 `C:\Users\사용자`를 가리킵니다.
- **프로젝트 메모리:** 프로젝트 루트의 `C:\Users\사용자\my-project\CLAUDE.md`. **그 프로젝트에서 일할 때만** 적용되며, 보통 저장소에 함께 담아 팀과 공유합니다.
- **하위 폴더 메모리:** 프로젝트 안의 특정 폴더(예: `my-project\frontend\CLAUDE.md`)에 둔 메모리. **그 폴더의 파일을 다룰 때** 추가로 읽혀, 그 영역에만 필요한 규칙을 담습니다.

여기에 더해, 회사가 배포하는 **조직(관리형) 메모리**가 가장 바깥에 있을 수 있습니다. 이는 관리자가 정한 조직 차원의 안내로, 개인이 함부로 지울 수 없습니다(자세한 거버넌스는 10장에서 다룹니다).

우선순위 병합 (priority merge)

우선순위 병합(priority merge)이란, 여러 메모리가 합쳐질 때 **충돌하는 규칙이 있으면 더 구체적인(작업에 가까운) 메모리를 우선**으로 따르는 방식을 말합니다. 핵심은 두 가지입니다. 첫째, 충돌하지 않는 내용은 **모두 더해져** 함께 적용됩니다. 둘째, 같은 주제에서 지시가 엇갈리면 **더 가까운 쪽**(하위 폴더 > 프로젝트 > 개인)이 이깁니다.

예를 들어 개인 메모리에 "들여쓰기는 2칸"이라 적혀 있어도, 지금 일하는 프로젝트의 CLAUDE.md가 "들여쓰기는 4칸"이라 하면 **그 프로젝트에서는 4칸**이 적용됩니다. 프로젝트 규칙이 더 구체적이기 때문입니다.



기초 단계에서 흔한 오해 둘을 짚겠습니다. 하나는 **"메모리 파일은 한 개만 적용된다"**는 생각입니다. 실제로는 여러 파일이 함께 읽혀 하나로 합쳐집니다. 다른 하나는 **"전역(개인) 설정이 늘 프로젝트보다 세다"**는 오해입니다. 오히려 반대로, 충돌 시에는 더 가까운 프로젝트·하위 폴더 쪽이 우선합니다.

- **왜(why) 계층이 있나:** 모든 규칙을 한 파일에 욱여넣으면, 내 개인 취향과 팀 표준과 특정 폴더 규칙이 뒤섞입니다. 계층으로 나누면 **각자 자기 자리에서** 관리되어 깔끔하고, 공유할 것과 개인적인 것을 분리할 수 있습니다.
- **언제(when) 하위 폴더 메모리를 쓰나:** 한 프로젝트 안에서 영역마다 규칙이 다를 때입니다. 예를 들어 `frontend` 폴더는 화면 규칙, `backend` 폴더는 데이터 규칙이 다를 수 있습니다.

- **어떻게(how) 확인하나:** 어떤 메모리가 실제로 합쳐졌는지는 `/memory` 명령으로 볼 수 있습니다(5절에서 다룹니다).

이렇게 합쳐집니다

다음은 세 위치에 놓인 메모리가 어떻게 한 데 모이는지를 보여 주는 예상 구성입니다. 실제 파일들의 내용 일부를 나란히 둔 것입니다.

C:\Users\사용자\.claude\CLAUDE.md (개인 - 모든 프로젝트 공통)

- 답변은 한국어로 해 줘.
- 들여쓰기는 2칸을 선호해.

C:\Users\사용자\my-project\CLAUDE.md (프로젝트 - 팀 공유)

- 이 프로젝트는 들여쓰기 4칸을 씁니다.
- 커밋 메시지는 Conventional Commits 형식을 따릅니다.

C:\Users\사용자\my-project\frontend\CLAUDE.md (하위 폴더)

- 이 폴더의 컴포넌트 파일 이름은 PascalCase로 짓습니다.

frontend 폴더의 파일을 다룰 때 실제로 적용되는 **유효 컨텍스트**는 다음과 같이 정리됩니다.

[frontend 폴더에서 일할 때 합쳐진 규칙]

- 답변은 한국어로 (개인 메모리에서 - 충돌 없음, 그대로 적용)
- 들여쓰기는 4칸 (개인 2칸과 충돌 -> 더 가까운 프로젝트 4칸 우선)
- 커밋 메시지는 Conventional (프로젝트에서 - 충돌 없음, 그대로 적용)
- 컴포넌트 이름은 PascalCase (하위 폴더에서 - 이 폴더에서만 추가 적용)

세 계층 메모리 비교

계층	위치(Windows)	적용 범위	충돌 시 우선순위
개인(전역)	C:\Users\사용자\.claude\CLAUDE.md	내 모든 프로젝트	가장 약함(넓고 일반적)
프로젝트	C:\Users\사용자\my-project\CLAUDE.md	그 프로젝트 전체	개인보다 우선
하위 폴더	...\my-project\frontend\CLAUDE.md	그 폴더 영역	가장 강함(가장 구체적)

팁: 외우기 쉬운 한 문장은 "**좁을수록 세다**"입니다. 작업에 더 가까운(범위가 좁은) 메모리일수록 충돌 시 우선합니다. 그래서 나만의 일반 취향은 개인 메모리에, 팀 공통 규칙은 프로젝트 메모리에, 특정 영역 예외는 하위 폴더 메모리에 두면 자연스럽게 의도대로 합쳐집니다.

흔한 실수: 프로젝트에서 의도대로 동작하지 않는데 원인을 못 찾는 경우, 사실은 **개인 메모리에 적어 둔 오래된 취향**이 끼어든 것일 때가 많습니다. "분명 프로젝트엔 안 적었는데 왜 이러지?" 싶으면 개인 메모리(`~/claude/CLAUDE.md`)에 충돌하는 줄이 있는지 함께 살핍니다.

절 요약

- 여러 위치의 CLAUDE.md가 동시에 로드되어 하나의 유효 컨텍스트로 합쳐집니다.
- 충돌하지 않는 규칙은 모두 더해지고, 충돌하면 더 가까운(구체적) 메모리가 우선합니다.
- 개인·프로젝트·하위 폴더로 나눠 두면 공유할 것과 개인적인 것을 깔끔히 분리할 수 있습니다.

잘 쓰는 CLAUDE.md - 구조·규칙·금지·임포트

도입

같은 매뉴얼이라도 잘 정리된 것은 한눈에 들어오고, 두서없는 것은 읽다가 지칩니다. CLAUDE.md도 마찬가지입니다. Claude Code가 빠르게 핵심을 잡으려면 **구조**가 필요합니다. 이 절에서는 효과적인 CLAUDE.md에 무엇을 어떤 순서로 담는지, 그리고 다른 문서를 불러오는 임포트(@) 기능을 배웁니다.

핵심 개념 설명

프로젝트 규칙 (project rule)

프로젝트 규칙(project rule)이란, 그 프로젝트에서 일관되게 지켜야 할 약속을 CLAUDE.md에 명문으로 적은 것입니다. 좋은 CLAUDE.md는 보통 네 갈래를 담습니다. 첫째 **소개**(이 프로젝트가 무엇을 하는지), 둘째 **규칙**(어떻게 해야 하는지), 셋째 **금지**(무엇을 하면 안 되는지), 넷째 **검증 명령**(끝났는지 어떻게 확인하는지)입니다.

특히 **금지**를 또렷하게 적는 것이 중요합니다. 사람은 "당연히 안 그러겠지" 싶은 것도 도구에게는 명시해 줘야 안전합니다. "`secrets` 폴더와 `.env` 파일은 절대 수정하지 마", "내 허락 없이 외부로 푸시하지 마" 같은 한 줄이 사고를 크게 줄여 줍니다.

메모리 임포트 (memory import)

메모리 임포트(memory import)란, CLAUDE.md 안에서 @경로 표기로 다른 문서를 불러와 메모리에 포함하는 기능입니다. 문서에 다른 문서를 링크처럼 끼워 한꺼번에 참고하게 하는 것과 같습니다. 예를 들어 코딩 규칙이 길면 별도 파일로 빼 두고, CLAUDE.md에서는 @docs/coding-style.md 한 줄로 불러올 수 있습니다.

임포트는 불러온 문서가 또 다른 문서를 부르는 식으로 여러 단계까지 이어질 수 있지만(보통 다섯 단계까지), 무한정 부풀리지 않도록 주의합니다. 불러온 내용도 결국 매 세션 함께 읽혀 **컨텍스트 공간을 차지하기** 때문입니다.

- **왜(why) 구조를 두나:** 소개·규칙·금지·검증으로 나누면 도구가 "지금 보는 줄이 규칙인지 금지인지"를 빠르게 구분합니다. 뒤섞인 문단보다 표제가 붙은 목록이 훨씬 잘 지켜집니다.
- **언제(when) 임포트를 쓰나:** 규칙이 길어졌거나, 여러 프로젝트에서 같은 표준 문서를 공통으로 쓰고 싶을 때입니다.
- **어떻게(how) 적나:** 짧은 명령형 문장과 글머리표를 쓰고, 긴 배경 설명 대신 핵심만 남깁니다. 공통 문서는 @경로 로 분리합니다.

이렇게 작성합니다

다음은 구조·규칙·금지·검증·임포트를 모두 갖춘 실전형 CLAUDE.md 예시입니다.

```
# 일정 관리 앱 (CLAUDE.md)

## 개요
사내 일정 관리용 웹 앱입니다. 프론트는 React, 백엔드는 Node.js를 씁니다.

## 코딩 규칙
- 들여쓰기는 공백 4칸, 변수는 snake_case로 짓습니다.
- 새 기능에는 테스트를 함께 작성합니다.
- 자세한 스타일은 아래 문서를 따릅니다. @docs/coding-style.md

## 금지 (하지 말 것)
- `secrets/` 폴더와 `.env` 파일은 절대 읽거나 수정하지 않습니다.
- 내 명시적 허락 없이 원격 저장소로 push 하지 않습니다.

## 검증 명령
- 테스트: `npm test`
- 형식 검사: `npm run lint`
```

이렇게 적어 두면, 작업을 마칠 때 도구가 검증 명령까지 챙기는 흐름을 예상 화면에서 볼 수 있습니다.

> 일정 삭제 기능 추가해 줘

[Edit] scheduler.js 에 삭제 함수를 추가하겠습니다 (들여쓰기 4칸, 테스트 동반)

[Edit] scheduler.test.js 에 테스트를 추가하겠습니다

[Bash] npm test 를 실행해 확인할까요?

(CLAUDE.md의 '검증 명령'을 알고 있어, 끝낸 뒤 스스로 테스트 실행을 제안합니다.)

CLAUDE.md를 이루는 네 갈래 + импорт

갈래	무엇을 적나	효과
개요	프로젝트의 목적·기술 스택	작업의 큰 그림을 잡습니다
규칙	지켜야 할 약속(형식·절차)	일관된 결과를 냅니다
금지	하면 안 되는 일	위험·사고를 예방합니다
검증 명령	끝났는지 확인하는 방법	도구가 스스로 점검합니다
@경로 импорт	외부 문서 불러오기	긴 규칙을 분리·재사용합니다

베스트 프랙티스: 규칙은 "하라"보다 **"이렇게 하라 / 이걸 하지 마라"**의 짝으로 적으면 잘 지켜 집니다. 예: "커밋 메시지를 잘 써라"(모호함)보다 "커밋 메시지는 `feat:`, `fix:` 같은 접두어로 시작하라"(구체적)가 훨씬 효과적입니다.

흔한 실수: CLAUDE.md를 회사 위키처럼 길게 쓰기. 배경 설명, 회의록, 장황한 문단을 다 넣으면 매 세션 그만큼 공간을 잡아먹고 정작 핵심 규칙이 묻힙니다. 긴 자료는 별도 파일로 빼고 @경로로 필요할 때만 불러옵니다. CLAUDE.md 본문은 한 화면에 들어올 정도로 추립니다.

절 요약

- 효과적인 CLAUDE.md는 소개·규칙·금지·검증 명령의 네 갈래로 구조화합니다.
- 금지 항목을 또렷이 적는 것이 안전사고 예방에 특히 중요합니다.
- 긴 규칙은 별도 문서로 빼고 @경로 imports로 불러와 짧게 유지합니다.

개인 메모리와 팀 공유 메모리 분리

도입

같은 사무실을 쓰더라도, 게시판에 붙이는 공지와 내 책상 서랍에 둔 개인 메모는 성격이 다릅니다. 공지는 모두가 따라야 하고, 개인 메모는 나에게만 맞으면 됩니다. CLAUDE.md도 똑같이 **모두와 나눌 것과 나만 쓸 것을** 갈라 뉘야 합니다. 이 절에서는 그 경계를 어떻게 긋는지 배웁니다.

핵심 개념 설명

공유 메모리 (shared memory)

공유 메모리(shared memory)란, 저장소(repository, 코드와 파일을 함께 보관·공유하는 프로젝트 창고)에 함께 담아 **팀 전체가 똑같이 적용받는** 메모리를 말합니다. 프로젝트 루트의 CLAUDE.md가 대표적입니다. 이 파일을 저장소에 커밋(commit, 변경 내용을 저장소에 기록하는 일)해 두면, 동료가 그 프로젝트를 내려받는 순간 같은 규칙을 함께 갖게 됩니다.

여기에는 **팀이 합의한 표준만** 담습니다. 코딩 규칙, 금지 사항, 빌드·테스트 명령, 프로젝트 도메인 용어처럼 누가 일하든 동일하게 적용돼야 하는 것들입니다.

개인 메모리 (personal memory)

개인 메모리(personal memory)란, 저장소에 올리지 않고 **나에게만 적용되는** 메모리입니다. 두 자리가 있습니다. 하나는 내 모든 프로젝트에 공통으로 쓰는 전역 개인 메모리 C:\Users\사용자\.claude\CLAUDE.md이고, 다른 하나는 특정 프로젝트에서 나만 쓰고 싶은 규칙입니다.

여기에는 **개인 취향**을 담습니다. "답변은 한국어로", "설명은 짧게", "내가 자주 쓰는 별칭 명령" 같은 것들입니다. 이런 취향까지 공유 메모리에 넣으면 동료에게는 불필요한 강요가 되므로 분리해야 합니다.

- **왜(why) 분리하나:** 팀 표준과 개인 취향이 한 파일에 섞이면, 공유했을 때 동료가 내 취향까지 떠안게 되고, 거꾸로 내 취향을 빼려다 팀 규칙을 건드리는 사고가 납니다.
- **언제(when) 어디에 두나:** "누가 일하든 같아야 하나?"가 기준입니다. 그렇다면 프로젝트 CLAUDE.md (공유), 나에게만 맞으면 개인 메모리입니다.
- **어떻게(how) 가르나:** 공유 메모리는 저장소에 커밋하고, 개인적인 줄은 전역 개인 메모리로 옮기거나, 저장소가 추적하지 않는 파일(.gitignore에 등록한 로컬 파일)에 둡니다.

이렇게 나눕니다

다음은 같은 정보를 **공유**와 **개인**으로 갈라 둔 예시입니다. 어디에 무엇을 두는지가 핵심입니다.

```
# C:\Users\사용자\my-project\CLAUDE.md (저장소에 커밋 - 팀 공유)
## 코딩 규칙
- 들여쓰기 4칸, snake_case, 테스트 동반.
## 금지
- secrets/ 와 .env 는 수정 금지.
## 검증
- 테스트: `npm test`
```

```
# C:\Users\사용자\.claude\CLAUDE.md (커밋 안 함 - 나만의 취향)
- 답변과 코드 주석은 한국어로 해 줘.
- 설명은 핵심만 짧게.
- 코드를 고치기 전에 무엇을 바꿀지 한 줄로 먼저 알려 줘.
```

어떤 파일이 저장소에 올라가고 무엇이 빠지는지는 다음 화면처럼 확인할 수 있습니다(`git status` 로 추적 상태를 봅니다).

```
PS C:\Users\사용자\my-project> git status
```

추적되는 변경:

```
new file:   CLAUDE.md          <- 팀 공유 (커밋 대상)
```

추적 안 함 / 무시됨:

```
(개인 메모리는 C:\Users\사용자\.claude\ 안에 있어 이 저장소에 포함되지 않습니다)
```

무엇을 어디에 둘까

내용	두는 곳	저장소 공유	이유
코딩 규칙·금지·검증 명령	프로젝트 CLAUDE.md	함께 커밋	누가 일하든 같아야 함
프로젝트 도메인 용어	프로젝트 CLAUDE.md	함께 커밋	팀 공통 이해가 필요함
답변 언어·말투 취향	개인 ~/.claude/CLAUDE.md	공유 안 함	사람마다 다름
나만의 단축 습관	개인 메모리	공유 안 함	동료에게 불필요함

주의: 비밀번호·API 키·토큰 같은 **비밀값**을 **CLAUDE.md**에 **절대 적지 마십시오**. 공유 메모리는 저장소에 올라가 동료 모두가 보게 되고, 한 번 커밋되면 기록에 남아 지우기 어렵습니다. 비밀값은 메모리가 아니라 환경변수로 다루며, 이는 10장에서 자세히 배웁니다.

절 요약

- 공유 메모리(프로젝트 CLAUDE.md)에는 팀 표준만, 개인 메모리에는 내 취향만 담습니다.
- "누가 일하든 같아야 하나?"가 둘을 가르는 기준입니다.
- 비밀값은 어떤 메모리에도 적지 않습니다(환경변수로 분리).

메모리 디버깅 - 무엇이 로드됐는지 확인

도입

규칙을 분명히 적었는데도 도구가 엉뚱하게 동작할 때가 있습니다. 그럴 때 "왜 안 듣지?" 하고 막연히 답답해하기보다, **실제로 어떤 메모리가 로드됐는지** 눈으로 확인하는 편이 빠릅니다. 이 절에서는 `/memory` 명령으로 유효 컨텍스트를 점검하고, 기대대로 적용되지 않을 때의 점검 순서를 배웁니다.

핵심 개념 설명

메모리 점검 (`/memory`)

메모리 점검(`/memory`)이란, 세션 안에서 `/memory` 슬래시 명령을 실행해 **지금 어떤 CLAUDE.md 파일들이 로드됐고 어디에 있는지** 확인하고 편집할 수 있는 기능입니다. 슬래시 명령은 앞 장에서 본, 대화창에서 `/`로 시작해 실행하는 명령입니다. `/memory`를 치면 개인·프로젝트·하위 폴더 메모리의 목록과 경로가 한눈에 보이고, 그 자리에서 파일을 열어 고칠 수 있습니다.

유효 컨텍스트 확인 (effective context)

유효 컨텍스트(effective context)란, 여러 메모리가 합쳐진 끝에 **실제로 지금 적용되고 있는 최종 규칙 묶음**을 말합니다. 2절에서 본 '병합 결과'가 바로 이것입니다. 우리가 적은 것과 실제 적용되는 것이 어긋날 때, 유효 컨텍스트를 확인하면 어디서 규칙이 끼어들었는지(또는 빠졌는지) 알 수 있습니다.

- **왜(why) 점검하나:** 메모리는 여러 파일이 합쳐지므로, 문제가 보이지 않는 곳(예: 잊고 있던 개인 메모리)에서 올 수 있습니다. 추측 대신 목록을 봐야 원인을 빨리 좁힙니다.
- **언제(when) 점검하나:** 규칙을 적었는데 안 지켜질 때, 반대로 적은 적 없는 규칙이 적용될 때, 또는 파일을 고쳤는데 반영이 안 된 듯한 때입니다.

- **어떻게(how) 점검하나:** 아래 순서대로 좁혀 갑니다.

이렇게 점검합니다

다음은 `/memory` 를 실행했을 때의 예상 화면입니다. 어떤 파일이 어디서 로드됐는지 목록으로 보여 줍니다.

```
> /memory
```

로드된 메모리 파일:

1. 개인(전역) C:\Users\사용자\.claude\CLAUDE.md
2. 프로젝트 C:\Users\사용자\my-project\CLAUDE.md
3. 하위 폴더 C:\Users\사용자\my-project\frontend\CLAUDE.md
4. 임포트 @docs/coding-style.md (프로젝트에서 불러옴)

편집할 파일 번호를 고르세요 (취소: Esc) >

기대대로 적용되지 않을 때는 다음 순서로 점검하면 대부분 원인이 잡힙니다.

[메모리가 기대대로 안 들을 때 점검 순서]

- 1) `/memory` 로 그 파일이 실제로 목록에 있는가? (파일명·위치가 CLAUDE.md 인지)
- 2) 충돌하는 다른 메모리는 없는가? (특히 잊고 있던 개인 메모리)
- 3) 규칙 문장이 모호하지 않은가? ("잘 해라"가 아니라 구체적 지시인가)
- 4) 방금 고친 파일이면 `/memory` 로 다시 읽혔는지 확인 (필요하면 세션 재시작)

점검 순서가 짚는 것

단계	확인 내용	자주 나오는 원인
1	파일이 로드됐는가	파일명 오타(CLAUDE.md 가 아님)·엉뚱한 폴더
2	충돌 메모리 여부	개인 메모리의 오래된 규칙이 끼어듦
3	문장이 구체적인가	모호한 규칙이라 도구가 해석을 달리함
4	변경이 반영됐는가	고친 내용이 아직 안 읽힘

팁: 규칙을 추가하고 싶을 때, 파일을 직접 열지 않고도 대화창에서 `#` 로 시작하는 줄을 입력하면 그 내용을 메모리에 빠르게 적어 넣을지 물어봅니다. 어느 메모리(개인/프로젝트)에 저장할지 고를 수 있어, 떠오른 규칙을 잊기 전에 즉시 담아 두기 좋습니다.

흔한 실수: 파일 이름을 `claude.md` 나 `claude.MD` 처럼 잘못 적어 두고 "왜 안 읽히지?" 하기. 메모리 파일 이름은 정확히 `CLAUDE.md` 여야 합니다. `/memory` 목록에 그 파일이 보이지 않는다면, 가

장 먼저 파일명과 위치(올바른 폴더에 있는지)부터 확인합니다.

절 요약

- `/memory` 로 지금 로드된 메모리 파일의 목록·경로를 확인하고 편집할 수 있습니다.
- 유효 컨텍스트는 여러 메모리가 합쳐진 최종 적용 규칙입니다.
- 안 들을 때는 로드 여부 -> 충돌 -> 문장 모호성 -> 반영 여부 순으로 좁혀 봅니다.

실습 과제

해보기: 내 프로젝트 CLAUDE.md 작성하고 계층 실험하기

앞 장에서 만든 실습 폴더(`claude-practice`)를 프로젝트 삼아, 이 장에서 배운 메모리 계층을 직접 만들어 보고 `/memory` 로 확인합니다.

- **단계 1 (프로젝트 메모리):** 실습 폴더 루트에 `CLAUDE.md` 를 만들고 **소개·코딩 규칙·금지·검증 명령**을 각각 한두 줄씩 적습니다. 예: 규칙 "들여쓰기 4칸", 금지 "secrets/ 수정 금지".
- **단계 2 (개인 메모리):** `C:\Users\사용자\.claude\CLAUDE.md` 에 나만의 취향을 한 줄 적습니다. 예: "들여쓰기는 2칸 선호". (프로젝트 규칙과 **일부러 충돌**시켜 봅니다.)
- **단계 3 (하위 폴더 메모리):** 실습 폴더 안에 `sub` 폴더를 만들고 그 안에 `CLAUDE.md` 를 뒤 그 폴더에만 적용할 규칙을 한 줄 적습니다.
- **단계 4 (점검):** 세션을 열고 `/memory` 를 실행해 세 파일이 모두 목록에 뜨는지 확인합니다. 그다음 들여쓰기가 2칸과 4칸 중 무엇으로 적용되는지 도구에게 물어, **더 가까운 프로젝트 규칙(4칸)이 이기는지** 직접 확인합니다.
- **점검표:** 아래 표를 채워 보고, 충돌이 어떻게 해결됐는지 한 줄로 설명할 수 있으면 성공입니다.

메모리	적은 규칙	<code>/memory</code> 목록에 보였나	충돌 시 적용된 값
개인(전역)	들여쓰기 2칸	(예) 예	(예) 프로젝트에 밀림
프로젝트	들여쓰기 4칸	...	...
하위 폴더	...	...	...

FAQ

Q1. CLAUDE.md는 어떤 형식으로 적어야 하나요? 정해진 문법이 있나요? → A1. 특별한 문법은 없습니다. 사람이 읽는 **마크다운 텍스트**면 됩니다. 다만 #로 만든 제목으로 소개·규칙·금지·검증을 구분하고, 긴 문단보다 짧은 글머리표(-)로 적으면 도구가 핵심을 더 잘 잡습니다. 파일 이름만 정확히 CLAUDE.md로 맞추면 됩니다.

Q2. 개인 메모리와 프로젝트 메모리가 같은 규칙을 다르게 말하면 무엇이 적용되나요? → A2. 더 **가까운(구체적인)** 쪽이 우선합니다. 작업 중인 프로젝트의 CLAUDE.md가 개인 메모리를 이기고, 그 안의 하위 폴더 메모리는 프로젝트 메모리도 이깁니다. "좁을수록 세다"로 기억하면 쉽습니다. 충돌하지 않는 규칙은 모두 함께 적용됩니다.

Q3. 메모리를 고쳤는데 도구가 옛 규칙대로 행동합니다. 왜 그런가요? → A3. 세 가지를 확인합니다. 첫째 /memory로 그 파일이 실제 로드됐는지, 둘째 다른 메모리(특히 잊고 있던 개인 메모리)에 충돌 규칙이 없는지, 셋째 방금 고친 내용이 반영됐는지입니다. 반영이 안 된 듯하면 세션을 닫았다가 다시 열면 메모리를 새로 읽어 들입니다.

Q4. CLAUDE.md에 회사 보안 토크나 비밀번호를 적어 두면 편하지 않나요? → A4. 절대 안 됩니다. 프로젝트 CLAUDE.md는 저장소에 올라가 팀 전체가 보게 되고, 한 번 커밋되면 기록에 남아 지우기 어렵습니다. 비밀값은 메모리가 아니라 환경변수로 다뤄야 하며, 그 방법은 10장에서 배웁니다.

장 요약

이번 장에서는 같은 설명을 매번 되풀이하지 않게 해 주는 **메모리**, 곧 CLAUDE.md를 배웠습니다. 대화 맥락은 세션이 끝나면 사라지지만, CLAUDE.md는 파일로 남아 **세션마다 자동 로드되는 지속 컨텍스트**가 됩니다. 메모리는 한 겹이 아니라 **개인(전역)·프로젝트·하위 폴더**의 여러 겹으로 쌓여 하나의 유효 컨텍스트로 합쳐지며, 충돌하면 작업에 더 가까운(구체적인) 메모리가 우선합니다("좁을수록 세다"). 잘 쓰는 CLAUDE.md는 **소개·규칙·금지·검증 명령**으로 구조화하고, 긴 자료는 @경로 임포트로 분리해 짧게 유지합니다. 팀 표준은 저장소에 커밋하는 **공유 메모리**에, 내 취향은 **개인 메모리**에 두어 가르고, 비밀값은 어디에도 적지 않습니다. 마지막으로 기대대로 동작하지 않을 때는 /memory로 무엇이 로드됐는지 확인해 원인을 좁힙니다.

핵심 키워드

프로젝트 메모리(project memory), 지속 컨텍스트(persistent context), 메모리 계층(memory hierarchy), 우선순위 병합(priority merge), 프로젝트 규칙(project rule), 메모리 импорт(memory import), 개인 메모리(personal memory), 공유 메모리(shared memory), 메모리 점검(/memory)

다음 장 미리보기

다음 장에서는 자주 쓰는 작업을 한마디로 부르는 **슬래시 커맨드(slash command)** 를 다룹니다. `/help`·`/clear` 같은 내장 명령을 둘러보고, 나만의 반복 작업을 `.claude/commands` 폴더에 **커스텀 명령**으로 만들어 더 빠르게 일하는 법을 배웁니다.

슬래시 커맨드: 내장과 커스텀

학습 목표 - 자주 쓰는 내장 슬래시 커맨드(slash command)를 상황에 맞게 사용할 수 있습니다.
 - 프론트매터(frontmatter)를 갖춘 커스텀 명령을 `.claude/commands`에 만들 수 있습니다.
 - 인자 (\$ARGUMENTS)·파일참조(@)·bash(!)를 명령에 넣어 동적으로 동작시킬 수 있습니다.
 - 명령의 도구 권한·모델을 지정하고 플러그인(plugin)으로 팀에 배포할 수 있습니다.

앞 장까지 우리는 대화로 일을 시키고, 파일을 고치고, 권한을 다루는 기본 흐름을 익혔습니다. 그러다 보면 **똑같은 부탁을 매번 길게 되풀이하는 순간**이 옵니다. "변경된 내용을 요약해서 커밋 메시지를 만들어 줘"를 하루에도 몇 번씩 타이핑하게 되는 식입니다. 이 장은 그렇게 반복되는 요청을 **한마디 명령으로 묶는 법**을 다룹니다.

이 책은 Windows 11의 **PowerShell(파워셸)**을 기준으로 합니다. PowerShell은 글자로 명령을 입력해 컴퓨터에 일을 시키는 검은 화면(터미널)이며, 화면 맨 앞의 `PS C:\Users\사용자\my-project>` 표시는 "지금 `my-project` 폴더에서 명령을 기다리는 중"이라는 안내입니다. 이 장에서 만드는 명령 파일은 메모장이나 코드 편집기로 작성하는 **평범한 텍스트 파일**이므로, 프로그래밍을 해 본 적이 없어도 따라 할 수 있습니다.

슬래시 커맨드(slash command)와 내장 명령 둘러보기

도입

음식점 메뉴판에는 "오늘의 정식"처럼 자주 나가는 주문을 한 단어로 부를 수 있는 항목이 있습니다. 길게 설명하지 않아도 점원이 바로 알아듣습니다. **슬래시 커맨드**가 바로 그 단축 주문입니다. 이 절에서는 Claude Code에 기본으로 들어 있는 명령들을 둘러보고, 그것들이 일반 대화와 어떻게 다른지 살펴봅니다.

핵심 개념 설명

슬래시 커맨드 (slash command)

슬래시 커맨드(slash command)란, 대화창에서 빗금 기호 `/`로 시작해 실행하는 명령을 말합니다. `/`는 키보드에서 물음표(?)와 같은 자리에 있는 빗금 글자입니다. `/help`, `/clear`처럼

Claude Code에 기본으로 들어 있는 **내장 명령(built-in command)** 과, 사용자가 직접 만든 커스텀 명령(다음 절에서 다룹니다)이 있습니다.

여기서 가장 중요한 차이를 짚겠습니다. 일반 프롬프트(prompt)는 "이렇게 해 줘"라고 **부탁**하면 Claude가 알아서 해석합니다. 반면 슬래시 커맨드는 **정해진 동작을 곧바로 실행**하는 명령입니다. `/clear` 는 해석의 여지 없이 대화를 비우고, `/model` 은 모델을 바꾸는 화면을 엽니다. 즉 슬래시 커맨드는 "말로 부탁하는 것"이 아니라 "버튼을 누르는 것"에 가깝습니다.

흔한 오해는 "슬래시 커맨드도 그냥 프롬프트처럼 처리되겠지" 라는 생각입니다. 그렇지 않습니다. `/` 로 시작하면 Claude Code는 그 줄을 **명령으로 먼저 알아보고** 정해진 기능을 실행합니다. 그래서 같은 말을 길게 풀어 부탁하는 것보다 훨씬 빠르고 결과가 예측 가능합니다.

- **왜(why) 쓰나:** 자주 하는 조작(대화 비우기, 모델 바꾸기, 메모리 편집)을 매번 문장으로 설명할 필요 없이 한 단어로 끝냅니다.
- **언제(when) 쓰나:** 대화 맥락을 정리하고 싶을 때, 비용·상태를 확인할 때, 작업 흐름을 전환할 때입니다.
- **어떻게(how) 쓰나:** 입력창에 `/` 만 치면 사용 가능한 명령 목록이 자동으로 뜹니다. 거기서 고르거나 이어서 이름을 타이핑합니다.

자주 쓰는 내장 명령

입력창에서 `/` 를 치면 다음과 비슷한 목록이 펼쳐집니다. 아래 화면은 실제 입력이 아니라 어떤 모습인지 보여 주기 위한 예시입니다.

```
PS C:\Users\사용자\my-project> claude
> /

/help      사용법과 명령 목록을 봅니다
/clear     현재 대화 내용을 비웁니다
/compact   긴 대화를 요약해 컨텍스트 공간을 회복합니다
/model     사용할 모델을 고릅니다
/memory    프로젝트 메모리(CLAUDE.md)를 엽니다
/cost      이번 세션에서 쓴 사용량을 봅니다
...
```

특히 입문 단계에서 손에 익혀 두면 좋은 명령을 추려 비교합니다.

명령	하는 일	언제 쓰나
<code>/help</code>	사용법과 명령 목록을 보여 줍니다	무엇이 있는지 잊었을 때
<code>/clear</code>	지금까지의 대화 맥락을 깨끗이 비웁니다	전혀 다른 새 작업으로 넘어갈 때
<code>/compact</code>	긴 대화를 요약해 공간을 회복합니다	대화가 길어져 느려지거나 맥락이 가득 찼을 때
<code>/model</code>	사용할 모델을 고르는 화면을 엽니다	더 빠른/더 똑똑한 모델로 바꾸고 싶을 때
<code>/memory</code>	프로젝트 규칙 파일(CLAUDE.md)을 엽니다	코딩 규칙·약속을 적어 두고 싶을 때
<code>/login</code>	계정에 로그인합니다(인증)	처음 쓰거나 계정을 바꿀 때
<code>/cost</code>	이번 세션 사용량을 보여 줍니다	비용·토큰 사용을 확인하고 싶을 때

`/clear`와 `/compact`는 헷갈리기 쉬우니 한 번 더 구분하겠습니다. `/clear`는 **대화를 통째로 비워** 백지에서 새로 시작합니다(이전 맥락이 사라집니다). `/compact`는 **핵심만 요약해 남기고** 자리만 회복하므로 흐름을 이어 갈 수 있습니다. "방금까지 하던 일과 이어지나, 아예 새 일인가"로 골라 쓰면 됩니다.

팁: 명령 이름이 잘 기억나지 않으면 `/`만 치고 몇 글자를 더 입력해 보십시오. 목록이 입력한 글자에 맞춰 좁혀집니다. 외울 필요 없이 **고르는 방식**으로 쓰면 됩니다.

흔한 실수: 새 작업을 시작하면서 `/clear`를 하지 않아, 이전 작업의 맥락이 남아 엉뚱한 파일을 건드리는 경우가 있습니다. 전혀 다른 일로 넘어갈 때는 `/clear`로 한번 비워 주면 오해가 줄어듭니다. 반대로 하던 일을 이어 가야 하는데 `/clear`를 누르면 맥락이 사라지니 주의합니다.

절 요약

- 슬래시 커맨드는 `/`로 시작하는 명령으로, 일반 프롬프트와 달리 정해진 동작을 곧바로 실행합니다.
- `/help`로 목록을 보고, `/clear`·`/compact`로 맥락을 관리하며, `/model`·`/cost`로 환경과 사용량을 다룹니다.

- `/clear` 는 대화를 비우고, `/compact` 는 요약해 이어 갑니다 — 새 일이면 `/clear`, 이어 갈 일이면 `/compact` 입니다.

커스텀 명령 만들기(.claude/commands)

도입

자주 쓰는 계약서나 보고서는 매번 처음부터 쓰지 않습니다. **양식(템플릿)을 저장해 두고 빈칸만 채워** 씁니다. 커스텀 명령이 바로 그 양식입니다. 반복해서 부탁하던 긴 프롬프트를 파일 하나로 저장해 두면, 다음부터는 `/이름` 한마디로 그 프롬프트를 그대로 불러올 수 있습니다.

핵심 개념 설명

커스텀 명령 (custom command)

커스텀 명령(custom command) 이란, `.claude/commands` 폴더에 둔 **마크다운 파일**로 정의하는 나만의 슬래시 커맨드입니다. 마크다운(markdown)은 `#` 이나 `-` 같은 기호로 제목과 목록을 표시하는 **간단한 글쓰기 형식**으로, 특별한 프로그램 없이 메모장으로도 작성할 수 있는 보통 텍스트입니다. 파일 이름이 곧 명령 이름이 됩니다. 예를 들어 `commit.md` 라는 파일을 만들면 `/commit` 명령이 생깁니다.

가장 흔한 오해부터 풀겠습니다. "**커스텀 명령을 만들려면 코드를 짤 줄 알아야 한다**" 는 생각입니다. 전혀 그렇지 않습니다. 명령 파일의 본문은 **그냥 Claude에게 건넬 부탁의 글**입니다. 평소 대화창에 길게 적던 요청을 파일에 옮겨 적는 것이 전부입니다.

프롬프트 템플릿 (prompt template)

프롬프트 템플릿(prompt template) 이란, 빈칸을 갖춘 미리 짜 둔 요청문을 말합니다. 커스텀 명령의 본문이 바로 이 템플릿입니다. `/commit` 이라고 입력하면 Claude Code가 `commit.md` 의 본문을 꺼내 **그 내용을 마치 내가 방금 타이핑한 프롬프트인 것처럼** 대화에 펼쳐 넣고 실행합니다.

 사용자가 터미널에 `/이름 [인수]`를 입력하면, Claude Code가 `.claude/commands/이름.md` 파일을 탐색하고, 파일 본문의 `$ARGUMENTS`를 실제 인수로 치환하여 완성된 프롬프트를 펼친 뒤 Claude에게 전달해 실행하는 4단계 화살표 흐름 다이어그램

명령을 두는 두 곳: 프로젝트 vs 개인

명령 파일은 두 위치 중 하나에 둘 수 있고, 위치에 따라 누가 쓸 수 있는지가 달라집니다.

두는 곳	경로(Windows 11)	적용 범위
프로젝트 명령	C:\Users\사용자\my-project\.claude\commands\	그 프로젝트에서만, 팀원과 공유됨
개인 명령	C:\Users\사용자\.claude\commands\	내 모든 프로젝트에서, 나만 사용

규칙은 단순합니다. **팀이 함께 써야 하는 명령**(예: 우리 팀 커밋 규칙)은 프로젝트 폴더에 두어 저장소에 함께 올리고, **나만의 개인 습관**(예: 내가 좋아하는 코드 설명 방식)은 개인 폴더에 둡니다.

이렇게 만듭니다

PowerShell에서 프로젝트에 명령 폴더를 만들고, 첫 명령 파일을 준비하는 과정입니다. 아래 명령은 실제로 입력하는 내용입니다.

```
# 프로젝트 안에 .claude\commands 폴더를 만듭니다 (-Force: 이미 있어도 오류 없이 진행)
New-Item -ItemType Directory -Force -Path .\.claude\commands
```

이제 그 폴더에 `summary.md` 파일을 만들고 다음 내용을 적습니다. 이 파일이 곧 `/summary` 명령이 됩니다. 아래는 파일에 적는 **내용 그 자체**이며, 실행 코드가 아니라 Claude에게 건넬 부탁의 글입니다.

```
---
description: 최근 변경 내용을 한국어로 짧게 요약합니다
---

지금까지의 작업에서 무엇이 바뀌었는지 핵심만 3줄 이내로 요약해 주세요.
- 사용자 입장에서 무엇이 달라지는지 위주로 적어 주세요.
- 전문 용어는 풀어서 설명해 주세요.
```

맨 위 `---` 로 감싼 부분이 **프론트매터(frontmatter)** 입니다. 프론트매터는 파일 본문 앞에 두는 **설정 머리말**로, "이 명령이 무엇을 하는지" 같은 정보를 적는 칸입니다. 여기서는 `description` (설명) 한 줄만 넣었습니다. 그 아래 일반 글이 실제 Claude에게 전달될 프롬프트 템플릿입니다.

이제 Claude Code 안에서 `/summary` 를 입력하면 다음처럼 동작합니다.

```
PS C:\Users\사용자\my-project> claude
> /summary
```

(summary.md 본문이 프롬프트로 펼쳐져 실행됩니다)
이번 작업 요약입니다.

- 로그인 화면에 오류 메시지를 한국어로 표시하도록 바꿨습니다.
- 비밀번호 입력칸에서 글자가 보이지 않도록 가렸습니다.
- 잘못된 입력일 때 버튼이 비활성화되도록 했습니다.

명령 파일 구성 요소

구성 요소	위치	의미
파일 이름 (summary.md)	파일명	.md 를 뺀 summary 가 명령 이름이 됩니다 (/summary)
--- 사이 영역	본문 맨 위	프론트매터 — 명령의 설정 머리말
description	프론트매터 안	/ 목록에 보이는 한 줄 설명
그 아래 본문	프론트매터 다음	Claude에게 전달되는 프롬프트 템플릿

팁: 명령 이름은 짧고 동사형으로 지으면 기억하기 쉽습니다(/summary , /review , /fix). 폴더 안에 하위 폴더를 만들면 /frontend:fix 처럼 묶음으로 정리할 수도 있습니다. 처음에는 가장 자주 반복하는 부탁 하나만 명령으로 옮겨 보십시오.

흔한 실수: 파일을 .txt 로 저장하거나 commands 철자를 틀리면 명령이 목록에 나타나지 않습니다. 확장자는 반드시 .md , 폴더 이름은 정확히 .claude\commands 여야 합니다. 명령이 안 보이면 /help 로 목록에 댄지부터 확인합니다.

절 요약

- 커스텀 명령은 .claude\commands 폴더의 .md 파일로 만들며, 코드가 아니라 부탁하는 글입니다.
- 파일 이름이 명령 이름이 되고(summary.md → /summary), 본문은 프롬프트 템플릿으로 펼쳐집니다.
- 팀과 공유할 명령은 프로젝트 폴더에, 개인용은 사용자 홈의 .claude\commands 에 둡니다.

인자·파일참조·bash 실행을 명령에 넣기

도입

앞 절의 양식은 빈칸 없이 통째로 정해진 글이었습니다. 하지만 실제 양식에는 "받는 사람", "날짜"처럼 **그때그때 채우는 빈칸**이 있습니다. 또 "지난달 보고서를 첨부해서"처럼 다른 자료를 끌어오기도 합니다. 이 절에서는 명령에 **빈칸(인자)** 을 두고, **파일이나 명령 실행 결과를 끼워 넣어** 명령을 살아 움직이게 만드는 법을 배웁니다.

핵심 개념 설명

명령 인자 (command argument)

명령 인자(command argument) 란, 명령을 부를 때 함께 넘기는 값입니다. 명령 파일 안에 `$ARGUMENTS` 라고 적어 두면, 내가 명령 뒤에 붙여 입력한 글자가 그 자리에 **그대로 갈아끼워집니다**. 예를 들어 `/fix` 로그인 버튼이 안 눌려요 라고 입력하면, 파일 속 `$ARGUMENTS` 자리에 "로그인 버튼이 안 눌려요"가 채워집니다.

값을 여러 개 따로 받고 싶을 때는 `$1`, `$2` 처럼 **번호가 붙은 자리표시자**를 씁니다. `$1` 은 첫 번째 값, `$2` 는 두 번째 값입니다. 흔한 오해가 "인자는 하나만 받을 수 있다" 는 생각인데, `$1` · `$2` 를 쓰면 여러 값을 자리마다 나눠 받을 수 있습니다.

동적 컨텍스트 (@파일·!명령)

명령 본문에는 두 가지 특별한 기호로 **실행 시점의 정보**를 끌어올 수 있습니다.

- **@파일경로** — 그 파일의 **내용을 읽어 와** 명령에 함께 넣습니다. 예: `@README.md` .
- **!`명령`** — 백틱(`, 키보드 숫자 1 왼쪽의 작은 따옴표 모양)으로 감싼 셸 명령을 **실제로 실행해 그 결과를** 명령에 끼워 넣습니다. 예: `!`git status`` .

이 둘을 **동적 컨텍스트(dynamic context)** 라 부릅니다. 명령을 실행하는 그 순간의 파일 내용·명령 결과를 가져오므로, 같은 명령이라도 매번 최신 상황에 맞춰 동작합니다.

이렇게 만듭니다

값을 받아 동작하는 `/fix` 명령입니다. `$ARGUMENTS` 자리에 내가 설명한 증상이 들어갑니다.

```

---
description: 증상을 받아 원인을 찾아 고칩니다
argument-hint: [고치고 싶은 증상 설명]
---

```

다음 증상을 해결해 주세요: \$ARGUMENTS

먼저 원인을 찾고, 어떻게 고칠지 설명한 다음 변경 내용을 보여 주세요.

`argument-hint` (인자 힌트)는 명령을 고를 때 무엇을 입력하면 되는지 알려 주는 안내 문구입니다. 실제 동작에는 영향을 주지 않고, 사용자에게 입력 형태를 귀띔하는 용도입니다. 호출은 다음과 같습니다.

```
> /fix 로그인 버튼을 눌러도 아무 반응이 없습니다
```

원인을 살펴보겠습니다.

[Read] `src/login.js` 를 읽었습니다

버튼 클릭 이벤트가 연결되지 않은 것이 원인입니다. 다음과 같이 고치겠습니다 ...

이번에는 파일 참조(@)와 bash 실행(!)을 함께 쓴 `/commit` 명령입니다. 변경 상황을 자동으로 읽어 커밋 메시지를 만들어 줍니다.

```

---
description: 변경 내용을 보고 커밋 메시지를 제안합니다
allowed-tools: Bash(git status:*), Bash(git diff:*)
---

```

현재 변경 상황입니다.

- 상태: `!`git status --short``
- 변경 내용: `!`git diff --stat``
- 프로젝트 규칙: `@CLAUDE.md`

위 내용을 바탕으로 한국어 커밋 메시지 한 줄을 제안해 주세요.

명령 안에서 쓰는 치환 기호: `$ARGUMENTS` · `$1` `$2` · `@<파일경로>` · `!`<셸 명령>``

치환 기호 정리

기호	하는 일	예시
<code>\$ARGUMENTS</code>	명령 뒤 입력 전체를 그 자리에 넣습니다	<code>/fix 버튼 오류</code> → "버튼 오류"
<code>\$1, \$2</code>	입력을 빈칸 단위로 나눠 순서대로 넣습니다	<code>/rename old new</code> → <code>\$1 =old, \$2 =new</code>
<code>@파일</code>	그 파일의 내용을 읽어 와 끼워 넣습니다	<code>@README.md</code>
<code>!`명령`</code>	셸 명령을 실행해 그 결과를 끼워 넣습니다	<code>!`git status`</code>

주의: `!`명령`` 는 컴퓨터에서 **실제로 명령을 실행**합니다. 그래서 그 셸 명령을 쓸 수 있도록 프론트매터의 `allowed-tools` 에 허용을 적어 두어야 합니다(다음 절에서 자세히 다룹니다). `git status` 처럼 읽기만 하는 명령으로 시작하고, 파일을 지우거나 바꾸는 명령은 함부로 넣지 않습니다.

팁: 증상이나 대상이 매번 달라지는 명령에는 `$ARGUMENTS` 를, 현재 코드·변경 상황을 봐야 하는 명령에는 `@·!` 를 씁니다. 둘을 섞으면 "내가 말한 증상 + 지금의 실제 코드"를 함께 본 채로 일하게 되어 결과가 한결 정확해집니다.

절 요약

- `$ARGUMENTS` 는 입력 전체를, `$1·$2` 는 나눈 값을 자리마다 채워 명령을 동적으로 만듭니다.
- `@파일` 은 파일 내용을, `!`명령`` 은 셸 명령 실행 결과를 명령에 끼워 넣습니다.
- `!` 실행은 실제로 명령을 돌리므로 읽기 전용 명령부터 쓰고 권한을 좁혀 둡니다.

명령에서 도구 권한·모델 지정(frontmatter)

도입

회사에서 일을 맡길 때 "이 사람은 자료 열람만, 결제는 불가"처럼 **권한을 정해** 둡니다. 권한을 좁히면 실수로 큰일이 벌어질 위험이 줄어듭니다. 명령에도 같은 원리를 적용할 수 있습니다. 이

절에서는 프론트매터로 그 명령이 쓸 수 있는 도구를 제한하고, 일에 맞는 모델을 골라 실행하는 법을 배웁니다.

핵심 개념 설명

명령 프론트매터 (command frontmatter)

명령 프론트매터(command frontmatter)란, 명령 파일 맨 위 `---`로 감싼 설정 영역입니다. 앞에서 `description`을 적던 바로 그 자리입니다. 여기에는 설명뿐 아니라 **권한·모델 같은 동작 설정**도 적을 수 있습니다. 본문(프롬프트)이 "무엇을 부탁할지"라면, 프론트매터는 "어떤 조건으로 실행할지"를 정합니다.

도구 허용 목록 (allowed-tools)

도구 허용 목록(allowed-tools)이란, 그 명령을 실행하는 동안 Claude가 쓸 수 있는 도구를 한정하는 목록입니다. Claude Code는 파일 읽기(`Read`), 파일 편집(`Edit`), 셸 실행(`Bash`) 같은 여러 도구를 가지고 있는데, `allowed-tools`에 적은 것만 허용됩니다. 적지 않은 도구는 그 명령에서 막힙니다.

왜 굳이 좁힐까요. **검토용 명령은 파일을 고치면 안 되기** 때문입니다. 예를 들어 코드를 살펴보고 의견만 주는 `/review` 명령에 `Read`만 허용하면, 실수로라도 파일을 바꾸는 일이 원천적으로 막힙니다. 권한을 좁히는 것은 **안전장치**입니다.

이렇게 만듭니다

읽기만 하고 절대 고치지 않는 코드 검토 명령입니다. `allowed-tools`에 `Read`만 적어 편집을 막았습니다.

```
---
description: 코드를 읽고 개선점만 제안합니다(파일은 고치지 않음)
allowed-tools: Read
argument-hint: [검토할 파일 경로]
---
```

다음 파일을 검토해 주세요: @\$ARGUMENTS

읽기 쉬운지, 위험한 부분이 없는지 살펴보고 개선점을 목록으로 제안만 해 주세요.
파일을 직접 고치지는 마세요.

특정 모델을 지정할 수도 있습니다. 빠른 답이 필요한 가벼운 명령에는 빠른 모델을, 깊은 분석이 필요한 명령에는 더 똑똑한 모델을 골라 둡니다. `model` 한 줄을 추가합니다.


```

---
description: 변경 내용을 빠르게 한 줄로 요약합니다
allowed-tools: Read
model: claude-haiku-4-5
---
```

방금 바뀐 내용을 한 줄로만 요약해 주세요.

자주 쓰는 프론트매터 키

키	의미	예시 값
description	/ 목록에 보이는 한 줄 설명	코드를 검토합니다
argument-hint	입력 형태를 알려 주는 안내(동작엔 영향 없음)	[파일 경로]
allowed-tools	이 명령이 쓸 수 있는 도구 목록	Read, Bash(git diff:*)
model	이 명령을 실행할 모델 지정	claude-haiku-4-5

allowed-tools 에서 Bash(git diff:*) 처럼 괄호로 범위를 더 좁힐 수도 있습니다. 이는 "셸 실행은 허용하되 git diff 로 시작하는 명령만"이라는 뜻으로, git diff 는 변경 내용을 보여 주기만 할 뿐 파일을 건드리지 않으므로 안전합니다. 별표 * 는 "그 뒤에 무엇이 와도"라는 의미입니다.

베스트 프랙티스: 새 명령을 만들 때는 권한을 가장 좁게 시작하십시오. 검토·요약처럼 보기만 하는 명령은 Read 만, 변경 상황을 읽어야 하면 Bash(git status:*) · Bash(git diff:*) 처럼 읽기 전용 셸만 허용합니다. 정말 편집이 필요할 때만 Edit 를 더합니다. 좁게 시작해 필요할 때 넓히는 편이 안전합니다.

흔한 실수: allowed-tools 를 적지 않은 채 ! 로 셸 명령을 넣으면, 실행할 때마다 권한을 묻는 창이 떠 흐름이 끊깁니다. 자동으로 매끄럽게 돌리려면 그 명령에 필요한 도구를 allowed-tools 에 미리 적어 둡니다. 반대로 너무 넓게(Bash 전체) 열어 두면 위험하니 범위를 좁힌 형태를 권합니다.

절 요약

- 프론트매터에는 설명뿐 아니라 권한(allowed-tools)·모델(model)도 지정할 수 있습니다.

- `allowed-tools` 로 도구를 좁히면 검토용 명령이 실수로 파일을 고치는 일을 막을 수 있습니다.
- 권한은 가장 좁게 시작해 필요할 때만 넓히고, 셸은 `Bash(git diff:*)` 처럼 범위를 한정합니다.

팀과 명령 공유·플러그인(plugin)으로 배포

도입

좋은 양식을 혼자만 쓰면 아깝습니다. 잘 만든 명령은 **팀 전체가 같은 방식으로 일하게** 만드는 자산입니다. 이 절에서는 명령을 동료와 나누는 가장 쉬운 방법과, 여러 명령을 한 묶음으로 포장해 배포하는 **플러그인**을 살펴봅니다.

핵심 개념 설명

명령 공유 (command sharing)

명령 공유(command sharing)란, 내가 만든 커스텀 명령을 동료도 쓸 수 있게 나누는 것입니다. 방법은 의외로 간단합니다. 프로젝트의 `.claude\commands` 폴더는 프로젝트 안에 있는 보통 폴더이므로, **저장소(repository)에 함께 올리면** 됩니다. 저장소란 프로젝트 파일을 버전별로 보관하고 팀원과 주고받는 공동 보관소(예: Git 저장소)를 말합니다.

동료가 그 저장소를 내려받으면 `.claude\commands` 안의 명령이 함께 따라오고, 별도 설치 없이 곧바로 `/명령` 으로 쓸 수 있습니다. 즉 **명령이 프로젝트 코드와 함께 따라다니는** 구조입니다. 그래서 팀이 같은 커밋 규칙, 같은 검토 방식을 자연스럽게 공유하게 됩니다.

플러그인 (plugin)

플러그인(plugin)이란, 커스텀 명령·에이전트(agent)·설정 등 여러 조각을 **하나로 묶어 설치·배포하는 단위**입니다. 명령 파일 몇 개를 저장소에 올리는 것이 "양식 낱장을 건네는 것"이라면, 플러그인은 그 양식들을 **표지를 붙인 한 권의 묶음으로 포장**해 누구나 한 번에 설치하도록 만든 것입니다.

플러그인은 **마켓플레이스(marketplace)** 라는 목록을 통해 배포됩니다. 마켓플레이스는 설치 가능한 플러그인 목록을 담은 출처(보통 Git 저장소)로, 한 번 등록해 두면 그 안의 플러그인을 골라 설치할 수 있습니다.

이렇게 합니다

먼저 명령을 저장소에 함께 올려 공유하는 가장 기본적인 방법입니다. 프로젝트의 `.claude/commands` 를 다른 파일과 똑같이 버전 관리에 포함하면 됩니다.

```
# 명령 파일을 저장소에 포함해 함께 올립니다
git add .claude/commands
git commit -m "팀 공용 슬래시 커맨드 추가"
git push
```

동료는 이 저장소를 내려받기만 하면 됩니다. 별도 설치 과정이 없습니다.

```
PS C:\Users\동료\my-project> git pull
PS C:\Users\동료\my-project> claude
> /

    /summary   최근 변경 내용을 한국어로 짧게 요약합니다
    /commit    변경 내용을 보고 커밋 메시지를 제안합니다
    /review     코드를 읽고 개선점만 제안합니다
    ...
```

플러그인을 설치할 때는 Claude Code 안에서 슬래시 커맨드로 마켓플레이스를 등록하고 설치합니다.

```
> /plugin marketplace add 회사이름/claude-plugins
마켓플레이스를 등록했습니다.

> /plugin install team-commands@claude-plugins
team-commands 플러그인을 설치했습니다. 새 명령이 추가되었습니다.
```

플러그인 관련 명령

명령	하는 일
<code>/plugin marketplace add <출처></code>	플러그인 목록(마켓플레이스)을 등록합니다
<code>/plugin install <이름>@<마켓플레이스></code>	그 목록에서 플러그인을 골라 설치합니다
<code>/plugin</code>	설치된 플러그인을 보고 관리합니다

언제 무엇을 쓸지 정리합니다.

방법	적합한 상황	장점
저장소에 함께 올리 기	한 프로젝트 팀끼리 공유	가장 간단, 내려받으면 끝
플러그인으로 배포	여러 프로젝트·여러 팀에 배 포	한 번에 설치, 버전 관리·업데이트 편리

팁: 처음에는 저장소에 명령을 함께 올리는 방식으로 충분합니다. 같은 명령 묶음을 **여러 프로젝트에 반복 설치**하게 되거나, 외부에 공개·공유하고 싶어질 때 플러그인을 고려하면 됩니다. 단계를 건너뛰지 말고 필요해질 때 올라서면 됩니다.

주의: 플러그인이나 공유 명령에는 다른 사람이 만든 **! 셸 실행**이나 넓은 `allowed-tools` 가 들어 있을 수 있습니다. 출처를 모르는 플러그인을 설치할 때는 **어떤 도구 권한을 요구하는지** 먼저 확인하십시오. 신뢰할 수 있는 출처에서만 설치하는 것이 안전합니다.

절 요약

- 프로젝트의 `.claude\commands` 를 저장소에 함께 올리면 팀원이 내려받아 곧바로 명령을 공유합니다.
- 플러그인은 여러 명령·에이전트를 한 묶음으로 포장해 마켓플레이스를 통해 배포·설치하는 단위입니다.
- 외부 플러그인은 요구하는 도구 권한을 확인하고 신뢰할 수 있는 출처에서만 설치합니다.

실습 과제

해보기: 반복 작업용 커스텀 슬래시 커맨드 만들기

자주 하는 작업을 인자와 파일 참조를 활용한 커스텀 명령으로 만들고, 권한을 줬던 뒤 동작을 문서화합니다.

- 단계 1** — PowerShell에서 프로젝트로 이동해 명령 폴더를 만듭니다. `New-Item -ItemType Directory -Force -Path .\claude\commands`
- 단계 2** — `.claude\commands\diff-summary.md` 파일을 만들고, 프론트매터에 `description` 과 `allowed-tools: Bash(git diff:*)` 를 적습니다.
- 단계 3** — 본문에 `!`git diff --stat`` 으로 변경 상황을 끌어오고, `$ARGUMENTS` 로 "요약을 어떤 관점에서 할지"를 받아 한국어로 요약하도록 작성합니다.

- **단계 4** — Claude Code에서 `/diff-summary` 사용자 관점에서 처럼 입력해 동작을 확인합니다.
- **점검** — `/help` 목록에 명령이 보이는가, 인자가 본문에 채워지는가, `allowed-tools` 덕분에 권한 창 없이 매끄럽게 실행되는가를 확인합니다.
- **정리** — 이 명령이 무엇을 하고 어떤 권한을 쓰는지 한두 줄로 적어, 팀원이 보고 이해할 수 있게 문서화합니다.

FAQ

Q1. 슬래시 커맨드와 그냥 길게 부탁하는 프롬프트는 무엇이 다른가요? → A1. 일반 프롬프트는 Claude가 뜻을 해석해 처리하지만, 슬래시 커맨드는 `/`로 시작해 **정해진 동작을 곧바로 실행**합니다. 내장 명령(`/clear` 등)은 기능이 고정돼 있고, 커스텀 명령은 미리 저장해 둔 프롬프트 템플릿을 한 단어로 불러오는 것입니다.

Q2. 만든 명령이 `/` 목록에 안 보입니다. 무엇을 확인하나요? → A2. 세 가지를 봅니다. 파일 확장자가 `.md`가 맞는지, 폴더가 정확히 `.claude\commands` 인지(철자·점 포함), 그리고 파일을 둔 위치(프로젝트 폴더인지 사용자 홈인지)가 지금 작업 중인 곳과 맞는지입니다. `/help`로 목록을 다시 확인해 보십시오.

Q3. 명령에 여러 개의 값을 따로 받고 싶은데 가능한가요? → A3. 가능합니다. `$ARGUMENTS`는 입력 전체를 한 덩어리로 받지만, `$1`·`$2`처럼 번호가 붙은 자리표시자를 쓰면 빈칸으로 나뉜 값을 순서대로 받을 수 있습니다. 예를 들어 `/rename old new`에서 `$1`은 `old`, `$2`는 `new`가 됩니다.

Q4. `allowed-tools`를 꼭 적어야 하나요? → A4. 필수는 아니지만 권합니다. 적지 않으면 명령이 도구를 쓸 때마다 권한 창이 떠 흐름이 끊기고, 검토용 명령이 실수로 파일을 고칠 여지도 생깁니다. 보기만 하는 명령은 `Read`만, 변경 상황을 읽어야 하면 `Bash(git diff:*)`처럼 **가장 좁게** 적어 두는 편이 안전하고 매끄럽습니다.

장 요약

이번 장에서는 반복되는 요청을 한마디로 묶는 슬래시 커맨드를 배웠습니다.

`/help`·`/clear`·`/compact`·`/model` 같은 내장 명령으로 작업 흐름을 다루고, `.claude\commands` 폴더에 마크다운 파일을 두어 나만의 커스텀 명령을 만드는 법을 익혔습니다. 명령에는 `$ARGUMENTS`·`$1`로 값을 받고, `@파일`·`!`명령``으로 실행 시점의 정보를 끌어올 수 있으며, 프론트 매터의 `allowed-tools`·`model`로 권한과 모델을 정할 수 있습니다. 마지막으로 잘 만든 명령은 저

장소에 함께 올려 팀과 공유하거나 플러그인으로 묶어 배포할 수 있습니다. 핵심은 "자주 하는 일을 명령으로 만들고, 권한은 가장 좁게 시작하라"는 한 문장입니다.

핵심 키워드

슬래시 커맨드, 내장 명령, 커스텀 명령, 프롬프트 템플릿, \$ARGUMENTS, @파일 참조, 프론트매터, allowed-tools, 플러그인

다음 장 미리보기

다음 장에서는 Claude Code의 동작을 더 깊이 다듬는 설정(settings)과 메모리(CLAUDE.md)를 살펴봅니다. 명령을 넘어, 도구 전체가 내 작업 방식에 맞춰 움직이도록 길들이는 방법을 익힙니다.

서브에이전트 설계와 위임 (.claude/agents)

학습 목표 - 서브에이전트(subagent)가 독립 컨텍스트로 일해 메인 대화를 깨끗하게 지키는 원리를 설명할 수 있습니다. - `.claude/agents`의 에이전트 정의 구조(frontmatter의 `name · description · tools · model`)와 역할 프롬프트를 작성할 수 있습니다. - `description` 설계로 자동 위임(auto-delegation)과 명시적 호출(explicit invocation)을 의도대로 제어할 수 있습니다. - 에이전트별 도구 권한을 최소 권한(least privilege)으로 좁히고 역할 경계를 뚜렷이 그을 수 있습니다. - 멀티 에이전트 협업(에이전트 팀)의 병렬·순차 패턴과 비용·복잡도 한계를 판단할 수 있습니다.

5장에서 `CLAUDE.md`로 프로젝트의 기억을, 6장에서 슬래시 커맨드로 반복 작업을 자동화했습니다. 이 장은 한 걸음 더 나아가 **일을 사람이 아니라 또 다른 에이전트에게 맡기는 법**을 다룹니다. 지금까지는 Claude Code와 나, 단둘이 한 대화창에서 모든 일을 처리했습니다. 그런데 작업이 커지면 한 대화에 조사·구현·검증이 뒤섞여 맥락이 어지러워집니다. **서브에이전트(subagent)**는 이 문제를 "전문 담당자에게 나눠 맡기고 보고만 받기"로 푸는 장치입니다.

이 장은 중급 단계인 만큼 화면 사용법을 넘어 **왜 독립 컨텍스트가 필요한지, 자동 위임이 내부에서 어떻게 라우팅되는지** 같은 동작 원리까지 파고듭니다. 이 책은 Windows 11의 **PowerShell(파워셸)**을 기준으로 하며, 에이전트 정의 파일은 프로젝트 폴더 안의 `.claude\agents\` 경로(예: `C:\Users\사용자\my-project\.claude\agents\`)에 둡니다. 아래 코드 블록의 에이전트 파일과 화면은 모두 **참고용 예시**이며, 실제로 실행되는 프로그램이 아니라 여러분이 직접 작성·확인하는 설정과 그 결과 화면입니다.

서브에이전트(subagent)와 독립 컨텍스트의 원리

도입

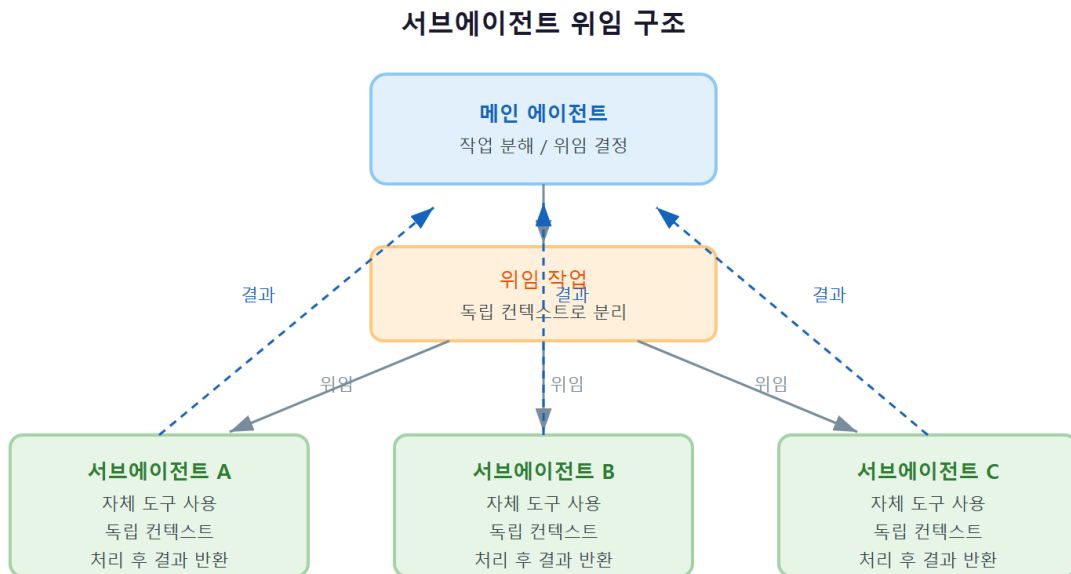
회사에서 팀장이 모든 일을 직접 하지는 않습니다. "이 보고서 사실관계만 검증해 줘", "이 코드 리뷰만 봐 줘"라고 담당자에게 맡기고, 담당자가 **핵심 결론만** 요약해 올리면 팀장은 그 결론으로 다음 판단을 합니다. 담당자가 자료를 뒤지며 본 수백 줄의 중간 과정은 팀장 책상에 쌓이지

않습니다. 서브에이전트가 하는 일이 정확히 이것입니다. 이 절에서는 서브에이전트가 무엇이고, **독립 컨텍스트(separate context)** 가 왜 품질을 끌어올리는지 그 원리를 봅니다.

핵심 개념 설명

서브에이전트 (subagent)

서브에이전트(subagent)란, 메인 에이전트가 특정 작업을 위임하는 **독립된 보조 에이전트**입니다. 메인 대화(내가 마주하고 있는 Claude Code)가 "팀장"이라면, 서브에이전트는 한 가지 일을 맡아 처리하고 **결과 요약만 돌려주는 담당자**입니다. 담당자는 자기만의 작업 책상(컨텍스트)에서 일하므로, 그가 읽은 파일과 시행착오가 팀장의 책상을 어지럽히지 않습니다.



독립 컨텍스트 (separate context)

독립 컨텍스트(separate context)란, 서브에이전트가 메인 대화와 **분리된 별도의 컨텍스트 윈도우(context window)**에서 동작한다는 뜻입니다. 여기서 컨텍스트 윈도우란 3장에서 배운 "한 번에 기억하는 대화·파일의 총량을 담는 그릇"입니다. 서브에이전트는 자기 그릇을 따로 들고 일하고, 다 끝나면 **요약 한 덩어리만** 메인 그릇에 담아 돌려줍니다.

여기서 가장 흔한 오해 하나를 짚겠습니다. "**서브에이전트가 메인 대화 맥락을 그대로 이어받는다**" 는 생각입니다. 그렇지 않습니다. 서브에이전트는 빈 책상에서 시작하며, 메인에 위임할 때 건네준 지시와 자신의 역할 프롬프트만 가지고 일을 시작합니다. 그래서 위임할 때는 **필요한 맥락을 명확히 적어 건네야** 합니다.

- **왜(why) 나누나:** 한 대화에 모든 중간 과정을 쌓으면 컨텍스트 원도가 빠르게 차고, 정작 중요한 지시가 옛 잡음에 묻힙니다. 위임하면 잡음은 담당자 책상에서 소모되고, 메인에는 결론만 남습니다.
- **언제(when) 쓰나:** (1) 코드베이스를 뒤져 답을 찾는 **탐색**처럼 중간 산출이 많지만 결론은 짧은 작업, (2) 작성한 코드를 다른 시각으로 **검증·리뷰**해 편향을 줄이고 싶은 작업이 대표적입니다.
- **어떻게(how) 동작하나:** 메인이 서버에이전트를 호출 → 서버에이전트가 자기 컨텍스트에서 도구를 써 가며 일함 → 결과 요약은 메인에 반환 → 메인은 그 요약만 받아 다음을 진행합니다.

시나리오로 보기

- **시나리오 A (탐색):** "이 프로젝트에서 사용자 인증이 어디서 처리되는지 찾아 줘." 탐색 담당 서버에이전트가 수십 개 파일을 읽어 가며 추적하고, 메인에는 "인증은 `auth/` 폴더의 세 파일에서 처리됩니다"라는 요약만 돌아옵니다. 메인 대화에는 읽은 수십 개 파일 내용이 쌓이지 않습니다.
- **시나리오 B (검증):** 메인이 방금 함수를 고쳤습니다. 같은 대화에서 "잘 고쳤나?"라고 물으면, 방금 자기가 고친 맥락에 끌려 후하게 평가하기 쉽습니다. 리뷰 담당 서버에이전트는 빈 책상에서 코드만 보고 평가하므로, 자기 확신 편향에서 자유로운 판단을 줍니다.

이렇게 동작합니다

다음은 메인 에이전트가 탐색 작업을 서버에이전트에게 위임하는 모습의 예시 화면입니다. > 줄이 내가 건넨 요청이고, `[code-explorer]` 같은 대괄호 표시는 지금 어떤 서버에이전트가 일하는 중인지 알려 주는 안내입니다.

```
PS C:\Users\사용자\my-project> claude
> 이 프로젝트에서 사용자 인증을 어디서 처리하는지 찾아서 핵심만 알려 줘.

[code-explorer 서버에이전트에게 위임] 독립 컨텍스트에서 탐색을 시작합니다
  [Read] src/auth/login.ts
  [Read] src/auth/session.ts
  [Grep] "verifyToken" (12 results)
  ... (중간 탐색 과정은 서버에이전트 컨텍스트 안에서만 진행됩니다)
[code-explorer 완료] 요약을 메인에 반환했습니다

인증은 src/auth/ 폴더의 세 파일에서 처리됩니다.
- login.ts: 로그인 요청 처리
- session.ts: 세션 토큰 발급·검증
- middleware.ts: 요청마다 토큰 확인
```

위임 과정에서 일어난 일

화면 표시	의미
[code-explorer 서버에이전트에게 위임]	메인이 탐색 작업을 서버에이전트에게 넘겼습니다
[Read] · [Grep] (들여쓰기됨)	서브에이전트가 자기 컨텍스트 안에서 도구를 사용한 흔적입니다
[code-explorer 완료] 요약물 메인에 반환	중간 과정은 버려지고 요약만 메인으로 돌아왔습니다
마지막 요약 문단	메인 컨텍스트에 실제로 남는 것은 이 결론뿐입니다

팁: 위임은 "메인 대화를 아끼는 일"이기도 합니다. 긴 탐색·로그 분석처럼 토큰을 많이 먹지만 결론은 짧은 작업일수록 서버에이전트로 넘기면, 메인 컨텍스트 윈도우를 오래 깨끗하게 쓸 수 있습니다.

흔한 실수: 위임하면서 맥락을 충분히 안 건네기. 서버에이전트는 메인 대화를 못 봅니다. "아까 그 함수"라고만 하면 담당자는 그게 뭔지 모릅니다. 위임 요청에는 대상 파일·목표·제약을 **그 자체로 완결되게** 적습니다.

절 요약

- 서버에이전트는 메인이 일을 맡기는 독립 보조 에이전트이며, 결과 요약만 돌려줍니다.
- 독립 컨텍스트 덕분에 중간 과정의 잡음이 메인 대화를 오염시키지 않습니다.
- 서버에이전트는 메인 맥락을 이어받지 않으므로, 위임 시 필요한 맥락을 완결되게 건네야 합니다.

에이전트 정의하기 - frontmatter 구조

도입

새 담당자를 팀에 들이려면 **직무 기술서**가 필요합니다. 이름이 무엇이고, 어떤 상황에 부르며, 어떤 도구를 쓸 수 있고, 어떤 태도로 일하는지를 적어 두는 문서입니다. Claude Code에서 그 직무 기술서가 바로 `.claude\agents` 폴더의 에이전트 정의 파일입니다. 이 절에서는 그 파일이 어떤 구조이고 각 칸이 무엇을 결정하는지 봅니다.

핵심 개념 설명

에이전트 정의 (agent definition)

에이전트 정의(agent definition) 란, `.claude\agents` 폴더 안의 **마크다운 파일 하나**로 서버에이전트를 만드는 구성입니다. 파일은 두 부분으로 나뉩니다. 위쪽의 **프론트매터(frontmatter)** 와 아래쪽의 **본문**입니다.

- **프론트매터(frontmatter)**: 파일 맨 위에 `---` 두 줄 사이에 적는 설정 영역입니다. 6장의 커스텀 명령에서 본 것과 같은 형식이며, 여기에 `name · description · tools · model` 같은 항목을 적습니다.
- **본문(역할 프롬프트)**: 프론트매터 아래의 모든 글이 그 에이전트의 **시스템 프롬프트(system prompt)**, 즉 "너는 누구이고 어떻게 일하라"는 역할 지침이 됩니다.

역할 프롬프트 (persona prompt)

역할 프롬프트(persona prompt) 란, 에이전트에게 **정체성과 일하는 방식**을 부여하는 본문 글입니다. "너는 꼼꼼한 코드 리뷰어다. 보안과 가독성을 우선으로 보고, 발견을 심각도순으로 정리하라" 같은 지침이 여기 들어갑니다. 같은 모델이라도 역할 프롬프트가 다르면 전혀 다른 담당자처럼 행동합니다.

여기서 흔한 오해는 "**파일만 만들어 두면 알아서 잘 동작한다**" 는 생각입니다. 역할 프롬프트가 비거나 모호하면 에이전트는 평범한 만능 비서처럼 행동해, 굳이 나눈 보람이 없습니다. 직무 기술서가 또렷할수록 담당자가 자기 일을 잘하듯, 역할 프롬프트가 구체적일수록 결과가 좋아집니다.

- **왜(why) 파일로 두나**: 파일이면 저장소에 커밋해 **팀과 공유**할 수 있고, 버전 관리로 변경이력도 남습니다.
- **언제(when) 만드나**: 같은 종류의 작업(리뷰·문서화·탐색)을 반복해서 시킬 때, 그때마다 긴 지시를 다시 쓰는 대신 정의로 굳혀 둡니다.
- **어떻게(how) 두나**: 프로젝트 전용은 `프로젝트\.claude\agents\`, 내 모든 프로젝트에서 쓸 개인용은 사용자 홈의 `~\.claude\agents\` (Windows에서는 `C:\Users\사용자\.claude\agents\`)에 둡니다.

이렇게 정의합니다

다음은 코드 리뷰 담당 서버에이전트를 정의한 파일 예시입니다. 파일 이름은 `code-reviewer.md`로 하고 `프로젝트\.claude\agents\` 폴더에 둡니다. 실행되는 코드가 아니라 여러분이 작성하는 **설정 문서**입니다.

```

---
name: code-reviewer
description: 코드 변경의 품질·보안·가독성을 리뷰할 때 사용합니다. 함수나 파일을 수정한 직후 검증
tools: Read, Grep, Glob
model: sonnet
---

```

너는 꼼꼼한 코드 리뷰어다. 주어진 변경만 검토하고 파일을 직접 고치지는 않는다.

검토 우선순위:

1. 보안 결함(입력 검증 누락, 비밀값 노출)
2. 명백한 버그와 예외 처리 누락
3. 가독성과 명명

발견을 심각도(높음/중간/낮음)순으로 정리하고, 각 항목에 위치와 한 줄 개선안을 붙여라.
문제가 없으면 "특이사항 없음"이라고만 답하라.

프론트매터 항목 설명

항목	의미
name	에이전트의 고유 이름입니다. 명시적으로 부를 때(<code>code-reviewer</code> 를 시켜 줘) 이 이름을 씁니다
description	언제 이 에이전트를 부를지 를 적습니다. 자동 위임의 판단 근거가 되므로 가장 중요합니다(다음 절에서 자세히 다룹니다)
tools	이 에이전트가 쓸 수 있는 도구 목록입니다. 적지 않으면 메인의 도구를 상속합니다
model	이 에이전트가 쓸 모델입니다(예: <code>sonnet</code> · <code>haiku</code> · <code>opus</code>). 가벼운 일은 빠른 모델로 둘 수 있습니다
본문(--- 아래)	역할 프롬프트(시스템 프롬프트)입니다. 정체성·우선순위·출력 형식을 적습니다

명령줄로 직접 만들지 않고 대화 안에서 관리하고 싶다면 `/agents` 명령을 씁니다. 다음은 그 화면 예시입니다.

```
> /agents
```

서브에이전트 관리

```
[Library] code-reviewer (project) 리뷰 담당
          doc-writer (project) 문서화 담당
          (+ 내장·사용자·플러그인 에이전트 목록)
[Running] (현재 실행 중인 서브에이전트가 여기 표시됩니다)
```

새로 만들기 / 편집 / 삭제 를 고를 수 있습니다.

팁: 처음에는 `/agents` 명령의 안내를 따라 만들면 frontmatter 형식을 틀릴 일이 적습니다. 익숙해지면 파일을 직접 작성해 저장소에 커밋하는 편이 팀 공유와 버전 관리에 유리합니다.

절 요약

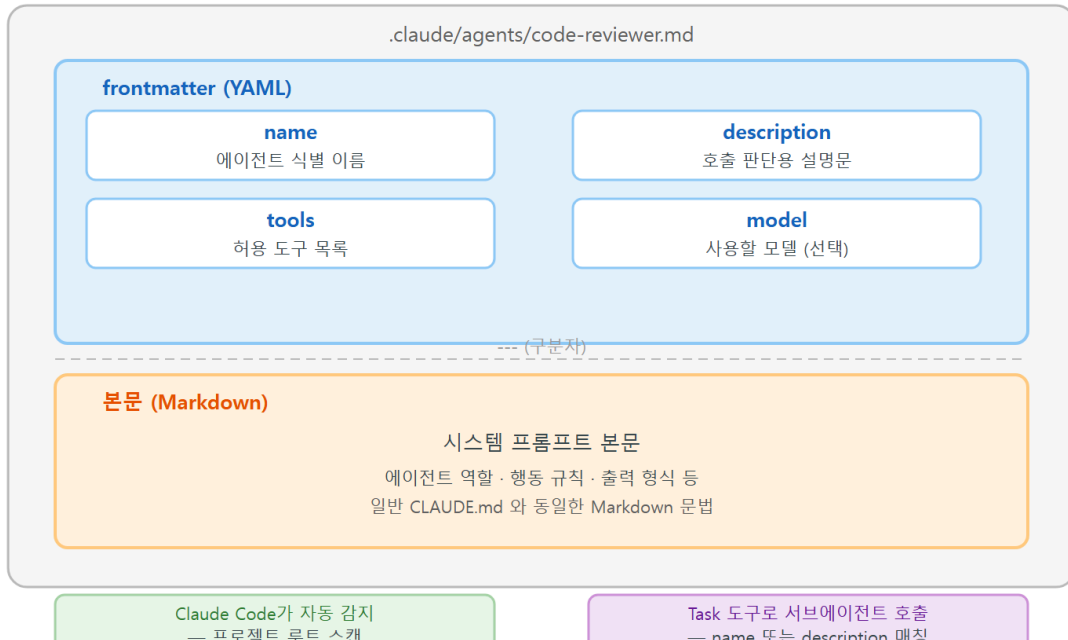
- 에이전트 정의는 `.claude\agents` 의 마크다운 파일이며, 위쪽 프론트매터와 아래쪽 역할 프롬프트로 구성됩니다.
- 프론트매터에는 `name · description · tools · model` 을, 본문에는 정체성·우선순위·출력 형식을 적습니다.
- 프로젝트 전용은 프로젝트 `.claude\agents`, 개인용은 홈의 `.claude\agents` 에 두며, `/agents` 로도 관리할 수 있습니다.

자동 위임 vs 명시 호출 - description 설계

도입

큰 건물 1층 안내데스크는 방문객의 용건을 듣고 알맞은 부서로 연결해 줍니다. 안내가 정확하려면 각 부서가 "우리는 이런 일을 합니다"라고 또렷이 알려 줘야 합니다. 부서 설명이 모호하면 엉뚱한 곳으로 연결되지요. 서브에이전트의 **자동 위임**도 똑같습니다. 이 절에서는 메인인 어떤 기준으로 에이전트를 고르는지, 그리고 그 선택을 좌우하는 `description` 설계를 봅니다.

.claude/agents/이름.md 파일 정의 구조



핵심 개념 설명

자동 위임 (auto-delegation)

자동 위임(auto-delegation)이란, 내가 특정 에이전트를 지목하지 않아도 **메인 에이전트가 작업 내용과 각 서브에이전트의 description을 견주어** 가장 적합한 담당자를 스스로 골라 맡기는 동작입니다. "방금 고친 코드 좀 봐 줘"라고만 해도, 메인인 `code-reviewer`의 `description`("코드 변경을 리뷰할 때 사용")과 맞다고 판단하면 알아서 그 에이전트에게 넘깁니다.

내부에서 일어나는 일(동작 원리): 메인 에이전트는 등록된 모든 서브에이전트의 `name`과 `description`을 일종의 "부서 안내판"으로 들고 있습니다. 새 작업이 들어오면, 그 작업 설명과 각 안내판을 의미적으로 견주어 가장 잘 맞는 후보를 고릅니다. 즉 자동 위임의 정확도는 **전적으로 description이 작업 상황을 얼마나 잘 묘사하느냐**에 달려 있습니다. `description`은 "이 에이전트가 무엇을 하는가"보다 **"언제 이 에이전트를 불러야 하는가"**를 적을수록 매칭이 정확해 집니다.

명시적 호출 (explicit invocation)

명시적 호출(explicit invocation)이란, 자동 판단에 맡기지 않고 내가 **에이전트를 직접 지목해 부르는** 방식입니다. "`code-reviewer` 서브에이전트로 이 변경을 검토해 줘"처럼 이름을 대면, 메인 매칭을 거치지 않고 그 에이전트를 부릅니다. 어떤 담당자에게 맡길지 내가 확실히 알 때 쓰면 더 빠르고 예측 가능합니다.

여기서 가장 흔한 오해는 "에이전트를 만들어 두기만 하면 무조건 자동으로 불린다" 는 생각입니다. description이 모호하거나 다른 에이전트와 겹치면, 메인이 엉뚱한 담당자를 고르거나 아예 직접 처리해 버립니다. 자동 위임이 자꾸 빗나가면, 에이전트를 더 만들 게 아니라 **description을 다듬어야** 합니다.

- **왜(why) description이 중요한가:** 자동 위임의 라우팅 근거가 오직 description이기 때문입니다. 여기가 흐릿하면 모든 자동 호출이 흔들립니다.
- **언제(when) 명시 호출을 쓰나:** 담당자가 확실할 때, 또는 자동 위임이 자주 빗나가 결과가 들쭉날쭉할 때 명시 호출로 고정합니다.
- **어떻게(how) 잘 쓰나:** description에 **트리거 상황**("...할 때 사용", "...한 직후 적합")을 넣고, 필요하면 "...에는 사용하지 않음"으로 경계를 칩니다.

이렇게 설계합니다

다음은 같은 코드 리뷰 에이전트의 description을 **모호하게** 쓴 경우와 **또렷하게** 쓴 경우를 나란히 둔 비교입니다.

빗나가기 쉬운 description (모호함)

description: 코드를 돕습니다.

잘 라우팅되는 description (트리거 상황이 또렷함)

description: 코드 변경의 품질·보안·가독성을 리뷰할 때 사용합니다. 함수나 파일을 수정한 직후, 적용 전에 검증이 필요할 때 적합합니다. 새 기능 구현 자체에는 쓰지 않습니다.

다음은 자동 위임이 일어나는 화면 예시입니다. 내가 에이전트를 지목하지 않았는데도 메인이 알맞은 담당자를 골랐습니다.

> 방금 고친 login.ts 변경이 괜찮은지 봐 줘.

[자동 위임 판단] 작업이 'code-reviewer'의 description과 가장 잘 맞습니다
[code-reviewer 서버에이전트에게 위임] 독립 컨텍스트에서 검토를 시작합니다
[code-reviewer 완료]

리뷰 결과 (심각도순)

- [중간] login.ts:42 입력값 검증 없이 토큰을 신뢰합니다. 길이·형식 확인을 권합니다.
- [낮음] 변수명 d 는 의미가 모호합니다. decodedToken 을 제안합니다.

비교를 위해, 명시적으로 부르는 경우는 다음과 같습니다.

> code-reviewer 서버에이전트로 login.ts 변경을 검토해 줘.

좋은 description의 재료

재료	모호한 예	또렷한 예
무슨 일	"코드를 돕습니다"	"품질·보안·가독성을 리뷰합니다"
언제(트리거)	(없음)	"함수·파일 수정 직후, 적용 전 검증이 필요할 때"
경계(언제 아님)	(없음)	"새 기능 구현 자체에는 쓰지 않습니다"

흔한 실수: 여러 에이전트의 description이 서로 겹치게 쓰기. 예를 들어 `code-reviewer` 와 `security-checker` 가 둘 다 "코드를 점검합니다"라고만 적혀 있으면, 메인이 어느 쪽을 부를지 혼들립니다. 각 에이전트의 트리거 상황이 **서로 겹치지 않게** 경계를 나눠 적습니다.

팁: 자동 위임이 자주 빛나가면 "방금 이 작업은 code-reviewer가 했어야 했어"처럼 메인에 알려주고, 그 피드백을 바탕으로 description의 트리거 문구를 고치는 식으로 다듬어 갑니다. 만들고-시험하고-다듬는 반복이 정상입니다.

절 요약

- 자동 위임은 메인이 작업 설명과 각 에이전트의 description 을 건주어 담당자를 고르는 동작입니다.
- description은 "무엇을 하는가"보다 "언제 부르는가(트리거)"와 "언제 안 부르는가(경계)"를 적을수록 정확합니다.
- 담당자가 확실하거나 자동 위임이 빛나갈 때는 이름을 지목하는 명시적 호출로 고정합니다.

도구 권한 최소화와 역할 프롬프트 설계

도입

회사에서 모든 직원에게 서버실 마스터키를 주지는 않습니다. 각자 자기 일에 필요한 출입증만 받습니다. 그래야 사고가 나도 피해 범위가 좁고, 누가 무엇을 할 수 있는지 명확합니다. 서버에 이전트의 도구 권한도 같은 원칙으로 설계합니다. 이 절에서는 **최소 권한(least privilege)** 으로 도구를 좁히는 법과, 한 가지 일을 잘하게 만드는 역할 프롬프트를 봅니다.

핵심 개념 설명

최소 권한 (least privilege)

최소 권한(least privilege)이란, 에이전트에게 **자기 역할에 꼭 필요한 도구만** 허용하는 설계 원칙입니다. 코드를 읽고 평가만 하는 리뷰어에게 파일을 고치는 `Edit` 나 명령을 실행하는 `Bash` 까지 줄 이유가 없습니다. 리뷰어에게는 `Read · Grep · Glob` 처럼 **읽기·검색 도구만** 주면 충분합니다.

frontmatter의 `tools` 항목을 적지 않으면 에이전트는 **메인의 도구를 그대로 상속**합니다. 즉 아무것도 안 적는 것은 "마스터키를 준 것"과 같습니다. 권한을 좁히려면 `tools` 에 허용할 도구를 **명시적으로 나열**해야 합니다.

- **왜(why) 좁히나:** (1) **안전** — 읽기 전용 에이전트라면 실수로든 오작동으로든 파일을 망가뜨릴 수 없습니다. (2) **집중** — 도구가 적으면 에이전트가 역할 밖으로 새지 않고 본업에 집중합니다.
- **언제(when) 어떤 도구를:** 리뷰·탐색은 읽기·검색만, 문서화는 읽기와 문서 파일 쓰기만, 구현은 편집·실행까지처럼 역할에 맞춰 정합니다.
- **어떻게(how) 적나:** `tools: Read, Grep, Glob` 처럼 쉼표로 나열합니다. 위험 도구 (`Bash · Edit · Write`)는 정말 필요한 역할에만 줍니다.

역할 경계 (role boundary)

역할 경계(role boundary)란, 한 에이전트가 **맡은 일의 범위**를 뚜렷이 그어 두는 것입니다. 도구 권한이 "할 수 있는 것"의 물리적 한계라면, 역할 프롬프트의 경계 문장은 "해야 할 것"의 방향입니다. 둘을 함께 써야 에이전트가 한 우물만 깊게 팝니다.

여기서 흔한 오해는 "**권한은 넉넉히 줘야 일을 잘한다**" 는 생각입니다. 오히려 반대입니다. 만능 에이전트는 어떤 일이든 어설프게 하지만, 도구와 역할이 좁은 에이전트는 그 좁은 일을 깊고 일관되게 해냅니다. 하나를 잘하는 담당자 여럿이 만능 한 명보다 낫습니다.

시나리오로 보기

- **시나리오 A (안전한 리뷰어):** 리뷰어에게 `tools: Read, Grep, Glob` 만 줍니다. 설령 역할 프롬프트가 흔들려 "고쳐 버릴까" 하는 충동이 생겨도, `Edit` 도구가 없으니 **물리적으로 파일을 못 바꿉니다**. 안전이 권한 단계에서 보장됩니다.
- **시나리오 B (역할 경계로 집중):** 문서화 담당에게 "코드 로직은 바꾸지 말고 README와 주석만 손대라"는 경계를 적고, `tools` 도 읽기와 문서 파일 쓰기로 좁힙니다. 그러면 문서를 고치다 코드까지 건드리는 일이 생기지 않습니다.

이렇게 설계합니다

다음은 문서화 담당 에이전트 `doc-writer.md` 의 예시입니다. 도구를 좁히고 역할 경계를 본문에 명시한 점에 주목합니다.

```
---
name: doc-writer
description: 코드 변경 후 README·주석·사용 설명 문서를 갱신할 때 사용합니다.
  기능 구현이나 버그 수정 자체에는 쓰지 않습니다.
tools: Read, Glob, Edit
model: haiku
---
```

너는 기술 문서 작성 담당이다. 다음 경계를 반드시 지켜라.

할 일:

- 코드의 동작을 읽고 README·주석·사용 예시를 명확한 합니다체로 갱신한다.
- 사용자가 따라 할 수 있도록 단계와 예시를 구체적으로 적는다.

하지 말 것(역할 경계):

- 코드 로직 자체는 수정하지 않는다(기능 변경은 다른 담당의 몫이다).
- 실행 명령(Bash)은 쓰지 않는다.

작업 끝에는 어떤 문서의 어느 부분을 왜 고쳤는지 두세 줄로 요약하라.

최소 권한 설계 점검표

설계 요소	이 예시에서	효과
<code>tools</code> 좁히기	Read, Glob, Edit 만(Bash 없음)	명령 실행 사고 원천 차단
역할 경계 문장	"코드 로직은 수정하지 않는다"	문서 작업이 코드로 새지 않음
가벼운 <code>model</code>	haiku	단순 반복 작업의 비용·속도 최적화
출력 형식 지정	"두세 줄로 요약"	메인이 받기 좋은 결과 형태

다음은 도구 권한이 좁아 위험 작업이 막히는 화면 예시입니다.

```
[doc-writer 서브에이전트 작업 중]
[Read] README.md
[Edit] README.md 사용법 섹션 갱신
(Bash 도구는 이 에이전트에 허용되지 않아 시도조차 하지 않습니다)
[doc-writer 완료] README.md 사용법 섹션을 최신 옵션에 맞춰 갱신했습니다.
```

베스트 프랙티스: 에이전트 설계는 "한 에이전트=한 가지 일"을 원칙으로 합니다. 도구는 그 일에 필요한 최소만 주고, 역할 프롬프트에는 **할 일**과 **하지 말 것**을 함께 적습니다. 권한(물리적 한계)과 경계(방향)를 둘 다 갖추면 에이전트가 일관되게 동작합니다.

흔한 실수: `tools` 를 비워 두고 안전하다고 여기기. 비워 두면 메인의 모든 도구를 상속하므로, 읽기 전용으로 만들려던 리뷰어가 `Edit · Bash` 까지 주게 됩니다. **읽기 전용이 목적이면 반드시 읽기·검색 도구만 명시합니다.**

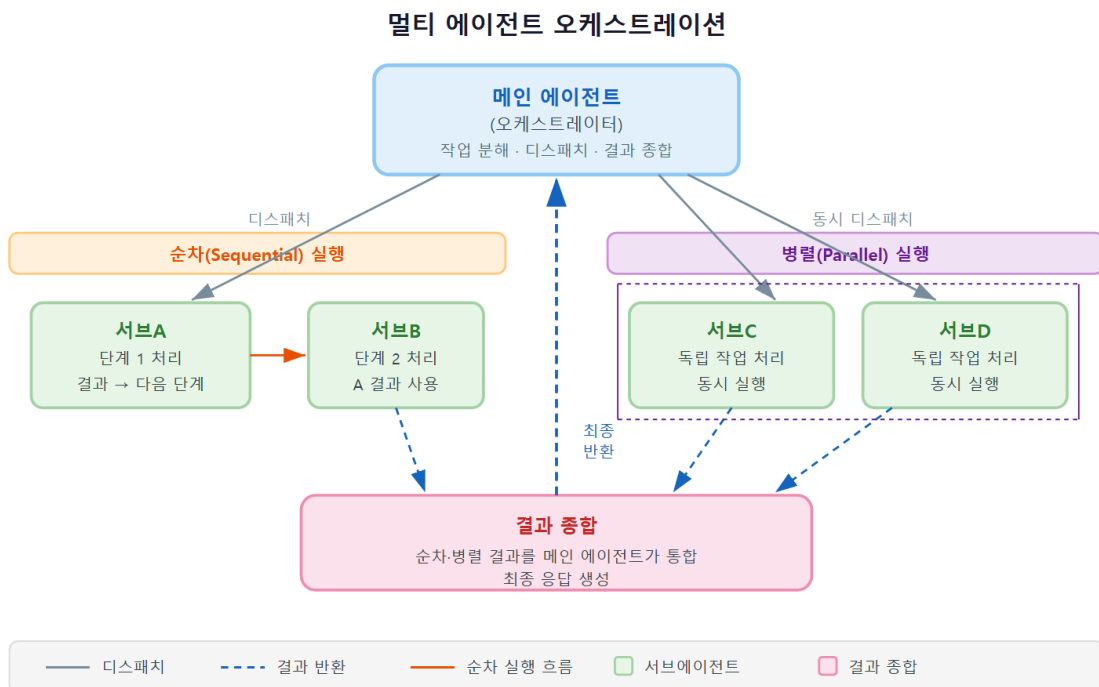
절 요약

- 최소 권한은 역할에 꼭 필요한 도구만 `tools` 에 명시하는 원칙이며, 비워 두면 메인 도구를 모두 상속합니다.
- 도구 권한은 "할 수 있는 것"의 한계, 역할 경계 문장은 "해야 할 것"의 방향으로 함께 써야 합니다.
- 만능 에이전트보다 한 가지를 잘하는 좁은 에이전트 여럿이 더 일관됩니다.

멀티 에이전트 협업 패턴(에이전트 팀)과 한계

도입

규모 있는 프로젝트는 한 사람이 아니라 팀으로 굴러갑니다. 프로젝트 매니저(PM)가 조사·구현·검수 담당에게 일을 나눠 주고, 결과를 모아 종합합니다. 서브에이전트도 여럿을 엮으면 이런 팀처럼 일할 수 있습니다. 다만 팀이 커질수록 조율 비용과 인건비가 늘듯, 에이전트 팀에도 분명한 한계가 있습니다. 이 절에서는 협업 패턴과 **언제 멈춰야 하는지**를 함께 봅니다.



핵심 개념 설명

에이전트 팀 (agent team)

에이전트 팀(agent team)이란, **오케스트레이터(orchestrator)** 역할의 메인 에이전트가 여러 전문 서브에이전트에게 작업을 나눠 맡기고 결과를 종합하는 멀티에이전트 협업 패턴입니다. 여기서 오케스트레이터란 "지휘자"라는 뜻으로, 직접 연주하기보다 단원들을 조율해 하나의 곡을 완성하는 역할입니다.

오케스트레이션 (orchestration)

오케스트레이션(orchestration)이란, 어떤 작업을 어떤 순서로 어느 에이전트에게 맡기고 결과를 어떻게 합칠지 **조율하는 일**입니다. 크게 두 패턴이 있습니다.

- **병렬(parallel) 디스패치**: 서로 의존하지 않는 작업을 **여러 서브에이전트에게 동시에** 던집니다. 예를 들어 "프론트엔드 코드 조사"와 "백엔드 코드 조사"를 동시에 보내면, 둘이 각자 독립 컨텍스트에서 동시에 일하고 메인이 두 요약 한 번에 받습니다. 벽시계 기준 시간이 줄어듭니다.
- **순차(sequential) 위임**: 앞 작업 결과가 뒷 작업의 입력이 될 때 차례로 맡깁니다. "조사 → 그 결과로 구현 → 구현물 검증"처럼 의존이 있으면 순서대로 흘립니다.

내부 동작과 비용 원리: 서브에이전트는 각자 **자기 컨텍스트와 토큰**을 씁니다. 담당자를 늘릴수록 일이 나뉘어 빨라질 수 있지만, 동시에 **총 토큰 사용량(따라서 비용)**이 **합산되어 늘어납니다**. 또 오케스트레이터가 여러 결과를 받아 종합하는 단계 자체도 토큰을 씁니다. 즉 멀티에이전트는 "공짜로 빨라지는 마법"이 아니라 **비용을 더 써서 시간·품질을 사는 거래**입니다.

여기서 가장 흔한 오해는 **"에이전트를 많이 쓸수록 항상 빠르고 싸진다"**는 생각입니다. 사실은 반대일 때가 많습니다. 작업이 작거나 의존이 강하면, 나누는 오버헤드가 이득보다 커서 **더 느리고 더 비싸집니다**. 한 사람이 5분이면 할 일을 굳이 세 명에게 나눠 회의까지 하면 더 오래 걸리는 것과 같습니다.

- **왜(why) 나누나**: 독립적이고 양이 많은 작업을 동시에 처리해 시간을 줄이거나, 서로 다른 전문 시각(구현 vs 검증)으로 품질을 높이려 할 때입니다.
- **언제(when) 쓰지 말아야 하나**: 작업이 작을 때, 단계 간 의존이 강해 어차피 순서대로 해야 할 때, 비용에 민감할 때는 단일 에이전트가 낫습니다.
- **어떻게(how) 운영하나**: 작업을 독립 단위로 쪼갤 수 있는지 먼저 보고, 쪼개지면 병렬, 의존하면 순차로 흘리며, 에이전트 수는 **필요한 만큼만** 늘립니다.

이렇게 협업합니다

다음은 오케스트레이터가 조사·구현·검증을 나눠 맡기는 화면 예시입니다. 앞 두 조사는 병렬로, 검증은 구현 뒤 순차로 진행됩니다.

> 결제 모듈을 리팩터링하고 싶어. 현 구조 조사부터 검증까지 진행해 줘.

[병렬 위임] 두 서브에이전트를 동시에 디스패치합니다

├─ [code-explorer] 결제 모듈 현재 구조 조사 (독립 컨텍스트)

└─ [dep-explorer] 결제가 의존하는 외부 모듈 조사 (독립 컨텍스트)

[병렬 완료] 두 요약을 받았습니다

[순차 위임] 조사 결과를 바탕으로 구현 → 검증 순으로 진행합니다

→ [refactorer] 조사 요약을 입력받아 리팩터링 적용

→ [code-reviewer] 변경된 코드를 독립 컨텍스트에서 검토

[종합] 메인이 세 결과를 모아 정리했습니다

요약: 결제 모듈을 2개 파일로 분리했고, 리뷰에서 발견된 중간 1건을 반영했습니다.

이 협업에서 무슨 일이 일어났나

단계	패턴	무슨 일
구조·의존 조사	병렬	의존 없는 두 조사를 동시에 처리해 시간 절약
리팩터링	순차	조사 요약이 있어야 시작 가능(앞 결과에 의존)
코드 리뷰	순차	구현 결과가 있어야 검토 가능
종합	오케스트레이터	메인이 세 요약을 모아 최종 정리(이 단계도 토큰 사용)

한계와 과용의 위험: 에이전트를 늘릴수록 (1) **토큰·비용이 합산되어 증가**하고, (2) 누가 무엇을 했는지 따라가기 어려워 **디버깅이 복잡**해지며, (3) 위임·종합 단계의 오버헤드가 쌓입니다. "에이전트를 쓰니 멋지다"는 이유만으로 나누지 말고, **나눠서 분명히 이득인 작업에만** 적용합니다.

베스트 프랙티스: 멀티 에이전트는 "독립적이고 양 많은 병렬 작업" 또는 "구현과 검증을 분리해 편향을 줄이는 작업"에서 가장 빛납니다. 의심스러우면 단일 에이전트로 시작하고, 한 대화가 너무 길어지거나 같은 일을 반복할 때 비로소 팀으로 키웁니다.

절 요약

- 에이전트 팀은 오케스트레이터가 전문 서브에이전트들에게 일을 나눠 맡기고 결과를 종합하는 패턴입니다.

- 의존 없는 작업은 병렬로, 앞 결과가 필요한 작업은 순차로 실행됩니다.
- 에이전트를 늘리면 토큰 비용과 복잡도가 함께 늘므로, 나눠서 분명히 이득인 작업에만 적용합니다.

실습 과제

해보기: 전문 서브에이전트 2종 설계·검증하기

실습 프로젝트(예: `C:\Users\사용자\my-project`)에 코드 리뷰 담당과 문서화 담당 서브에이전트를 만들고, 자동 위임이 의도대로 일어나는지 시험합니다.

- **단계 1 (정의):** 프로젝트\`.claude\agents\` 폴더에 `code-reviewer.md` 와 `doc-writer.md` 를 만듭니다. 각 파일에 `name · description · tools · model` 을 적고, 본문에 역할 프롬프트(할 일·하지 말 것)를 씁니다.
- **단계 2 (최소 권한):** 리뷰어의 `tools` 는 `Read`, `Grep`, `Glob` (읽기 전용)으로, 문서화 담당은 `Read`, `Glob`, `Edit` 로 줍니다. `Bash` 는 두 에이전트 모두 주지 않습니다.
- **단계 3 (자동 위임 시험):** 에이전트를 지목하지 않고 "방금 고친 코드 좀 검토해 줘"라고 요청해, 메인이 `code-reviewer` 로 자동 위임하는지 봅니다.
- **단계 4 (다듬기):** 만약 빗나가거나 메인이 직접 처리하면, `description` 의 트리거 문구("...할 때 사용", "...에는 쓰지 않음")를 고쳐 다시 시험합니다.
- **점검:** 아래 표를 채워, 두 에이전트가 의도한 상황에서 각각 자동으로 불렀으면 성공입니다.

에이전트	의도한 트리거	시험한 요청	자동 위임 됐나
code-reviewer	코드 수정 직후 검증	(예) "이 변경 봐 줘"	예 / 아니오
doc-writer	코드 변경 후 문서 갱신	...	...

FAQ

Q1. 서브에이전트는 메인 대화 내용을 그대로 볼 수 있나요? → **A1.** 아닙니다. 서브에이전트는 독립 컨텍스트(빈 책상) 에서 시작하며, 메인 대화를 그대로 이어받지 않습니다. 메인이 위임할

때 건넨 지시와 그 에이전트의 역할 프롬프트만 가지고 일합니다. 그래서 위임할 때 대상 파일·목표·제약을 그 자체로 완결되게 적어 줘야 합니다.

Q2. 에이전트 파일만 만들면 자동으로 불리나요? 자꾸 안 불립니다. → A2. 자동 위임은 오직 `description` 을 근거로 판단합니다. "코드를 돕습니다"처럼 모호하면 빗나갑니다. **"언제 부르는가(트리거)"와 "언제 안 부르는가(경계)"** 를 또렷이 적으세요. 그래도 확실히 부르고 싶으면 "code-reviewer 서브에이전트로 ...해 줘"처럼 이름을 지목하는 명시적 호출을 쓰면 됩니다.

Q3. `tools` 를 안 적으면 어떻게 되나요? 안 적는 게 편한데요. → A3. 비워 두면 에이전트가 **메인의 모든 도구를 상속**합니다. 즉 읽기 전용으로 만들려던 리뷰어가 `Edit · Bash` 까지 주게 됩니다. 읽기·검색만 시킬 거라면 `tools: Read, Grep, Glob` 처럼 **반드시 명시해** 최소 권한으로 좁히세요. 이것이 안전과 집중 둘 다를 보장합니다.

Q4. 에이전트를 많이 만들수록 빠르고 좋아지나요? → A4. 항상 그렇지는 않습니다. 서브에이전트마다 자기 토큰을 쓰므로 **수를 늘리면 비용이 합산되어** 늘고, 종합·디버깅도 복잡해집니다. 멀티 에이전트는 "독립적이고 양 많은 병렬 작업"이나 "구현과 검증을 분리하는 작업"에서만 분명히 이득입니다. 작은 일은 단일 에이전트가 더 빠르고 쌉니다.

장 요약

이번 장에서는 일을 또 다른 에이전트에게 맡기는 **서브에이전트**를 배웠습니다. 핵심은 **독립 컨텍스트**입니다. 서브에이전트는 자기 책상에서 일하고 결과 요약만 돌려주므로, 탐색·검증 같은 작업의 잡음이 메인 대화를 오염시키지 않습니다. 에이전트는 `.claude\agents` 의 마크다운 파일로 정의하며, 위쪽 **프론트매터**(`name · description · tools · model`)와 아래쪽 **역할 프롬프트**로 구성됩니다. 자동 위임의 정확도는 전적으로 **description**에 달려 있어, "무엇을 하는가"보다 "언제 부르는가"를 적어야 라우팅이 정확하고, 확실할 때는 이름을 지목하는 명시적 호출로 고정합니다. 도구는 역할에 필요한 **최소 권한**만 주고(비우면 메인 도구를 모두 상속함을 기억), 역할 경계를 본문에 함께 적어 한 우물만 깊게 파게 합니다. 마지막으로 여러 에이전트를 **병렬·순차**로 엮으면 팀처럼 일하지만, 토큰·비용과 복잡도가 함께 늘므로 **나눠서 분명히 이득인 작업에만** 적용합니다.

핵심 키워드

서브에이전트(subagent), 독립 컨텍스트(separate context), 에이전트 정의(agent definition), 프론트매터(frontmatter), 역할 프롬프트(persona prompt), 자동 위임(auto-delegation), 명시적 호

출(explicit invocation), 최소 권한(least privilege), 역할 경계(role boundary), 에이전트 팀(agent team), 오케스트레이션(orchestration)

다음 장 미리보기

다음 장에서는 에이전트의 판단에 맡기지 않고 **코드로 강제하는 가드레일**, 훅(hook)을 다룹니다. 특정 도구를 쓰기 직전·직후 같은 정해진 시점에 내 스크립트를 자동 실행해, 민감 파일 편집을 차단하거나 저장 시 포매팅을 거는 확정적 자동화를 배웁니다.

Hooks: 이벤트 자동화와 가드레일

학습 목표 - 훅(hook)이 에이전트 판단 밖에서 코드로 강제되는 확정적 가드레일임을 설명할 수 있습니다. - 주요 훅 이벤트의 발화 시점을 에이전트 작업 흐름 타임라인 위에서 구분할 수 있습니다. - 훅의 JSON 입출력·종료 코드(exit code)·결정(decision) 계약을 이해하고 작업을 차단할 수 있습니다. - 실전 가드·포매팅·로깅 훅을 만들고 Windows 환경의 인코딩·경로 함정을 피할 수 있습니다.

앞 장들에서 권한 모드와 설정 파일(`settings.json`)을 다루며 "Claude Code가 무엇을 해도 되는지"를 미리 정해 두는 법을 배웠습니다. 그런데 권한 설정은 "이 도구는 허용/거부" 수준의 큰 칸막이입니다. "특정 폴더만은 절대 못 고치게 한다"거나 "파일을 고칠 때마다 자동으로 형식을 정리한다" 같은 **세밀하고 반드시 지켜져야 하는 규칙**은 권한 설정만으로는 표현하기 어렵습니다.

이 장에서 다루는 **훅(hook)** 이 바로 그 빈칸을 메웁니다. 훅은 정해진 순간에 Claude Code가 자동으로 실행하는 내 명령(스크립트)이며, 모델의 그날 기분이나 판단과 무관하게 **코드로 규칙을 강제**합니다. 이 장은 중급 단계인 만큼, 훅이 화면 뒤에서 어떤 데이터를 주고받으며 어떻게 작업을 막거나 통과시키는지 그 **내부 동작 원리**까지 들여다봅니다. 이 책은 Windows 11의 **PowerShell(파워셸)** 을 기준으로 하므로, 훅 스크립트와 경로도 Windows 기준으로 다룹니다.

훅(hook)이란 - 확정적 가드레일의 원리

도입

건물 현관의 센서등을 떠올려 봅시다. 사람이 지나가면 **반드시** 켜집니다. 경비원이 "오늘은 피곤하니 그냥 두자"라고 마음대로 끄지 않습니다. 정해진 조건에서 무조건 작동하는 이 확실함이 바로 훅의 핵심입니다. 이 절에서는 훅이 무엇이고, 왜 "모델에게 부탁하는 것"과 근본적으로 다른지를 봅니다.

핵심 개념 설명

훅 (hook)

훅(hook)이란, 특정 이벤트가 일어날 때 Claude Code가 자동으로 실행하는 외부 명령입니다. 여기서 '외부 명령'이란 내가 직접 작성한 작은 프로그램이나 스크립트(예: PowerShell 스크립트, 파이썬 스크립트)를 말합니다. 훅의 본질은 **모델의 판단이 아니라 코드로 규칙을 확정적으로 강제한다**는 데 있습니다.

이 차이를 일상에 빗대 보겠습니다. 모델에게 "secrets 폴더는 건드리지 마"라고 프롬프트나 CLAUDE.md 에 적어 두는 것은, 동료에게 "그 서랍은 열지 말아 줘"라고 *부탁*하는 것과 같습니다. 대부분은 잘 지키지만, 맥락이 길어지거나 지시를 잊으면 실수로 열 수도 있습니다. 반면 훅은 그 서랍에 **자물쇠**를 다는 것입니다. 부탁이 아니라 물리적 차단이라, 잊거나 봐주는 일이 없습니다.

확정적 자동화 (deterministic automation)

확정적 자동화(deterministic automation)란, 같은 조건이면 *언제나 같은 결과*가 나오도록 코드로 동작을 고정하는 방식입니다. '확정적'은 "결과가 미리 정해져 흔들리지 않는다"는 뜻입니다. 모델의 출력은 같은 질문에도 표현이 조금씩 달라지는 **확률적(probabilistic)** 성질이 있는 반면, 훅은 입력이 같으면 결과가 항상 같습니다.

여기서 중급 단계의 흔한 오해 하나를 짚겠습니다. **"훅도 모델이 상황을 봐서 건너뛸 수 있다"**는 생각입니다. 그렇지 않습니다. 훅은 모델이 호출하는 기능이 아니라, Claude Code라는 프로그램이 **이벤트 발생 시 직접 실행하는 바깥 장치**입니다. 모델은 훅이 돌아간다는 사실조차 선택할 수 없습니다. 그래서 "반드시"가 보장됩니다.

- **왜(why) 훅이 필요한가:** 안전과 일관성에 직결되는 규칙은 "대체로 지켜짐"으로는 부족합니다. 민감 파일 보호나 코드 형식 통일처럼 *한 번이라도 어긋나면 곤란한* 규칙은 코드로 못박아야 안심할 수 있습니다.
- **언제(when) 쓰나:** (1) 위험 작업을 무조건 막아야 할 때(가드레일), (2) 매번 반복되는 정리 작업을 자동화할 때(포매팅·로깅), (3) 팀 전체가 동일한 규칙을 따르게 강제할 때입니다.
- **어떻게(how) 동작하나:** settings.json 의 hooks 항목에 "어떤 이벤트에, 어떤 조건으로, 어떤 명령을 실행할지"를 등록합니다. 그 이벤트가 일어나면 Claude Code가 등록된 명령을 자동으로 돌립니다.

이렇게 등록합니다

훅은 설정 파일에 등록합니다. 다음은 파일을 편집([Edit])하기 직전에 guard.py 라는 내 스크립트를 실행하도록 등록한 최소 예시입니다(실제로 입력/작성하는 설정 내용이며, 동작 원리는 뒤 절에서 자세히 다룹니다).

```
// C:\Users\사용자\my-project\.claude\settings.json (참고용 설정 — 비실행)
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          { "type": "command", "command": "python C:\\Users\\사용자\\my-project\\.claude\\guard."
        ]
      }
    ]
  }
}
```

설정 키 설명

키	의미
hooks	훅 전체를 담는 묶음입니다
PreToolUse	"도구 실행 직전"이라는 이벤트 이름입니다(다음 절에서 전체 목록을 봅니다)
matcher	어떤 도구에 반응할지 고르는 패턴입니다. "Edit Write" 는 편집·쓰기 도구를 뜻합니다
type	훅의 종류입니다. "command" 는 외부 명령을 실행한다는 뜻입니다
command	실제로 실행할 명령입니다. 여기서는 파이썬으로 내 스크립트를 돌립니다

팁: 설정 파일은 두 종류가 있습니다. `.claude/settings.json` 은 git에 올려 **팀과 공유**하는 설정이고, `.claude/settings.local.json` 은 git에서 제외되어 **나만 쓰는** 개인 설정입니다. 팀 전체가 지켜야 할 가드 혹은 공유 파일에, 내 PC에만 맞춘 임시 혹은 로컬 파일에 두면 깔끔합니다.

흔한 실수: 훅을 "모델에게 주는 강한 지시" 정도로 여기기. 훅은 모델 바깥에서 도는 장치라, 프롬프트로 잘 타이르는 것과는 차원이 다른 강제력을 가집니다. 반대로 말하면, 훅을 잘못 등록하면 모델이 아무리 옳게 판단해도 작업이 막히므로, 등록 내용은 신중하게 설계해야 합니다.

절 요약

- 훅은 정해진 이벤트에 Claude Code가 자동 실행하는 외부 명령으로, 모델 판단이 아니라 코드로 규칙을 강제합니다.

- 확정적 자동화란 같은 조건이면 항상 같은 결과가 나오도록 동작을 고정하는 것이며, 혹은 모델이 건너뛸 수 없습니다.
- 혹은 `settings.json` 의 `hooks` 에 "이벤트·matcher·command"로 등록합니다.

혹 이벤트 전체 지도

(PreToolUse·PostToolUse·UserPromptSubmit 외)

도입

공장 컨베이어 벨트에는 지점마다 점검 센서가 달려 있습니다. 입구 센서, 가공 직전 센서, 가공 직후 센서, 출구 센서가 각자 정해진 위치에서만 작동합니다. 혹도 똑같습니다. Claude Code가 일하는 흐름에는 여러 길목이 있고, **이벤트(event)** 마다 다른 혹이 발화합니다. 이 절에서는 그 길목 전체를 지도처럼 펼쳐 봅니다.

핵심 개념 설명

혹 이벤트 (hook event)

혹 이벤트(hook event)란, 혹이 발화하는 시/점입니다. Claude Code의 한 작업은 "세션 시작 → 사용자가 입력 → 모델이 도구 호출 → 도구 실행 → 응답 표시 → 정지"라는 수명주기(lifecycle)를 따라 흐르는데, 각 길목마다 이름이 붙어 있습니다.

수명주기 이벤트 (lifecycle event)

수명주기 이벤트(lifecycle event)란, 위처럼 작업의 시작부터 끝까지 이어지는 흐름 위의 각 단계를 가리키는 이벤트입니다. 컨베이어 비유에서 "입구·가공 전·가공 후·출구"가 각각 하나의 수명주기 이벤트에 해당합니다. 어느 이벤트에 혹을 걸지에 따라 "막을 수 있는 것"과 "할 수 있는 일"이 달라집니다.

가장 많이 쓰이는 다섯 가지부터 익히면 됩니다. **PreToolUse**(도구 직전), **PostToolUse**(도구 직후), **UserPromptSubmit**(사용자 입력 제출 직후), **Stop**(작업 정지 시), **SubagentStop**(보조 에이전트 종료 시)입니다.

흑 이벤트 타임라인

한 턴(Turn) 내 흑 발화 순서 및 시점



□ 흑 이벤트 (등록 가능)

□ 클로드 내부 처리 (흑 없음)

※ PreToolUse 에서 비0 종료코드를 반환하면 도구 실행 자체가 차단된다.

- **왜(why) 시점이 중요한가**: 작업을 **막으려면** 그 작업이 일어나기 **전에** 발화하는 이벤트라야 합니다. 위험 명령 차단은 도구가 실행되기 전인 **PreToolUse** 라야 의미가 있고, 이미 끝난 뒤인 **PostToolUse** 로는 막을 수 없습니다.
- **언제(when) 어떤 이벤트를 고르나**: "차단"이 목적이면 **Pre** 계열, "뒷정리·기록"이 목적이면 **Post** 계열, **Stop** 계열을 고릅니다.
- **어떻게(how) 흐름을 읽나**: 위 다이어그램의 타임라인을 왼쪽(세션 시작)에서 오른쪽(정지)으로 따라가며, 내가 끼어들고 싶은 길목 하나를 고른다고 생각하면 됩니다.

주요 이벤트 한눈에 보기

흑 이벤트와 발화 시점

이벤트	발화 시점	작업을 막을 수 있나	대표 용도
SessionStart	세션이 시작될 때	아니오	시작 시 환경 점검·컨텍스트 주입
UserPromptSubmit	사용자가 입력을 제출한 직후	예	금지어 검사, 입력에 정보 덧 붙이기
PreToolUse	모델이 도구를 호출하기 직전	예	위험 명령·보호 경로 차단(가드레일)
PostToolUse	도구가 성공적으로 끝난 직후	아니오	자동 포매팅, 감사 로깅
Stop	메인 작업이 정지하려 할 때	예	마무리 검증("테스트 통과 전엔 못 멈춤")
SubagentStop	보조 에이전트가 끝났을 때	예	보조 작업 결과 검증·정리
MessageDisplay	응답을 화면에 표시하기 직전	아니오	표시 직전 후처리·마스킹

MessageDisplay 나 SubagentStop 같은 이벤트는 비교적 나중에 추가된 길목입니다.

MessageDisplay 는 모델의 답이 화면에 *그러지기* 직전이라, 예를 들어 응답에 섞여 나올 수 있는 민감 문자열을 가리는 후처리에 쓸 수 있습니다. SubagentStop 은 큰 작업을 잘게 나눠 처리하는 **보조 에이전트(subagent)** 가 자기 몫을 끝냈을 때 발화하므로, 그 중간 결과를 점검하는 데 적합합니다.

"막을 수 있나" 열을 읽는 법

구분	의미
막을 수 있음(Pre · Stop · Submit 계열)	발화 시점이 해당 행동 전 이라, 혹은 "거부"하면 그 행동이 일어나지 않습니다
막을 수 없음 (Post · Display · SessionStart)	이미 일어난 일에 대한 <i>반응</i> 이라, 되돌릴 수는 없고 뒤처리만 가능합니다

팁: 새 훅을 설계할 때 먼저 던질 질문은 "막고 싶은가, 기록하고 싶은가"입니다. 막고 싶다면 그 행동 *이전* 이벤트(`PreToolUse` 등)를, 기록·정리만 원한다면 *이후* 이벤트(`PostToolUse` 등)를 고릅니다. 이 한 가지 판단만으로 이벤트 선택의 절반이 끝납니다.

흔한 실수: "훅은 도구 실행 전에만 작동한다"고 생각해 모든 것을 `PreToolUse` 에 몰아넣기.

`PreToolUse` 에서 포매팅을 시도하면 정작 파일이 아직 바뀌기 전이라 헛일이 됩니다. 포매팅·로깅처럼 결과를 다루는 일은 반드시 `PostToolUse` 에 뒤야 합니다.

절 요약

- 훅 이벤트는 작업 수명주기의 각 길목(세션 시작·입력 제출·도구 전후·정지·표시)을 가리킵니다.
- 작업을 막으려면 그 행동 *전에* 발화하는 이벤트(`PreToolUse` 등)를, 기록·정리는 *후* 이벤트(`PostToolUse` 등)를 씁니다.
- `MessageDisplay` · `SubagentStop` 처럼 나중에 추가된 이벤트는 표시 후처리·보조 작업 검증에 쓰입니다.

훅 입출력 계약 - JSON·exit code·decision

도입

검문소에 도착한 차량은 운전자가 신분증과 행선지를 적은 서류를 내밀고, 검문관은 그 서류를 보고 "통과" 또는 "정지" 도장을 돌려줍니다. 훅과 Claude Code 사이에도 똑같은 약속된 절차가 있습니다. 이 절에서는 둘이 정확히 어떤 형식으로 데이터를 주고받아 작업을 진행하거나 차단하는지, 그 **입출력 계약**을 들여다봅니다.

핵심 개념 설명

훅 입출력 계약 (hook I/O contract)

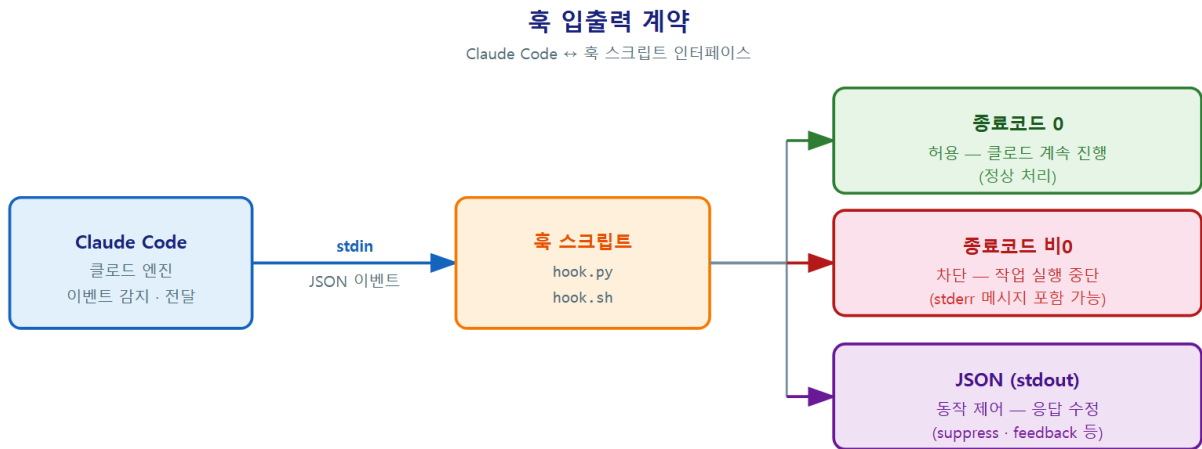
훅 입출력 계약(hook I/O contract) 이란, Claude Code와 훅 스크립트가 데이터를 주고받는 약속된 규약입니다. 흐름은 세 단계입니다. (1) Claude Code가 훅에게 상황 정보를 **JSON으로 stdin(표준 입력)** 에 흘려보냅니다. (2) 훅은 그 JSON을 읽고 판단합니다. (3) 훅이 **종료 코드(exit code)** 와 **stdout(표준 출력)의 JSON** 으로 결과를 돌려줍니다.

여기서 'stdin'은 프로그램이 입력을 받는 통로, 'stdout'은 출력을 내보내는 통로를 말합니다. '종료 코드'는 프로그램이 끝날 때 남기는 숫자 신호로, 보통 0은 정상, 0이 아니면 문제를 뜻합니다.

차단 결정 (deny decision)

차단 결정(deny decision)이란, 혹은 "이 작업을 하지 못하게 막아라"라고 Claude Code에 돌려주는 응답입니다. 혹은 차단을 표현하는 방법은 두 가지입니다.

- **종료 코드 2로 끝내기:** 혹은 `exit 2`로 종료하면 Claude Code는 작업을 막고, 혹은 `stderr`(표준 오류 출력)에 적은 메시지를 모델에게 전달해 "왜 막혔는지" 알려 줍니다. 빠르고 간단한 차단 방식입니다.
- **stdout에 결정 JSON 내보내기:** 더 세밀하게 제어하려면, 혹은 종료 코드 0으로 끝내되 `stdout`에 결정을 담은 JSON을 출력합니다. `PreToolUse`에서는 `permissionDecision` 값으로 "allow" (허용)·"deny" (차단)·"ask" (사용자에게 물어봄)를 줄 수 있습니다.



※ 혹은은 Claude Code가 stdin으로 JSON 이벤트를 전달하여 실행하며, 종료코드 또는 JSON stdout으로 동작을 제어한다.

여기서 중급 단계의 흔한 오해를 짚겠습니다. "형식과 무관하게 그냥 메시지를 출력하면 차단된다"는 생각입니다. 그렇지 않습니다. Claude Code는 **정해진 형식의 JSON만** 결정으로 해석합니다. 형식이 어긋난 출력은 그냥 무시되거나 잡음으로 취급됩니다. 또 한 가지, **stdout의 JSON은 종료 코드가 0일 때만** 해석됩니다. 차단 JSON을 내보내면서 동시에 `exit 2`로 끝내면 신호가 충돌하니, JSON 방식을 쓸 때는 `exit 0`으로 끝내야 합니다.

- **왜(why) JSON으로 주고받나:** 사람이 보는 글이 아니라 프로그램이 정확히 해석할 데이터라야 "막아라/통과시켜라"를 오해 없이 전달할 수 있기 때문입니다.
- **언제(when) 두 방식을 나눠 쓰나:** 단순히 막기만 하면 되면 `exit 2`가 간편하고, 허용·차단·되묻기를 골라야 하거나 모델에게 구체적 사유를 전하고 싶으면 결정 JSON을 씁니다.

- **어떻게(how) 사유를 전하나:** 결정 JSON의 `permissionDecisionReason` 에 적은 문장이 모델에게 전달되어, 모델이 다른 방법을 찾도록 안내합니다.

주고받는 데이터의 모양

먼저 Claude Code가 흑에게 **stdin**으로 건네는 JSON의 예상 모양입니다(편집 도구를 쓰려 할 때).

```
// Claude Code가 PreToolUse 흑의 stdin으로 보내는 입력 (예상 형태)
{
  "hook_event_name": "PreToolUse",
  "tool_name": "Edit",
  "tool_input": {
    "file_path": "C:\\Users\\사용자\\my-project\\secrets\\api_key.txt",
    "old_string": "...",
    "new_string": "..."
  }
}
```

흑은 이 JSON에서 `tool_input.file_path` 를 읽어 판단한 뒤, 차단하려면 **stdout**으로 다음과 같은 결정 JSON을 내보냅니다.

```
// 흑이 stdout으로 돌려주는 차단 결정 (exit code 0 과 함께)
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason": "보호 경로(secrets)는 흑 정책상 편집할 수 없습니다."
  }
}
```

이 결정을 받으면 Claude Code는 편집을 진행하지 않고, `permissionDecisionReason` 의 문장을 모델에게 전달합니다.

입출력 계약의 핵심 요소

요소	방향	의미
stdin JSON	Claude Code → 혹	어떤 이벤트·도구·인자인지 알려 주는 상황 정보
tool_input.file_path	입력 안의 값	혹이 판단 근거로 읽는 대상 경로
종료 코드 0	혹 → Claude Code	정상 종료. stdout JSON을 결정으로 해석합니다
종료 코드 2	혹 → Claude Code	작업 차단. stderr 메시지를 모델에 전달합니다
permissionDecision	stdout JSON 안의 값	allow · deny · ask 중 하나로 진행 여부 결정
permissionDecisionReason	stdout JSON 안의 값	차단·허용 사유. 모델에게 전달되는 설명

차단 신호 두 가지 (요약) - 간단 차단: 혹을 `exit 2`로 종료 + 사유는 stderr에 출력 - 세밀 차단: `exit 0` + stdout에 `{"hookSpecificOutput": {"hookEventName": "...", "permissionDecision": "deny", "permissionDecisionReason": "..."} }`

흔한 실수: stdout에 진단용 `print` 문이나 디버그 메시지를 섞어 내보내기. `PreToolUse`의 결정 JSON을 쓸 때 stdout에는 **JSON 객체만** 있어야 합니다. "지금 검사 중..." 같은 안내 문구를 같이 출력하면 JSON 해석이 깨집니다. 디버그 메시지는 반드시 stderr로 보내십시오.

주의: 종료 코드와 결정 JSON을 뒤섞지 마십시오. `exit 2`(즉시 차단)와 `permissionDecision: "deny"`(JSON 차단)는 서로 다른 경로입니다. JSON 방식을 쓰기로 했으면 종료 코드는 `0`으로 두어야 stdout이 해석됩니다. 둘을 동시에 쓰면 의도와 다르게 동작할 수 있습니다.

절 요약

- 혹 입출력 계약은 "stdin JSON으로 상황을 받고, 종료 코드와 stdout JSON으로 결과를 돌려주는" 약속입니다.
- 차단은 `exit 2`(간단)나 결정 JSON의 `permissionDecision: "deny"`(세밀) 두 방식으로 표현합니다.
- 결정 JSON은 종료 코드 `0`일 때만 해석되며, stdout에는 JSON만 있어야 합니다(디버그는 stderr로).

실전 훅 - 민감파일 차단·저장 시 포매팅·로깅

도입

지금까지 배운 이벤트와 입출력 계약을 실제 업무 규칙으로 묶어 보겠습니다. 현장에서 가장 자주 쓰는 훅은 세 가지입니다. **위험한 편집을 막는 가드 훅**, **고친 파일을 자동으로 정리하는 포매팅 훅**, **누가 무엇을 고쳤는지 남기는 로깅 훅**입니다. 이 절에서는 각각을 Windows 기준 설정과 스크립트로 살펴봅니다.

핵심 개념 설명

가드 훅 (guard hook)

가드 훅(guard hook)이란, 위험하거나 금지된 작업을 `PreToolUse` 시점에서 차단하는 훅입니다. '가드(guard)'는 "지키는 문지기"라는 뜻으로, 정해진 조건에 맞는 작업이 들어오면 통과시키지 않습니다. 대표 용도는 `secrets · .env` 같은 **보호 경로** 편집 차단입니다.

포매팅 훅 (format hook)

포매팅 훅(format hook)이란, 파일이 편집된 직후(`PostToolUse`)에 코드 정리 도구(포매퍼)를 자동으로 돌려 형식을 통일하는 훅입니다. 사람이 매번 "정리해 줘"라고 시키지 않아도, 편집이 끝나면 늘 같은 형식으로 맞춰집니다. 이것이 앞에서 말한 '일관성'의 대표 사례입니다.

- **왜(why) 가드와 포매팅을 분리하나:** 막는 일은 행동 전, 정리는 행동 후에 해야 하기 때문입니다. 그래서 가드 훅은 `PreToolUse`, 포매팅 훅은 `PostToolUse`에 등록합니다.
- **언제(when) 로깅을 더하나:** 팀 작업이나 규정 준수가 필요할 때, "언제 어떤 파일이 편집됐는지"를 파일에 남겨 두면 나중에 추적할 수 있습니다. 로깅 역시 결과를 다루므로 `PostToolUse`에 둡니다.
- **어떻게(how) 묶나:** 한 `settings.json`에 `PreToolUse` (가드)와 `PostToolUse` (포매팅·로깅)를 함께 등록해, 편집 한 번에 "검사 → 적용 → 정리·기록"이 자동으로 이어지게 만듭니다.

시나리오 1 - 보호 경로 편집 차단

다음은 `secrets · .env`를 건드리려는 편집을 막는 가드 훅의 등록 설정과 스크립트입니다. 먼저 설정입니다.

```
// .claude/settings.json — 가드 훅 등록 (비실행 — 참고용)
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write|MultiEdit",
        "hooks": [
          { "type": "command", "command": "python .claude/hooks/guard_paths.py" }
        ]
      }
    ]
  }
}
```

이 설정이 가리키는 파이썬 스크립트는 다음과 같은 모양입니다. stdin으로 들어온 JSON에서 대상 경로를 읽어, 보호 경로면 차단 JSON을 내보냅니다.

```
# .claude/hooks/guard_paths.py (참고용 스크립트)
import sys
import json

# 보호할 경로 조각 목록
PROTECTED = ["secrets", ".env"]

def main() -> None:
    # 표준 입력으로 들어온 JSON을 읽습니다
    try:
        data = json.load(sys.stdin)
    except json.JSONDecodeError:
        # 입력이 깨졌으면 막지 않고 통과(정상 종료)
        sys.exit(0)

    path = data.get("tool_input", {}).get("file_path", "")
    lowered = path.replace("\\", "/").lower()

    # 보호 경로가 포함되면 차단 결정 JSON을 stdout으로 출력
    if any(p in lowered for p in PROTECTED):
        decision = {
            "hookSpecificOutput": {
                "hookEventName": "PreToolUse",
                "permissionDecision": "deny",
                "permissionDecisionReason": "보호 경로(secrets/.env)는 편집할 수 없습니다.",
            }
        }
        print(json.dumps(decision, ensure_ascii=False))
        sys.exit(0) # JSON 방식이므로 정상 종료

    # 보호 경로가 아니면 아무것도 출력하지 않고 통과
    sys.exit(0)

if __name__ == "__main__":
    main()
```

이 훅이 작동하면 화면에는 다음과 비슷하게 차단 결과가 보입니다.

```
> .env 파일에 새 API 키를 추가해 줘

[Edit] .env 를 수정하려고 합니다 ...
혹에 의해 차단되었습니다: 보호 경로(secrets/.env)는 편집할 수 없습니다.
대신 .env.example 에 키 이름만 적어 두는 방법을 제안합니다.
```

가드 스크립트가 한 일

단계	코드	설명
입력 읽기	<code>json.load(sys.stdin)</code>	Claude Code가 보낸 상황 JSON을 읽습니다
경로 추출	<code>data.get("tool_input", {}).get("file_path", "")</code>	편집 대상 경로를 꺼냅니다
경로 정규화	<code>path.replace("\\", "/").lower()</code>	역슬래시·대소문자 차이를 없애 비교를 안정화합니다
차단 판단	<code>any(p in lowered for p in PROTECTED)</code>	보호 조각이 경로에 들어 있는지 확인합니다
차단 출력	<code>print(json.dumps(decision, ...))</code>	차단 결정 JSON을 내보냅니다

시나리오 2 - 편집 후 포매팅·감사 로깅

다음은 편집이 끝난 뒤 포맷터를 돌리고, 동시에 편집 내역을 로그 파일에 남기는 설정입니다. `PostToolUse` 에 두 개의 훅을 나란히 등록합니다.

```
// .claude/settings.json — 편집 후 포매팅 + 로깅 (비실행 — 참고용)
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write|MultiEdit",
        "hooks": [
          { "type": "command", "command": "python .claude/hooks/format_py.py" },
          { "type": "command", "command": "python .claude/hooks/audit_log.py" }
        ]
      }
    ]
  }
}
```

로깅 스크립트는 차단할 필요가 없으므로, 결정 JSON 없이 파일에만 기록하고 정상 종료합니다.


```
# .claude/hooks/audit_log.py (참고용 스크립트)
import sys
import json
from datetime import datetime
from pathlib import Path

def main() -> None:
    try:
        data = json.load(sys.stdin)
    except json.JSONDecodeError:
        sys.exit(0)

    path = data.get("tool_input", {}).get("file_path", "(알 수 없음)")
    stamp = datetime.now().isoformat(timespec="seconds")

    # 로그 파일에 한 줄 append (UTF-8 고정)
    log = Path(".claude/edit_audit.log")
    with log.open("a", encoding="utf-8") as f:
        f.write(f"{stamp}\tEDIT\t{path}\n")

    sys.exit(0)

if __name__ == "__main__":
    main()
```

```
# .claude/edit_audit.log 에 쌓이는 기록 (예상 모양)
2026-06-16T14:02:11 EDIT    C:/Users/사용자/my-project/src/app.py
2026-06-16T14:05:48 EDIT    C:/Users/사용자/my-project/src/util.py
```

베스트 프랙티스: 가드 혹은 보호 경로 목록은 스크립트 안에 하드코딩하기보다 한곳에 모아 관리하는 편이 안전합니다. 팀이 공유하는 `.claude/settings.json` 에 가드 혹은 보호 경로 규칙을 코드 리뷰로 함께 관리하면 "누가 몰래 보호 목록을 비활성화"하는 일을 막을 수 있습니다.

흔한 실수: 포매팅 혹은 포매터가 없을 때(설치 안 됨) 오류로 종료해 작업 전체를 망가뜨리기. `PostToolUse` 혹은 비정상 종료하면 불필요한 잡음이 모델에 전달됩니다. 포매터 실행은 "있으면 돌리고 없으면 조용히 통과"하도록 방어적으로 작성하십시오.

절 요약

- 가드 혹은 `PreToolUse` 에서 보호 경로 편집을 차단하고, 포매팅·로깅 혹은 `PostToolUse` 에서 결과를 정리·기록합니다.
- 한 `settings.json` 에 `Pre` (검사)와 `Post` (정리·기록)를 함께 등록하면 편집 한 번에 자동으로 이어집니다.
- 로깅은 차단이 목적이 아니므로 결정 JSON 없이 파일에 기록하고 정상 종료합니다.

Windows에서 훅 작성 시 주의(PowerShell·경로·인코딩)

도입

같은 요리법이라도 주방 설비가 다르면 결과가 달라집니다. 훅도 마찬가지입니다. 인터넷에서 본 훅 예시는 대개 macOS·리눅스의 셸(bash)을 기준으로 적혀 있어, Windows에서 그대로 붙여 넣으면 동작하지 않는 경우가 많습니다. 이 절에서는 Windows 11에서 훅을 만들 때 부딪히는 셸·경로·인코딩 함정을 정리합니다.

핵심 개념 설명

크로스플랫폼 훅 (cross-platform hook)

크로스플랫폼 훅(cross-platform hook)이란, Windows·macOS·리눅스 어디서나 동작하도록 작성한 훅입니다. 가장 확실한 방법은 셸에 의존하는 명령(bash 전용 문법) 대신, 어느 OS에나 똑같이 깔리는 **파이썬 같은 언어로 스크립트를 작성**하는 것입니다. 이 책의 예시가 PowerShell 명령을 직접 길게 쓰지 않고 `python .claude/hooks/...py`로 호출한 이유가 바로 이것입니다.

인코딩·경로 함정 (encoding/path pitfalls)

인코딩·경로 함정(encoding/path pitfalls)이란, Windows 특유의 문자 인코딩과 경로 표기 때문에 훅이 깨지거나 오작동하는 문제들입니다. 두 가지가 대표적입니다.

- **인코딩:** Windows의 일부 콘솔은 기본 인코딩이 **CP949**(한국어 Windows의 기본 문자 집합)라, 한글이 섞인 JSON을 읽고 쓸 때 깨질 수 있습니다. 스크립트에서 파일을 열 때 `encoding="utf-8"`을 명시하고, 콘솔 출력 인코딩도 UTF-8로 고정하면 안전합니다.
- **경로 구분자:** Windows 경로는 역슬래시(\)를 쓰는데, 역슬래시는 JSON·문자열에서 특수 문자로 해석됩니다. JSON에 경로를 적을 때는 `C:\\Users\\...`처럼 **두 번씩** 쓰거나, 비교할 때는 슬래시(/)로 바꿔 처리하는 편이 안전합니다(앞 절 가드 스크립트의 `path.replace("\\", "/")`가 그 예입니다).
- **왜(why) Windows에서 더 주의하나:** 대다수 훅 예시가 bash·UTF-8 환경을 전제로 하므로, Windows의 기본값(CP949, 역슬래시)과 어긋나 조용히 깨지는 일이 잦기 때문입니다.
- **언제(when) 문제가 드러나나:** 한글 경로·한글 사유 메시지를 다룰 때, 그리고 경로를 문자열로 비교·기록할 때 주로 드러납니다.
- **어떻게(how) 예방하나:** (1) 셸 명령 대신 파이썬 스크립트로 작성, (2) 파일 입출력에 `encoding="utf-8"` 명시, (3) 경로는 정규화해서 비교합니다.

이렇게 다룹니다

PowerShell로 직접 명령을 거는 경우, 인용과 인코딩을 명시해야 합니다. 다음은 Windows에서 콘솔 인코딩 문제를 줄이기 위해 파이썬을 UTF-8 모드로 호출하는 설정 예시입니다.

```
// .claude/settings.json — Windows에서 인코딩 안전하게 호출 (비실행 — 참고용)
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          { "type": "command", "command": "python -X utf8 .claude/hooks/guard_paths.py" }
        ]
      }
    ]
  }
}
```

PowerShell 스크립트 (.ps1)로 훅을 작성한다면, 다음처럼 출력 인코딩을 UTF-8로 먼저 고정합니다.

```
# .claude/hooks/guard.ps1 (참고용 — 출력 인코딩 고정 예시)
# 콘솔 출력 인코딩을 UTF-8로 고정해 한글 깨짐을 막습니다
[Console]::OutputEncoding = [System.Text.Encoding]::UTF8

# 표준 입력으로 들어온 JSON을 읽습니다
$raw = [Console]::In.ReadToEnd()
$data = $raw | ConvertFrom-Json
$path = $data.tool_input.file_path

if ($path -match "secrets|\.env") {
  # 간단 차단: stderr에 사유를 적고 종료 코드 2로 종료
  [Console]::Error.WriteLine("보호 경로는 편집할 수 없습니다.")
  exit 2
}
exit 0
```

```
# 인코딩을 고정하지 않았을 때 흔히 보이는 깨짐 (예상)
# 혹은 사유: ?췘췘 췘췘췘... (한글이 깨져 추적이 어려워짐)
```

Windows 훅 점검표

항목	권장 처리	이유
셸 의존	파이썬 스크립트로 호출	bash 전용 문법은 PowerShell에서 안 됩니다
파일 입출력	<code>encoding="utf-8"</code> 명시	CP949 기본값에서 한글 깨짐 방지
콘솔 출력	<code>python -X utf8</code> 또는 <code>OutputEncoding=UTF8</code>	차단 사유의 한글 보존
JSON 안의 경로	<code>\\</code> 로 이스케이프	역슬래시가 특수 문자로 해석됨
경로 비교	<code>/</code> 로 정규화 후 비교	구분자·대소문자 차이 흡수

주의: 혹 명령에 적은 스크립트 경로가 틀리면 혹이 조용히 실행되지 않아, 가드가 작동하는 줄 알았는데 안 막히는 위험한 상태가 됩니다. 등록 직후 일부러 보호 경로를 건드려 보고 **실제로 차단되는지 한 번 확인**하십시오. 가드 혹은 "등록했다"가 아니라 "막히는 걸 봤다"까지 확인해야 신뢰할 수 있습니다.

팁: 혹이 의심스럽게 동작하면 스크립트에서 디버그 정보를 **stderr로만** 출력하도록 추가해 흐름을 추적하십시오. stdout은 결정 JSON 전용이므로 디버그를 섞으면 안 되지만, stderr는 자유롭게 써도 결정 해석을 깨뜨리지 않습니다.

절 요약

- Windows에서는 bash 전용 예시가 자주 깨지므로, 파이썬 스크립트로 작성해 크로스플랫폼으로 만드는 편이 안전합니다.
- 인코딩은 `encoding="utf-8"` · `-X utf8` 로 고정하고, 경로는 `\\` 이스케이프와 `/` 정규화로 다룹니다.
- 가드 혹은 등록 후 일부러 차단을 유발해 실제로 막히는지 확인해야 신뢰할 수 있습니다.

실습 과제

해보기: 보호 경로 차단 + 편집 후 로깅 혹 만들기

내 실습 프로젝트에 가드 혹은 로깅 혹은 직접 등록하고, Windows에서 인코딩 경로 문제 없이 동작하는지 시험합니다.

- **단계 1 (준비):** 프로젝트 루트에 `.claude/hooks` 폴더를 만들고, 이 장의 `guard_paths.py` 와 `audit_log.py` 를 참고해 두 스크립트를 작성합니다. 보호 경로 목록에는 `secrets` 와 `.env` 를 넣습니다.
- **단계 2 (등록):** `.claude/settings.json` 의 `hooks` 에 `PreToolUse` (가드)와 `PostToolUse` (로깅) 를 등록합니다. Windows 인코딩을 위해 호출은 `python -X utf8 ...` 형태로 적습니다.
- **단계 3 (차단 확인):** Claude Code에게 "`.env` 파일에 줄 하나 추가해 줘"라고 요청하고, **실제로 차단되는지** 그리고 차단 사유 한글이 깨지지 않는지 봅니다.
- **단계 4 (로깅 확인):** 보호되지 않은 일반 파일을 하나 고치게 한 뒤, `.claude/edit_audit.log` 에 시각과 경로가 한 줄 쌓였는지 확인합니다.
- **점검:** 아래 표를 채울 수 있으면 성공입니다.

시험	기대 결과	실제 결과
<code>.env</code> 편집 요청	차단됨(사유 표시)	...
일반 파일 편집	통과 + 로그 1줄 추가	...
차단 사유 한글	깨지지 않음	...

FAQ

Q1. 혹은 등록했는데 아무 일도 안 일어납니다. 왜 그런가요? → A1. 가장 흔한 원인은 (1) `command` 에 적은 스크립트 경로가 틀렸거나, (2) `matcher` 패턴이 실제 도구 이름과 맞지 않는 경우입니다. 편집을 막으려면 `matcher` 에 `Edit·Write` 가 포함돼야 합니다. 또 스크립트가 조용히 오류로 끝나도 혹은 안 도는 것처럼 보이므로, `stderr`로 디버그 메시지를 찍어 흐름을 확인하십시오.

Q2. `exit 2`로 막는 것과 결정 JSON으로 막는 것, 어느 쪽을 써야 하나요? → A2. 단순히 "막기만" 하면 되면 `exit 2`가 간편합니다(사유는 `stderr`로). 허용·차단·되묻기(`ask`)를 골라야 하거나 모델에게 구체적 사유를 전달해 다른 길을 찾게 하고 싶으면 결정 JSON(`permissionDecision`)을 쓰십시오. 단, JSON 방식은 종료 코드를 `0`으로 두고 `stdout`에 JSON만 출력해야 합니다.

Q3. 혹은 차단해도 모델이 우회해서 결국 해 버리지 않나요? → A3. 아닙니다. 혹은 모델 바깥에서 Claude Code 프로그램이 직접 실행하는 장치라, 모델이 끄거나 건너뛸 수 없습니다.

`PreToolUse` 에서 차단된 도구 호출은 실행 자체가 일어나지 않습니다. 모델은 차단 사유를 받아 다른 방법을 제안할 뿐, 막힌 그 작업을 그대로 강행할 수는 없습니다.

장 요약

이번 장에서는 **훅(hook)** 으로 Claude Code의 동작에 확정적 규칙을 심는 법을 익혔습니다. 훅은 모델에게 부탁하는 프롬프트가 아니라, 정해진 이벤트에 Claude Code가 자동 실행하는 외부 명령이라 **모델이 건너뛸 수 없는 가드레일**이 됩니다. 작업 수명주기에는 여러 길목이 있어, 막고 싶으면 행동 전 이벤트(`PreToolUse` 등)를, 정리·기록은 후 이벤트(`PostToolUse` 등)를 고릅니다. 훅과 Claude Code는 **stdin JSON으로 상황을 받고, 종료 코드와 stdout JSON으로 결과를 돌려주는** 입출력 계약을 따르며, 차단은 `exit 2` 또는 `permissionDecision: "deny"` 로 표현합니다. 실전에서는 보호 경로를 막는 가드 훅, 편집 후 형식을 맞추는 포매팅 훅, 내역을 남기는 로깅 훅을 함께 등록해 "검사 → 적용 → 정리·기록"을 자동화합니다. 마지막으로 Windows에서는 셸 의존을 피해 파이썬으로 작성하고, 인코딩을 UTF-8로 고정하며, 경로를 이스케이프·정규화하는 습관으로 함정을 피합니다.

핵심 키워드

훅(hook), 확정적 자동화(deterministic automation), 훅 이벤트(hook event), 수명주기 이벤트(lifecycle event), 훅 입출력 계약(hook I/O contract), 차단 결정(deny decision), 가드 훅(guard hook), 포매팅 훅(format hook)

다음 장 미리보기

다음 장에서는 여러 개의 보조 에이전트(subagent)에게 일을 나눠 맡기고 조율하는 법을 다룹니다. 이 장에서 만난 `SubagentStop` 훅이 그 보조 작업의 마무리를 어떻게 점검하는지도 함께 이어집니다.

MCP 서버 연동과 직접 작성

학습 목표 - MCP(Model Context Protocol)가 외부 도구를 연결하는 표준 어댑터임을 설명할 수 있습니다. - stdio·SSE·HTTP 전송과 local·project·user 범위를 구분해 서버를 추가할 수 있습니다. - 연결된 MCP 도구(tool)·리소스(resource)·프롬프트(prompt)를 실제로 사용할 수 있습니다. - 최소 MCP 서버를 직접 만들고 Claude Code에 등록해 호출까지 확인할 수 있습니다. - 신뢰·권한·비밀값 원칙을 적용해 MCP를 안전하게 운영할 수 있습니다.

지금까지 Claude Code는 주로 **내 PC 안의 파일과 명령**을 다뤘습니다. 하지만 실제 업무는 PC 밖에도 있습니다. 깃허브(GitHub)의 이슈, 슬랙(Slack)의 메시지, 회사 데이터베이스(database)의 기록 같은 것들입니다. 이 장에서는 Claude Code가 이런 **바깥세상의 도구·데이터와 표준 방식**으로 연결되는 **통로인 MCP**를 다룹니다.

이 장은 중급 단계입니다. 단순히 "어떻게 켜는가"를 넘어, **MCP가 내부에서 어떻게 동작하는지**, 등록 정보가 **어디에 저장되는지**, 그리고 직접 서버를 만들 때 무엇을 정의해야 하는지까지 들여다봅니다. 이 책은 Windows 11의 **PowerShell(파워셸)**을 기준으로 하며, 명령은 `PS C:\Users\사용자\my-project>` 같은 프롬프트에서 입력합니다.

MCP(Model Context Protocol)의 구조와 의미

도입

새 노트북을 사면 마우스, 키보드, 외장하드를 따로 공부하지 않아도 USB 단자에 꽂기만 하면 동작합니다. **USB라는 공통 규격** 덕분입니다. MCP도 똑같은 발상입니다. 깃허브든 데이터베이스든 브라우저든, **하나의 표준 규격**으로 Claude Code에 꽂아 쓰게 해 줍니다. 이 절에서는 그 규격이 무엇이고, 무엇을 주고받는지 살펴봅니다.

핵심 개념 설명

MCP (Model Context Protocol)

MCP(Model Context Protocol)란, Claude Code 같은 AI 도구가 외부 도구·데이터에 **표준 방식**으로 연결하기 위한 **개방형 약속(프로토콜)**입니다. 우리말로 풀면 "모델이 바깥 정보를 가져다"

쓰기 위한 공통 연결 규격"입니다. 여기서 '프로토콜(protocol)'은 "서로 어떻게 대화할지 미리 정해 둔 규칙"을 뜻합니다.

핵심은 **표준**이라는 데 있습니다. MCP가 없다면 깃허브 연결, 슬랙 연결, 데이터베이스 연결을 **각각 다른 방식**으로 만들어야 합니다. MCP가 있으면 모두 같은 규격을 따르므로, 한 번 배운 연결 방법을 어디서나 똑같이 씁니다.

여기서 가장 흔한 오해 하나를 짚겠습니다. **"MCP는 특정 회사 전용 연결 방식"**이라는 생각입니다. 그렇지 않습니다. MCP는 누구나 규격에 맞춰 서버를 만들 수 있는 **개방형 표준**이라서, 공식 서버뿐 아니라 내가 직접 만든 서버도 같은 방식으로 연결됩니다(이 장 4절에서 직접 만듭니다).

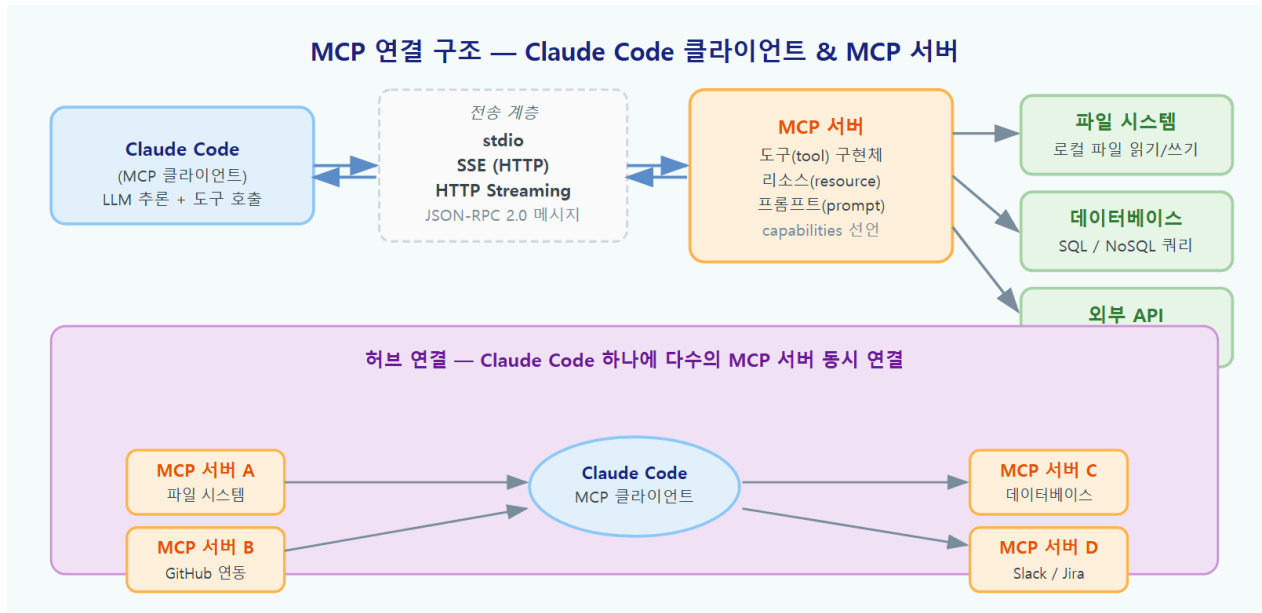
표준 어댑터 (standard adapter)

연결 구조에는 두 역할이 있습니다. **MCP 클라이언트(client)**와 **MCP 서버(server)**입니다.

Claude Code가 클라이언트(부탁하는 쪽)이고, 깃허브·데이터베이스 같은 바깥 기능을 규격에 맞춰 제공하는 프로그램이 서버(들어주는 쪽)입니다. 클라이언트 하나가 여러 서버에 동시에 꽂힐 수 있어서, 멀티탭에 여러 기기를 꽂듯 도구를 눌러 갑니다.

MCP 서버는 Claude Code에게 세 가지를 **노출(제공)** 합니다.

- **도구(tool)**: Claude Code가 **실행할 수 있는 동작**입니다. 예: "이 저장소에 이슈를 만든다", "이 표를 조회한다". 함수(function)를 호출하듯 인자를 넣어 부릅니다.
- **리소스(resource)**: Claude Code가 **읽어 올 수 있는 데이터**입니다. 예: 위키 문서, 설정 파일 내용. 파일을 첨부하듯 대화 맥락에 넣습니다.
- **프롬프트(prompt)**: 서버가 미리 만들어 둔 **재사용 가능한 지시문** 틀입니다. 슬래시 커맨드처럼 불러 씁니다.
- **왜(why) 중요한가**: 표준이 있으면 도구를 새로 붙일 때마다 연결 방법을 다시 배우지 않아도 됩니다. 생태계 전체가 같은 규격을 공유하므로 선택지가 빠르게 늘어납니다.
- **언제(when) 쓰나**: Claude Code가 내 PC 밖의 정보를 읽거나, 외부 서비스에 동작을 일으켜야 할 때입니다. 예: 이슈 트래커 조회, 사내 문서 검색, 브라우저 자동화.
- **어떻게(how) 동작하나**: 클라이언트와 서버는 **JSON-RPC**라는 형식으로 메시지를 주고받습니다. 사람이 직접 다룰 일은 거의 없고, "서버를 등록하면 그 서버의 도구가 Claude Code의 도구 목록에 더해진다"고 이해하면 충분합니다.



이렇게 보입니다

서버를 등록한 뒤 세션 안에서 `/mcp` 를 입력하면, 지금 어떤 MCP 서버가 연결돼 있고 무엇을 제공하는지 보여 줍니다. 아래는 예상 화면입니다(실제 입력이 아니라 구조를 보여 주는 예시입니다).

```
PS C:\Users\사용자\my-project> claude
> /mcp
```

MCP 서버 (2개 연결됨)

```
github      연결됨   도구 12 · 리소스 0 · 프롬프트 2
glossary    연결됨   도구 1  · 리소스 1 · 프롬프트 0
```

서버를 선택하면 제공하는 도구·리소스 목록을 볼 수 있습니다.

연결된 서버의 도구는 `mcp__서버이름__도구이름` 형태의 이름을 가집니다. 예를 들어 깃허브 서버의 이슈 생성 도구는 `mcp__github__create_issue` 처럼 표시됩니다.

`/mcp` 화면에서 읽어야 할 것

표시	의미
github / glossary	등록된 MCP 서버의 이름입니다
연결됨	서버가 정상적으로 켜져 응답하는 상태입니다
도구 12	그 서버가 노출한 실행 가능한 동작(tool) 개수입니다
리소스 1	읽어 올 수 있는 데이터(resource) 개수입니다
프롬프트 2	미리 만들어 둔 지시문 틀(prompt) 개수입니다

팁: 새 서버를 붙인 직후에는 항상 `/mcp` 로 **연결됨** 상태인지 먼저 확인합니다. "연결 안 됨"으로 보이면 도구를 호출해 봐야 헛수고이므로, 이 단계에서 문제를 먼저 잡는 편이 빠릅니다.

절 요약

- MCP는 외부 도구·데이터를 표준 규격으로 연결하는 개방형 프로토콜이며, USB 단자에 비유할 수 있습니다.
- Claude Code는 클라이언트, 외부 기능을 제공하는 프로그램은 서버이며, 서버는 도구·리소스·프롬프트를 노출합니다.
- `/mcp` 로 연결 상태와 제공물 개수를 확인할 수 있습니다.

MCP 서버 추가와 설정 범위(stdio·SSE·HTTP)

도입

같은 메모라도 어디에 붙이느냐에 따라 보는 사람이 달라집니다. **내 책상** 위에만 두면 나만 보고, **팀 게시판**에 붙이면 팀 전체가 보며, **회사 공지**로 올리면 모든 부서가 봅니다. MCP 서버 등록도 똑같이 "어디에 저장하느냐(범위)"를 고릅니다. 이 절에서는 **전송 방식(어떻게 연결하는가)** 과 **설정 범위(어디에 저장하는가)** 라는 두 축을 함께 살펴봅니다.

핵심 개념 설명

전송 방식 (transport)

전송 방식(transport)이란, Claude Code(클라이언트)와 MCP 서버가 **물리적으로 어떤 통로로 메시지를 주고받는지**를 정하는 방법입니다. 같은 MCP 규격이라도 통로는 두 종류로 나뉩니다.

- **stdio(표준 입출력)**: 가장 흔한 방식입니다. Claude Code가 **내 PC에서 서버 프로그램을 직접 실행**하고, 그 프로그램의 입력(stdin)과 출력(stdout)으로 메시지를 흘려보냅니다. 별도 서버 주소가 필요 없고, 로컬 파일·명령에 접근하는 도구에 적합합니다.
- **SSE / HTTP(원격)**: 이미 인터넷 어딘가에서 돌고 있는 서버에 **주소(URL)로 접속**합니다. SSE(Server-Sent Events)와 HTTP는 원격 서버가 응답을 흘려보내는 방식으로, 클라우드 서비스나 팀이 함께 쓰는 공용 서버에 적합합니다.

쉽게 말해 **stdio**는 "**내 PC에서 프로그램을 켜서 직접 대화**", **SSE-HTTP**는 "**멀리 있는 서버에 전화로 연결**" 이라고 이해하면 됩니다.

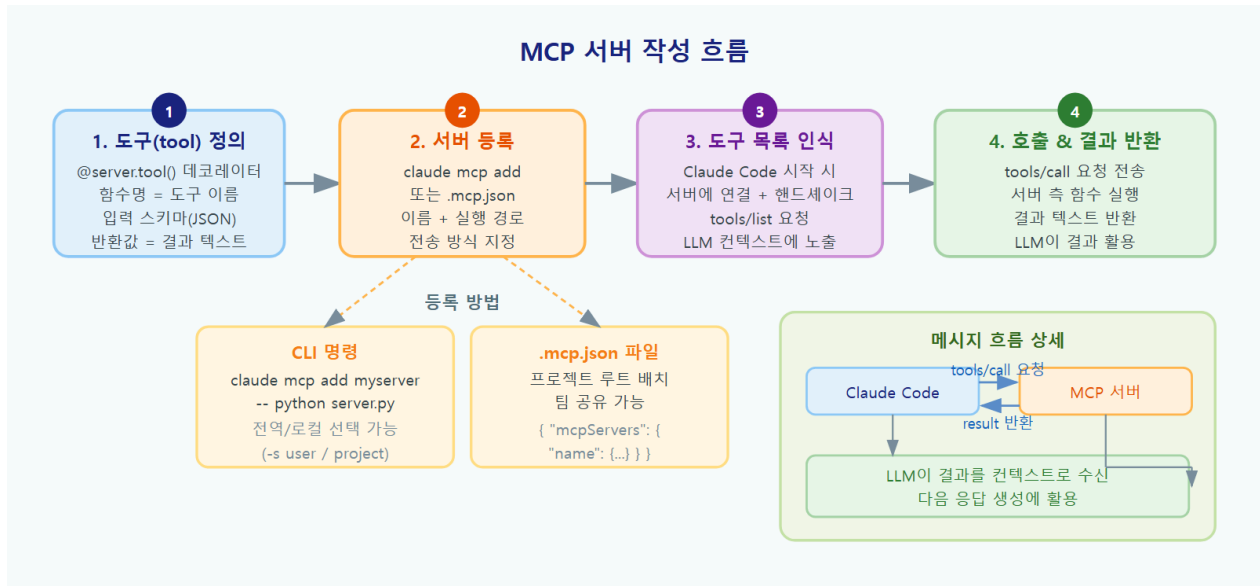
설정 범위 (config scope)

설정 범위(config scope)란, MCP 서버 등록 정보를 **어느 위치에 저장해 누가 쓰게 할지** 정하는 구분입니다. Claude Code는 세 범위를 둡니다.

- **local(로컬, 기본값)**: *나만, 지금 이 프로젝트에서만* 쓰는 등록입니다. 홈 폴더의 개인 설정 (C:\Users\사용자\.claude.json) 안에 현재 프로젝트용으로 저장되며, 저장소에 공유되지 않습니다.
- **project(프로젝트)**: *이 프로젝트를 함께 쓰는 모두가* 공유하는 등록입니다. 프로젝트 루트의 `.mcp.json` 파일에 저장되어 Git으로 커밋·공유됩니다. 팀이 같은 도구를 쓰게 만들 때 씁니다.
- **user(사용자/전역)**: *내 모든 프로젝트에서* 쓰는 등록입니다. 어느 폴더에서 Claude Code를 열어도 따라옵니다.

여기서 흔한 오해는 "**한 번 등록한 서버는 모든 프로젝트에서 항상 보인다**" 는 생각입니다. 그렇지 않습니다. 기본값인 local·project 범위는 **그 프로젝트 안에서만** 보입니다. 모든 프로젝트에서 쓰고 싶다면 user 범위로 등록해야 합니다.

- **왜(why) 범위를 나누나**: 개인 실험용 서버를 팀 저장소에 섞으면 혼란스럽고, 반대로 팀 표준 서버를 개인 설정에만 두면 동료가 못 씁니다. 범위를 나눠 **공유 대상을 통제**합니다.
- **언제(when) 무엇을 쓰나**: 혼자 시험할 때는 local, 팀과 공유할 때는 project, 어디서나 쓰는 개인 도구는 user입니다.
- **어떻게(how) 정하나**: `claude mcp add` 명령에 `--scope` 옵션으로 지정합니다. 생략하면 local입니다.



이렇게 설정/실행합니다

서버 추가는 PowerShell에서 `claude mcp add` 명령으로 합니다. 먼저 **원격(HTTP) 서버**를 **project** 범위로 추가하는 예입니다. 팀이 함께 쓸 서버이므로 `.mcp.json` 에 저장되도록 합니다.

```
# 원격 HTTP 서버를 project 범위로 등록 (팀 공유: .mcp.json 에 저장)
claude mcp add --scope project --transport http github https://example.com/mcp
```

다음은 **로컬 프로그램(stdio) 서버**를 추가하는 예입니다. `--` 뒤에 서버를 실행할 명령을 적습니다. stdio는 기본 전송 방식이라 `--transport` 를 생략해도 됩니다.

```
# 로컬 stdio 서버 등록: -- 뒤는 서버를 실행하는 명령
claude mcp add glossary -- python C:\mcp\glossary_server.py
```

project 범위로 등록하면 프로젝트 루트에 `.mcp.json` 파일이 만들어집니다. 직접 열어 보면 다음과 비슷합니다(참고용 설정이며, 손으로 편집해도 됩니다).

```
{
  "mcpServers": {
    "github": {
      "type": "http",
      "url": "https://example.com/mcp"
    },
    "glossary": {
      "type": "stdio",
      "command": "python",
      "args": ["C:\\mcp\\glossary_server.py"]
    }
  }
}
```

등록 후 확인과 삭제는 다음 명령으로 합니다.

```
claude mcp list          # 등록된 서버 목록 보기
claude mcp get glossary  # 특정 서버의 상세 설정 보기
claude mcp remove glossary # 등록 해제
```

`.mcp.json` 설정 키 설명

키	의미
<code>mcpServers</code>	이 프로젝트에서 공유하는 서버들의 모음입니다
<code>type</code>	전송 방식입니다(<code>stdio</code> 로컬 프로세스 / <code>http-sse</code> 원격)
<code>url</code>	원격 서버일 때 접속할 주소입니다
<code>command</code>	stdio 서버를 실행할 프로그램입니다(예: <code>python</code>)
<code>args</code>	실행 프로그램에 넘길 인자 목록입니다(경로의 <code>\</code> 는 JSON에서 <code>\\</code> 로 적습니다)

Windows에서 흔한 실수: `.mcp.json` 의 경로에서 역슬래시(`\`)를 한 개만 쓰기. JSON에서는 `\` 가 특수문자라서 `C:\mcp\...` 는 `C:\\mcp\\...` 처럼 **두 개씩** 적어야 합니다. 한 개만 쓰면 서버가 "연결 안 됨"으로 뜹니다. `claude mcp add` 명령으로 등록하면 이 변환을 도구가 알아서 처리해 줍니다.

베스트 프랙티스: 팀과 공유할 서버는 `--scope project` 로 등록해 `.mcp.json` 을 저장소에 커밋합니다. 그러면 동료가 저장소를 받기만 해도 같은 도구가 갖춰집니다. 단, 개인 실험용 서버는 local로 두어 팀 설정을 깨끗하게 유지합니다.

절 요약

- 전송 방식은 통로를 정하며, stdio(내 PC에서 프로그램 실행)와 SSE-HTTP(원격 주소 접속)로 나뉩니다.
- 설정 범위는 저장 위치와 공유 대상을 정하며, local(나·이 프로젝트)·project(팀 공유, .mcp.json)·user(내 전체 프로젝트)가 있습니다.
- `claude mcp add --scope ... --transport ...` 로 등록하고, `list · get · remove` 로 관리합니다.

MCP 도구·리소스·프롬프트 사용하기

도입

도구를 새로 연결했다면, 이제 실제로 써 볼 차례입니다. 좋은 소식은 **특별한 명령을 외울 필요가 없다**는 점입니다. 평소처럼 한국어로 부탁하면 Claude Code가 적절한 MCP 도구를 알아서 골라 씁니다. 이 절에서는 도구를 호출하고, 리소스를 대화에 끌어오고, 서버가 제공한 프롬프트를 슬래시 커맨드로 쓰는 세 가지 사용법을 봅니다.

핵심 개념 설명

MCP 도구 (MCP tool)

MCP 도구(MCP tool)란, 연결된 서버가 노출한 **실행 가능한 동작**입니다. Claude Code 입장에서는 내장 도구(파일 읽기·편집·명령 실행)에 **새 도구가 더 붙은 것**과 같습니다. 사용자는 도구 이름을 직접 칠 필요 없이 "깃허브에 이슈 만들어 줘"처럼 자연어로 부탁하면, Claude Code가 `mcp_github__create_issue` 같은 도구를 골라 호출합니다.

중요한 점은 MCP 도구도 **권한 모델의 통제를 받는다**는 것입니다(4장 내용). 외부에 영향을 주는 도구라면, 실행 전에 권한 요청 창이 떠서 사람이 승인합니다. 즉 도구가 늘어나도 "사람이 위험한 동작을 막을 수 있는" 안전장치는 그대로 작동합니다.

MCP 리소스 (MCP resource)

MCP 리소스(MCP resource)란, 서버가 노출한 **읽어 올 수 있는 데이터**입니다. 도구가 "동작"이라면 리소스는 "읽을거리"입니다. 위키 문서, 설정 값, 데이터 한 묶음 같은 것이 리소스가 됩니다.

다. 사용자는 파일을 첨부하듯 **@** 기호로 리소스를 대화 맥락에 끌어옵니다. 예:

`@github:issue://123` 처럼 적으면 그 이슈 내용이 대화에 들어옵니다.

서버가 제공한 **프롬프트(prompt)** 는 슬래시 커맨드 형태로 쓰입니다. `/mcp__서버이름__프롬프트` 이름 으로 호출하며, 서버가 미리 다듬어 둔 지시문이 대화에 펼쳐집니다.

- **왜(why) 편한가:** 도구·리소스·프롬프트가 모두 **Claude Code의 기존 사용법(자연어 부탁, @ 첨부, / 명령)**에 그대로 녹아든다는 점입니다. 새 문법을 배우지 않아도 됩니다.
- **언제(when) 무엇을 쓰나:** 외부에 동작을 일으킬 때는 도구(자연어 부탁), 외부 데이터를 읽어 판단 근거로 쓸 때는 리소스(@), 정해진 작업 틀을 반복할 때는 프롬프트(/)입니다.
- **어떻게(how) 확인하나:** 어떤 도구가 호출됐는지는 세션 화면의 `[mcp__...]` 표시로 보입니다.

이렇게 사용합니다

먼저 **도구 호출**입니다. 자연어로 부탁하면 Claude Code가 MCP 도구를 골라 쓰고, 외부에 영향을 주는 동작이라 권한 요청이 함께 뜹니다.

> 이 저장소에 "로그인 버튼이 모바일에서 안 눌린다"는 버그 이슈를 만들어 줘

[mcp__github__create_issue] 다음 이슈를 생성하려고 합니다:

제목: 로그인 버튼이 모바일에서 안 눌린다

저장소: my-org/my-project

실행할까요? [1. 이번만 허용 2. 항상 허용 3. 거부]

다음은 **리소스 @** 로 끌어와 판단 근거로 삼는 경우입니다.

> @github:issue://123 이 이슈 내용을 읽고, 무엇을 고쳐야 하는지 요약해 줘

[mcp__github] 리소스 issue://123 을 읽었습니다

이 이슈는 결제 화면에서 합계가 음수로 표시되는 문제입니다.

원인은 할인 금액을 두 번 빼는 계산으로 보이며, 다음 두 곳을 확인하면 됩니다 ...

마지막으로 **서버가 제공한 프롬프트**를 슬래시 커맨드로 부르는 경우입니다.

> /mcp__github__open_pr_review

(서버가 정의한 PR 리뷰용 지시문이 대화에 펼쳐집니다)

세 가지 사용 방식 비교

제공물	부르는 법	쓰임	권한 요청
도구(tool)	자연어로 부탁	외부에 동작 일으키기(이슈 생성 등)	영향 있으면 뚝
리소스(resource)	@서버:주소 로 첨부	외부 데이터를 맥락에 넣기	보통 읽기라 안 뚝
프롬프트(prompt)	/mcp__서버__이름	정해진 지시문 틀 재사용	내용에 따라 다름

팁: 어떤 도구를 쓸 수 있는지 헛갈리면 그냥 한국어로 부탁해 보십시오. Claude Code가 연결된 MCP 도구 중 알맞은 것을 고릅니다. 적절한 도구가 없으면 "그 작업을 할 도구가 연결돼 있지 않다"고 알려 주므로, 그때 서버를 추가하면 됩니다.

흔한 실수: 외부에 동작을 일으키는 MCP 도구에 무심코 "항상 허용"을 고르기. 이슈 생성 정도는 괜찮지만, 메시지 발송·데이터 삭제처럼 되돌리기 어려운 도구는 매번 내용을 보고 이번만 허용으로 다루는 편이 안전합니다.

절 요약

- MCP 도구는 자연어 부탁으로, 리소스는 @ 첨부로, 프롬프트는 /mcp__서버__이름 으로 사용합니다.
- MCP 도구도 권한 모델의 통제를 받아, 영향 있는 동작 전에는 승인 창이 뜹니다.
- 새 문법 없이 기존 사용법(부탁·첨부·슬래시)에 그대로 녹아듭니다.

직접 만드는 MCP 서버 - 최소 구현

도입

공식 서버를 붙여 쓰는 것만으로도 강력하지만, 진짜 가치는 **내 업무에만 있는 기능**을 붙일 때 나옵니다. 예를 들어 "사내 약어를 풀어 주는 도구"는 어떤 공식 서버에도 없습니다. 다행히 MCP는 개방형 표준이라 **누구나 작은 서버를 만들 수 있습니다**. 이 절에서는 도구 하나를 노출하는 최소 MCP 서버를 만들고 Claude Code에 등록하는 흐름을 봅니다.

핵심 개념 설명

MCP 서버 작성 (building an MCP server)

MCP 서버 작성(building an MCP server) 이란, MCP SDK(소프트웨어 개발 키트, 만들기 도구 모음) 를 써서 도구·리소스를 정의하고, 전송 방식을 정해 Claude Code에 등록함으로써 **직접 만든 기능을 노출**하는 작업입니다. SDK는 파이썬(Python)·타입스크립트(TypeScript) 등으로 제공되며, 규격에 맞는 메시지 처리를 대신 해 주므로 우리는 **"도구가 무슨 일을 하는지"** 만 적으면 됩니다.

여기서 흔한 오해는 **"MCP 서버는 큰 인프라(서버 장비)가 있어야 만든다"** 는 생각입니다. 그렇지 않습니다. stdio 전송을 쓰면 **내 PC에서 돌아가는 작은 프로그램 파일 하나**가 곧 MCP 서버입니다. 별도 호스팅이나 네트워크 설정이 필요 없습니다.

최소 서버를 만드는 단계는 세 가지입니다.

1. **도구를 정의한다**: 함수 하나를 만들고 "무엇을 받아 무엇을 돌려주는지"와 한 줄 설명을 적습니다. 이 설명을 보고 Claude Code가 도구를 고릅니다.
2. **서버를 실행 형태로 만든다**: stdio로 동작하도록 설정합니다.
3. **Claude Code에 등록한다**: `claude mcp add` 로 이 프로그램을 실행하도록 연결합니다.
4. **왜(why) 직접 만드나**: 사내 용어, 내부 API, 회사 규칙 조회처럼 **우리에게만 있는 기능**은 직접 만들어야 Claude Code가 정확히 답합니다.
5. **언제(when) 만드나**: 같은 질문·조회를 반복하는데 적당한 공식 서버가 없을 때입니다.
6. **어떻게(how) 시작하나**: 도구 하나짜리 최소 서버부터 만들어 동작을 확인한 뒤, 필요하면 도구를 늘려 갑니다.

이렇게 작성/등록합니다

다음은 **사내 용어를 풀어 주는 도구** 하나를 노출하는 최소 MCP 서버입니다. 파이썬 MCP SDK의 간편 인터페이스를 사용했습니다. 아래는 구조를 설명하기 위한 **참고용(비실행) 예시**입니다.

```
# glossary_server.py - 사내 용어를 알려 주는 최소 MCP 서버 (참고용 예시)
from mcp.server.fastmcp import FastMCP

# 서버 이름을 지정합니다. Claude Code 도구 목록에 이 이름이 붙습니다.
mcp = FastMCP("glossary")

@mcp.tool()
def lookup_term(term: str) -> str:
    """사내 약어의 뜻을 돌려줍니다."""
    glossary = {
        "WIP": "처리 중인 작업(Work In Progress)",
        "MAU": "월간 활성 사용자 수(Monthly Active Users)",
    }
    # 등록된 약어면 뜻을, 아니면 안내 문구를 돌려줍니다.
    return glossary.get(term, "등록되지 않은 용어입니다.")

if __name__ == "__main__":
    # 기본 전송 방식인 stdio 로 서버를 실행합니다.
    mcp.run()
```

이 파일을 `C:\mcp\glossary_server.py` 에 두었다면, PowerShell에서 다음과 같이 등록합니다.

```
# 직접 만든 stdio 서버를 등록 (이 프로젝트에서 사용)
claude mcp add glossary -- python C:\mcp\glossary_server.py
```

등록 후 세션에서 자연어로 부탁하면, Claude Code가 이 도구를 골라 호출합니다.

```
> WIP가 무슨 뜻인지 사내 용어로 알려 줘

[mcp__glossary__lookup_term] term="WIP" 로 호출했습니다
WIP는 "처리 중인 작업(Work In Progress)"을 뜻합니다.
```

최소 서버 코드 구성 요소

코드 조각	하는 일
<code>FastMCP("glossary")</code>	서버를 만들고 이름을 붙입니다
<code>@mcp.tool()</code>	바로 아래 함수를 MCP 도구로 노출합니다
<code>def lookup_term(term: str) -> str</code>	도구가 받는 인자(<code>term</code>)와 돌려줄 형태를 정합니다
<code>"""사내 약어의 뜻을..."""</code>	도구 설명입니다. Claude Code가 이 설명을 보고 도구를 고릅니다
<code>mcp.run()</code>	stdio 전송으로 서버를 가동합니다

베스트 프랙티스: 도구 설명(함수 바로 아래 문서 문자열)을 **또렷하게** 적습니다. 이 설명이 7장의 서브에이전트 `description` 처럼 **자동 선택의 근거**가 됩니다. "용어를 조회한다"보다 "사내 약어의 정식 명칭과 뜻을 돌려준다"가 더 정확히 호출됩니다.

흔한 실수: 등록 명령에서 `--` 를 빼먹기. `claude mcp add glossary python`

`C:\mcp\glossary_server.py` 처럼 적으면 `python ...` 을 옵션으로 오해할 수 있습니다. **서버 실행 명령 앞에는 반드시 `--` 를 두어**, "여기부터는 서버를 실행할 명령"임을 알려야 합니다.

절 요약

- 최소 MCP 서버는 도구 정의 → stdio 실행 → `claude mcp add` 등록 세 단계로 만듭니다.
- 큰 인프라 없이 내 PC에서 돌아가는 작은 프로그램 하나면 충분합니다.
- 도구 설명을 또렷이 적어야 Claude Code가 정확히 호출합니다.

MCP 보안 - 신뢰·권한·비밀값

도입

MCP는 Claude Code의 손을 바깥세상까지 뻗게 해 줍니다. 힘이 커진 만큼 **위험도 함께** 커집니다. 검증되지 않은 서버는 내 데이터를 엉뚱한 곳으로 보낼 수 있고, 비밀번호 같은 값을 잘못 다루면 새어 나갑니다. 이 절에서는 MCP를 안전하게 쓰는 세 가지 원칙, 즉 **신뢰·권한·비밀값**을 다룹니다.

핵심 개념 설명

MCP 신뢰 (server trust)

MCP 신뢰(server trust)란, 연결할 MCP 서버가 **믿을 만한 출처인지 먼저 확인하는 원칙**입니다. MCP 서버는 내 PC에서 프로그램을 실행하거나(stdio), 내 데이터를 외부로 보낼 수 있는(원격) 힘을 가집니다. 그래서 **출처가 분명하고 코드를 확인할 수 있는 서버만** 연결해야 합니다.

특히 project 범위로 등록된 서버(`.mcp.json`)는 저장소를 받은 모두에게 적용됩니다. 그래서 Claude Code는 새 `.mcp.json` 서버를 처음 마주치면 **"신뢰하고 사용할지" 한 번 확인**합니다. 모르는 저장소를 열었을 때 이 확인 창이 뜨면, 서버 목록을 살펴보고 의심스러우면 거부합니다.

비밀값 주입 (secret injection)

비밀값(secret)이란 API 키, 토큰(token, 인증용 출입증), 비밀번호처럼 **새어 나가면 안 되는 값**입니다. **비밀값 주입(secret injection)**이란 이런 값을 코드나 설정 파일에 직접 적지 않고, **환경변수(environment variable)**를 통해 서버에 전달하는 방법입니다. 환경변수는 운영체제가 들고 있는 "이름=값" 보관함으로, 파일에 박아 두지 않고도 프로그램에 값을 넘길 수 있습니다.

여기서 가장 위험한 실수는 **토큰을 `.mcp.json`에 그대로 적어 저장소에 커밋하기**입니다. 저장소에 한 번 올라간 비밀값은 기록에 영구히 남아, 사실상 유출된 것으로 봐야 합니다. 그래서 비밀값은 항상 **설정에는 변수 이름만, 실제 값은 환경변수로** 둡니다.

- **왜(why) 조심하나:** MCP 서버는 외부 연결과 코드 실행 능력을 갖기 때문에, 악의적이거나 허술한 서버는 데이터 유출·시스템 손상으로 이어질 수 있습니다.
- **언제(when) 점검하나:** 새 서버를 등록할 때(출처 확인), `.mcp.json` 신뢰 확인 창이 뜰 때, 토큰이 필요한 서버를 설정할 때입니다.
- **어떻게(how) 적용하나:** 출처가 분명한 서버만 쓰고, 토큰은 환경변수로 주입하며, 4장에서 배운 권한 규칙으로 MCP 도구에도 경계를 둡니다.

이렇게 설정합니다

비밀값은 환경변수로 둡니다. 먼저 PowerShell에서 환경변수를 설정합니다(아래 `$env:` 방식은 현재 세션에만 적용됩니다).

```
# 현재 PowerShell 세션에 토큰을 환경변수로 설정 (파일에 적지 않음)
$env:GITHUB_TOKEN = "여기에_실제_토큰"
```

그런 다음 등록 시 **값이 아니라 변수 이름을 참조**하도록 합니다. `.mcp.json` 에는 다음처럼 환경 변수 참조만 남깁니다(실제 토큰 문자열은 들어가지 않습니다).

```
{
  "mcpServers": {
    "github": {
      "type": "stdio",
      "command": "some-mcp-server",
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    }
  }
}
```

이렇게 하면 `.mcp.json` 을 저장소에 커밋해도 **토큰 값은 어디에도 적히지 않습니다**. 새 서버를 처음 만나면 다음과 비슷한 신뢰 확인 창이 뜹니다.

이 프로젝트의 `.mcp.json` 에 새 MCP 서버가 있습니다:

github (stdio) some-mcp-server

이 서버를 신뢰하고 사용하시겠습니까?

1. 이번 세션에서 사용 2. 항상 신뢰 3. 사용 안 함

비밀값 설정에서 확인할 것

항목	안전한 방식
토큰 값	설정 파일이 아니라 환경변수로 둡니다
<code>.mcp.json</code> 의 env	실제 값이 아니라 <code>\${변수이름}</code> 참조만 적습니다
저장소 커밋	변수 참조만 있으므로 커밋해도 값이 새지 않습니다
신뢰 확인 창	모르는 서버면 "사용 안 함" 을 고릅니다

주의: 신원이 불분명한 MCP 서버, 특히 코드를 확인할 수 없는 서버는 함부로 "항상 신뢰"로 등록하지 마십시오. MCP 서버는 내 PC에서 명령을 실행하거나 데이터를 외부로 보낼 수 있습니다. 출처가 의심스러우면 **사용 안 함**을 고르고, 꼭 필요하면 코드와 권한을 확인한 뒤 연결합니다.

베스트 프랙티스: 4장에서 배운 권한 규칙을 MCP 도구에도 적용합니다. 외부에 동작을 일으키는 MCP 도구는 기본적으로 "매번 묻기"로 두고, 읽기 전용 조회 도구만 선별적으로 자동 허용합

니다. 비밀값은 10장에서 다룰 환경변수·관리형 정책과 함께 팀 차원에서 일관되게 관리하면 더 안전합니다.

절 요약

- MCP 서버는 강력한 만큼 위험하므로, 출처가 분명한 서버만 신뢰해 연결합니다.
- 비밀값은 설정 파일에 적지 않고 환경변수로 주입하며, `.mcp.json` 에는 `${변수이름}` 참조만 둡니다.
- 새 `.mcp.json` 서버의 신뢰 확인 창에서 모르는 서버는 거부하고, MCP 도구에도 권한 경계를 둡니다.

실습 과제

해보기: MCP 서버 연결 + 간단 서버 작성하기

공식 서버를 붙여 보고, 이어서 도구 하나짜리 최소 서버를 직접 만들어 연결까지 해 봅니다.

- **단계 1 (공식 서버 연결):** 신뢰할 수 있는 공식 MCP 서버 하나를 골라 `claude mcp add --scope project ...` 로 등록합니다. `/mcp` 로 "연결됨" 상태를 확인하고, 제공하는 도구를 한 가지 자연어로 호출해 봅니다.
- **단계 2 (범위 확인):** 등록 후 만들어진 `.mcp.json` 을 열어 어떤 키가 들어갔는지 살펴보고, `claude mcp list` 로 범위가 project로 잡혔는지 확인합니다.
- **단계 3 (내 서버 작성):** 도구 하나(예: 사내 용어 조회)를 노출하는 최소 MCP 서버 파일을 만들고 `claude mcp add 이름 -- python 경로` 로 등록합니다. 자연어로 부탁해 도구가 호출되는지 확인합니다.
- **단계 4 (비밀값 안전):** 토큰이 필요한 서버라면 값을 환경변수로 두고 `.mcp.json` 에는 `${변수이름}` 참조만 남겼는지 점검합니다.
- **점검:** 아래 표를 채워 보고, 각 서버의 전송 방식·범위·신뢰 처리 방법을 적을 수 있으면 성공입니다.

서버	전송 방식(stdio/http)	범위(local/project/user)	비밀값 처리
(공식 서버)	...	project	...
(내 서버)	stdio	...	환경변수 / 없음

FAQ

Q1. MCP 서버를 추가하면 Claude Code가 느려지거나 위험해지지 않나요? → A1. 서버를 등록해도 그 도구를 실제로 호출할 때만 동작하므로 평소 속도에는 거의 영향이 없습니다. 위험은 "어떤 서버를 믿느냐"에 달려 있습니다. 출처가 분명한 서버만 연결하고, 외부에 영향을 주는 도구는 권한 요청으로 매번 확인하면 안전하게 쓸 수 있습니다.

Q2. local·project·user 범위 중 무엇을 골라야 할지 모르겠습니다. → A2. 기준은 "누가 쓸 것인가"입니다. 혼자 시험할 때는 local, 이 프로젝트를 함께 쓰는 팀과 공유하려면 project(`.mcp.json` 커밋), 내 모든 프로젝트에서 쓰고 싶은 개인 도구는 user입니다. 헛갈리면 기본값인 local로 시작해 보고, 공유가 필요해지면 옮기면 됩니다.

Q3. 프로그래밍을 잘 못해도 MCP 서버를 만들 수 있나요? → A3. 도구 하나짜리 최소 서버는 함수 하나와 짧은 설명만 적으면 되므로, 4절 예시 수준의 코드면 충분합니다. SDK가 규격 처리를 대신해 주기 때문입니다. 더 나아가 "이런 도구를 노출하는 MCP 서버를 만들어 줘"라고 Claude Code에게 직접 부탁해 초안을 받은 뒤, 내용을 확인하며 다듬는 방법도 좋습니다.

장 요약

이번 장에서는 Claude Code가 바깥세상과 연결되는 통로인 **MCP(Model Context Protocol)**를 다뤘습니다. MCP는 USB 단자처럼 어떤 외부 도구든 같은 규격으로 꽂아 쓰게 해 주는 **개방형 표준**이며, Claude Code(클라이언트)가 여러 서버에 연결되어 **도구·리소스·프롬프트**를 받아 씁니다. 서버를 붙일 때는 통로를 정하는 **전송 방식(stdio·SSE·HTTP)**과 저장 위치·공유 대상을 정하는 **설정 범위(local·project·user)**를 함께 골라 `claude mcp add`로 등록합니다. 연결된 도구는 자연어 부탁으로, 리소스는 `@`첨부로, 프롬프트는 슬래시 커맨드로 쓰며, MCP 도구도 권한 모델의 통제를 받습니다. 나아가 SDK로 **도구 하나짜리 최소 서버**를 직접 만들어 내 업무 기능을 노출할 수 있습니다. 마지막으로 MCP는 힘이 큰 만큼 위험도 크므로, **신뢰할 수 있는 서버만 연결하고, 비밀값은 환경변수로 주입하며, 권한 경계를 두는 세 원칙**으로 안전하게 운영합니다.

핵심 키워드

MCP(Model Context Protocol), 표준 어댑터(standard adapter), 전송 방식(transport), 설정 범위(config scope), MCP 도구(MCP tool), MCP 리소스(MCP resource), MCP 서버 작성(building an MCP server), MCP 신뢰(server trust), 비밀값 주입(secret injection)

다음 장 미리보기

다음 장부터는 실무 단계로 들어갑니다. 먼저 `settings.json` 의 전체 구조와 병합 우선순위, 환경변수·비밀값 관리, 그리고 조직 차원에서 권한을 강제하는 **관리형 정책**까지 다뤄, 개인을 넘어 팀과 조직이 Claude Code를 안전하게 공유하는 거버넌스를 배웁니다.

settings.json 거버넌스·환경변수·팀 공유 정책

학습 목표 - settings.json의 네 가지 종류와 병합 우선순위를 설명하고 최종 유효 설정을 예측할 수 있습니다. - 비밀값(토큰 등)을 settings.json이 아니라 환경변수(environment variable)로 안전하게 다루는 원칙을 적용할 수 있습니다. - 관리형 정책(managed policy)으로 조직 차원의 허용·거부를 강제하는 거버넌스(governance) 구조를 이해합니다. - 기본 모델·상태 줄(status line)·출력 스타일(output style)·도구 권한을 설정으로 다듬을 수 있습니다. - 공유 설정 (settings.json)과 로컬 오버라이드(settings.local.json)를 분리해 팀과 안전하게 공유할 수 있습니다.

지금까지는 한 사람이 Claude Code를 쓰는 관점이었습니다. 이 장부터는 **여러 사람이 같은 프로젝트에서, 같은 규칙으로** 쓰는 실무 관점으로 넘어갑니다. 핵심 도구는 settings.json 하나입니다. 권한 규칙(4장), 메모리(5장), 훅(8장)에서 부분부분 만났던 그 파일을, 이번에는 **거버넌스(governance)** 관점에서 통째로 다룹니다. 거버넌스란 "누가 무엇을 할 수 있는지를 조직 차원에서 정해 강제하는 통제 체계"를 뜻합니다.

이 장은 실무 밴드입니다. 단순히 "어떤 키가 있다"를 넘어, **여러 설정 파일이 충돌할 때 무엇이 이기는지**, 비밀값을 어디에 두어야 사고가 안 나는지, 팀 표준을 어떻게 강제하면서도 개인 자유를 남길지 같은 **트레이드오프(trade-off, 득실 맞교환)** 를 다룹니다. 이 책은 Windows 11의 **PowerShell(파워셸)** 을 기준으로 하며, 경로는 C:\Users\사용자\my-project 형태, 사용자 홈은 \$env:USERPROFILE (보통 C:\Users\사용자)로 표기합니다.

settings.json 전체 구조와 병합 우선순위

도입

회사에서 일할 때 우리는 동시에 여러 규칙의 적용을 받습니다. 국가 법, 회사 취업규칙, 우리 팀 그라운드 룰, 그리고 내 개인 메모입니다. 평소엔 겹치지 않지만 **충돌하면 누가 이기는지**가 중요합니다. Claude Code의 설정도 똑같습니다. 설정 파일이 한 개가 아니라 네 종류이고, 같은 항목이 겹치면 정해진 우선순위로 하나의 **유효 설정(effective settings)** 이 결정됩니다. 이 절에서 그 구조와 순서를 끝까지 들여다봅니다.

핵심 개념 설명

설정 파일 (settings.json)

`settings.json` 은 Claude Code의 동작 방식을 JSON 형식으로 적어 두는 구성 파일입니다. 권한(permissions), 환경변수(env), 훅(hooks), 기본 모델(model), 상태 줄(statusLine) 같은 항목을 한곳에 모읍니다. 대화로 매번 "이 명령은 허용해", "이 모델 써" 하고 말하는 대신, 파일에 적어 두면 세션을 열 때마다 자동으로 적용됩니다.

중요한 점은 이 파일이 **하나가 아니라 네 위치**에 존재할 수 있다는 것입니다. 위치마다 목적과 적용 범위가 다릅니다.

- **사용자 전역 설정:** `$env:USERPROFILE\.claude\settings.json` (예: `C:\Users\사용자\.claude\settings.json`). 내가 쓰는 모든 프로젝트에 공통 적용되는 개인 기본값입니다.
- **공유 프로젝트 설정:** 프로젝트 폴더의 `.claude\settings.json`. **저장소에 커밋(commit)** 해 팀 전체가 함께 쓰는 표준입니다.
- **로컬 프로젝트 설정:** 프로젝트 폴더의 `.claude\settings.local.json`. **나만 쓰는** 개인 오버라이드로, 저장소에 올리지 않습니다(자동으로 `.gitignore` 에 등록됩니다).
- **관리형 정책 설정:** 조직 IT가 PC에 배포하는 최상위 정책 파일입니다(3절에서 자세히 다룹니다).

설정 병합 (settings precedence)

설정 병합(settings precedence) 이란, 위 네 종류 설정이 겹칠 때 정해진 우선순위로 합쳐져 하나의 유효 설정이 결정되는 규칙입니다. 가장 흔한 오해가 "**가장 최근에 만든 설정이 항상 이긴다**" 인데, 그렇지 않습니다. 만든 시점이 아니라 **종류(위치)** 가 순서를 정합니다.

우선순위는 높은 쪽부터 다음과 같습니다(높은 쪽이 충돌 시 이깁니다).

1. **관리형 정책**(조직 배포, 최상위 — 하위가 절대 못 덮음)
2. **명령줄 인자**(그 세션에서만 임시 적용, 예: `claude --model ...`)
3. **로컬 프로젝트 설정**(`.claude\settings.local.json`, 개인 비공유)
4. **공유 프로젝트 설정**(`.claude\settings.json`, 팀 공유)
5. **사용자 전역 설정**(`$env:USERPROFILE\.claude\settings.json`)
6. **왜(why) 이 순서인가:** 조직이 정한 보안 경계는 개인이 풀 수 없어야 하므로 관리형 정책이 맨 위입니다. 반대로 "내 PC에서만 잠깐 다르게" 하고 싶을 땐 로컬·명령줄이 팀 설정보다 우선해, 팀 표준을 건드리지 않고도 개인 실험이 가능합니다.

7. **언제(when) 의식하나**: 설정을 분명히 적었는데 적용이 안 될 때입니다. 대개 상위 설정이 같은 항목을 덮고 있는 경우입니다.
8. **어떻게(how) 합쳐지나**: 대부분의 객체 항목은 **병합(merge)** 됩니다. 즉 사용자 설정의 `permissions.allow`와 프로젝트 설정의 `permissions.allow`가 둘 다 살아 합쳐집니다. 다만 **deny (거부)**는 어느 위치에 있든 가장 강합니다. 허용과 거부가 겹치면 거부가 이깁니다.

settings 병합 우선순위



이렇게 설정합니다

다음은 팀이 저장소에 커밋하는 **공유 프로젝트 설정**의 예시입니다(실제로 `.claude/settings.json`에 적는 내용이며, 실행 코드가 아니라 구성 파일입니다).

```
// .claude/settings.json (팀 공유 — 저장소에 커밋)
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "model": "claude-sonnet-4-5",
  "permissions": {
    "allow": [
      "Read",
      "Edit",
      "Bash(npm run test:*)",
      "Bash(git status)"
    ],
    "deny": [
      "Read(./env)",
      "Read(./secrets/**)",
      "Bash(git push --force:*)"
    ],
    "defaultMode": "acceptEdits"
  },
  "env": {
    "BASH_DEFAULT_TIMEOUT_MS": "120000"
  }
}
```

같은 PC에서 나만 다르게 쓰고 싶은 부분은 **로컬 설정**으로 따로 둡니다. 아래 파일은 저장소에 올리지 않습니다.

```
// .claude/settings.local.json (개인 — 커밋하지 않음)
{
  "model": "claude-opus-4-1",
  "permissions": {
    "allow": ["Bash(npm run lint:*)"]
  }
}
```

이 두 파일이 함께 있으면 유효 설정은 다음처럼 합쳐집니다. 모델은 로컬이 공유보다 우선하므로 `claude-opus-4-1` 이 되고, `allow` 목록은 양쪽이 병합되어 테스트·git·lint가 모두 허용되며, `deny` 는 그대로 유지됩니다. 세션 안에서 실제 적용된 결과는 다음 화면처럼 확인합니다.

```
PS C:\Users\사용자\my-project> claude
> /permissions

Permission mode: acceptEdits
Allow:  Read, Edit, Bash(npm run test:*), Bash(git status), Bash(npm run lint:*)
Deny:  Read(./env), Read(./secrets/**), Bash(git push --force:*)
Model:  claude-opus-4-1 (from .claude/settings.local.json)
```

설정 키 설명

키	의미
<code>\$schema</code>	편집기에 자동완성·검증을 제공하는 스키마 주소입니다(동작에는 영향 없음).
<code>model</code>	이 프로젝트에서 기본으로 쓸 모델 이름입니다. 상위 설정이 덮어쓸 수 있습니다.
<code>permissions.allow</code>	문지 않고 자동 허용할 도구·명령 패턴 목록입니다. 위치별로 병합됩니다.
<code>permissions.deny</code>	항상 차단할 패턴입니다. 어느 위치에 있든 <code>allow</code> 보다 우선합니다.
<code>permissions.defaultMode</code>	세션 시작 시 승인 모드입니다(<code>default</code> · <code>acceptEdits</code> · <code>plan</code> 등).
<code>env</code>	세션에 적용할 환경변수입니다(2절에서 자세히 다룹니다).

주의: `deny` 는 합쳐진 모든 설정 중 가장 강합니다. 팀 공유 설정에서 `Read(/secrets/**)` 를 거부했다면, 개인이 로컬 설정에서 같은 경로를 `allow` 에 넣어도 **풀리지 않습니다**. 보안 항목을 풀려고 로컬에서 씨름하지 말고, 정책 자체를 팀과 논의해 바꾸십시오.

절 요약

- `settings.json` 은 사용자 전역·공유 프로젝트·로컬 프로젝트·관리형 정책 네 위치에 존재할 수 있습니다.
- 우선순위는 관리형 > 명령줄 인자 > 로컬 프로젝트(`.local`) > 공유 프로젝트 > 사용자 전역 순입니다.
- 대부분 항목은 병합되지만 `deny` 는 어디에 있든 가장 강해 `allow` 를 이깁니다.

환경변수와 비밀값 관리

도입

집 비밀번호를 현관 앞 메모지에 붙여 두는 사람은 없습니다. 그런데 코드와 설정 파일에서는 이 실수가 의외로 자주 일어납니다. API 키(외부 서비스 이용 자격을 증명하는 긴 문자열)를 `settings.json` 에 그대로 적고, 그 파일을 저장소에 커밋해 버리는 것입니다. 한 번 올라가면 사

실상 공개된 것이나 마찬가지입니다. 이 절에서는 비밀값을 설정 파일이 아니라 **환경변수**로 다루는 원칙을 익힙니다.

핵심 개념 설명

환경변수 (environment variable)

환경변수(environment variable)란, 설정값이나 비밀값을 코드·저장소가 아니라 **운영체제의 환경에 담아** 프로그램에 전달하는 방식입니다. 프로그램은 실행될 때 그 값을 꺼내 쓰지만, 값 자체는 소스 코드 어디에도 적혀 있지 않습니다. 비밀번호를 메모지에 적는 대신 금고에 넣어 두고 필요할 때만 꺼내는 것과 같습니다.

Claude Code는 두 가지 경로로 환경변수를 다룹니다.

- **OS 환경변수:** PowerShell이나 Windows 시스템 환경변수로 미리 설정해 둔 값입니다. 비밀값(토큰)은 반드시 이쪽에 둡니다.
- **settings.json의 env 블록:** 비밀이 *아닌* 동작 설정값(예: 타임아웃, 텔레메트리 끄기)을 팀과 공유하기 위해 적는 곳입니다. **여기에는 비밀값을 적지 않습니다.**

자주 쓰는 환경변수는 다음과 같습니다.

- **ANTHROPIC_API_KEY:** API 키 인증 시 사용하는 키입니다(비밀값 — OS 환경변수로만).
- **ANTHROPIC_MODEL:** 기본 모델을 환경변수로 지정합니다.
- **BASH_DEFAULT_TIMEOUT_MS / BASH_MAX_TIMEOUT_MS:** 명령 실행 제한 시간입니다.
- **DISABLE_TELEMETRY:** 사용 통계 전송을 끕니다.
- **HTTPS_PROXY:** 사내 프록시를 통해 통신하도록 지정합니다.

비밀값 관리 (secret management)

비밀값 관리(secret management)란, 토큰·키처럼 유출되면 안 되는 정보를 저장소·코드에 남기지 않고 안전하게 주입(injection)하는 원칙입니다. 핵심 규칙은 단순합니다. **비밀값은 환경변수로, 설정값은 settings.json으로.**

- **왜(why) 그런가:** settings.json (특히 공유본)은 저장소에 올라갑니다. 거기에 키를 적으면 저장소 접근 권한이 있는 모든 사람과, 저장소가 유출될 경우 외부인까지 키를 보게 됩니다. 키 하나가 새면 그 키로 할 수 있는 모든 일이 노출됩니다.
- **언제(when) 신경 쓰나:** API 키 방식으로 인증할 때, 사내 MCP 서버에 토큰을 넘길 때(9장), CI 같은 자동화에서 인증할 때입니다.
- **어떻게(how) 하나:** 내 PC에서는 OS 환경변수에 키를 넣고, CI에서는 "시크릿(secret)" 저장소(예: GitHub Actions Secrets)에 넣어 실행 시점에만 환경변수로 주입합니다.

이렇게 설정합니다

먼저 비밀값입니다. PowerShell에서 **현재 세션에만** 임시로 넣으려면 다음처럼 합니다(따옴표 안은 예시 값입니다).

```
PS C:\Users\사용자\my-project> $env:ANTHROPIC_API_KEY = "sk-ant-실제키값"
PS C:\Users\사용자\my-project> claude
```

매번 입력하기 번거롭다면 **사용자 단위로 영구 등록**합니다. 이 값은 내 계정에만 저장되며 저장소와 무관합니다.

```
PS C:\Users\사용자> [Environment]::SetEnvironmentVariable("ANTHROPIC_API_KEY", "sk-ant-실제키값")
```

반면 **비밀이 아닌 동작 설정**은 팀과 공유하기 위해 `settings.json`의 `env`에 적습니다.

```
// .claude/settings.json (팀 공유 — 비밀값 아님만)
{
  "env": {
    "BASH_DEFAULT_TIMEOUT_MS": "120000",
    "DISABLE_TELEMETRY": "1"
  }
}
```

설정/명령 설명

항목	의미
<code>\$env:NAME = "값"</code>	PowerShell에서 현재 세션에만 유효한 환경변수를 설정합니다.
<code>[Environment]::SetEnvironmentVariable(..., "User")</code>	사용자 계정에 영구 저장합니다(새 터미널부터 적용).
<code>env.BASH_DEFAULT_TIMEOUT_MS</code>	명령 기본 제한 시간(밀리초)입니다. 비밀이 아니라 공유해도 안전합니다.
<code>env.DISABLE_TELEMETRY</code>	1 이면 사용 통계 전송을 끕니다.

위험: `ANTHROPIC_API_KEY` 같은 비밀값을 `.claude/settings.json`의 `env`에 적고 커밋하면, 키가 저장소 이력에 영구히 남습니다. 나중에 파일에서 지워도 과거 커밋에는 그대로 남으므로, 이런

사고가 났다면 **즉시 키를 폐기(revoke)하고 새로 발급**해야 합니다. 비밀값은 처음부터 환경변수로만 다루십시오.

팁: 어떤 값이 "비밀"인지 헷갈릴 때의 기준은 간단합니다. **그 값이 유출되면 누군가 내 권한으로 무언가를 할 수 있는가?** 그렇다면 비밀값이니 환경변수로, 아니라면 공유 설정으로 두면 됩니다.

절 요약

- 비밀값은 OS 환경변수로, 비밀이 아닌 동작 설정은 `settings.json`의 `env`로 다룹니다.
- 키를 `settings.json`에 적어 커밋하면 유출 위험이 크며, 새면 즉시 폐기·재발급해야 합니다.
- "유출 시 내 권한으로 무언가 할 수 있는 값"이면 비밀값입니다.

관리형 정책으로 조직 거버넌스 강제

도입

팀 공유 `settings.json`에는 한 가지 한계가 있습니다. 누구나 자기 PC에서 `settings.local.json`이나 명령줄 인자로 덮어쓸 수 있다는 점입니다. 평소엔 편리하지만, "고객 데이터 폴더는 누구도 읽지 못하게" 같은 **양보 불가능한 규칙**에는 부족합니다. 개인이 풀 수 없는 경계가 필요할 때 쓰는 것이 **관리형 정책**입니다. 이 절에서 조직 차원의 강제 통제를 다룹니다.

핵심 개념 설명

관리형 정책 (managed policy)

관리형 정책(managed policy)이란, 조직의 IT 관리자가 구성원 PC에 배포하는 **최상위 설정 파일**입니다. 우선순위가 가장 높아 사용자·프로젝트·로컬 설정이 **절대 덮어쓸 수 없습니다**. 건물 전체에 적용되는 소방 규정을 개별 입주자가 마음대로 풀 수 없는 것과 같습니다.

가장 흔한 오해는 **"개인이 로컬 설정으로 조직 정책을 풀 수 있다"**입니다. 그럴 수 없습니다. 관리형 정책의 `deny`는 하위 어디에서도 해제되지 않으며, 하위 설정은 정책을 **더 좁힐 수만**(더 엄격하게) 있습니다.

관리형 정책 파일의 위치는 OS마다 다릅니다. Windows에서는 다음 경로입니다.

- **Windows:** `C:\Program Files\ClaudeCode\managed-settings.json`

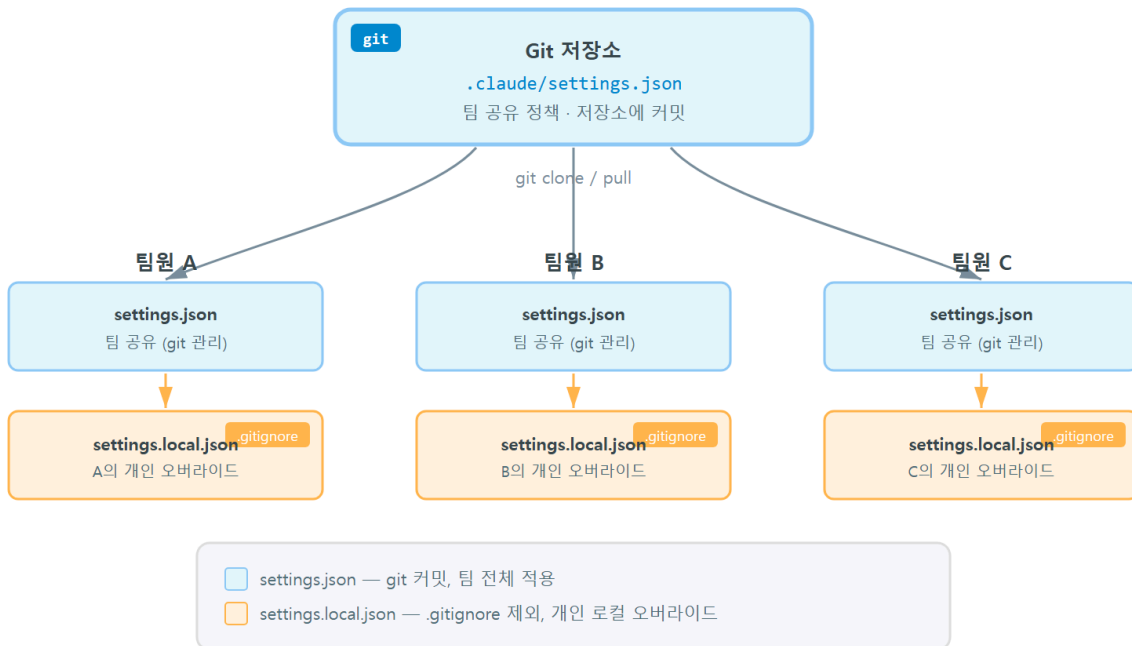
이 경로는 일반 사용자 권한으로는 쓸 수 없는 시스템 영역이라, 관리자만 배포할 수 있습니다. (참고: 예전 버전에서 쓰이던 `C:\ProgramData\ClaudeCode\...` 경로는 최신 버전에서 더 이상 쓰이지 않습니다. 조직 표준 경로는 IT 담당과 확인하십시오.)

거버넌스 (governance)

거버넌스(governance)란, 개인의 선의에 기대지 않고 **규칙을 구조로 강제**해 조직 전체의 안전·일관성을 지키는 통제 방식입니다. 관리형 정책은 거버넌스를 구현하는 가장 강한 수단입니다.

- **왜(why) 필요한가:** 사람은 실수하고 설정을 잊습니다. "각자 알아서 secrets 폴더를 막읍시다"는 한 명만 빠뜨려도 사고가 납니다. 구조로 강제하면 실수 여지가 사라집니다.
- **언제(when) 쓰나:** 규제 산업(금융·의료), 고객 데이터 취급, 위험 명령(강제 푸시·대량 삭제) 일괄 차단, 권한 우회 모드(bypass) 자체를 막아야 할 때입니다.
- **어떻게(how) 설계하나:** 관리형 정책에는 **모두에게 양보 불가능한 최소한만** 담습니다. 너무 많이 막으면 업무가 멈추고, 사람들이 Claude Code를 우회해 더 위험한 길로 갑니다. 이것이 거버넌스의 핵심 **트레이드오프**입니다 — 안전과 생산성의 균형입니다.

팀 공유 정책 — 저장소 설정과 개인 오버라이드



이렇게 설정합니다

다음은 조직이 배포하는 관리형 정책의 예시입니다. 위험 명령과 민감 경로를 전사 차원에서 막고, 권한 우회 모드 자체를 비활성화합니다.

```
// C:\Program Files\ClaudeCode\managed-settings.json (IT 관리자 배포)
{
  "permissions": {
    "deny": [
      "Read(/secrets/**)",
      "Read(/**/*.pem)",
      "Bash(rm -rf:*)",
      "Bash(git push --force:*)"
    ],
    "disableBypassPermissionsMode": "disable"
  }
}
```

이 정책이 깔린 PC에서 개인이 로컬 설정으로 `secrets` 읽기를 허용하려 해도 통하지 않습니다. 세션에서 시도하면 다음처럼 차단됩니다.

```
PS C:\Users\사용자\my-project> claude
> secrets/api.key 파일 내용을 보여 줘

[Read] secrets/api.key
차단됨: 조직 관리형 정책의 deny 규칙에 해당합니다.
이 규칙은 로컬 설정으로 해제할 수 없습니다.
```

설정 키 설명

키	의미
<code>permissions.deny</code>	전사 차원에서 항상 차단할 패턴입니다. 하위 설정이 해제 불가능합니다.
<code>Bash(rm -rf:*)</code>	<code>rm -rf</code> 로 시작하는 파괴적 삭제 명령을 차단합니다.
<code>disableBypassPermissionsMode</code>	"disable" 이면 권한 우회 모드(모든 확인을 건너뛰는 모드) 사용을 막습니다.

주의: 관리형 정책은 강력한 만큼 **과용하면 역효과**가 납니다. 정상 업무 명령까지 광범위하게 막으면 구성원이 매번 막혀 생산성이 떨어지고, 결국 Claude Code를 끄고 수작업으로 돌아가 통제 밖으로 새어 나갑니다. 정책에는 "절대 안 되는 것"만 최소로 담고, 나머지는 팀 공유 `settings.json` 으로 권고하십시오.

절 요약

- 관리형 정책은 최상위 설정으로, 사용자·프로젝트·로컬 설정이 덮어쓸 수 없습니다.

- Windows 경로는 `C:\Program Files\ClaudeCode\managed-settings.json`이며 관리자만 배포합니다.
- 정책에는 양보 불가능한 최소한만 담아 안전과 생산성의 균형을 맞춥니다.

모델·상태 줄·출력 스타일·도구 권한 다듬기

도입

거버넌스의 골격을 세웠다면, 이제 팀의 일상 경험을 다듬을 차례입니다. 어떤 모델을 기본으로 쓸지, 화면 하단 상태 줄에 무엇을 띄울지, 답변 스타일을 어떻게 할지 같은 항목입니다. 사소해 보이지만, 팀 전체가 같은 기본값을 공유하면 "왜 너는 다른 모델로 돌렸어?" 같은 혼선이 사라집니다. 이 절에서 자주 쓰는 다듬기 항목을 모읍니다.

핵심 개념 설명

기본 모델 설정 (default model)

기본 모델 설정(default model)은 세션을 열 때 자동으로 선택되는 모델을 `settings.json`의 `model` 키로 지정하는 것입니다. 프로젝트마다 적합한 모델이 다를 수 있습니다. 빠른 반복 작업엔 가벼운 모델, 어려운 설계·디버깅엔 강한 모델이 어울립니다.

- **왜(why):** 팀이 같은 기본 모델을 쓰면 결과의 일관성과 비용 예측이 쉬워집니다. 비용·속도·품질 사이의 트레이드오프를 팀 차원에서 한 번 정해 두는 셈입니다.
- **언제(when):** 프로젝트 성격이 뚜렷할 때(예: 비용 민감한 대량 작업 vs 정밀 설계)입니다.

출력 스타일 (output style)

출력 스타일(output style)은 Claude Code가 답변하는 톤과 형식을 조정하는 설정입니다. 예를 들어 설명을 자세히 풀지, 요점만 간결히 낼지를 프로젝트 성격에 맞춰 정합니다. **상태 줄(status line)**은 화면 하단에 모델·작업 폴더·컨텍스트 사용량 등을 보여 주는 표시줄로, `statusLine`에 명령을 지정해 원하는 정보를 띄울 수 있습니다.

이렇게 설정합니다

```
// .claude/settings.json (팀 공유 — 일상 기본값)
{
  "model": "claude-sonnet-4-5",
  "outputStyle": "concise",
  "statusLine": {
    "type": "command",
    "command": "echo \"[${git branch --show-current}] 모델:$CLAUDE_MODEL\""
  },
  "permissions": {
    "ask": [
      "Bash(git push:*)"
    ]
  }
}
```

상태 줄은 화면 하단에 다음처럼 나타납니다(예상 화면입니다).

```
[main] 모델:claude-sonnet-4-5
```

명령줄에서 그 세션만 다른 모델로 임시 실행할 수도 있습니다. 명령줄 인자는 `settings.json` 보다 우선합니다.

```
claude --model claude-opus-4-1
```

설정 키 설명

키	의미
<code>model</code>	세션 기본 모델입니다. 명령줄 <code>--model</code> 이 우선합니다.
<code>outputStyle</code>	답변 톤·형식입니다(예: 간결).
<code>statusLine.type</code> / <code>command</code>	상태 줄을 명령 결과로 채웁니다. <code>command</code> 의 출력이 그대로 표시됩니다.
<code>permissions.ask</code>	자동 허용·거부가 아니라 항상 한 번 물어볼 작업입니다.

팁: 권한은 `allow` (자동 허용)·`ask` (항상 묻기)·`deny` (항상 차단) 세 단계로 나뉩니다. 되돌리기 어려운 작업(`git push` 등)은 자동 허용 대신 `ask` 에 두면, 자동화의 편리함과 사람 확인의 안전을 함께 얻습니다.

절 요약

- `model` 로 팀 기본 모델을, `outputStyle` 로 답변 형식을, `statusLine` 으로 화면 표시를 정합니다.
- 권한은 `allow · ask · deny` 세 단계로 세밀하게 조정합니다.
- 명령줄 인자는 그 세션에 한해 `settings.json` 보다 우선합니다.

설정을 저장소에 담기 - 공유 vs 로컬 분리

도입

마지막 관문은 "무엇을 저장소에 올리고 무엇을 안 올릴까"입니다. 이 분리를 잘못하면 두 가지 사고가 납니다. 비밀값이 딸려 올라가 유출되거나, 반대로 팀 표준이 각자 PC에만 있어 사람마다 동작이 달라집니다. 이 절에서 공유본과 로컬본을 가르는 실무 기준을 정합니다.

핵심 개념 설명

공유 설정 (shared settings)

공유 설정(shared settings) 은 `.claude\settings.json` 에 담아 **저장소에 커밋**하는 팀 표준입니다. 팀 모두가 같은 권한 규칙, 같은 기본 모델, 같은 혹은 쓰게 만드는 단일 출처입니다. 새 팀원이 저장소를 내려받는 순간 동일한 환경이 따라옵니다.

로컬 오버라이드 (.local override)

로컬 오버라이드(.local override) 는 `.claude\settings.local.json` 에 담은 **개인 전용** 설정으로, 저장소에 올리지 않습니다(Claude Code가 자동으로 `.gitignore` 에 등록합니다). 내 취향의 모델, 내 PC에서만 허용할 명령, 개인 실험용 설정이 여기 들어갑니다.

- **무엇을 공유본에:** 팀 권한 규칙(`allow / deny / ask`), 기본 모델, 공통 혹은, 비밀이 아닌 `env`.
- **무엇을 로컬본에:** 개인 모델 취향, 개인 도구 허용, 임시 실험 설정.
- **무엇을 어느 쪽에도 두지 않기:** 비밀값(환경변수로만 — 2절).
- **왜(why):** 공유본은 일관성을, 로컬본은 개인 자유를 줍니다. 둘을 섞으면 비밀 유출 또는 표준 붕괴로 이어집니다.
- **트레이드오프:** 공유본에 너무 많이 넣으면 개인이 답답해 로컬에서 덮어쓰느라 표준이 무력화되고, 너무 적게 넣으면 표준이 사라집니다. "팀에 꼭 필요한 것만 공유본, 나머지는

로컬본"이 균형점입니다.

이렇게 설정합니다

저장소에는 다음처럼 정리합니다. 공유본은 커밋하고, 로컬본은 무시 목록에 둡니다.

```
my-project\
  .claude\
    settings.json      <- 커밋 (팀 공유 표준)
    settings.local.json <- 커밋 안 함 (개인)
  .gitignore
```

.gitignore 에는 로컬본이 빠지도록 적어 둡니다(Claude Code가 자동으로 추가하지만, 확인해 두면 안전합니다).

```
# .gitignore
.claude/settings.local.json
```

새 팀원이 저장소를 받은 뒤 자기 PC 환경만 로컬본으로 더하는 흐름은 다음과 같습니다.

```
PS C:\Users\사용자> git clone https://github.com/our-team/my-project.git
PS C:\Users\사용자> cd my-project
PS C:\Users\사용자\my-project> claude
```

베스트 프랙티스: 공유 settings.json 을 바꿀 때는 일반 코드처럼 **풀 리퀘스트(pull request, 변경 제안 검토 요청)** 로 리뷰를 거치십시오. 권한 규칙 변경은 팀 전체의 안전에 영향을 주므로, 한 사람이 조용히 바꾸지 않고 함께 확인하는 절차가 중요합니다.

주의: settings.local.json 을 실수로 커밋하지 않았는지 주기적으로 점검하십시오. git status 에 이 파일이 보인다면 .gitignore 설정이 빠진 것입니다. 한 번 올라가면 개인 설정이 팀에 새어 표준을 흐립니다.

절 요약

- 팀 표준은 공유 settings.json 에 커밋하고, 개인 설정은 settings.local.json 으로 분리합니다.
- 비밀값은 어느 설정 파일에도 두지 않고 환경변수로만 다룹니다.
- 공유본 변경은 풀 리퀘스트로 리뷰하고, 로컬본이 커밋되지 않는지 점검합니다.

실습 과제

해보기: 팀 공유용 settings.json 정책 세트 만들기

실습 폴더 `C:\Users\사용자\claude-lab\team-project` 에서 다음을 직접 구성해 봅니다.

- 단계 1: `.claude\settings.json` (공유본)을 만들고 ① 허용할 명령 3가지(`allow`), ② 항상 물어볼 작업 1가지(`ask`), ③ 절대 거부할 경로 2가지(`deny`), ④ 기본 모델(`model`)을 적습니다.
- 단계 2: `.claude\settings.local.json` (로컬본)에 본인 취향 모델과 개인 허용 명령 1가지를 적고, `.gitignore` 에 로컬본을 추가합니다.
- 단계 3: 세션에서 `/permissions` 로 두 파일이 어떻게 병합됐는지 확인하고, 로컬본의 모델이 공유본을 덮었는지 점검합니다.
- 단계 4: "관리형 정책이 있다고 가정한 거버넌스 경계 문서"를 한 페이지로 작성합니다 — 전사 차원에서 절대 막을 항목, 팀 공유본에 둘 항목, 개인 자유로 남길 항목을 표로 구분합니다.
- 점검: 비밀값이 어느 설정 파일에도 들어가지 않았는지, `deny` 가 의도대로 `allow` 를 이기는지 확인합니다.

FAQ

Q1. 분명히 `allow` 에 명령을 넣었는데 계속 차단됩니다. 왜 그런가요?

→ A1. 상위 설정이나 같은 설정의 `deny` 가 그 명령을 막고 있을 가능성이 큼니다. `deny` 는 위치와 무관하게 `allow` 를 이깁니다. 특히 조직 관리형 정책의 `deny` 는 개인이 해제할 수 없습니다. `/permissions` 로 합쳐진 유효 규칙을 먼저 확인하고, `deny` 에 겹치는 패턴이 있는지 보십시오.

Q2. API 키를 팀과 공유하고 싶은데 `settings.json` 에 적어 커밋하면 안 되나요?

→ A2. 안 됩니다. 공유 `settings.json` 은 저장소에 올라가므로 키가 그대로 노출되고 커밋 이력에 영구히 남습니다. 비밀값은 각자 OS 환경변수로, 자동화 환경에서는 시크릿 저장소로 주입하십시오. 팀이 공유할 것은 키 자체가 아니라 "어떤 환경변수 이름을 쓰는지"에 대한 약속입니다.

Q3. 사용자 전역 설정과 프로젝트 공유 설정 중 무엇이 우선인가요?

→ A3. 프로젝트 공유 설정(`.claude\settings.json`)이 사용자 전역 설정보다 우선합니다. 우선순위는 높은 쪽부터 관리형 정책 > 명령줄 인자 > 로컬 프로젝트(`.local`) > 공유 프로젝트 > 사

용자 전역 순입니다. 다만 대부분 항목은 병합되므로, 충돌하지 않는 설정은 양쪽이 모두 살아 합쳐집니다.

Q4. 관리형 정책이 깔린 회사 PC인데 개인 작업에 너무 제약이 많습니다. 제가 풀 수 있나요?

→ A4. 로컬 설정으로는 풀 수 없습니다. 관리형 정책은 최상위라 하위에서 해제가 불가능하고, 더 좁힐 수만 있습니다. 업무에 꼭 필요한 명령이 막혀 있다면 임의로 우회하지 말고 IT·보안 담당과 정책 조정을 논의하십시오. 우회 시도 자체가 거버넌스 위반일 수 있습니다.

장 요약

이 장에서는 `settings.json` 을 거버넌스 관점에서 통째로 다뤘습니다. 설정은 사용자 전역·공유 프로젝트·로컬 프로젝트·관리형 정책 네 위치에 존재하며, 관리형 > 명령줄 인자 > 로컬 > 공유 > 사용자 전역 순으로 병합되어 최종 유효 설정이 결정됩니다. `deny` 는 어디에 있든 가장 강하다는 점, 비밀값은 설정 파일이 아닌 환경변수로만 다룬다는 점이 가장 중요한 두 원칙입니다. 관리형 정책으로 조직의 양보 불가능한 경계를 강제하되, 과용하면 생산성을 해친다는 트레이드 오프를 기억해야 합니다. 마지막으로 팀 표준은 공유본에 커밋하고 개인 설정은 로컬본으로 분리해, 일관성과 개인 자유를 동시에 지킵니다.

핵심 키워드

`settings.json`, 설정 병합(settings precedence), 환경변수(environment variable), 비밀값 관리(secret management), 관리형 정책(managed policy), 거버넌스(governance), 공유 설정(shared settings), 로컬 오버라이드(.local override)

다음 장 미리보기

다음 장에서는 지금까지 다진 설정 위에서 여러 인터페이스(CLI·IDE·데스크톱·웹)를 오가며 일하고, 비대화형 실행(headless)으로 CI 파이프라인에 Claude Code를 통합하는 팀 협업·자동화 실무를 배웁니다.

여러 인터페이스와 팀 협업·CI 자동화

학습 목표 - 작업 상황에 맞춰 CLI·IDE·데스크톱·웹 인터페이스를 선택하고 설정 연속성을 유지할 수 있습니다. - 헤드리스 실행(headless mode, `claude -p`)과 구조화 출력(JSON)으로 사람 없이 작업을 자동화할 수 있습니다. - GitHub Actions에서 PR 자동 코드 리뷰 워크플로를 토큰·권한까지 안전하게 구성할 수 있습니다. - 슬래시 커맨드·서브에이전트·훅을 저장소와 플러그인으로 팀에 표준화해 배포할 수 있습니다. - 토큰·비용을 관측하고 감사 로그로 사용량을 거버넌스할 수 있습니다.

지금까지는 한 사람이 터미널 앞에 앉아 Claude Code와 대화하는 모습을 그려 왔습니다. 하지만 실무에서는 다릅니다. 같은 사람이 아침에는 터미널(CLI)에서, 낮에는 편집기(IDE) 안에서, 이동 중에는 웹에서 작업을 이어 가고, 밤에는 **사람이 없는 사이에 CI 서버가 코드를 자동 점검**합니다. 나아가 팀 전체가 같은 규칙·명령·자동화를 공유해야 합니다.

이 장은 **실무 밴드**입니다. 단순히 "어떻게 켜는가"를 넘어, 여러 표면을 하나의 설정으로 묶는 원리, 사람 없이 도는 자동화의 권한·비밀값 통제, 그리고 팀·조직 차원의 표준화와 거버넌스(governance, 조직 차원의 통제·관리)를 다룹니다. 사용자가 직접 입력하는 명령은 Windows 11의 **PowerShell(파워셸)** 프롬프트 `PS C:\Users\사용자\my-project>` 를 기준으로 적되, CI 서버에서 도는 워크플로 파일은 그 서버가 보통 리눅스(Linux)이므로 별도로 표시합니다.

여기서 자주 나오는 약어를 먼저 풀어 둡니다. **CI(Continuous Integration, 지속적 통합)** 는 코드를 올릴 때마다 서버가 자동으로 검사·빌드·테스트를 돌리는 방식입니다. **헤드리스(headless)** 는 "머리(화면·대화창)가 없는" 실행, 즉 사람과 대화하지 않고 한 번에 결과만 내는 실행을 뜻합니다.

인터페이스별 강점과 연속성(CLI·IDE·데스크톱·웹)

도입

어느 기기에서 로그인하든 같은 메일함·사진·연락처가 따라오는 클라우드 계정을 떠올려 보십시오. 노트북에서 쓰던 문서를 휴대폰에서 이어서 보고, 다시 데스크톱에서 마무리합니다. Claude Code의 여러 인터페이스도 똑같습니다. **하나의 엔진**을 터미널·편집기·데스크톱·웹이라는 여러

창구로 쓰는 것일 뿐, 그 밑의 **설정·메모리·권한은 공유**됩니다. 이 절에서는 각 표면이 언제 강점을 발휘하는지, 그리고 어떻게 끊김 없이 이어 가는지 살펴봅니다.

핵심 개념 설명

인터페이스 선택 (interface choice)

인터페이스 선택(interface choice)이란, 같은 Claude Code 작업을 어느 창구에서 할지 고르는 일입니다. 1장에서 본 것처럼 CLI·IDE 확장·데스크톱 앱·웹은 모두 같은 엔진을 쓰는 **표면(surface)**이며, 표면마다 잘하는 상황이 다릅니다.

- **CLI(터미널)**: 가장 빠르고 자동화에 강합니다. 명령을 스크립트로 묶거나, 헤드리스로 돌리거나, 다른 도구와 파이프(|)로 연결할 수 있습니다. 이 책이 CLI를 중심으로 삼은 이유입니다.
- **IDE 확장(VS Code·JetBrains)**: 편집 중인 코드 옆에서 변경 미리보기(diff)를 시각적으로 검토하기에 좋습니다. 지금 보고 있는 파일·선택 영역이 자동으로 맥락에 들어가, "이 코드"라고만 해도 통합니다.
- **데스크톱 앱**: 여러 작업을 나란히 띄우거나, 터미널이 익숙지 않은 사람이 시작하기에 편합니다.
- **웹**: 설치 없이 브라우저에서 접근하며, 장시간 작업을 클라우드에 맡겨 두고 이동 중에 진행 상황을 확인하기에 좋습니다.

설정 연속성 (config continuity)

설정 연속성(config continuity)이란, 어느 표면에서 작업하든 같은 `CLAUDE.md`·권한 규칙·설정이 따라와 끊김 없이 이어지는 성질입니다. 5장의 메모리(CLAUDE.md)와 10장의 `settings.json`은 특정 표면에 묶이지 않습니다. 프로젝트 폴더에 들어 있는 이 파일들을, 어느 표면이든 그 프로젝트를 열면 똑같이 읽어 들입니다.

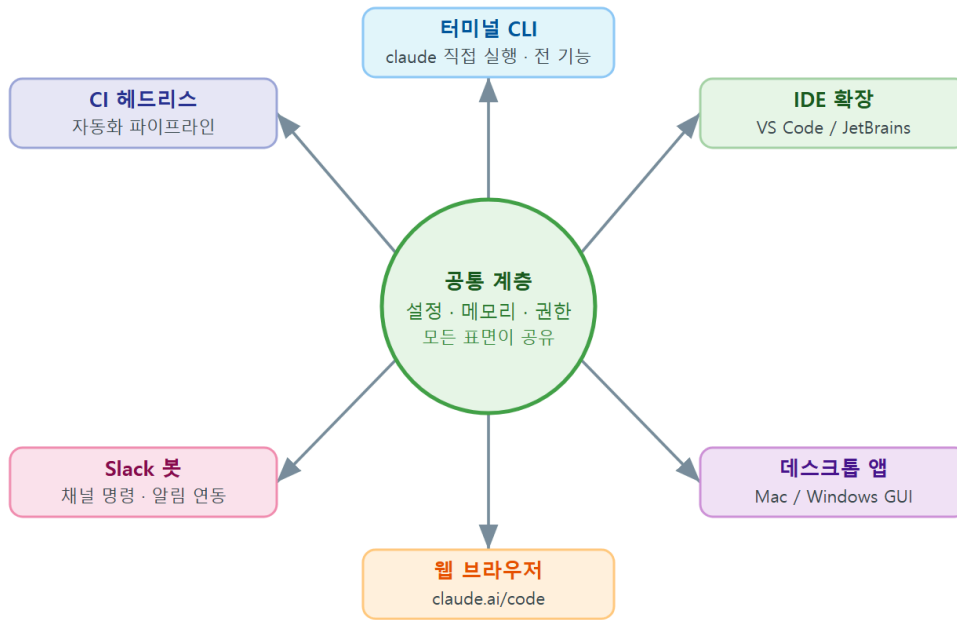
여기서 가장 흔한 오해는 "**표면을 바꾸면 설정을 처음부터 다시 해야 한다**"는 생각입니다. 그렇지 않습니다. 설정·메모리·권한은 **프로젝트(폴더)와 계정에 붙어 있지, 창구에 붙어 있지 않습니다**. CLI에서 만든 `.claude/commands`의 명령은 IDE에서도 보이고, 프로젝트 `settings.json`의 거부 규칙은 웹에서도 그대로 적용됩니다.

- **왜(why) 중요한가**: 표면마다 설정이 따로라면 규칙이 어긋나 위험합니다. CLI에서는 막은 위험 명령이 IDE에서는 풀려 있다면 안전장치가 무너집니다. 연속성 덕분에 **한 번 정한 안전 경계가 모든 창구에 일관되게 적용**됩니다.
- **언제(when) 무엇을 쓰나**: 빠른 반복·자동화는 CLI, 코드 리뷰·정밀 편집은 IDE, 가벼운 시작이나 비개발 직군 동료는 데스크톱, 이동 중 확인은 웹입니다. 한 작업 안에서도 표면을

바뀌 가며 강점을 취합니다.

- **어떻게(how) 이어지나**: 공유되는 것(프로젝트 메모리· settings.json ·MCP 서버 등록)과 표면마다 다른 것(단축키·화면 배치 같은 표시 방식)을 구분하면 됩니다. **무엇을 시킬지(규칙·권한)**는 공유되고, **어떻게 보일지(UI)**는 표면마다 다릅니다.

멀티 표면 연속성 — 공통 계층 공유 구조



ch_11_s01 · 멀티 표면 연속성(multi-surface continuity) · 허브-스포크(hub-spoke)

이렇게 이어 갑니다

같은 프로젝트를 두 표면에서 여는 모습을 비교해 보겠습니다. 먼저 CLI입니다.

```
# 터미널에서 프로젝트 폴더를 열어 세션 시작
PS C:\Users\사용자\my-project> claude
> /memory
```

로드된 메모리 (이 표면에서 적용 중)

프로젝트: C:\Users\사용자\my-project\CLAUDE.md
 개인(전역): C:\Users\사용자\.claude\CLAUDE.md
 설정: .claude\settings.json (공유) + settings.local.json (개인)

같은 프로젝트 폴더를 IDE(VS Code)에서 열고 Claude Code 패널에서 `/memory` 를 실행해도, **로드되는 메모리·설정 파일은 동일합니다**. 표면만 바뀌었을 뿐 읽어 들이는 자료는 같기 때문입니다.

표면 사이에서 공유되는 것과 다른 것

항목	공유되나	비고
프로젝트 메모리(<code>CLAUDE.md</code>)	공유됨	폴더에 들어 있어 어느 표면이든 로드
권한 규칙· <code>settings.json</code>	공유됨	안전 경계가 모든 창구에 일관 적용
MCP 서버 등록(<code>.mcp.json</code>)	공유됨	project 범위면 표면 무관하게 사용
단축키·테마·화면 배치	다름	표시 방식은 표면마다 별도
현재 열린 파일·선택 영역	다름	IDE는 편집 중 파일을 자동 맥락에 포함

팁: 표면을 옮길 때 "왜 IDE에서는 규칙이 다르지?"라는 의문이 들면, 먼저 **같은 프로젝트 폴더를 열었는지** 확인합니다. 대개는 다른 폴더(상위 폴더나 다른 클론)를 열어 다른 `CLAUDE.md` 가 로드된 경우입니다. `/memory` 로 실제 로드된 경로를 보면 바로 드러납니다.

베스트 프랙티스: 팀이 공유해야 하는 규칙은 사람·표면이 아니라 **저장소에** 둡니다(프로젝트 `CLAUDE.md`, 공유 `settings.json`, `.mcp.json`). 그래야 누가 어떤 창구를 쓰든 같은 안전 경계와 맥락에서 일합니다. 개인 취향(단축키·개인 메모리)만 각자 관리합니다.

절 요약

- Claude Code의 CLI·IDE·데스크톱·웹은 같은 엔진을 쓰는 표면이며, 자동화는 CLI, 정밀 편집은 IDE처럼 상황별 강점이 다릅니다.
- 설정 연속성 덕분에 메모리·권한·설정은 표면이 아니라 프로젝트·계정에 붙어 따라오므로, 표면을 바꿔도 다시 설정할 필요가 없습니다.
- 공유할 규칙은 저장소에 두고, 표시 방식(단축키·배치)만 표면마다 따로 관리합니다.

비대화형 실행(headless)과 CI 파이프라인 통합

도입

지금까지의 Claude Code는 사람이 옆에서 묻고 답하며 함께 일하는 동료였습니다. 그런데 매일 밤 정해진 시각에 누군가 자동으로 회의록을 정리해 둔다면 어떨까요. 사람이 버튼을 누르지 않아도 정해진 조건에서 돌아가는 **자동 배치 작업**처럼, Claude Code도 대화창 없이 한 번에 실행

할 수 있습니다. 이 절에서는 그 방식인 **헤드리스 실행**과, 결과를 기계가 읽기 좋게 받는 **구조화 출력**을 다룹니다.

핵심 개념 설명

헤드리스 실행 (headless mode)

헤드리스 실행(headless mode)이란, 대화창을 띄우지 않고 `claude -p "할 일"` 형태로 **작업을 한 번에 시켜 결과만 받는 비대화형 방식**입니다. 여기서 `-p`는 프롬프트(prompt)를 뜻하며, "이 요청을 처리하고 끝내라"는 의미입니다. 사람이 중간에 답하지 않으므로, 스크립트·예약 작업·CI 파이프라인에 끼워 넣을 수 있습니다.

대화형과 가장 크게 다른 점은 **사람이 권한 요청에 답할 수 없다**는 것입니다. 대화형에서는 위험한 작업 직전에 "허용하시겠습니까?" 창이 떴지만, 헤드리스에는 답할 사람이 없습니다. 그래서 **무엇을 허용할지 미리 규칙으로 정해 두어야** 합니다(4·10장의 권한 규칙). 정해 두지 않으면 작업이 권한 대기에서 멈추거나 실패합니다.

여기서 가장 위험한 오해는 **"헤드리스니까 권한 통제가 필요 없다"**는 생각입니다. 정반대입니다. 사람이 막아 줄 수 없으므로 **권한 통제가 더 중요합니다**. "전부 우회"로 돌리면 편하지만, 잘못된 명령 하나가 사람의 제동 없이 그대로 실행됩니다.

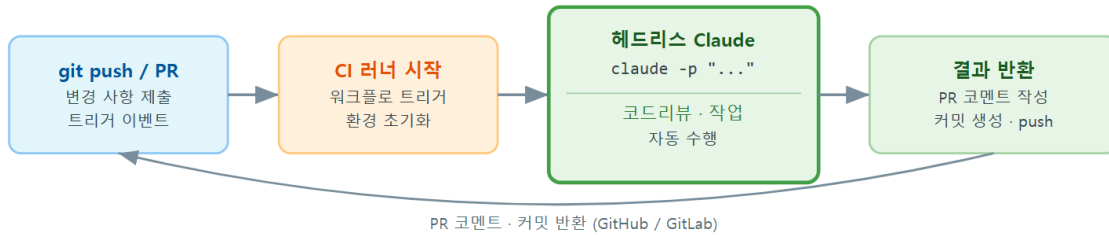
비대화형 출력 (structured output)

구조화 출력(structured output)이란, 헤드리스 실행 결과를 사람이 읽는 글이 아니라 **기계**가 읽기 좋은 형식(JSON)으로 받는 것입니다. `--output-format` 옵션으로 고릅니다.

- **text(기본)**: 사람이 읽는 평범한 글입니다.
- **json**: 결과 본문에 더해 사용한 토큰 수, 비용, 소요 시간, 세션 ID 같은 **메타데이터(metadata, 결과에 딸린 부가 정보)**를 함께 담은 하나의 묶음으로 돌려줍니다. 후속 단계에서 값을 꺼내 쓰기 좋습니다.
- **stream-json**: 진행 상황을 한 줄씩 흘려보내는 형식으로, 실시간으로 처리하거나 로그로 남길 때 씁니다.
- **왜(why) JSON인가**: 자동화에서는 결과를 다음 단계가 **프로그램으로 읽어 판단**해야 합니다. "리뷰에서 문제를 찾았는가", "토큰을 얼마나 썼는가" 같은 값을 글에서 긁어내는 것보다, JSON에서 정해진 자리의 값을 꺼내는 편이 안정적입니다.
- **언제(when) 쓰나**: CI에서 결과로 후속 동작(댓글 달기·실패 처리·비용 기록)을 정할 때, 또는 여러 실행 결과를 모아 집계할 때입니다.

- **어떻게(how) 통제하나:** `--allowedTools` 로 쓸 도구를 좁히고, `--permission-mode` 로 승인 방식을 정하며, `--max-turns` 로 무한 반복을 막습니다.

CI 자동화 파이프라인 — 헤드리스 Claude Code



ch_11_s02 · CI 헤드리스 자동화 · claude -p (프린트 모드, 헤드리스 실행)

이렇게 설정/실행합니다

가장 단순한 헤드리스 실행입니다. 변경 사항을 한 줄로 요약하게 시키고 결과만 받습니다 (PowerShell 기준).

```
# 한 번에 실행하고 결과만 받기 (대화창 없음)
PS C:\Users\사용자\my-project> claude -p "git에 스테이징된 변경을 한 문장으로 요약해 줘"
```

자동화에서 결과를 프로그램으로 다루려면 JSON으로 받습니다. 또한 사람이 답할 수 없으므로 **읽기 전용 도구만 허용**해 안전하게 제한합니다.

```
# JSON으로 받고, 읽기·검색·git 조회만 허용 (위험한 쓰기/실행은 막음)
PS C:\Users\사용자\my-project> claude -p "이 PR의 위험 요소를 점검해 줘" `
  --output-format json `
  --allowedTools "Read,Grep,Glob,Bash(git diff:*),Bash(git log:*)" `
  --max-turns 5
```

JSON 출력은 다음과 비슷한 구조입니다(값은 예시입니다).

```
{
  "type": "result",
  "subtype": "success",
  "is_error": false,
  "num_turns": 3,
  "result": "스테이징된 변경은 로그인 검증 로직을 수정합니다. 입력값 검사가 빠진 곳이 한 군데 보
  "session_id": "f3a1c9e2-...",
  "total_cost_usd": 0.0123,
  "usage": { "input_tokens": 18450, "output_tokens": 320 }
}
```

헤드리스 실행 주요 옵션

옵션	의미
<code>-p "..."</code>	비대화형으로 이 요청을 처리하고 종료합니다
<code>--output-format text\ json\ stream- json</code>	결과 형식을 정합니다(자동화는 json)
<code>--allowedTools "..."</code>	허용할 도구를 심표로 나열해 한정합니다
<code>--permission-mode <mode></code>	승인 방식을 정합니다(예: plan 은 변경 없이 계 획만)
<code>--max-turns <n></code>	에이전트 루프 반복 횟수 상한으로 폭주를 막습 니다
<code>--model <name></code>	사용할 모델을 지정합니다

JSON 출력에서 자동화가 꺼내 쓰는 값

키	쓰임
<code>result</code>	작업 결과 본문(요약·리뷰 내용 등)입니다
<code>is_error</code>	실패 여부로, CI 단계의 성공·실패 판정에 씁니다
<code>total_cost_usd</code>	이번 실행 비용으로, 비용 기록·한도 관리에 씁니다
<code>usage</code>	입력·출력 토큰 수로, 사용량 집계에 씁니다
<code>session_id</code>	이어서 실행할 때 참조하는 세션 식별자입니다

주의: 헤드리스에서 `--dangerously-skip-permissions` (모든 권한 우회)나 광범위한 `Bash(*)` 허용을 함부로 쓰지 마십시오. 사람이 막아 줄 수 없는 환경에서 위험 명령이 그대로 실행됩니다. 자동화는 **꼭 필요한 도구만 좁게 허용**하고, 쓰기·삭제·푸시가 필요하면 별도의 가드(8장 혹은, 10장 거부 규칙)와 함께 격리된 환경에서만 돌립니다.

팁: 무엇을 바꾸는지 먼저 보고 싶을 때는 `--permission-mode plan` 으로 돌립니다. 계획만 세우고 실제 변경은 하지 않으므로, 자동화 워크플로를 처음 만들 때 **빈 손으로 동작을 검증**하기에 안전합니다.

절 요약

- 헤드리스 실행은 `claude -p` 로 대화창 없이 작업을 한 번에 처리하는 비대화형 방식이며, 사람이 권한에 답할 수 없으므로 허용 도구를 미리 정해야 합니다.
- 구조화 출력(`--output-format json`)은 결과 본문과 비용·토큰·성공 여부 같은 메타데이터를 함께 줘서 후속 단계가 프로그램으로 다루기 좋습니다.
- `--allowedTools` · `--max-turns` · `--permission-mode` 로 자동화의 권한과 범위를 좁게 통제합니다.

GitHub Actions·자동 코드 리뷰 워크플로

도입

헤드리스 실행을 손으로 한 번 돌리는 것과, 그것을 **누가 코드를 올릴 때마다 자동으로** 돌게 하는 것은 다릅니다. 후자가 CI 통합입니다. 가장 널리 쓰이는 **깃허브 액션(GitHub Actions)** 을 예로, PR(Pull Request, 변경을 합쳐 달라는 요청)이 올라오면 Claude Code가 자동으로 코드를 검

토해 댓글을 다는 워크플로를 만들어 봅니다. 이 절의 핵심은 동작 자체보다 **토큰과 권한을 어떻게 새지 않게 다루느냐**입니다.

핵심 개념 설명

CI 통합 (CI integration)

CI 통합(CI integration)이란, 헤드리스 Claude Code를 **CI 파이프라인의 한 단계로 끼워 넣어, 코드 변경 같은 사건이 생길 때 자동으로 실행되게 하는 것**입니다. CI는 코드를 올릴 때마다 서버가 정해진 검사를 자동으로 돌리는 구조이므로, 그 검사 목록에 "Claude Code로 리뷰" 한 줄을 더하는 셈입니다.

깃허브에는 이 연결을 대신 처리해 주는 **공식 액션** `anthropics/claude-code-action@v1` 이 있습니다. PR의 변경 내용과 맥락을 모아 헤드리스 실행에 넘기고, 결과를 PR 댓글이나 커밋으로 되돌려 주는 배선(plumbing)을 자동으로 해 줍니다. 우리는 "언제 돌릴지"와 "무엇을 시킬지", 그리고 "권한·토큰"만 정하면 됩니다.

자동 코드 리뷰 (automated review)

자동 코드 리뷰(automated review)란, PR이 열리거나 갱신될 때 Claude Code가 변경된 코드를 읽고 **버그·보안·스타일 관점에서 점검 의견을 자동으로 남기는 것**입니다. 사람 리뷰어를 대체하는 것이 아니라, 사람이 보기 전에 **명백한 문제를 먼저 걸러 주는 1차 점검**입니다.

여기서 가장 중요한 안전 원칙은 **비밀값 관리**입니다. Claude Code가 깃허브에서 동작하려면 인증용 키(API 키 또는 토큰)가 필요한데, 이 값을 워크플로 파일에 그대로 적으면 저장소를 보는 모두에게 노출됩니다. 그래서 항상 **깃허브 시크릿(Secrets, 암호화 보관함)**에 넣고, 워크플로에 서는 `{{ secrets.이름 }}`으로 **이름만 참조**합니다(10장 환경변수 원칙과 같습니다).

- **왜(why) 쓰나**: 사람 리뷰어의 시간을 아끼고, 리뷰 누락을 줄이며, 팀 전체에 **일관된 점검 기준**을 적용할 수 있습니다.
- **언제(when) 쓰나**: PR이 열리거나 갱신될 때 자동으로, 또는 PR 댓글에서 `@claude` 처럼 호출했을 때입니다.
- **어떻게(how) 안전하게 하나**: 토큰은 시크릿으로 주입하고, 리뷰는 **읽기 위주 권한**으로 제한하며, 워크플로의 깃허브 권한(permissions)도 필요한 만큼만 엽니다.

이렇게 설정/실행합니다

저장소의 `.github/workflows/claude-review.yml`에 워크플로를 둡니다. CI 서버는 보통 리눅스에 서 도므로, 이 파일과 그 안의 명령은 PowerShell이 아니라 깃허브 액션의 YAML 형식입니다.

```
# .github/workflows/claude-review.yml - PR 자동 코드 리뷰 (참고용 예시)
name: Claude Code Review
on:
  pull_request:
    types: [opened, synchronize] # PR 열림·갱신 시 실행

permissions:
  contents: read # 코드 읽기만 허용
  pull-requests: write # PR에 댓글 달기 허용 (그 이상은 없음)

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run Claude Code review
        uses: anthropics/claude-code-action@v1
        with:
          anthropic_api_key: ${ secrets.ANTHROPIC_API_KEY }
          prompt: "이 PR의 변경을 리뷰하고 버그·보안 위험을 한국어로 짚어 줘"
          claude_args: "--allowedTools Read,Grep,Glob --max-turns 8"
```

직접 헤드리스 명령을 워크플로 단계에서 부르는 방식도 있습니다. 이때도 키는 환경변수로만 넘깁니다.

```
- name: Headless review (직접 호출 방식)
  env:
    ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
  run: |
    claude -p "변경된 코드를 리뷰해 줘" \
      --output-format json \
      --allowedTools "Read,Grep,Glob,Bash(git diff:*)" \
      --max-turns 6 > review.json
```

토큰은 깃허브 저장소의 Settings에서 시크릿으로 등록합니다(이 작업은 브라우저에서 합니다).

저장소 → Settings → Secrets and variables → Actions → New repository secret

Name: ANTHROPIC_API_KEY
Secret: (실제 키 값 — 저장 후 다시 볼 수 없음)

워크플로에서 안전을 좌우하는 키

항목	의미·안전 포인트
<code>on: pull_request</code>	언제 자동 실행할지(PR 열림·갱신)를 정합니다
<code>permissions</code>	이 워크플로가 깃허브에 쓸 수 있는 범위입니다. 필요한 만큼만 엽니다
<code>secrets.ANTHROPIC_API_KEY</code>	키를 파일에 적지 않고 암호화 보관함에서 이름으로 참조합니다
<code>claude_args / --allowedTools</code>	Claude Code가 쓸 도구를 읽기 위주로 좁힙니다
<code>--max-turns</code>	자동 실행이 폭주하지 않도록 반복을 제한합니다

주의: 외부 기여자가 보낸 PR에서 자동 워크플로를 돌릴 때는 특히 조심해야 합니다. 신뢰할 수 없는 코드가 워크플로 권한을 악용해 시크릿을 빼낼 수 있기 때문입니다(이런 공격을 막으려면 포크 PR에는 시크릿을 노출하지 않거나 승인 후 실행으로 제한합니다). 또한 `permissions` 는 항상 **필요한 최소**로 두고, `write` 권한을 습관적으로 열지 않습니다.

흔한 실수: 자동 리뷰에 쓰기·푸시 권한을 줘서 Claude Code가 PR을 직접 고치게 두기. 편해 보이지만, 사람 확인 없이 코드가 바뀌면 추적이 어렵고 트레이드오프(편의 대 통제력)에서 통제력을 잃습니다. 자동 단계는 **의견을 댓글로 남기는 데까지**로 두고, 실제 수정은 사람이 검토 후 적용하는 편이 안전합니다.

절 요약

- CI 통합은 헤드리스 Claude Code를 CI 단계로 끼워 코드 변경 시 자동 실행되게 하는 것이며, 깃허브는 공식 액션 `anthropics/claude-code-action@v1` 로 배선을 대신 처리합니다.
- 자동 코드 리뷰는 사람 리뷰 전 1차 점검이며, 키는 깃허브 시크릿으로 주입하고 `${{ secrets.이름 }}` 으로 이름만 참조합니다.
- 워크플로 `permissions` 와 `--allowedTools` 는 최소로 좁히고, 외부 기여자 PR과 쓰기 권한은 특히 신중하게 다룹니다.

팀 표준화 - 명령·에이전트·훅·플러그인 배포

도입

한 사람이 좋은 슬래시 커맨드와 서브에이전트, 안전 혹은 잘 만들어 두었다고 합시다. 그런데 동료들은 그 존재를 모릅니다. 모든 직원이 같은 업무 템플릿과 절차를 쓰도록 사내 표준을 배포하듯, Claude Code의 설정 자산도 **팀 전체가 같은 방식으로 일하도록 공유**해야 합니다. 이 절에서는 저장소를 통한 공유와, 그것을 깔끔하게 묶어 배포하는 플러그인을 다룹니다.

핵심 개념 설명

팀 표준화 (team standardization)

팀 표준화(team standardization)란, `CLAUDE.md`·슬래시 커맨드·서브에이전트·혹 같은 **설정 자산을 팀이 공유해 모두가 일관된 방식으로 일하게 만드는 활동**입니다. 핵심 발상은 단순합니다. 이 자산들은 모두 **프로젝트 폴더 안의 파일**(`.claude/` 폴더와 `CLAUDE.md`)이므로, **저장소에 커밋해 함께 받으면 곧 공유**됩니다. 동료가 저장소를 클론(복제)하기만 해도 같은 명령·에이전트·규칙이 갖춰집니다.

무엇을 공유하고 무엇을 개인으로 둘지 구분이 중요합니다. 팀 표준은

`.claude/commands`·`.claude/agents`·공유 `settings.json`·프로젝트 `CLAUDE.md`처럼 **저장소에 커밋**합니다. 개인 취향이나 비밀값은 `settings.local.json` (10장)처럼 **커밋하지 않습니다**.

플러그인 배포 (plugin distribution)

플러그인(plugin)이란, 슬래시 커맨드·서브에이전트·혹·MCP 서버 설정을 **하나의 묶음으로 포장**한 것입니다. **플러그인 배포(plugin distribution)**는 이 묶음을 **마켓플레이스(marketplace, 플러그인을 모아 나눠 주는 저장소)**를 통해 설치·갱신하게 하는 방식입니다.

저장소에 파일을 직접 커밋하는 방식과의 차이는 **재사용 범위**입니다. 한 프로젝트에만 쓸 자산은 그 저장소에 커밋하면 충분합니다. 그러나 여러 프로젝트·여러 팀이 같은 도구 세트를 쓰게 하려면, 매번 파일을 복사해 붙이는 대신 **플러그인 하나로 묶어 설치·버전 관리**하는 편이 깔끔합니다.

- **왜(why) 표준화하나**: "설정은 각자 알아서"가 효율적이라는 생각은 오해입니다. 사람마다 규칙이 다르면 결과가 들쭉날쭉하고, 안전 혹은 누구는 켜고 누구는 끄면 사고가 납니다. 표준화는 **품질과 안전의 하한선**을 모두에게 보장합니다.
- **언제(when) 무엇을 쓰나**: 한 프로젝트 안 공유는 저장소 커밋, 조직 전반의 재사용은 플러그인·마켓플레이스입니다.
- **어떻게(how) 시작하나**: 먼저 잘 쓰는 자산을 저장소에 커밋해 한 프로젝트에서 안정화한 뒤, 여러 곳에서 필요해지면 플러그인으로 승격합니다.

이렇게 설정/배포합니다

팀과 공유할 자산은 프로젝트의 `.claude/` 폴더 아래에 두고 저장소에 커밋합니다. 전형적인 구조입니다.

```
my-project/
  CLAUDE.md           # 팀 공유 프로젝트 규칙 (커밋)
  .mcp.json           # 팀 공유 MCP 서버 등록 (커밋)
  .claude/
    settings.json      # 팀 공유 설정·권한·흑 (커밋)
    settings.local.json # 개인 설정 (커밋 안 함 / .gitignore)
    commands/
      review.md         # 공유 슬래시 커맨드 /review
      summarize.md      # 공유 슬래시 커맨드 /summarize
    agents/
      code-reviewer.md  # 공유 서브에이전트
```

저장소를 처음 받는 동료가 무엇이 공유되고 무엇이 개인용인지 헷갈리지 않도록, `.gitignore`로 개인 파일을 제외합니다.

```
# .gitignore - 개인·비밀 파일은 공유에서 제외
.claude/settings.local.json
.env
```

조직 차원에서 여러 저장소가 같은 도구 세트를 쓰게 하려면 플러그인 마켓플레이스를 등록해 설치합니다(세션 안에서).

```
# 팀 플러그인 마켓플레이스를 등록하고 플러그인 설치
PS C:\Users\사용자\my-project> claude
> /plugin marketplace add our-org/claude-plugins
> /plugin install team-standards@our-org
```

공유 대상 구분

파일	커밋 여부	이유
CLAUDE.md	커밋	팀 공통 규칙·맥락
.claude/settings.json	커밋	공유 권한·혹 등 안전 경계
.claude/commands/* · agents/*	커밋	팀 공통 명령·에이전트
.mcp.json	커밋	팀 공통 외부 도구 연결
.claude/settings.local.json	제외	개인 설정·비밀값

베스트 프랙티스: 공유 자산도 코드처럼 **리뷰를 거쳐 변경**합니다. 안전 ·혹·권한 규칙은 모두에게 영향을 주므로, 누군가 임의로 바꾸면 팀 전체의 안전 경계가 흔들립니다.

settings.json · CLAUDE.md 변경도 PR로 검토하고, 새 명령·에이전트는 한 프로젝트에서 검증한 뒤 플러그인으로 승격합니다.

흔한 실수: 개인 settings.local.json 이나 토큰이 든 .env 를 실수로 커밋하기. 한 번 저장소 기록에 올라간 비밀값은 사실상 유출로 봐야 합니다. 공유 자산을 정리할 때 **무엇이 커밋되는지 항상 확인**하고, .gitignore 에 개인·비밀 파일을 먼저 등록한 뒤 작업합니다.

절 요약

- 팀 표준화는 CLAUDE.md ·명령·에이전트·혹 같은 설정 자산을 저장소에 커밋해 팀이 일관되게 일하도록 공유하는 활동입니다.
- 공유 자산(.claude/ 커밋)과 개인·비밀 파일(settings.local.json · .env 제외)을 .gitignore 로 분명히 구분합니다.
- 여러 프로젝트·팀이 같은 세트를 쓰게 하려면 플러그인·마켓플레이스로 묶어 설치·버전 관리하며, 공유 자산 변경도 리뷰를 거칩니다.

비용·관측·감사(audit)와 사용량 거버넌스

도입

팀이 Claude Code를 본격적으로 쓰기 시작하면 새로운 질문이 생깁니다. "이번 달에 얼마나 썼지", "누가 무엇을 했지", "비용이 갑자기 튀면 어떻게 알지". 회사가 경비 지출과 출입 기록을 관

리하듯, AI 도구도 얼마나 쓰는지 들여다보고(관측), 누가 무엇을 했는지 남기는(감사) 거버넌스가 필요합니다. 이 절에서는 토큰·비용 관측과 감사 로그를 다룹니다.

핵심 개념 설명

사용량 관측 (usage observability)

사용량 관측(usage observability)이란, Claude Code가 **토큰을 얼마나 쓰고 비용이 얼마나 드는지 들여다볼 수 있게 만드는 것**입니다. 여기서 **토큰(token)**은 모델이 글을 처리하는 기본 단위로, 대략 단어 조각이라고 보면 됩니다. 입력·출력 토큰 수에 따라 비용이 매겨지므로, 토큰 사용량이 곧 비용의 근거입니다.

개인은 세션 안에서 `/cost` 로 현재 세션의 비용을 즉시 볼 수 있습니다. 팀·조직 차원에서는 **OpenTelemetry(오픈텔레메트리, 시스템 사용 지표를 표준 방식으로 내보내는 규격, 줄여서 OTel)**로 토큰·비용 지표를 모니터링 도구에 보내 대시보드로 추적합니다. 헤드리스 실행에서는 앞 절에서 본 JSON 출력의 `total_cost_usd`·`usage` 값을 모아 집계할 수도 있습니다.

감사 로그 (audit log)

감사 로그(audit log)란, 누가·언제·무엇을 했는지 기록으로 남기는 것입니다. 출입 기록부처럼, 나중에 "이 변경은 누가 시켰나", "이 위험 명령은 왜 실행됐나"를 되짚을 수 있게 합니다.

Claude Code에서는 8장에서 배운 **훅(hook)**으로 직접 감사 로그를 남길 수 있습니다.

`PreToolUse`·`PostToolUse` 훅이 도구 사용 후엔 시각·도구·대상을 파일에 기록하게 하면 됩니다.

여기서 흔한 오해는 "**관측과 감사는 대기업만 필요하다**"는 생각입니다. 규모와 무관하게, 비용이 새는지 모르고 지나가거나 사고의 원인을 못 찾는 일은 작은 팀에서 더 뼈아픕니다. 가벼운 비용 확인과 간단한 로깅 훅만으로도 큰 사고를 예방합니다.

- **왜(why) 필요한가**: 비용은 모르면 통제할 수 없고, 기록이 없으면 사고를 추적할 수 없습니다. 관측·감사는 **통제와 책임 추적**의 토대입니다.
- **언제(when) 보나**: 비용은 주기적으로(대시보드·`/cost`), 감사 로그는 사고가 의심될 때와 정기 점검 때 봅니다.
- **어떻게(how) 갖추나**: 개인은 `/cost`, 팀은 OTel 지표 내보내기, 책임 추적은 훅 기반 감사 로깅으로 단계적으로 갖추니다.

이렇게 설정/실행합니다

먼저 개인 차원에서 현재 세션 비용을 확인합니다.

```
> /cost
```

이번 세션 사용량

입력 토큰: 124,300

출력 토큰: 8,420

비용: \$0.21

팀 차원에서는 환경변수로 텔레메트리(사용 지표 내보내기)를 켜고 수집 도구 주소를 지정합니다(PowerShell 기준, 현재 세션에만 적용).

사용 지표 내보내기 활성화 (모니터링 도구로 토큰·비용 전송)

```
PS C:\Users\사용자\my-project> $env:CLAUDE_CODE_ENABLE_TELEMETRY = "1"
```

```
PS C:\Users\사용자\my-project> $env:OTEL_METRICS_EXPORTER = "otlp"
```

```
PS C:\Users\사용자\my-project> $env:OTEL_EXPORTER_OTLP_ENDPOINT = "http://collector.our-org:4317"
```

감사 로그는 흑으로 남깁니다. 공유 `settings.json` 에 도구 사용 후 기록 흑을 등록합니다.

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write|Bash",
        "hooks": [
          {
            "type": "command",
            "command": "pwsh -File C:\\team\\hooks\\audit-log.ps1"
          }
        ]
      }
    ]
  }
}
```

관측·감사 설정 요소

항목	의미
<code>/cost</code>	현재 세션의 토큰·비용을 즉시 보여 줍니다
<code>CLAUDE_CODE_ENABLE_TELEMETRY</code>	사용 지표 내보내기를 켭니다(1 이면 활성화)
<code>OTEL_EXPORTER_OTLP_ENDPOINT</code>	지표를 보낼 수집 도구 주소입니다
<code>hooks.PostToolUse</code>	도구 사용 직후 실행할 감사 혹은 등록합니다
<code>matcher: "Edit\ Write\ Bash"</code>	편집·쓰기·명령 실행을 기록 대상으로 한정합니다

베스트 프랙티스: 감사 로깅 혹은 8장의 Windows 주의사항을 따릅니다. PowerShell 스크립트를 혹은으로 부를 때 경로의 역슬래시는 JSON에서 `\\` 로 적고, 로그 파일은 UTF-8로 기록해 한글이 깨지지 않게 합니다. 비용 관측은 `--output-format json` 의 `total_cost_usd` 를 CI에서 함께 모으면, 자동화 사용량까지 한곳에서 집계할 수 있습니다.

흔한 실수: 감사 로그에 비밀번호·민감 정보를 그대로 남기기. 도구 입력에는 토큰·경로 같은 민감 정보가 섞일 수 있으므로, 로깅 혹은에서 **무엇을 남길지 추리고 비밀번호는 가리도록(마스킹)** 합니다. 기록이 또 다른 유출 통로가 되지 않게 합니다.

절 요약

- 사용량 관측은 토큰·비용을 들여다보는 것이며, 개인은 `/cost` , 팀은 OpenTelemetry 지표 내보내기, 자동화는 JSON 출력의 비용 값으로 집계합니다.
- 감사 로그는 누가·언제·무엇을 했는지 남기는 것이며, `PostToolUse` 혹은으로 도구 사용 내역을 기록합니다.
- 관측·감사는 규모와 무관하게 통제·책임 추적의 토대이며, 로그에 비밀번호가 새지 않게 가립니다.

실습 과제

해보기: PR 자동 점검 워크플로 설계하기

헤드리스 실행으로 변경을 점검하는 자동화를 설계하고, 팀 공유와 거버넌스 계획까지 문서로 정리합니다(실제 키 없이 설계·점검 위주로 진행해도 됩니다).

- **단계 1 (헤드리스 손맛 보기):** 실습 폴더에서 `claude -p "스테이징된 변경을 요약해 줘" --output-format json` 을 돌려, JSON 출력에서 `result · total_cost_usd · is_error` 위치를 확인합니다.
- **단계 2 (권한 좁히기):** 같은 실행에 `--allowedTools "Read,Grep,Glob"` 과 `--max-turns 5` 를 더해, 읽기 위주로 제한했을 때 동작이 어떻게 달라지는지 적습니다.
- **단계 3 (워크플로 설계):** `.github/workflows/claude-review.yml` 초안을 작성합니다. `on: pull_request`, 최소 `permissions`, 키는 `${{ secrets.ANTHROPIC_API_KEY }}` 참조로 둡니다.
- **단계 4 (팀 표준화):** 공유할 자산 (`CLAUDE.md · .claude/commands · .claude/agents · settings.json`)과 제외할 개인·비밀 파일 (`settings.local.json · .env`)을 `.gitignore` 기준으로 분류합니다.
- **단계 5 (거버넌스):** `/cost` 로 비용을 확인하고, `PostToolUse` 감사 혹은 어떤 대상 (`Edit|Write|Bash`)에 걸지 정리합니다.
- **점검:** 아래 표를 채울 수 있으면 성공입니다.

항목	내 설계
자동 실행 시점(on)	...
허용 도구(allowedTools)	...
깃허브 permissions	...
키 주입 방식	secrets 참조
공유 자산 / 제외 파일	... / ...
비용·감사 방법	...

FAQ

Q1. 헤드리스(`claude -p`) 자동화에서 권한 요청이 뜨면 작업이 멈추지 않나요? → A1. 그렇습니다. 사람이 답할 수 없으므로 권한 대기에서 멈추거나 실패합니다. 그래서 자동화는 `--allowedTools` 로 쓸 도구를 미리 허용하거나, 변경이 필요 없으면 `--permission-mode plan` 으로 계획만 세우게 합니다. "전부 우회"는 편하지만 사람의 제동이 사라지므로, 격리된 환경에서 꼭 필요한 도구만 좁게 여는 편이 안전합니다.

Q2. CI에서 쓰는 API 키나 토큰은 어디에 뒀야 안전한가요? → A2. 워크플로 파일이나 코드에 직접 적지 않고 깃허브 시크릿(Secrets)에 넣은 뒤 `{{ secrets.이름 }}` 으로 이름만 참조합니다 (10장 환경변수 원칙과 같습니다). 워크플로의 `permissions` 도 필요한 만큼만 열고, 외부 기여자가 보낸 PR에는 시크릿이 노출되지 않도록 승인 후 실행 등으로 제한합니다.

Q3. 팀에 공유한 슬래시 커맨드·에이전트·혹을 동료가 쓰려면 따로 설치해야 하나요? → A3. 저장소에 커밋된 `.claude/commands` · `.claude/agents` · 공유 `settings.json` 은 동료가 저장소를 클론해 그 폴더에서 Claude Code를 열면 자동으로 적용됩니다. 별도 설치가 필요 없습니다. 다만 여러 프로젝트에서 같은 세트를 쓰려면 플러그인으로 묶어 `/plugin install` 로 배포하는 편이 깔끔합니다. 개인 설정(`settings.local.json`)은 공유되지 않습니다.

장 요약

이번 장에서는 한 사람의 대화를 넘어 **여러 인터페이스·팀·자동화**로 Claude Code를 확장했습니다. CLI·IDE·데스크톱·웹은 같은 엔진을 쓰는 표면이라 **설정·메모리·권한이 공유**되므로, 상황에 맞는 표면을 골라 끊김 없이 이어 갈 수 있습니다(설정 연속성). 사람 없이 도는 자동화는 **헤드리스 실행**(`claude -p`) 과 **구조화 출력(JSON)** 으로 만들되, 권한에 답할 사람이 없으므로 허용 도구를 미리 좁게 정하는 것이 핵심입니다. 이를 **깃허브 액션**에 끼워 PR 자동 코드 리뷰를 구성할 때는, 토큰을 시크릿으로 주입하고 권한을 최소로 여는 안전이 동작보다 중요합니다. 팀 차원에서는 명령·에이전트·혹을 저장소와 **플러그인**으로 표준화해 모두가 일관되게 일하게 하고, 개인·비밀 파일은 분명히 분리합니다. 마지막으로 토큰·비용을 들여다보는 **사용량 관측**과 누가 무엇을 했는지 남기는 **감사 로그**로, 개인을 넘어 팀·조직이 Claude Code를 안전하게 통제·책임 추적하는 거버넌스를 갖춥니다.

핵심 키워드

인터페이스 선택(interface choice), 설정 연속성(config continuity), 헤드리스 실행(headless mode), 비대화형 출력(structured output), CI 통합(CI integration), 자동 코드 리뷰(automated review), 팀 표준화(team standardization), 플러그인 배포(plugin distribution), 사용량 관측(usage observability), 감사 로그(audit log)

다음 장 미리보기

다음 장은 이 책의 마지막으로, Claude Code 엔진을 코드에서 직접 부르는 **Agent SDK**와 계획 모드·백그라운드·멀티에이전트 같은 고급 워크플로, 그리고 인증·권한·MCP·컨텍스트 문제를 증상별로 진단하는 트러블슈팅까지 다뤄 실무 역량을 마무리합니다.

Agent SDK 실전·고급 워크플로·트러블슈팅

학습 목표 - 계획 모드(plan mode)·백그라운드 작업(background task)·멀티에이전트를 엮어 고급 워크플로를 설계할 수 있습니다. - Agent SDK(에이전트 개발 키트)가 Claude Code와 같은 엔진을 코드에서 부르는 도구임을 이해하고 `query()` 루프를 설명할 수 있습니다. - SDK 옵션으로 도구·권한·서브에이전트(subagent)를 갖춘 커스텀 에이전트를 구성할 수 있습니다. - 증상별 진단 순서로 인증·권한·MCP 연결·컨텍스트 문제를 스스로 좁혀 해결할 수 있습니다. - 이 책 이후에 무엇을 더 배울지 학습 로드맵을 세울 수 있습니다.

여기까지 우리는 Claude Code를 **사람이 직접 대화창에서 쓰는 도구**로 다뤄 왔습니다. 이 마지막 장에서는 한 걸음 더 나아갑니다. 같은 엔진을 **계획적으로·동시에·자동으로** 돌리는 고급 워크플로를 보고, 그 엔진을 **코드에서 직접 호출**하는 Agent SDK를 익히며, 일이 어긋날 때 **증상으로 원인을 좁히는** 트러블슈팅까지 다룹니다.

이 장은 이 책의 가장 깊은 실무 단계입니다. 단순한 사용법을 넘어 **언제 쓰고(when)**, **왜 그렇게 하며(why)**, **어떻게 안전하게 적용하는가(how)** 와 함께, 실무에서 만나는 **트레이드오프(장단점 맞교환)** 를 짚습니다. 이 책은 Windows 11의 **PowerShell(파워셸)** 을 기준으로 하며, 명령은 `PS C:\Users\사용자\my-project>` 같은 프롬프트에서 입력합니다. 참고로 **SDK(Software Development Kit)** 는 "소프트웨어를 만들 때 쓰는 도구 모음"을 뜻하는 약어입니다.

고급 워크플로 - 계획 모드·백그라운드·멀티에이전트

도입

집을 고칠 때 곧바로 벽을 부수는 사람은 없습니다. 먼저 **설계도를 그려 검토받고**, 큰 공사는 **전문 팀에 나눠 맡기며**, 오래 걸리는 작업은 **따로 돌려 두고** 다른 일을 합니다. Claude Code의 고급 워크플로도 똑같습니다. 이 절에서는 실행 전에 계획을 검토하는 **계획 모드**, 장시간 작업을 떼어 돌리는 **백그라운드 작업**, 여러 에이전트가 동시에 일하는 **멀티에이전트**를 하나로 엮는 법을 봅니다.

핵심 개념 설명

계획 모드 (plan mode)

계획 모드(plan mode)란, 파일을 실제로 바꾸기 *전에* Claude Code가 "무엇을 어떤 순서로 할지" **실행 계획을 먼저 글로 제시하고, 사람이 검토·승인한 뒤에야** 실행으로 넘어가는 작업 방식입니다. 4장에서 본 승인 모드(approval mode)의 한 종류로, 공사 전 설계도 검토에 해당합니다.

여기서 흔한 오해 하나를 짚습니다. "**계획 모드에서도 파일이 곧바로 바뀐다**" 는 생각입니다. 그렇지 않습니다. 계획 모드에서는 Claude Code가 코드베이스를 **읽기만** 하고, 편집·명령 실행 같은 영향 있는 동작은 **하지 않습니다**. 계획을 승인해야 비로소 실행 단계로 전환됩니다. 그래서 큰 변경 앞에서 "먼저 방향을 맞추는" 안전한 출발점이 됩니다.

- **왜(why) 쓰나**: 큰 리팩터링이나 낫선 코드베이스에서는 곧바로 편집하게 두면 엉뚱한 방향으로 많이 고쳐 놓을 위험이 있습니다. 계획을 먼저 보면 **방향이 틀렸을 때 토큰과 시간을 낭비하기 전에** 바로잡을 수 있습니다.
- **언제(when) 쓰나**: 변경 범위가 넓을 때, 코드베이스를 처음 다룰 때, 되돌리기 어려운 작업(마이그레이션 등) 전입니다. 반대로 사소한 한 줄 수정에는 과합니다.
- **어떻게(how) 커나**: 세션에서 `Shift+Tab` 으로 승인 모드를 순환해 "plan" 으로 맞추거나, 시작할 때 `claude --permission-mode plan` 으로 켭니다.

백그라운드 작업 (background task)

백그라운드 작업(background task)이란, 테스트 실행이나 빌드처럼 **오래 걸리는 명령을 따로 떼어 돌려 두고**, 그 사이 Claude Code와 다른 작업을 계속 이어 가는 방식입니다. 작업이 끝나면 결과를 가져와 확인합니다. 빨래를 세탁기에 돌려 두고 설거지를 하는 것과 같습니다.

- **왜(why) 쓰나**: 장시간 명령이 끝날 때까지 대화가 멈춰 있으면 비효율적입니다. 백그라운드로 돌리면 **대기 시간에 다른 진전을** 만듭니다.
- **언제(when) 쓰나**: 개발 서버 띄우기, 전체 테스트 스위트 실행, 긴 빌드처럼 **몇 분 이상 걸리는 명령일** 때입니다.
- **어떻게(how) 하나**: 긴 명령은 백그라운드로 보내고(예: `Ctrl+B` 로 실행 중 명령을 백그라운드 전환), 진행 중 작업은 `/bashes` 로 목록을 보고 결과를 회수합니다.

멀티에이전트와 오케스트레이션 (orchestration)

멀티에이전트는 7장에서 본 **서브에이전트**를 여러 개 동시에 굴리는 패턴입니다. 메인 에이전트가 **오케스트레이터(지휘자)**가 되어, 조사·구현·검증 같은 일을 각자 독립 컨텍스트를 가진 서브에이전트에게 **병렬로** 나눠 주고 결과를 모읍니다. 악단의 지휘자가 파트별 연주자에게 동시에 신호를 주는 모습과 같습니다.

다만 실무에서 가장 중요한 것은 **트레이드오프**입니다. 에이전트를 늘리면 동시에 일이 진행돼 빨라지는 듯하지만, 에이전트마다 별도 컨텍스트를 채우느라 **토큰과 비용이 함께 늘고**, 결과를

합치는 과정에서 복잡도도 커집니다. "많이 쓸수록 항상 빠르고 싸진다"는 생각은 오해입니다. 작업이 독립적으로 나뉘고 각 부분이 충분히 무거울 때만 멀티에이전트가 이득입니다.



ch_12_s01 · Agent SDK · query() 호출 구조 · 도구 실행 흐름

이렇게 실행합니다

세 가지를 엮은 전형적인 실무 흐름은 "계획 → 승인 → (백그라운드 + 멀티에이전트) 실행"입니다. 아래는 계획 모드로 시작하는 예상 화면입니다(실제 입력이 아니라 흐름을 보여 주는 예시입니다).

```

PS C:\Users\사용자\my-project> claude --permission-mode plan
> 결제 모듈을 신규 통화 단위(KRW)까지 지원하도록 리팩터링해 줘

(plan) 코드베이스를 읽었습니다. 다음 계획을 제안합니다. 실행하지 않습니다.

1. price 계산 함수 3곳을 통화 인자를 받도록 수정
2. 통화별 반올림 규칙 테이블 추가
3. 기존 테스트를 통화 파라미터로 확장하고 전체 테스트 실행

이 계획대로 진행할까요? [ 1. 승인하고 실행 2. 계획 수정 3. 취소 ]
    
```

계획을 승인하면 실행 단계로 넘어갑니다. 이때 시간이 오래 걸리는 전체 테스트는 백그라운드로 돌리고, 그 사이 다른 점검을 진행할 수 있습니다.

> 1

[Edit] price.py 수정 완료

[Bash] pytest 전체 실행을 백그라운드로 시작했습니다 (id: bash_1)

> 테스트가 도는 동안 README의 통화 설명도 갱신해 줘
... (백그라운드 테스트와 별개로 문서 작업 진행)

> /bashes

bash_1 running pytest -q 2분 12초 경과

고급 워크플로 도구 정리

표시·명령	의미
<code>--permission-mode plan</code>	계획 모드로 세션을 시작합니다(읽기만, 변경 안 함)
(plan) 프롬프트	지금 이 계획 단계라 파일이 바뀌지 않음을 알립니다
<code>Shift+Tab</code>	세션 중 승인 모드를 순환 전환합니다
백그라운드 (id: bash_1)	장시간 명령을 떼어 돌릴 때 붙는 식별표입니다
<code>/bashes</code>	백그라운드로 돌고 있는 작업 목록과 상태를 봅니다

팁: 낯선 코드베이스에 처음 손댈 때는 습관처럼 계획 모드로 시작합니다. 계획만 받아 봐도 Claude Code가 코드 구조를 어떻게 이해했는지 드러나므로, 잘못 이해했다면 변경을 시작하기 전에 바로잡을 수 있습니다.

흔한 실수: "빨리 끝내려고" 작은 작업에까지 멀티에이전트를 동원하기. 서브에이전트는 저마다 컨텍스트를 새로 채우므로 **고정 비용**이 있습니다. 한두 파일을 고치는 일이라면 단일 에이전트가 더 빠르고 쌉니다. 멀티에이전트는 "조사 따로, 구현 따로"처럼 **일이 분명히 나뉠 때만** 씁니다.

절 요약

- 계획 모드는 변경 전에 실행 계획을 검토·승인받는 안전한 출발점이며, 큰 변경·낯선 코드에서 특히 유용합니다.
- 백그라운드 작업은 장시간 명령을 떼어 돌려 대기 시간을 줄입니다(`/bashes` 로 관리).
- 멀티에이전트는 일이 독립적으로 나뉠 때만 이득이며, 토큰·비용·복잡도라는 트레이드오프가 따릅니다.

Agent SDK 개요(Python·TypeScript)와 query 루프

도입

지금까지는 사람이 대화창에 앉아 Claude Code를 부렸습니다. 그런데 "매일 밤 로그를 요약해 슬랙에 올린다" 같은 일은 사람이 매번 앉아 있을 수 없습니다. 이럴 때 **같은 엔진을 코드로 부르는** 방법이 Agent SDK입니다. 리모컨 API에 비유하면, 손으로 누르던 가전을 코드가 대신 조작하게 해 주는 셈입니다. 이 절에서는 SDK가 무엇이고, 그 핵심인 `query()` 루프가 어떻게 도는지 봅니다.

핵심 개념 설명

Agent SDK (에이전트 SDK)

Agent SDK(에이전트 SDK)란, Claude Code와 똑같은 **에이전트 루프·도구·컨텍스트** 관리를 파이썬(Python)·타입스크립트(TypeScript) 코드에서 **프로그래밍 방식으로 부를 수 있게** 해 주는 개발 라이브러리입니다. 예전에는 "Claude Code SDK"라고 불렸고, 지금은 **Agent SDK**라는 이름으로 정리되었습니다.

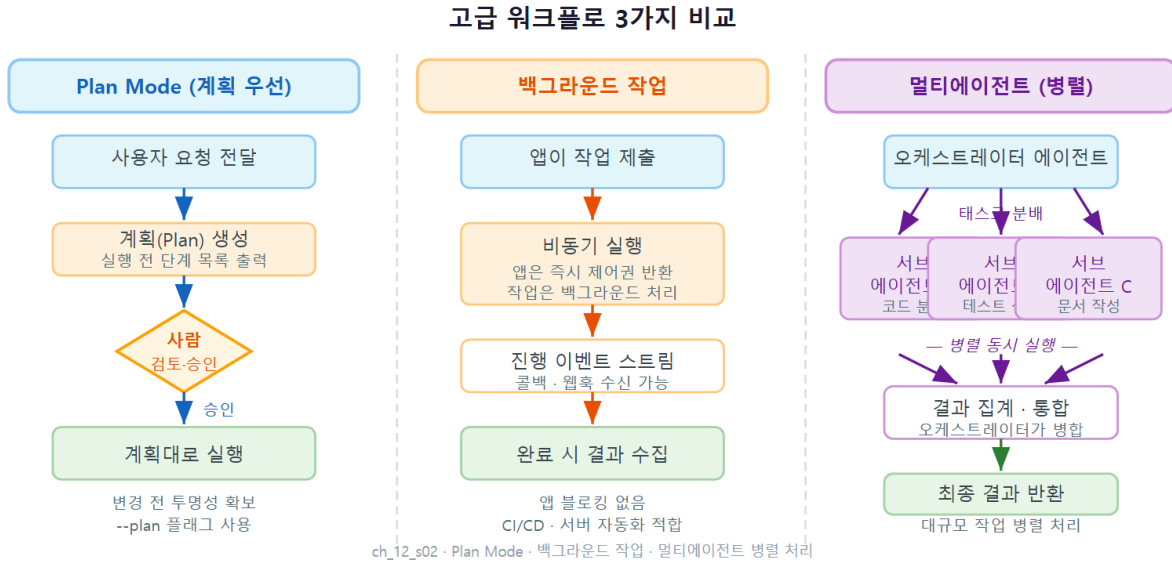
여기서 가장 중요한 오해를 바로잡습니다. **"SDK는 CLI와 전혀 다른 별개 엔진"**이라는 생각입니다. 그렇지 않습니다. **엔진은 같습니다.** 대화창(CLI)에서 쓰던 권한 모델, CLAUDE.md 메모리, 서버에이전트, MCP 연결이 SDK에서도 그대로 작동합니다. 차이는 오직 **입구**입니다. CLI는 사람이 키보드로, SDK는 프로그램이 코드로 같은 엔진을 부릅니다.

- **왜(why) 쓰나:** 정해진 시각의 자동 실행, CI 파이프라인 통합, 사내 웹 서비스에 에이전트 기능 내장처럼 **사람 없이 반복·자동화**해야 할 때입니다.
- **언제(when) CLI 대신 SDK인가:** 한 번 손으로 하는 일이면 CLI가, 같은 일을 **반복·무인·대규모**로 돌려야 하면 SDK가 맞습니다.
- **어떻게(how) 시작하나:** 파이썬은 `pip install claude-agent-sdk`, 타입스크립트는 `npm install @anthropic-ai/claude-agent-sdk`로 설치합니다. 인증은 CLI와 같은 방식(구독 로그인 또는 API 키)을 씁니다.

query 루프 (query loop)

`query()`는 SDK의 가장 기본 입구입니다. 프롬프트(지시)와 옵션을 넘기면, Claude Code와 동일한 **에이전트 루프**(모델 턴 → 도구 호출 → 도구 결과 → 다시 모델 턴)가 돌면서, 중간에 일어나는 일들을 **스트림(stream, 흐름)**으로 하나씩 돌려줍니다. 즉 결과만 한 번에 받는 게 아니라, "지금 파일을 읽었다", "이 명령을 실행하려 한다" 같은 **중간 메시지**를 차례로 받습니다.

이 스트림 구조가 중요한 이유는 **에이전트 루프가 본래 여러 턴을 돈다**는 데 있습니다(1장). 한번 부탁에 한 번 응답이 아니라, 목표를 이룰 때까지 읽고·고치고·실행하며 반복합니다. `query()` 는 그 반복의 각 단계를 프로그램이 지켜보고 개입할 수 있게 열어 줍니다.



명령·함수 시그니처

```
async def query(*, prompt, options=None) -> AsyncIterator[Message]
```

`query()` 는 비동기(async) 함수로, 메시지를 하나씩 흘려보내는 **비동기 반복자**를 돌려줍니다. 그래서 `async for` 로 메시지를 한 건씩 받아 처리합니다.

이렇게 작성합니다

아래는 폴더의 변경 사항을 한 줄로 요약하게 하는 최소 파이썬 예시입니다. 구조를 보여 주기 위한 **참고용(비실행) 코드**입니다.

```
# summarize.py - 변경 요약을 받아 출력하는 최소 SDK 예시 (참고용)
import anyio
from claude_agent_sdk import query, ClaudeAgentOptions

async def main() -> None:
    # 읽기 계열 도구만 허용해 안전하게 돕니다.
    options = ClaudeAgentOptions(
        allowed_tools=["Read", "Grep", "Bash"],
        permission_mode="plan", # 변경은 막고 분석만 시킵니다
        cwd="C:\\Users\\사용자\\my-project",
    )

    # query()가 돌려주는 메시지를 하나씩 받아 처리합니다.
    async for message in query(
        prompt="최근 변경된 파일들을 읽고 한 줄로 요약해 줘",
        options=options,
    ):
        print(message)

if __name__ == "__main__":
    anyio.run(main)
```

타입스크립트도 발상이 같습니다. `query()` 가 돌려주는 흐름을 `for await` 로 받습니다.

```
// summarize.ts - 같은 일을 하는 타입스크립트 최소 예시 (참고용)
import { query } from "@anthropic-ai/claude-agent-sdk";

const stream = query({
    prompt: "최근 변경된 파일들을 읽고 한 줄로 요약해 줘",
    options: {
        allowedTools: ["Read", "Grep", "Bash"],
        permissionMode: "plan",
    },
});

for await (const message of stream) {
    console.log(message);
}
```

실행하면 결과만 한 번에 나오지 않고, 루프의 각 단계가 메시지로 흘러나옵니다(예상 출력 형태).

```
PS C:\Users\사용자\my-project> python summarize.py
[assistant] 변경된 파일을 살펴보겠습니다.
[tool_use] Bash: git diff --name-only
[tool_result] price.py, README.md
[tool_use] Read: price.py
[assistant] 결제 계산에 통화 인자를 추가하고 README에 통화 설명을 보강했습니다.
[result] 완료 (turns=3, 소요 8.2초)
```

query() 옵션 키 설명

키	의미
<code>prompt</code>	에이전트에게 줄 지시(자연어)입니다
<code>allowed_tools</code>	사람 확인 없이 자동 허용할 도구 목록입니다(예: <code>Read</code>)
<code>permission_mode</code>	권한 처리 방식입니다(<code>plan</code> 분석만 / <code>acceptEdits</code> 편집 자동수락 등)
<code>cwd</code>	에이전트가 작업할 폴더입니다(Windows 경로는 <code>\\</code> 로 적습니다)
<code>async for</code> / <code>for</code> <code>await</code>	루프가 흘려보내는 메시지를 한 건씩 받는 구문입니다

베스트 프랙티스: 무인 자동화에서는 `permission_mode` 를 함부로 `bypassPermissions` (전체 우회) 로 두지 않습니다. 대신 `allowed_tools` 로 **꼭 필요한 도구만** 자동 허용하고, 위험한 도구는 빼둡니다. 사람이 지켜보지 않는 만큼 권한은 더 좁혀야 안전합니다.

흔한 실수: `query()` 가 한 번에 최종 답만 준다고 기대하기. `query()` 는 루프의 **중간 메시지까지 모두** 흘려보냅니다. 최종 결과만 필요하다면 메시지 종류를 보고 `result` 유형만 추려 쓰거나, 마지막 메시지를 모아 처리해야 합니다.

절 요약

- Agent SDK는 CLI와 **같은 엔진**을 코드(Python·TypeScript)에서 부르는 라이브러리로, 입구만 다릅니다.
- `query()` 는 프롬프트·옵션을 받아 에이전트 루프를 돌리고, 중간 메시지를 스트림으로 돌려줍니다.
- 무인 실행일수록 `allowed_tools` · `permission_mode` 로 권한을 좁혀 안전하게 둡니다.

SDK로 커스텀 에이전트 만들기 - 도구·권한·서브에이전트

도입

앞 절의 `query()` 가 "엔진에 시동을 거는" 단계라면, 이번 절은 "운전 방식을 내 손에 맞게 설정하는" 단계입니다. 어떤 도구를 쓸지, 위험한 작업은 누가 승인할지, 어떤 전문 서브에이전트를 둘지를 **코드 옵션**으로 정합니다. 이렇게 하면 "우리 업무에만 맞는" 작은 에이전트를 직접 빚어 낼 수 있습니다.

핵심 개념 설명

프로그래밍 가능한 에이전트 (programmable agent)

프로그래밍 가능한 에이전트(programmable agent)란, 도구·권한·역할·서브에이전트 구성을 **코드 옵션**으로 지정해 만든 맞춤 에이전트입니다. CLI에서

CLAUDE.md · `.claude/agents` · `settings.json`으로 하던 설정을, SDK에서는 `ClaudeAgentOptions` (타입스크립트는 `options`) 한곳에 모아 코드로 정의합니다.

- **왜(why) 만드나**: "폴더 정리 요약 봇", "사내 문서 질의 봇"처럼 **한 가지 일을 정해진 권한으로 반복**시키려면, 매번 손으로 설정하기보다 코드에 굳혀 두는 편이 일관되고 안전합니다.
- **언제(when) 만드나**: 같은 작업을 여러 번·여러 사람·자동 일정으로 돌릴 때, 또는 다른 프로그램 안에 에이전트 기능을 끼워 넣을 때입니다.
- **어떻게(how) 구성하나**: ① 시스템 프롬프트로 역할을 부여하고 ② `allowed_tools` 로 도구를 좁히고 ③ 필요하면 서브에이전트(`agents`)를 정의하며 ④ 위험 작업은 권한 콜백으로 사람이 확인하게 합니다.

권한 콜백 (permission callback)과 사람 확인 (human-in-the-loop)

권한 콜백(permission callback)이란, 에이전트가 도구를 쓰려 할 때마다 **내 코드의 함수를 거치게 해, 허용·수정·거부를 프로그램이 직접 결정**하게 하는 장치입니다. CLI에서 사람이 보던 권한 요청 창을, SDK에서는 **내가 짰 함수**가 대신 판단합니다. 파이썬에서는 `can_use_tool` 함수를 옵션에 넘깁니다.

이것이 곧 **사람 확인(human-in-the-loop) 체크포인트**의 토대입니다. 무인으로 돌리되, 되돌리기 어려운 작업(파일 삭제·외부 발송 등)만은 콜백 안에서 멈춰 사람에게 묻거나, 규칙에 따라 자

동 거부할 수 있습니다. "전부 자동" 과 "전부 수동" 사이에서 **위험한 지점에만 사람을 끼워 넣는** 설계입니다.

흔한 오해는 **"에이전트 정의에 권한을 안 정해도 안전하다"** 는 생각입니다(7장에서도 같은 함정을 봤습니다). 도구를 열어 둔 채 무인으로 돌리면, 의도치 않은 삭제나 외부 호출이 사람 눈 없이 일어날 수 있습니다. **최소 권한(least privilege)** 원칙은 SDK에서 더욱 중요합니다.

이렇게 구성합니다

아래는 도구를 좁히고, 위험 작업은 콜백으로 거르며, 문서화 전용 서브에이전트를 둔 파이썬 예시입니다. 구조 설명용 **참고용(비실행) 코드**입니다.

```
# cleanup_agent.py - 도구·권한·서브에이전트를 갖춘 커스텀 에이전트 (참고용)
import anyio
from claude_agent_sdk import query, ClaudeAgentOptions

# 권한 콜백: 위험한 도구는 사람 확인 또는 자동 거부로 거릅니다.
async def can_use_tool(tool_name, tool_input, context):
    # 삭제 계열 명령은 무인 실행에서 막습니다.
    if tool_name == "Bash" and "rm" in tool_input.get("command", ""):
        return {"behavior": "deny", "message": "삭제 명령은 자동 실행에서 금지됩니다."}
    return {"behavior": "allow"}

async def main() -> None:
    options = ClaudeAgentOptions(
        system_prompt="너는 폴더를 분석해 정리 제안을 요약하는 보조자다.",
        allowed_tools=["Read", "Grep", "Bash"],
        can_use_tool=can_use_tool,          # 도구 사용 전 내 함수를 거칩니다
        agents={                            # 전용 서브에이전트 정의
            "doc_writer": {
                "description": "정리 결과를 마크다운 보고서로 작성하는 담당",
                "tools": ["Read", "Write"],
            }
        },
        cwd="C:\\Users\\사용자\\my-project",
    )

    async for message in query(
        prompt="downloads 폴더를 분석해 정리안을 요약하고 보고서로 정리해 줘",
        options=options,
    ):
        print(message)

if __name__ == "__main__":
    anyio.run(main)
```

실행하면 권한 콜백이 위험 명령을 가로채는 모습을 볼 수 있습니다(예상 출력).

```
PS C:\Users\사용자\my-project> python cleanup_agent.py
[assistant] downloads 폴더를 살펴봅니다.
[tool_use] Bash: ls downloads
[tool_result] (파일 목록)
[tool_use] Bash: rm downloads/old.zip
[permission] deny: 삭제 명령은 자동 실행에서 금지됩니다.
[assistant] 삭제는 보류하고, 정리 대상만 보고서로 정리하겠습니다.
[subagent: doc_writer] cleanup-report.md 작성 완료
```

커스텀 에이전트 옵션 설명

옵션	의미
<code>system_prompt</code>	에이전트의 역할·말투를 정하는 지침입니다
<code>allowed_tools</code>	자동 허용할 도구만 추려 권한을 줍니다
<code>can_use_tool</code>	도구 사용 전 호출되는 권한 콜백 함수입니다(<code>allow</code> / <code>deny</code>)
<code>agents</code>	코드에서 정의하는 전용 서브에이전트 모음입니다
<code>behavior: "deny"</code>	그 도구 사용을 차단하고 사유를 돌려줍니다

주의: 권한 콜백 없이 `allowed_tools` 에 `Bash` 를 넣고 무인으로 돌리면, 에이전트가 임의의 셸 명령을 사람 확인 없이 실행할 수 있습니다. 자동화에서 셸을 허용해야 한다면 **반드시 권한 콜백으로 위험 명령(삭제·강제 푸시·외부 전송 등)을 거르거나**, 8장의 훅(hook)으로 한 번 더 가드 레일을 칩니다.

팁: SDK용 MCP 도구는 `create_sdk_mcp_server` 로 **프로그램 안에 직접** 만들어 붙일 수 있습니다. 9장에서 별도 프로세스로 띄우던 MCP 서버를, SDK에서는 같은 프로그램 안의 함수로 노출해 더 가볍게 쓸 수 있습니다.

절 요약

- 커스텀 에이전트는 시스템 프롬프트·도구·권한·서브에이전트를 코드 옵션으로 모아 정의합니다.
- 권한 콜백(`can_use_tool`)은 사람 확인 창을 대신해 위험 작업을 코드로 허용·거부합니다.
- 무인 실행일수록 최소 권한이 중요하며, 콜백·훅으로 위험 지점에만 사람·가드레일을 끼워 넣습니다.

트러블슈팅 - 인증·권한·MCP·컨텍스트 진단

도입

차가 안 걸린다고 곧장 엔진을 통째로 갈지는 않습니다. 배터리 → 연료 → 점화 순으로 **원인을 좁혀** 갑니다. Claude Code 문제도 똑같습니다. 오류가 났다고 무작정 재설치하기 전에, **증상을 단서로 원인 후보를 줄여** 가는 순서를 익히면 대부분 몇 분 안에 해결됩니다. 이 절에서는 가장 자주 만나는 네 갈래(인증·권한·MCP·컨텍스트)의 진단 순서를 봅니다.

핵심 개념 설명

문제 해결 (troubleshooting)과 진단 순서 (diagnostic flow)

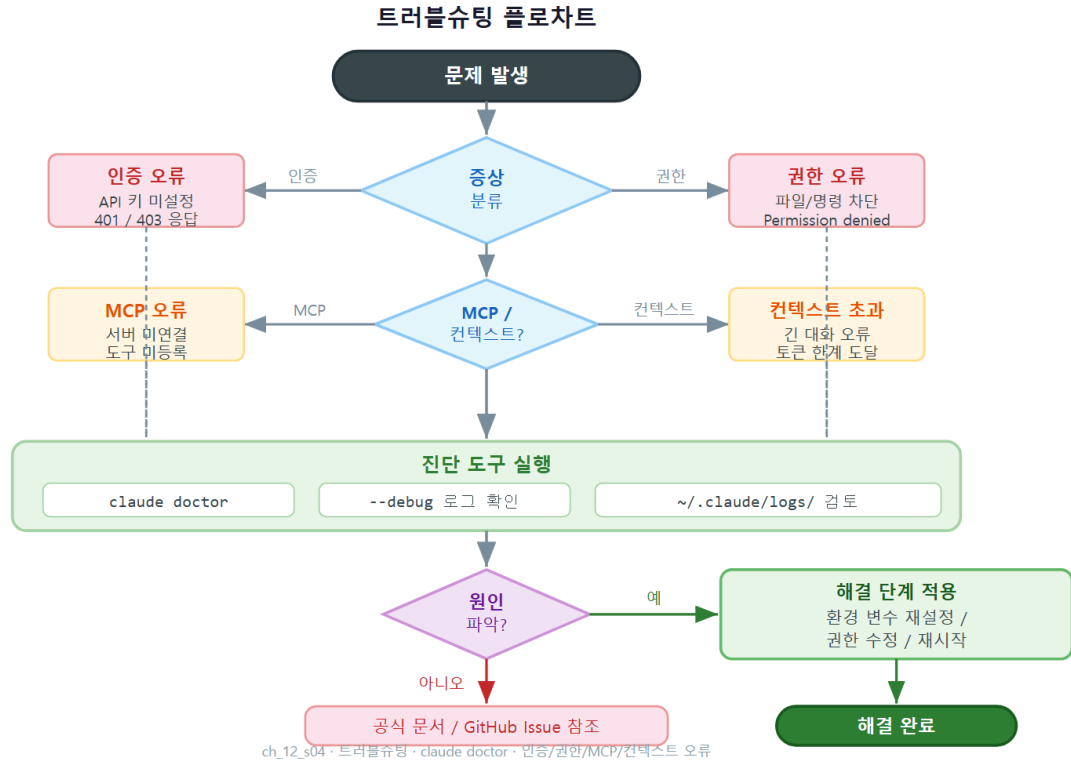
문제 해결(troubleshooting)이란, 증상을 단서로 **원인 후보를 단계적으로 좁혀** 로그·설정을 점검해 해결하는 과정입니다. 핵심은 **순서**입니다. 흔하고·확인이 쉬운 원인부터 점검해, 비싼 조치(재설치 등)는 마지막에 둡니다.

가장 먼저 쓰는 만능 진단 명령은 `claude doctor` 입니다. 설치 상태·버전·인증·설정 위치 등을 한번에 점검해, "어디가 문제인지"의 첫 단서를 줍니다. 차의 계기판 경고등을 읽는 것과 같습니다.

흔한 오해는 "**오류가 나면 무조건 재설치부터**" 입니다. 대부분의 문제는 인증 만료·권한 규칙·경로·컨텍스트 한도처럼 **설정 수준**에서 생기므로, 재설치는 거의 마지막 수단입니다.

아래는 네 갈래 증상의 진단 순서입니다.

- **인증 실패(로그인 안 됨·401 오류):** ① `claude doctor` 로 인증 상태 확인 → ② `/login` 으로 재로그인(토큰 만료가 가장 흔함) → ③ API 키 방식이면 환경변수(`ANTHROPIC_API_KEY`)가 비었는지 확인 → ④ 회사망이면 프록시·방화벽 점검.
- **도구가 자꾸 거부됨(권한):** ① 지금 승인 모드 확인(계획 모드면 편집이 막힘) → ② `settings.json`의 `deny` 규칙에 걸렸는지 확인 → ③ 관리형 정책(10장)이 조직 차원에서 막는지 확인. 관리형 정책은 개인이 못 풉니다.
- **MCP 연결 안 됨:** ① `/mcp` 로 "연결됨" 인지 확인 → ② `.mcp.json` 의 경로·역슬래시(`\\`)·`command` 점검(9장) → ③ 서버 실행 명령을 PowerShell에서 직접 돌려 보고 자체 오류 확인 → ④ 토큰이 필요한 서버면 환경변수 주입 확인.
- **컨텍스트 부족(대화가 길어 느려지거나 잘림):** ① 상태 줄의 컨텍스트 사용량 확인(2장) → ② `/compact` 로 요약 압축, 새 주제면 `/clear` → ③ 큰 파일을 통째로 넣지 말고 필요한 부분만 참조.



이렇게 진단합니다

먼저 만능 점검 명령입니다. 어떤 문제든 막히면 여기서 시작합니다.

```
# 설치·버전·인증·설정 위치를 한 번에 점검
claude doctor
```

예상 화면은 다음과 같습니다. 경고(!) 표시가 첫 단서입니다.

```
PS C:\Users\사용자\my-project> claude doctor
```

Claude Code 진단

```
설치      OK   버전 x.y.z (최신)
인증      !   토큰이 만료되었습니다. /login 으로 다시 로그인하세요.
설정 파일 OK   C:\Users\사용자\.claude\settings.json
MCP 서버  OK   2개 연결됨
```

권장 조치: /login 후 다시 시도하세요.

증상별로 이어지는 점검 명령을 모았습니다.

```

claude doctor      # 1) 항상 여기서 시작
claude /login      # 인증: 토큰 만료 시 재로그인
claude mcp list    # MCP: 등록·연결 상태 확인
claude --debug     # 자세한 로그를 보며 원인 추적

```

세션 안에서 권한·컨텍스트 문제는 슬래시 커맨드로 점검합니다.

```

> /status      (현재 모델·승인 모드·컨텍스트 사용량 확인)
> /permissions (적용 중인 allow/ask/deny 규칙 확인)
> /compact     (컨텍스트가 차오를 때 요약 압축)

```

증상 → 첫 점검 빠른 표

증상	먼저 의심할 원인	첫 점검
로그인·401 오류	토큰 만료	<code>claude doctor</code> → <code>/login</code>
도구가 계속 거부됨	계획 모드·deny 규칙·관리형 정책	<code>/status</code> → <code>/permissions</code>
MCP 도구 안 보임	경로·역슬래시·서버 실행 실패	<code>/mcp</code> → <code>.mcp.json</code> 확인
응답이 느리거나 맥락 없음	컨텍스트 한도 도달	<code>/status</code> → <code>/compact</code> 또는 <code>/clear</code>

Windows에서 흔한 실수: MCP가 "연결 안 됨"일 때 원인의 상당수는 `.mcp.json` 경로의 역슬래시입니다. JSON에서는 `C:\mcp\server.py` 가 아니라 `C:\\mcp\\server.py` 처럼 **두 개씩** 적어야 합니다(9장). 또한 인코딩 문제로 한글 경로·CP949가 섞이면 실행이 깨질 수 있으니, 실습 폴더는 영문 경로(`C:\claude-lab`)를 권장합니다.

베스트 프랙티스: 팀이라면 자주 겪는 증상·원인·해결을 **개인 또는 팀 트러블슈팅 플레이북**으로 적어 둡니다. 같은 문제를 두 번째 만났을 때 진단 시간이 크게 줄고, 신규 합류자에게도 그대로 전수됩니다. 이 장의 실습이 바로 그 플레이북의 첫 페이지입니다.

절 요약

- 트러블슈팅의 핵심은 순서입니다. 흔하고 확인 쉬운 원인부터 점검하고 재설치는 마지막에 둡니다.

- `claude doctor` 가 만능 첫 점검이며, 증상별로 인증(`/login`)-권한(`/permissions`)-MCP(`/mcp`)-컨텍스트(`/compact`)로 줍니다.
- Windows에서는 경로 역슬래시·인코딩이 잦은 함정이니 영문 경로와 `\\` 표기를 지킵니다.

다음 단계 - 심화 자료·커뮤니티·계속 학습

도입

이 절은 책의 마지막입니다. 운전 면허를 딴 뒤 진짜 운전 실력은 도로에서 늘듯, Claude Code 도 자기 업무에 매일 써 보며 깊어집니다. 이 절에서는 책을 덮은 뒤 무엇을, 어떤 순서로 더 익히면 좋을지 로드맵을 정리합니다.

핵심 개념 설명

학습 로드맵 (learning roadmap)과 심화 자료 (advanced resources)

학습 로드맵(learning roadmap)이란, 지금 수준에서 다음 단계로 나아가기 위한 **학습 순서와 목표의 지도**입니다. 무작정 모든 기능을 한꺼번에 쫓기보다, **내 업무에 가까운 것부터** 한 가지씩 굳히는 편이 빠릅니다.

- **왜(why) 순서가 중요한가:** 기능이 많아 한 번에 다 익히려 하면 어느 것도 손에 붙지 않습니다. 쓰는 것부터 깊게 하면 그 기반 위에 다음 기능이 자연스럽게 엹니다.
- **언제(when) 무엇을 더 배우나:** 반복 작업이 보이면 슬래시 커맨드·혹으로 자동화(6·8장), 팀에 퍼뜨릴 때가 오면 settings 거버넌스(10장), 무인·대규모가 필요해지면 SDK(이 장)로 확장합니다.
- **어떻게(how) 최신을 따라가나:** Claude Code와 Agent SDK는 빠르게 바뀝니다. 공식 문서와 변경 기록(릴리스 노트)을 주기적으로 확인하고, 새 버전의 `claude doctor` 로 환경을 점검하는 습관을 들입니다.

추천 다음 단계를 단계별로 정리하면 다음과 같습니다.

1. **굳히기:** 이 책의 핵심(권한·CLAUDE.md·슬래시 커맨드)을 실제 프로젝트에서 2주간 매일 써 봅니다.
2. **자동화:** 반복되는 작업 한 가지를 커스텀 명령 또는 혹으로 자동화합니다.
3. **확장:** 사내 도구를 MCP로 붙이거나, 무인 작업을 SDK로 만들어 봅니다.
4. **공유:** 익힌 설정을 팀 저장소·플러그인으로 표준화해 함께 씁니다.

참고할 곳

대화창 없이도 공식 문서·커뮤니티를 통해 계속 배울 수 있습니다. 아래는 어디서 무엇을 얻는지 정리한 표입니다.

계속 학습 자료 지도

자료	어떤 때 보나
공식 문서(Claude Code Docs)	기능 정확한 사용법·옵션 확인
Agent SDK 레퍼런스(Python·TypeScript)	코드로 에이전트를 만들 때
릴리스 노트·변경 기록	새 기능·바뀐 동작 추적
커뮤니티(포럼·디스코드 등)	막혔을 때 사례·해결 사례 검색
<code>claude doctor · /help</code>	내 환경에서 바로 점검·도움말

팁: 무엇을 배울지 막막하면 Claude Code에게 직접 물어봅니다. "이 프로젝트에서 자주 반복하는 작업을 자동화할 방법을 제안해 줘"처럼 부탁하면, 지금 내 상황에 맞는 다음 단계를 짚어 줍니다.

절 요약

- 학습 로드맵은 "쓰는 것부터 깊게"가 원칙이며, 굳히기 → 자동화 → 확장 → 공유 순으로 넓힙니다.
- 도구는 빠르게 바뀌므로 공식 문서·릴리스 노트·`claude doctor` 로 최신 상태를 따라갑니다.
- 막히면 커뮤니티 사례를 찾고, 다음 단계는 Claude Code에게 직접 물어 정할 수 있습니다.

실습 과제

해보기: SDK 미니 에이전트 설계 + 트러블슈팅 플레이북 작성

이 책의 마무리 과제입니다. 작은 자동화 에이전트를 **설계**하고, 자주 겪을 문제의 **해결 플레이북**을 만들어 둡니다.

- **단계 1 (목표 정하기):** 무인으로 반복할 작업 하나를 고릅니다(예: "downloads 폴더 정리 안 요약"). 사람이 매번 앉지 않아도 되는 일이 좋습니다.
- **단계 2 (권한 설계):** 그 작업에 꼭 필요한 도구만 `allowed_tools` 로 적고, 위험 작업(삭제·외부 전송 등)은 권한 콜백으로 막을지 표로 정리합니다.
- **단계 3 (서브에이전트 판단):** 멀티에이전트가 정말 이득인지 판단합니다. 일이 분명히 나뉘지 않으면 단일 에이전트로 둡니다(트레이드오프 메모).
- **단계 4 (트러블슈팅 플레이북):** 인증·권한·MCP·컨텍스트 네 갈래에 대해 "증상 → 먼저 의심할 원인 → 첫 점검 명령"을 표로 채웁니다. 막혔을 때 바로 꺼내 쓸 나만의 점검표입니다.
- **점검:** 아래 두 표를 채울 수 있으면 이 책을 완주한 것입니다.

항목	내 설계
자동화 작업	...
허용 도구(allowed_tools)	...
콜백으로 막을 작업	...
단일/멀티에이전트(이유)	...

증상	먼저 의심할 원인	첫 점검
로그인 실패	...	<code>claude doctor</code> → <code>/login</code>
도구 거부	...	<code>/status</code> → <code>/permissions</code>
MCP 안 됨	...	<code>/mcp</code> → <code>.mcp.json</code>
컨텍스트 부족	...	<code>/status</code> → <code>/compact</code>

FAQ

Q1. Agent SDK를 쓰려면 개발자여야 하나요? 비개발 직군도 가능한가요? → **A1.** 일상 사용(대화창·슬래시 커맨드·CLAUDE.md)에는 SDK가 필요 없습니다. SDK는 "사람 없이 반복·자동으로" 돌려야 할 때 쓰는 코드 도구라서, 파이썬·타입스크립트를 어느 정도 다룰 수 있어야 합니다. 다

만 이 장 예시 수준의 최소 코드는 Claude Code에게 "이런 일을 하는 SDK 스크립트 초안을 만들어 줘"라고 부탁해 받은 뒤, 권한 설정만 직접 확인·조정하는 방식으로 시작할 수 있습니다.

Q2. 계획 모드와 SDK의 `permission_mode="plan"` 은 같은 건가요? → A2. 같은 개념입니다. 대화창에서 `Shift+Tab` 으로 켜는 계획 모드를, SDK에서는 옵션으로 지정할 뿐입니다. 둘 다 "읽기만 하고 변경은 하지 않으며, 계획을 먼저 내놓는" 동작입니다. 엔진이 같기 때문에 동작도 동일합니다.

Q3. 오류가 났는데 어디서부터 봐야 할지 모르겠습니다. → A3. 항상 `claude doctor` 로 시작하십시오. 설치·인증·설정·MCP 상태를 한 번에 점검해 첫 단서를 줍니다. 그다음 증상에 맞춰 인증이면 `/login`, 권한이면 `/permissions`, MCP면 `/mcp`, 컨텍스트면 `/compact` 로 좁혀 갑니다. 재설치는 설정 수준 점검이 모두 실패한 마지막 수단으로 둡니다.

Q4. 멀티에이전트를 쓰면 항상 더 빠르고 좋은가요? → A4. 아닙니다. 서브에이전트는 각자 컨텍스트를 새로 채우므로 토큰·비용·복잡도라는 고정 비용이 듭니다. 일이 "조사 따로, 구현 따로"처럼 독립적으로 나뉘고 각 부분이 충분히 무거울 때만 이득입니다. 한두 파일 수정 같은 가벼운 작업은 단일 에이전트가 더 빠르고 쌉니다.

장 요약

이번 장에서는 Claude Code를 실무 수준으로 끌어올리는 세 축을 다뤘습니다. 첫째, **고급 워크플로**로서 변경 전 계획을 검토하는 **계획 모드**, 장시간 작업을 떼어 돌리는 **백그라운드 작업**, 그리고 일이 독립적으로 나뉠 때만 이득인 **멀티에이전트**를 트레이드오프와 함께 살펴봤습니다. 둘째, **Agent SDK**가 CLI와 **같은 엔진**을 코드에서 부르는 도구임을 이해하고, `query()` 루프가 에이전트의 각 단계를 스트림으로 돌려준다는 점, 그리고 `allowed_tools`·`permission_mode`·권한 콜백·서브에이전트 옵션으로 **최소 권한의 커스텀 에이전트**를 빙는 법을 배웠습니다. 셋째, **트러블슈팅**에서는 `claude doctor` 를 첫 점검으로, 인증·권한·MCP·컨텍스트 네 갈래의 **증상 → 원인 → 해결** 순서를 익혔습니다. 마지막으로 책을 덮은 뒤의 **학습 로드맵**(굳히기 → 자동화 → 확장 → 공유)을 그렸습니다. 이로써 입문에서 실무까지, Claude Code를 안전하고 일관되게 다루는 한 바퀴를 마칩니다.

핵심 키워드

계획 모드(plan mode), 백그라운드 작업(background task), 오케스트레이션(orchestration), Agent SDK(에이전트 SDK), query 루프(query loop), 프로그래밍 가능한 에이전트(programmable agent),

권한 콜백(permission callback), 사람 확인(human-in-the-loop), 문제 해결(troubleshooting), 학습 로드맵(learning roadmap)

다음 장 미리보기

이 장이 이 책의 마지막입니다. 다음 단계는 더 읽을 장이 아니라 **직접 쓰는 일**입니다. 위 실습 과제로 나만의 미니 에이전트와 트러블슈팅 플레이북을 완성하고, 오늘부터 실제 업무에 한 가지씩 적용해 보십시오. 그것이 이 책 다음의 진짜 1장입니다.